Developer Guide

SAP UI development toolkit for HTML5 (SAPUI5) is a cross-browser JavaScript library for building rich internet applications. The key features are:

- well-designed API, easy to consume and use
- extensible UI component model, including tooling support
- high performance, SAP product standard compliant
- powerful theming support based on CSS
- provides Ajax capabilities
- based on open standards like OpenAjax, CSS, HTML 5, etc.
- using and including the popular jQuery library

Get Started

Lets get started with SAPUI5: read an overview, write and run your first SAPUI5 application:

- Introduction
- Writing a simple SAPUI5 Application ("Hello World")
- Writing an advanced SAPUI5 Application

General Information

General information of SAPUI5 - for the Runtime and Development Tools - can be found here:

- Browser Support
- Deployment Scenarios
- Security Information
- Compatibility Rules
- Release Notes
- Terminology

SAPUI5 Runtime

This section provides the documentation of the SAPUI5 Runtime.

- API Reference
- Runtime Concepts

SAPUI5 for Mobile

This section provides the documentation for the mobile control libraries `sap.m` and `sap.makit`

- SAPUI5 for Mobile

SAPUI5 Theme Designer

This section provides the documentation for the Application Theme Designer

- Theme Designer

SAPUI5 Development Tools

This section provides the documentation of the SAPUI5 Development Tools for Eclipse which support

application developers to build SAPUI5 applications.

SAPUI5 Application Tool

- Develop a Simple Application with SAPUI5 Tools
- Develop a View according to the MVC Concept
- Local Testing in Eclipse

Synchronizing with ABAP-based Repository

- Synchronizing with UI5 Repository based on BSP Repository in Suite systems

The UI development toolkit for HTML5

The toolkit for HTML5 is a JavaScript OpenAjax-compliant library for building rich internet applications. It fully supports SAP's product standards as well as RIA-like client-side features. Apart of this, it is extensible regarding custom controls, application services, theming, or custom branding. The framework is lightweight and easy to consume, so that you can combine it easily with 3rd-party JS libraries to design an HTML page and integrate it into various application frameworks or server platforms.

The Artifacts in the Framework

The top-level structural unit is called a *UI library*. Libraries are the master artifacts in the extensibility concept. They bundle a set of controls and related types and make them consumable by web applications. There are predefined and standard libraries, like `sap.ui.commons` with many commonly used controls. At the same time, it treats custom UI libraries as first-class citizens, making it easy for you to write and use your own controls alongside the predefined ones.

A *UI element* is the basic building block of our user interfaces; it is a reusable entity with *properties*, *events*, *methods*, and relations. The most important relations are *aggregations* to other UI elements, and in this way a tree structure of elements can be created.

From a developer point of view, a *control* (e.g. `Button`, `Label`, `TextField`, or `Table`) is the most important artifact. It is an object which controls the appearance and user interaction of a rectangular screen region. It is a special kind of user interface element which can be used as the root of such a tree structure. In this way, it serves as an entry point, especially for rendering. Besides controls, there are also other *non-control elements*, which cannot be used as the root of such a tree structure, but only as a dependent part within it (e.g. `TableRow`, `TableCell`).

A *property* has a name and an associated data type. It has a well-defined default value expressed as a literal of that data type. Properties are accessible to application code via the element's API as getters and setters, but are also used by a control's renderer in a read-only way.

Data types are first class entities in the meta model. This allows reuse of types across libraries and extensibility of the type system. The core library (technically, this is the `sap.ui.core` library) already defines a core set of types that can be used in other libraries.

An *aggregation* is a special relation between two UI element types. It is used to define the parent/child relationship within the above mentioned tree structure. The parent end of the aggregation has cardinality 0..1, while the child end may have 0..1 or 0..*. The element's API - as generated by the tools - offers convenient and consistent methods to deal with aggregations (e.g. to get, set, or remove target elements). Examples are table rows and cells, or the content of a table cell.

An *association* is another type of relation between two UI element types which is independent of the parent/child relationship within the above mentioned tree structure. Directed outgoing associations to a target of cardinality 0..1 are supported. They represent a loose coupling only and are thus implemented by storing the target element instance's ID. The most prominent example is the association between a label and its field.

An *event* has a name as well as any number of parameters. The element's API offers support to manage event subscriptions.

How to Develop an Application

As an application developer, you need to declare the list of used UI libraries when including the framework's bootstrap script. After that, you can use all controls provided by these libraries to construct one or more control trees and connect these to parts of your HTML page. You program against a well-designed control API, mostly calling setters for properties and aggregations and managing event subscriptions. The framework also supports the JavaScript Object Notation (JSON) to initialize controls with a reduced typing effort. Within an event handler, you may want to retrieve control instances by ID

and use getters for properties. Of course, you are free to modify a control's state from within an event handler. Rerendering will be done automatically by the framework.
This way an application can be created in 20 seconds.

The SAPUI5 Tools for Eclipse provide wizards and support for creating SAPUI5 Applications in an easy way, which generates the necessary Application skeleton.

How to Develop a UI Control

As a control developer, you create or modify UI libraries and their pieces, i.e. controls and types. You define the set of properties your control provides as well as events or aggregations. A major task is the implementation of a control-specific renderer, which knows how to create suitable HTML markup for a given control instance, taking its current state into account. A renderer is written in JavaScript and produces HTML output which is styled by means of CSS. Such style sheets are another important part of a UI library.

- Introductio
- Writing a simple Applicatior
- Writing an advanced Applicatior

SAPUI5 Hello World

This page explains **how to create and run a simple SAPUI5 application from scratch within twenty seconds** (with some practice... the current record is 16 seconds).
If you are interested in exactly doing this without reading too much, you can **jump right [down to the next chapter]** on this page.

Explanation

As SAPUI5 is a client-side web UI library (i.e.: runs in the browser), a SAPUI5 application is typically an HTML page (plus potentially many more files).

UI5 is implemented in JavaScript, so for loading UI5, its bootstrap just needs to be included with a `<script>` tag. The last two attributes select the visual design to apply initially (other choices would be "sap_hcb" or "sap_platinum") and the UI control library/libraries to use ("sap.ui.dev" would be another one). In your scenario you need to make sure the URL points to a SAPUI5 installation.

```
<script id="sap-ui-bootstrap"
        src="resources/sap-ui-core.js"
        data-sap-ui-theme="sap_goldreflection"
        data-sap-ui-libs="sap.ui.commons"></script>
```

SAPUI5 UI elements are created and modified programmatically:

```
// create the button instance
var myButton = new sap.ui.commons.Button("btn");

// set properties, e.g. the text (there is also a shorter way of setting several properties)
myButton.setText("Hello World!");

// attach an action to the button's "press" event (use jQuery to fade out the button)
myButton.attachPress(function(){$("#btn").fadeOut()});
```

There is also a shorthand notation based on JSON for setting multiple properties; you could also write:

```
var myButton = new sap.ui.commons.Button({text:"Hello World!",tooltip:"Hello Tooltip!"});
```

Finally you need to tell UI5 where the UI control should be placed. You can just give the ID of an element in the page to do so:

```
// place the button into the HTML element defined below
myButton.placeAt("uiArea");
```

This element must exist somewhere in the HTML page, so you need to put the following code to the desired place within the `<body>`:

```
<div id="uiArea"></div>
```

(As of now you can only put *one* UI5 control into a parent; for adding more UI5 controls you need to either define more parents or to use a UI5 layout control which can arrange many children.)

An alternative way to create and initialize the control in a more jQuery-style manner is also available:

```
$(function(){
   $("#uiArea").sapui("Button", "btn", {
      text:"Hello World!",
      press:function(){$("#btn").fadeOut();}
   });
});
```

As a minor detail, the `<body>` should have a certain CSS class, so the page background and some other styles are properly set:

```
<body class="sapUiBody">
```

And two meta-Tags at the beginning of the <head>: the first to ensure that Internet Explorer 8+ uses

its most-standards-compliant rendering mode. And the second to let all browsers treat the file as UTF-8 encoded (assuming that you use this encoding when editing/saving the file):

```
<meta http-equiv="X-UA-Compatible" content="IE=edge" />
<meta http-equiv="Content-Type" content="text/html;charset=UTF-8"/>
```

And how to do it in 20 seconds?

Assumption for these instructions to work exactly as described: you have a Windows Computer (other OS will work similarly), Internet Explorer 9+ (having the security option: access data across domains turned on for the proper zone), FireFox 13+, Safari 5+ or Chrome 20+ and you know where you can refer to SAPUI5 on some server.

Important: the URL in the script tag is pre-filled as " https://sapui5.netweaver.ondemand.com/resources/sap-ui-core.js" (which was at least available at the time of writing this document). Test this URL first: if it does not work, you need to find a different installation or deploy SAPUI5 on your own server.
Also note that the version of SAPUI5 deployed on https://sapui5.netweaver.ondemand.com/ may be updated with a delay of some days or weeks after a new release of SAPUI5, even though we try to keep them in sync. This example will work nevertheless.

(In Internet Explorer you might have to accept a security question or even enable the security option: access data across domains turned on for the proper zone.)

1. Right-click your desktop, select "New" → "Text Document"
2. Name the new file e.g. "ui5.html" and accept the extension change warning
3. Right-click the new file and select "Edit" (make sure it opens in Notepad or so, NOT in MS Word!)
4. Copy&Paste the below HTML code and save the file (e.g. press Ctrl-S) (remember: the SAPUI5 URL must be adapted)
5. Drag the file to this browser window
6. End. Nothing else. That was it.

```
<!DOCTYPE html>
<html>
<head>
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta http-equiv="Content-Type" content="text/html;charset=UTF-8"/>
    <title>SAPUI5 in 20 Seconds</title>

    <!-- 1.) Load SAPUI5 (from a remote server), select theme and control library -->
    <script id="sap-ui-bootstrap"
        src="https://sapui5.netweaver.ondemand.com/resources/sap-ui-core.js"
        data-sap-ui-theme="sap_goldreflection"
        data-sap-ui-libs="sap.ui.commons"></script>


    <!-- 2.) Create a UI5 button and place it onto the page -->
    <script>
        // create the button instance
        var myButton = new sap.ui.commons.Button("btn");

        // set properties, e.g. the text (there is also a shorter way of setting several
properties)
        myButton.setText("Hello World!");

        // attach an action to the button's "press" event (use jQuery to fade out the button)
        myButton.attachPress(function(){$("#btn").fadeOut()});

        // place the button into the HTML element defined below
        myButton.placeAt("uiArea");

        // an alternative, more jQuery-like notation for the same is:
        /*
        $(function(){
            $("#uiArea").sapui("Button", "btn", {
                text:"Hello World!",
                press:function(){$("#btn").fadeOut();}
            });
        });
        */
```

```
    </script>

</head>
<body class="sapUiBody">

    <!-- This is where you place the UI5 button -->
    <div id="uiArea"></div>

</body>
</html>
```

Result

If you followed the steps above you should now see a button like this which fades out when clicked:

Next Steps

You could now...

- ...add more buttons
- ...let them do trickier things
- ...use an different visual theme mentioned above, e.g. "sap_ux"
- ... find out about further properties and events of button controls and use those
- ... find out about further controls and add those

Writing an Advanced SAPUI5 Application

Via this procedure, you develop a Web application that uses the UI5 framework. You create the basic entities which are the UI5 application project as well as the UI5 application itself which consists - from the design-time and wizard approach - of the skeleton containing view(s) and controller.

Prerequisites

- You have installed the *SAPUI5 Application Development* feature in your Eclipse installation.

Using the Application Wizard

1. Open your Eclipse IDE where you have installed SAPUI5 tools.

2. Create a UI5 application project by triggering the corresponding wizard entry via *File → New → Other → SAPUI5 Application Development → Application Project*.

3. Make the required entries in the application project wizard, keep *Desktop* and *Create an Initial View* selected, and go to the wizard screen for view type definitions.

4. Choose one of the following view types: *JSview, XMLview, JSONview, HTMLview*.

5. Finish the wizard.

As a result, the following application parts are created:

- A view file of the chosen type
- A controller file of type JS
- An index.html file containing references for the libraries sap.ui.core and sap.ui.commons, the used theme where the default theme is sap_goldreflection, as well as information for the script type and the script id. In a second script block, the index file refers to project name, view type and name; a content default for placing the controls on the UI later on; and the ARIA application role.

Type and Content of the View Files

The view file is named `viewname.view.viewtype.` It contains a getter method for the function which returns the view, as well as a method for content creation. Depending on the chosen view type, the generated view file looks as following:

```
sap.ui.jsview("myproject.myview", {
      getControllerName : function() {
         return "myproject.myview";
      },
      createContent : function(oController) {
      }
});
```

```
<core:View xmlns:core="sap.ui.core" xmlns:mvc="sap.ui.core.mvc" xmlns="sap.ui.commons"
           controllerName="myproject2.myview2" xmlns:html="http://www.w3.org/1999/xhtml">
</core:View>
```

```
{
  "Type":"sap.ui.core.mvc.JSONView",
  "controllerName":"myproject3.myview3",
  "content":[{
  }]
}
```

The Controller File

The controller file is of same type and content for each view type. The onInit function method is called when the controller is instantiated and the controls have been created. The method can be used for

event handler bindings, other one-time initializations, and view modification (before the first rendering). The onBeforeRendering method is invoked before the view is re-rendered. onAfterRendering is used for post-rendering manipulations of the HTML, for example. onExit is used to free resources, for example.

```
sap.ui.controller( "myproject.myview" ,{

// onInit: function(){
//
//},

// onBeforeRendering: function(){
//
//},

// onAfterRendering: function(){
//
//},

// onExit: function(){
//
//}
});
```

index.html

```
<!DOCTYPE HTML>
<html>
	<head>
		<meta http-equiv="X-UA-Compatible" content="IE=edge">
		<meta http-equiv="Content-Type" content="text/html;charset=UTF-8"/>

		<script src="resources/sap-ui-core.js"
			id="sap-ui-bootstrap"
			data-sap-ui-libs="sap.ui.commons"
			data-sap-ui-theme="sap_goldreflection" >
		</script>
		<!-- add sap.ui.table,sap.ui.ux3 and/or other libraries to 'data-sap-ui-libs' if
required -->

		<script>
			sap.ui.localResources("myproject");
			var view = sap.ui.view({id:"idmyview1", viewName:"myproject.myview",
                                        type:sap.ui.core.mvc.ViewType.JS});
			view.placeAt("content");
		</script>

	</head>
	<body class="sapUiBody" role="application">
		<div id="content"></div>
	</body>
</html>
```

Next Steps

- Read more details about Creating an Application Project and Creating a View
- Try out Code Completion, Templates and Snippets
- Testing the SAPUI5 Application

Browser Support in SAPUI5

SAPUI5 Browser Support Note

See also the SAPUI5 Browser Support Note **1716423**

SAPUI5 for Desktop (Core and Standard Libraries)

As SAPUI5 for Desktop is a control library that depends on the availability of CSS3, HTML5, new JavaScript API and JavaScript speed, mainly the HTML5 capable browsers are supported.

You can see the official browser support within the product availability matrix (PAM) under http://service.sap.com/PAM, specifically the one for NW UI Extensions.

Supported Browsers

- only for Windows, not for MAC
  - Internet Explorer 9 and upwards, so including IE10
  - Firefox 17 (aka Firefox Extended Support Release - ESR) and latest version
  - Chrome latest version (currently version 24)
- only for MAC OS X, not for Windows
  - Safari 5.1 and upwards

Browsers With Restricted Support

- Internet Explorer 8: There are degradations in visual design and over time also restricted functionality. So far, there is no functional restriction but when HTML5 features are added to SAPUI5 which are not available in IE8, there is not necessarily an alternative solution created by the SAPUI5 teams.

Browsers Which Aren't Supported

- Internet Explorer 6 and 7 and in general all browsers which are not mentioned above are not supported.

Browser Support Policies By The Different Vendors

- Internet Explorer:
  - In general, IE is seen as a "component" of a "parent major product" = Windows. Every IE version is supported on a particular Windows version for as long as the Windows version is supported Explanation: "Internet Explorer is considered as a component of the operating system (OS) for which it was released. The support timelines for IE are inherited from the OS and its associated service packs. Basically, this means that the versions of Internet Explorer that shipped for a specific OS or service pack will be supported with the support lifecycle of the OS or service pack. Support for older versions of IE will not end unless we ship a replacement version of IE in a future OS service pack....As per the policy, we will not end support previous versions of Internet Explorer on supported operating system versions."
  - IE6 is support is coupled with the XP lifetime even though Microsoft strongly encourages users to switch to newer browsers.

- Firefox: SAPUI5 supports ESR and the current version (at the time of writing, it is version 13)
  - Firefox ESR can be downloaded but it "will be maintained with security and stability updates" only. Features and performance updates will take more time to get into ESR version.

- Safari: Apple doesn't seem to issue any information which browser version is supported until when.

- There is a concept of vintage products that have been discontinued more than 7 years ago, but this refers to hardware and not software
- The assumption is that Apple only really supports the last version of their browser

SAPUI5 for Desktop (Contributions)

Browser Support for VIZ Charting

The VIZ charting library (sap.viz) relies on the open source component D3 which in turn relies on the availability of Scalable Vector Graphics (SVG). As SVG is not supported by IE8 and not fully supported by FF ESR, the VIZ charting library is also not supported on those browsers.

SAPUI5 for Mobile

Control Library sap.m

The supported platforms depend on the available themes: SAP Blue Crystal, SAP Mobile Visual Identity

The theme 'SAP Blue Crystal' is **not supported** by the controls: Carousel

The theme 'SAP Mobile Visual Identity' is **only supported** by a subset of the controls: ActionSheet, ActionListItem, App, BusyDialog, BusyIndication, Button, CheckBox, Carousel, CustomListItem, DateTimeInput, Dialog, DisplayListItem, FlexBox, GrowingList, Input, Image, InputListItem, Label, List, NavContainer, MessageToast, Page, Bar, Popover, PullToRefresh, RadioButton, SegmentedButton, StandardListItem, ScrollContainer, SearchField, Select, Slider, Switch, SplitApp, Text, TextArea

Control Library sap.makit

The library `sap.makit` supports the same platforms as `sap.m` in theme `Mobile Visual Identity`.

SAPUI5 Runtime

All parts of SAPUI5 runtime resources are compatible with all browsers mentioned above, i.e. the ones that are mentioned for desktop and mobile libraries. Exceptions to this rule are the Open Source components that are placed in SAPUI5's thirdparty folder, which is to be found under resources\sap\ui\thirdparty. SAPUI5 guarantees that the parts of these components which are directly used within the SAPUI5 runtime are compatible to all browsers supported by the desktop and mobile library unless stated differently. However, not all parts of the components under the thirdparty folder might support all browsers, in particular Internet Explorer's version 8 might not be supported by all components. For the general compatibility of the components under thirdparty check their corresponding web pages.

Deployment Scenarios

A Web application developed using the SAPUI5 framework with runtime and tools can run on various platforms:

- SAP NW ABAP Server,
- SAP NW Java Server,
- SAP NW Cloud,
- Open Source Java Application Server, and
- static Open Source Web server.

Depending on the chosen development platforms provided by SAP, you find the required deployment steps in the corresponding platform documentation for SAP ABAP Server or SAP NW Cloud.

Using a SAP Server

SAP NW ABAP Server

For the deployment on the ABAP Site you can use the SAPUI5 ABAP Repository Team Provider feature to store your application artifacts in the SAPUI5 repository based on BSP repository. After that you can run your application from the ABAP system.
With the deployment on the sophisticated ABAP development infrastructure you benefit from the ABAP transport infrastructure as well as from the integrated translation infrastructure for the language-dependent application .properties files.

SAP NW Java Server

Beginning with SAP NW Java 731 the SCA *UISAPUI5_JAVA* is available which can be used to install the SAPUI5 Java and JavaScript runtime components. This will install the SAPUI5 components as application *sapui5*. Other applications can refer to SAPUI5 with a server absolute URL - e.g.: `/sapui5/resources/sap-ui-core.js`.

SAP NW Cloud

For applications which shall run in the cloud and which are using the SAPUI5 user interface technology, see topic Deploying Using SAP NW Cloud Console Client in the NW documentation. Or go to Deploying an Application on SAP NW Cloud (same NW documentation platform). For the open source platforms, the SAPUI5 Framework supports the standard deployment procedures as they are available.

Using Open Source Servers

Java Application Server

To deploy an application on a Java Web Container, our recommendation is the Apache Tomcat Server where version 6.x is supported. You find information on the procedure on Apache's homepage http://tomcat.apache.org/tomcat-6.0-doc/deployer-howto.html. Generally, you would proceed an application .war file which you can manually create at any time.

Static Web Server

As Web servers, you can use the Apache HTTPD server (download it from the Apache.org homepage); or Microsoft's Internet Information Services (IIS), for this go via Windows Program Manager. In Apache HTTPD, the static archives can easily be unzipped to the /htdocs folder of the HTTPD server. In the Microsoft IIS web server, you would extract a static archive into the /wwwroot folder.

SAPUI5 Security Information

Introduction

Security With SAPUI5

This guide covers security aspects of the usage of SAPUI5. It is targeted to SAPUI5 application and control developers as well as to system administrators running applications based on SAPUI5.

It is important to understand that SAPUI5 is a clientside JavaScript library, so while the library itself is designed and tested to be secure, it can not ensure the application to be secure. Unlike Web Dynpro, where the application is built against an abstract programming model and the framework handles all the HTML rendering, JavaScript code and communication with the browser, in SAPUI5 the application controls the HTML output, it provides own JavaScript code which is executed on the client and it handles client/server communication.

While this brings a lot of freedom and possibilities for the application, it comes with a lot of responsibility with regards to security. Application developers need to understand the security threats and actively prohibit exploitation. Also the correct configuration of the HTTP server which is used is important.

This also means common security mechanisms, which are taken for granted, like User Authentication, Session Handling, Authorization Handling or Encryption are not part of SAPUI5 and need to be handled by the server side framework and/or custom code of the application.

Why Is Security Necessary?

With the increasing use of distributed systems and the Internet for managing business data, the demands on security are also on the rise. When using a distributed system, you need to be sure that your data and processes support your business needs without allowing unauthorized access to critical information. Protection of the user's personal data must be guaranteed and legal regulations regarding this must be complied with. User errors, negligence, or attempted manipulation on your system should not result in loss of information or processing time. These demands on security apply likewise to SAPUI5 . To assist you in securing SAPUI5, we provide this Security Guide.

About this Document

The security guide comprises the following main sections:

- Before You Start Links to general security guides
- Architectural Overview How SAPUI5 is embedded and interfacing with the application
- Browser Security Client related security aspects
- Transport Security Security of data transport between client and server
- Server Security Serverside security considerations with SAPUI5
- Third Party Libraries jQuery and datajs are part of SAPUI5
- Secure Programming Guide What needs to be done in the application
- Using the SAPUI5 Repository based on BSP Repository

Before You Start

SAPUI5 is not bound to any server implementation or serverside programming language, so can be used with Netweaver ABAP, Java, HANA XSEngine or any standard web server. Therefore, the corresponding Security Guides also apply to SAPUI5.

Fundamental Security Guides

Please also refer to the global and other security guides of involved technologies and platforms.

Additional Information

For more information about specific topics, see the Quick Links as shown in the table below.

Content	Quick Link on the SAP Service Marketplace or SDN
Security	http://sdn.sap.com/irj/sdn/security
Security Guides	http://service.sap.com/securityguide
Related SAP Notes	http://service.sap.com/notes
	http://service.sap.com/securitynotes
Released platforms	http://service.sap.com/pam
Network security	http://service.sap.com/securityguide
SAP Solution Manager	http://service.sap.com/solutionmanager
SAP NetWeaver	http://sdn.sap.com/irj/sdn/netweaver

Architectural Overview

SAPUI5 is a JavaScript library based on jQuery. It is embedded in the application using a script-tag and triggers additional requests for on-demand-loading of JavaScript classes, stylesheets and other resources. For Java and ABAP a special resource handler is offered, which provides extended capabilities, and is used for all SAPUI5 internal requests.

The application usually has a serverside part and a clientside part. The serverside part can be based on any web framework, the clientside part is a web application, which is utilizing SAPUI5 for its user interface.

Data Protection and Privacy

SAPUI5 does not store or provide access to any person-related data. If an application built with SAPUI5 deals with person-related data, it has to take care of the data protection rules of the target countries of the application. This includes usage of proper authentication, authorization and encryption (e.g. SSO and usage of https), as well properly securing and logging access to person-related data. For more inforamtion regarding data protection and privacy, please see the security guide of your server side framework.

Browser Security

First of all, the browser is an untrusted client by design. The server can not rely on any information sent from the browser, as a malicious user can use a JavaScript debugger to tamper with the client code or a proxy server like fiddler to modify request data. All input validation on the client is just for convenience, the server always has to validate all the data coming from the client again.

The browser also offers some possible attack vectors, especially Cross-Site-Scripting (XSS), which has to be taken care of by the application.

Cross-Site-Scripting

Cross-Site-Scripting is the most prominent security issue of web applications within the last years and also the mosts dangerous one, as it allows so many ways of exploitation. Once malicious code is running within your browser, it can be used to steal your session cookies, to trigger requests within the current session, or even to exploit a known browser vulnerability to do native code execution.

For SAPUI5 applications XSS vulnerabilities can exist on different levels:

- Within the HTML page or custom data transports sent to the browser from the server
- Within the JavaScript Code of the application, which is processing server responses
- Within the HTML renderers of SAPUI5 controls

SAPUI5 can only prevent cross site scripting in the processing and rendering of controls. For that purpose there is input validation on all typed Element properties and output encoding done in the renderer class of Controls (see Secure Programming Guide for Control Developers). Please be aware that there are exceptions to this, for controls which are especially built to include arbitrary HTML (like sap.ui.core.HTML).

Application is responsible for proper output encoding of all content embedded into the HTML page itself, as well as for encoding of JSON or XML data sent to the client and secure processing of this data. Of course the application also has to take care of security of custom controls provided by the application.

HTML5

HTML5 offers a lot of new functionality which also brings a lot of potential new security issues. This just an overview of some of the new features and possible security issues when they are used.

Local Storage

All browsers are now offering a local storage API. This API can be used to store a limited amount of data on the browser. Access to this data is limited to JavaScript code running from the same domain as it has been stored. SAPUI5 offers helper functions to access the local storage on different browsers.

The local storage of browsers is not a secure storage, so while it can be used for static data, like enumerations, applications must not store any user or application data within the local storage.

By default SAPUI5 is not using the local storage, but it can be enabled for the history-capability of Dropdown- and ComboBoxes for uncritical data.

WEBGL

While more and more browsers are supporting WEBGL by default, WEBGL allows accessing the graphics API of the computer on a very low level, which may also lead to low level exploits. This is the main reason Internet Explorer has no support for WebGL at all, Microsoft recently stated, that they are not going to support WebGL as they think it can not be implemented in a secure way.

SAPUI5 is currently not using WEBGL.

WebSockets

While WebSockets offer great new possibilities for the client/server communication of web applications, there have been many security issues rising while the first implementations were done by the browser vendors. Standardization of WebSockets has reached a stable state with RFC 6455 and is now implemented beginning with Chrome 16, Firefox 11 and Internet Explorer 10. Even if the browser implementations themselfes prove to be secure, using Web Sockets may require additional security measures on the client.

SAPUI5 is currently not using WebSockets.

postMessage/onMessage

This is another great feature in the HTML5 area, which can lead to massive security issues, when not used correctly. postMessage allows inter-window-communication between windows from different domains. So basically this opens a hole in the same origin policy currently implemented in the browser. As soon you are subscribing to the onMessage event, you can receive messages from any other browser window, the application is responsible to check the originating domain and only to process messages which have been sent by trusted domains.

SAPUI5 is utilizing postMessage for its debugging and tracing functionality.

Transport Security

Of course the best security on the client and server doesn't help, if the data transport between client and server can be read, intercepted or even modified by an attacker. Per default HTTP communication is stateless and unencrypted, which makes it necessary to configure it in a way that it is using encrypted connections and to add session handling on top using either cookies or URL rewriting.

Encryption

Sending the HTTP protocol over a SSL secured connection is not only standardized, but also required for SAP applications.

SAPUI5 fully supports usage of HTTPS, but there are some restrictions regarding the CDN version of SAPUI5 when HTTPS is used. It is recommended to enable or at least to test SSL connections in an early stage of application development, as usually switching to HTTPS causing some issues. First of all, when the application is started using HTTPS, the SAPUI5 library also has to be loaded from an HTTPS server. Second Internet Explorer 8 and 9 have some additional restrictions regarding cross origin requests with HTTPS, which are related to the security zone concept.

Session security

Even if the data transport is secured using SSL, there are possibilities to hijack such a secure connection and sending malicious requests from the client. Cross site request forgery and session fixation are two of the prominent examples of this class of attacks.

SAPUI5 does only provide XSRF prevention for the data which is sent to the server by SAPUI5. Currently this only happens in the OData Model, where a XSRF token is read from the server and used for subsequent write requests.

Application is responsible for using XSRF header or other mechanisms to prevent XSRF for all other server communication triggered by the application.

Server Security

SAPUI5 only ships a small serverside part, to support loading of resources by the client framework. Usage of the resource handlers is not mandatory, SAPUI5 also offers a static version of the libraries, which can be used with an arbitrary HTTP server.

Cross Origin Resource Sharing

Usually the XMLHttpRequest for security reasons does only allow accessing resources from the same domain as the originating document. As their are a lot of web based services available today, starting with RSS or Atom feeds, WebServices or OData services, there was a need to be able to also access data sources from different domains within the browser, which was adressed with the CORS (Cross Origin Resource Sharing) standard. This allows the server to set special headers on their responses, which are telling the XMLHttpRequest object, whether it is allowed to process the requested data or not.

This CORS capability also plays an important role in SAPUI5 based applications. In case the application itself and the data visualized are coming from different servers, the CORS header has to be configured correctly on the data providing server, to allow the application server domain to access the data.

SAPUI5 is using CORS header on its CDN based library, to be able to load additional scripts, styles and resources from the CDN server.

Resource Handler (Java)

The Resource Handler for Java provides some configuration options (context parameters in the web.xml). It is possible to enable CORS, configuring the cache control and activating a development mode. Details about the configuration options can be found here: Resource Handling.

In case of productive usage of the Resource Handler you must not have the development mode activated. You have to remove the configuration option `com.sap.ui5.resource.DEV_MODE` in the web.xml.

Resource Handler (ABAP)

The Resource Handler for ABAP is not configurable. It is used to serve the resources from the mime repository.

Resource Handler for Application Resources (ABAP, NW 7.X)

The Resource Handler for Application Resources for ABAP, NW 7.X is not configurable. It is used to serve the resources from the SAPUI5 Repository based on BSP Repository, see also Executing SAPUI5 Applications from the SAPUI5 Repository

Proxy Servlet (Java)

SAPUI5 is providing a `SimpleProxyServlet`. This proxy servlet can be used on a local server only for local requests to access data from other domains. This is useful to avoid cross domain issues when fetching data from another domain for testing purposes. Also for the SAPUI5 tools this is required since the local testing needs to access data from the remote ABAP server. Due to security reasons this proxy servlet is limited to localhost usage only and cannot be used by requests from other domains.

Third-Party Libraries

Libraries shipped with SAPUI5

SAPUI5 ships with a handful of third party libraries. jQuery is mandatory as SAPUI5 is based on it and datajs is needed in case OData services should be used.

jQuery

jQuery does not have any security related documentation on their site, but they are known to be aware of security and usually reacting quickly in case security issues are found within their library.

SAPUI5 is including jQuery in different versions together with their own libraries, so also has the possibility to add custom security fixes to jQuery, if necessary.

datajs

datajs does not have any security related documentation on their site.

SAPUI5 is including the datajs library and can add custom security fixes, if necessary.

Libraries included by the application

Applications based on SAPUI5 are allowed from a technical point of view to include arbitrary custom libraries within their application. Of course SAPUI5 can not give any statement about the security of third party libraries and can not ensure security of third party libraries. The application has full responsibility for doing an security assessment of third party libraries before using them and for embedding and using them in a secure manner.

Secure Programming Guide

Input Validation

Application point of view: Input validaton of user input, must be done on the server, optional on the client, can be achieved using two way data binding and model types. The application can create custom types by extending the SimpleType? class. This is explained in more detail in the databinding documentation.

- `oInput.bindValue("/path", new sap.ui.model.type.Float())` Ensures only a Float value can be entered, otherwise a ParseError will be thrown
- `oInput.bindValue("/path", new sap.ui.model.type.String({}, {maxLength: 20}))` Ensures that maximal 20 characters can be entered, otherwise a ValidationError will be thrown

Control point of view: Input validation of control properties, so integer proeprties only accept integers, enumeration properties only accept an existing enumeration value. While this sounds obivous, in JavaScript it is not. The type system of JavaScript doesn't do type validation on assignment.

Try to avoid string properties whenever possible. SAPUI5 offers special types like CSSSize or URL for control properties, which should be used when applicable. If there is a fixed set of possible values, please create an enumeration.

Output Encoding

All data sent from the server must be properly output encoded according to the context they are contained in. For more information please see the XSS Secure Programming Guide. Content which is created on the client side, either for display within the browser or for data transport, needs to be properly output encoded with the encoding methods provided by SAPUI5. There are methods for encoding HTML, XML, JavaScript, CSS and URI components.

When developing custom controls, the following two methods should be used within the control renderer for HTML encoding:

- `RenderManager.writeEscaped(sString)` Encodes the given string and writes it to the HTML output as content
- `RenderManager.writeAttributeEscaped(sString, sString)` Encodes the given string and writes it to the HTML output as an attribute

The encoding library does encode according to the XSS Secure Programming Guide and should be used whenever encoding is needed by the application. It offers the following API:

- `jQuery.sap.encodeCSS(sString)` Encode the string for inclusion into CSS string literals or identifiers
- `jQuery.sap.encodeHTML(sString)` Encode the string for inclusion into HTML content/attribute
- `jQuery.sap.encodeJS(sString)` Encode the string for inclusion into a JS string literal
- `jQuery.sap.encodeURL(sString)` Encode the string for inclusion into an URL parameter
- `jQuery.sap.encodeXML(sString)` Encode the string for inclusion into XML content/attribute

All controls from SAPUI5 libraries properly encode their data, except for HTML-control and XMLView. The latter two are explicitely built to display arbitrary HTML content. If applications use these two controls AND provide unsecure HTML content to them, they have to check/validate the content on their own. Note that using an XMLView with application controlled, secure HTML content together with standard UI5 controls (other than HTML and XMLView) containing potentially unsecure data, is also safe. Only untrusted HTML content is critical.

SAPUI5 includes the CAJA HTML sanitizer, which is used by the HTML control and the RichTextEdit?.

URL Validation

URL validation should take place on the server side when possible. In case URLs are entered on the

client side or are loaded from an external service, SAPUI5 offers an URL validator, which can be used to whether a URL is well formed and properly encoded. It also contains a configurable white liste to restrict URLs to certain protocols or certain hosts. Initially, the white list only checks for the protocols `http`,`https` and `ftp` but nothing else. Applications should define their own whitelist.

SAPUI5 is using URL validation for outgoing requests (e.g. links). SAPUI5 offers the following API for URL validation within the application.

- `jQuery.sap.getUrlWhitelist()` Gets the whitelist for URL validation
- `jQuery.sap.addUrlWhitelist(protocol, host, port, path)` Adds a whitelist entry for URL valiadtion
- `jQuery.sap.removeUrlWhitelist(iIndex)` Removes a whitelist entry for URL validation
- `jQuery.sap.validateUrl(sUrl)` Validates an URL.
- `jQuery.sap.clearUrlWhitelist()` clears the whitelist for URL valiadtion

Cache Settings

The application has to take care, that caching of data is disabled, by setting appropriate HTTP Headers on the server side.

Static resources from SAPUI5 or from the application are not security relevant and are excluded from this rule, so they can safely be cached on the client.

User Management / Authentication

SAPUI5 doesn't provide any authorization or user management. An application which implements such facilities based on SAPUI5 has to make sure that SSL is enabled to prevent cleartext passwords sent over the wire. Applications must not store any logon information on the client.

Local Storage

The local storage of browsers is not a secure storage, so while it can be used for static data, like enumerations, applications must not store any user or application data within the local storage.

The following API is offered by SAPUI5 to access the local storage

- `jQuery.sap.storage.get(sId)` Retrieves the state string stored in the session under the key sStateStorageKeyPrefix + sId
- `jQuery.sap.storage.getType()` Returns the type of the storage.
- `jQuery.sap.storage.put(sId, sStateToStore)` Stores the passed state string in the session, under the key sStateStorageKeyPrefix + sId
- `jQuery.sap.storage.remove(sId)` Deletes the state string stored in the session under the key sStateStorageKeyPrefix + sId
- `jQuery.sap.storage.clear()` Deletes all the entries saved in the session. CAUTION: This method should be called only in very particular situations, when a global erasing of data is required. Given that the method deletes the data saved under any ID, it should not be called when managing data for specific controls.

Using the SAPUI5 Repository based on BSP Repository

Using the SAPUI5 Repository Team Provider

The SAPUI5 Repository Team Provider connected against a NW 7.31 ABAP system with UI-AddOn or a NW 7.40 (from SP1) can be used to synchronize SAPUI5 application resources between Eclipse and the SAPUI5 Repository on the ABAP system.

For usage of the SAPUI5 Repository Team Provider, also see the Security Guide for ABAP Development Tools which is part of the ABAP Development User Guide and the SAP NetWeaver Security Guide.

Needed Authorization Objects

Authorization Object	Description
S_DEVELOP	The authorization object S_DEVELOP is needed to create, update and delete SAPUI5 applications in the SAPUI5 Repository.
S_ICF_ADM	The authorization object S_ICF_ADM is needed to create the SAPUI5 application specific ICF node under */sap/bc/ui5_ui5/*.
S_TCODE	The authorization object S_TCODE is needed to create the SAPUI5 application specific ICF node under */sap/bc/ui5_ui5/*.
S_TRANSPRT	The authorization object S_TRANSPRT is used to create new transport request or new task.
S_CTS_ADMI	The authorization object S_CTS_ADMI is needed to transport SAPUI5 applications.
S_CTS_SADM	The authorization object S_CTS_SADM is needed to transport SAPUI5 applications.
S_ADT_RES	The authorization object S_ADT_RES is used for the communication between Eclipse and the ABAP Backend via the SAPUI5 Repository Team Provider.
S_RFC	The authorization object S_RFC is used for the communication between Eclipse and the ABAP Backend via the SAPUI5 Repository Team Provider.

For more information about authority checks and working with authorization objects, see the following: *SAP NetWeaver 7.0x Security Guides (Complete)* on SAP Service Marketplace at http://service.sap.com/securityguide

Delivered Virus Scan Profile

When uploading files to the SAPUI5 Repository, you can perform a virus scan.

As of SAP NetWeaver 7.00 with UI AddOn, SAP delivers the following virus scan profile for ABAP within the UI AddOn (from 7.40 SP1 on the virus scan profile is already part of the software component `SAP_UI`):

- /UI5/UI5_INFRA_APP/REP_DT_PUT

This profile is used by the SAPUI5 Repository API to store files in the SAPUI5 Repository based on BSP Repository.
For example: Upload of a local file using SAPUI5 Repository API /UI5/CL_UI5_REP_DT, method /UI5/IF_UI5_REP_DT~PUT_FILE from 7.00 on, or the SAPUI5 Team Repository Provider in 7.31 or 7.40.

The profile is deactivated when delivered. To activate it, first create at least one basis profile and save it as the default profile. You can then activate one of the delivered profiles. By default, it links to a reference profile, which is the default profile. For more information, see SAP Help Portal:

- ABAP-Specific Configuration of the Virus Scan Interface (7.00)
- ABAP-Specific Configuration of the Virus Scan Interface (7.31)

Executing SAPUI5 Applications from the SAPUI5 Repository

The SAPUI5 Application can be executed from the NW 7.X ABAP System by retrieving the SAPUI5 Application Resources from the SAPUI5 Repository based on BSP Repository with the help of an ICF Handler.

Delivered ICF Nodes

For the execution of SAPUI5 Applications from the SAPUI5 Repository, SAP delivers ICF node */sap/bc/ui5_ui5/*. For every SAPUI5 applications an ICF node below that exists.

Note that all services delivered by SAP (such as the */sap/bc/ui5_ui5/* service for executing SAPUI5

Applications) are initially inactive. Also each new service that you create has status inactive. Before you work with the ICF, you must activate the services you require.

For more information, see also Activating and Deactivating ICF Services (7.00 EhP3) and SAP Library for SAP NetWeaver on SAP Help Portal at http://help.sap.com *SAP NetWeaver → SAP NetWeaver Library → SAP NetWeaver by Key Capability → Application Platform by Key Capability → Connectivity → Components of SAP Communication Technology → Communication between ABAP and Non-ABAP Technologies → Internet Communication Framework → Development → Activating and Deactivating ICF Services*.
For more information about ICF security, see SAP NetWeaver Security Guide on SAP Service Marketplace at http://service.sap.com/securityguide *SAP NetWeaver 7.0x Security Guides (Complete) → SAP NetWeaver 7.0x Security Guides (Online Version) → Security Guides for Connectivity and Interoperability Technologies → RFC/ICF Security Guide*

Needed Authorization Objects

No specific authorization objects are needed to execute SAPUI5 Applications from the SAPUI5 Repository.

As for ICF service nodes in general, authorization for specific ICF service nodes can be restricted, see also Defining Service Data (7.00 EhP3) or SAP Library for SAP NetWeaver on SAP Help Portal at http://help.sap.com *SAP NetWeaver → SAP NetWeaver Library → SAP NetWeaver by Key Capability → Application Platform by Key Capability → Connectivity → Components of SAP Communication Technology → Communication between ABAP and Non-ABAP Technologies → Internet Communication Framework → Development → Server-Side Development → Creating and Configuring ICF Services → Create Service → Defining Service Data* and Authorization Object S_ICF (7.00 EhP3) or SAP Library for SAP NetWeaver on SAP Help Portal at http://help.sap.com *SAP NetWeaver → SAP NetWeaver Library → SAP NetWeaver by Key Capability → Application Platform by Key Capability → Connectivity → Components of SAP Communication Technology → Communication between ABAP and Non-ABAP Technologies → Internet Communication Framework → ICF Administration → Server Function Administration → Authorizations → Authorization Object S_ICF*.

Tracking coding changes and text changes in the SAPUI5 Repository

Code changes can be traced by using the usual ABAP version control of the corresponding resource file. A new version is created when a new transport is written.
Text changes can be traced by using the "Table History" transcation(SCU3), the relevant tables for UI5 texts are /UI5/TREP_TEXT and /UI5/TREP_TEXT_T for the translated text. Table logging has to be activated in the system for this functionality.

Compatibility Rules

This page provides an overview what compatibility means with regards to SAPUI5 development (application and control development).

SAPUI5 plans to be a "Release Independent Component" (RIC). One important constraint for such a RIC is that it must be as simple as possible to upgrade to a newer version. This makes compatibility rules so important.

Versioning Scheme

UI5 uses a 3 number version identifier inspired by the semantic versioning specification, for example **1.4.3**.

The left most number is the **major version** number (here 1), the middle one is called **minor version** number (here 4) and the last number is the **patch version** number (here 3). New SAPUI5 releases are described by a major and minor version number. The patch version number is only used to identify patches within a release.

API Evolution

Definition of 'API'

If not mentioned differently in the following paragraphs, **API** means **public API** (functions, classes, namespaces, controls with their declared properties, aggregations, ...). The solely definition of the SAPUI5 public API is the API Reference Documentation (aka JSDoc). If any feature (class, method, property, ...) is not mentioned in the JSDoc, it must be assumed **not to be part of the API**. If in doubt, get in contact with SAP.

A release with a new major version is called a **Major release** and can introduce new APIs but it also can make incompatible changes to existing APIs. If so, the release notes will describe these incompatibilities.

A **Minor release** (where only the minor version but not the major version differs) can introduce new APIs, but must not contain incompatible changes to APIs introduced by the same major release line.

Patch releases must only fix implementations, but must not introduce new features nor must they do incompatible API changes.

Exceptions from the above rules are possible, but only in rare, very urgent cases (e.g. security issues) and they have to be documented in the release notes.

Compatible changes

- Adding new libraries, controls, classes, properties, functions, namespaces, to the API ...
- Moving properties up in the inheritance hierarchy (generalize them)
- Changing the logging behavior (adding, removing log output, modifying the content of existing log output), logging is not part of the API
- Adding new values to enumeration types (this means for developers: when dealing with enum properties, always be prepared to accept new values, e.g. by implementing a "default" or "otherwise" path when reacting on enum values)
- Modifications to the markup / style (HTML/CSS) of controls

Incompatible changes

These changes might only occur in major version as described above:

- Removing an API (library, namespace, function, property, control, events, ...)
- Renaming an API (library, namespace, function, property, control, events, ...)
- Removing support for URL parameters
- Removing support for configuration entries
- Reducing the visibility of an API (doesn't break JavaScript applications, but changes the contract)
- Removing or reordering parameters in an API signature
- Reducing the accepted value range (e.g. parameter of a function)
- Broadening the value range of a return value (or property). **Except:** Enumerations
- Moving JavaScript artifacts (namespaces, functions, classes) between modules
- Replacing asserts by precondition checks
- Moving properties etc. down in the inheritance hierarchy
- Changing the name of enum values
- Changing defaults (properties, function parameters)
- Renaming / removement of files

Deprecation

- Whenever possible, instead of doing incompatible changes, SAPUI5 will mark the old artifacts as deprecated and (if applicable) new ones created
- As with Java, deprecation documentation (in the API Reference and maybe a log entry in the implementation) should explain why and when something has been deprecated and provide hints how to achieve the same functionality without deprecated functionality.

Experimental APIs

Some new functionality / controls are documented as being in an **experimental development state**. Such features are still under development and changes to their API are very likely (even incompatible changes in minor releases). They are included in public releases to collect feedback from early adopters. Consumers of such APIs must understand that their respective code might be broken (repeatedly) when upgrading to new SAPUI5 releases and they must be willing to adapt to such changes.

Third Party Open Source Libraries

SAPUI5 contains and uses several third party open source libraries (e.g. *jQuery* just to mention one). These libraries can also be used by applications and/or custom control libraries, but **the SAPUI5 compatibility policy** described within this document **does not apply to these third party libraries**.

Consumers that want to use the contained third party open source libraries can do so when respecting the following restrictions:

- SAPUI5 decides which version(s) and modules of the used libraries is / are provided.
- SAPUI5 can upgrade to higher version(s) of the used libraries even within a micro release.
- For important reasons (e.g. security) SAPUI5 can stop providing a library.
- The third party libraries are provided "as is". Extensions, adaptations and support are not done by SAPUI5.

For closed source libraries, a usage beyond the use of the corresponding SAPUI5 controls is not allowed.

Exceptions / Important to know

Not part of the public API are all functions, etc. which are not declared as public in the JSDoc. Additionally not part of the API is the rendered HTML and the CSS of the controls. This means in particular that the following scenarios can lead to problems (also when switching minor or micro version). The support for such problems by SAPUI5 can only be "limited" and adaptation of non-SAPUI5 coding might be necessary. JavaScript makes it easy to modify code that one isn't the owner of e.g. enrich prototypes of foreign classes, add properties/functions to existing objects. This should not be forbidden, but the consequences (risk of conflicts when SAPUI5 is enhanced) must be clear.

- Manipulation of HTML / CSS (e.g. via jQuery, Control.addStyleClass or directly via CSS).
- Using or overriding non-public functions, etc.
- Creation of new controls (e.g. "Notepad-Controls") based on existing ones (inheritance) or subclassing in general → new classes also depend on private functionality of the super classes, collision of generated functions for properties, aggregations are possible
- Pixel perfect compatibility not guaranteed for SAPUI5 owned controls. Therefore also not guaranteed for controls inheriting from SAPUI5 controls.

Release Notes

This page provides an overview of all new features and all corrections in the delivered releases for SAPUI5 Runtime, SAPUI5 Controls and SAPUI5 Tools.

Version 1.14.5 (October 2013)

Patch for the rel-1.14 code line. It contains the following fixes for the UI5 Core and Controls:

UI5 Tools:

- [FEATURE] When creating a new UI5 application project for mobile apps, Blue Crystal is used as theme instead of SAP MVI

Core

- [FIX] ArrayDiff: fix wrong calculation of diff and optimized performance
- [FIX] Avoid assertion failures for applySettings() with default model
- [FIX] Device API: fix Media Queries on iPad
- [FIX] Fix missing focus outline in new Chrome 30.
- [FIX] IconPool: make IconFonts work in IE9 (URL with Params)
- [FIX] ListBinding: store application filters correctly
- [FIX] ODataModel: fix incorrect submitBatch request handle

Desktop

- [FIX] ComboBox/DropdownBox: fix update of item Text
- [FIX] ComboBox: restore focus on rerendering
- [FIX] MenuButton: value change on mobile devices
- [FIX] DropdownBox: wrong item set in onAfterRendering
- [FIX] SimpleForm: insert new field before first label fails
- [FIX] ThingInspector: fix keyboard navigation
- [FIX] TextArea with MaxLength: DEL key do not work in Firefox
- [FIX] ToolPopup: forward closing to Shell
- [FIX] Button: fix iconSelected
- [FIX] ux3.Shell: closing of Opened ToolPopups
- [Fix] MatrixLayout: colspan is wrong in IE10
- [FIX] Menu: no special top style for Menu in the Blue Crystal Theme
- [FIX] UX3/Shell: fix minor issues in the Blue Crystal Theme

Version 1.14.4 (October 2013)

Core

- [FEATURE] Adds a private method to the Component.js and MockServer.js to refresh the service metadata document and the mockdata of all created mockservers
- [FEATURE] Component: Automatic activation/deactivation of customizing
  - The first instance of the component activates the customizing if a customizing configuration is available in the metadata. If the last instance of a component is destroyed the customizing will be deactivated
- [FEATURE] Controller Extensibility
- [FEATURE] I18N: support to enhance ResourceModels with custom props
- [FEATURE] NavContainer: add insertPreviousPage method
  - used to populate the history stack when deep-linking is used
- [FEATURE] Router fires routePatternMatched event now
  - now an app can determine the actual route being hit
- [FEATURE] UIComponent: added support for routing configuration
- [FEATURE] sap.m.DateTimeInput: Added Keyboard Navigation

- [FEATURE] sap.m.Dialog: padding in content area is removed for 1.16 compatible applications by default
    - Adding class sapUiPopupWithPadding to popups to enable additional padding in content area
    - ResponsivePopover now forwards the style change functions to inner popup instance to enable setting the additional padding

- [FEATURE] sap.ui.UX3 DataSet: Enabled MultiSelect
- [FEATURE] sap.ui.core.IconPool: IconPool is updated with icon version 2.3 and two new icons are added: multi-select and exit-full-screen
- [FIX] Consolidated UI5 Core & Runtime Demokit content
  Following topics have been touched:
    - Made API Documentation link consistent on all Demokit pages
    - Added BlueCrystal theme to the list of supportedThemes for UX3 controls
    - Fixed broken references

- [FIX] Customizing: actually ignore property modifications for properties other than "visible"
- [FIX] Device API now recognizes android stock browser
    - The android browser has no special name in its user-agent
    - It is now recognized when the browser is running on Android
    - Is based on Webkit and does not call itself "Chrome"

- [FIX] Handling of pcpFields with no value (SapPcpWebSocket)
- [FIX] Menu: Made Menu Usable in Popup

    if the menu was used in a popup. Especially within the ToolBar overflow it wasn't usable. Therefore the menu registers itself to the root-popup as additional focusable area. Even if the menu closes via its autoclose mechanism this close event is bubbled to the corresponding popup. Created a new test page to demonstrate this rare scenario.

- [FIX] ODataModel: Add function import metadata support
- [FIX] ODataModel: add global URL Params to batch request
- [FIX] ODataModel: small documentation fix for useBatch setter
- [FIX] ODataModel: use withCredentials option only for async requests.
- [FIX] Router - getUrl works with undefined
    - When a route had optional parameters it did throw an error
    - Added a testpage for rouepatterns

- [FIX] Routing: Allow subroutes to be defined as object
    - Property key will be set as name
    - Array is still compatible

- [FIX] Routing: Only remove aggregation - no destroy
- [FIX] XMLTemplateProcessor check for existence of sap.ui.core.Component.activateCustomizing before using it
- [FIX] XMLView: Replace children before onAfterRendering
    - When an XMLView is rerendered, it does only render its child controls in an hidden area and inserts them in the existing HTML structure. By default the onAfterRendering (which does this inserting) is executed after the onAfterRendering of the children. This was causing issues with chart controls and the RichTextEditor, so it was changed that the inserting now takes place before calling onAfterRendering of the child controls.

- [FIX] added a fix to get over the missing getControllerName error on the prototype
- [FIX] bindElement refreshes all bindings of the Model
    - Refresh only the bindings that are affected by the new context

- [FIX] bindElement refreshes all bindings of the Model
    - Refresh only the bindings that are affected by the new context

- [FIX] disallow disabling customizing via URL parameter
- [FIX] fixed view name recognition in ViewSerializer

[FIX] fixing IE 8 in History.js
- [FIX] jQuery.sap.xml fix serialize XML for IE9
- [FIX] jQuery.sap.xml fix serialize for IE
- [FIX] Removed context from router config
  - Merging model with routing is not the way to go

- [FIX] sap.ui.core.IconPool: fix the calculation of absolute font path in IE9
  - The relative path needs to be converted into absolute path when icon is used inside iframe in IE9
  - URI.js is used to correctly convert url to absolute url based on current window.location.href

- [FIX] sap.ui.core.routing Allow inheritance of targetControl
  - Routes don't have to specify a pattern anymore
  - If you set the targetControl to undefined in a subroute you get the targetControl of the parent

- [FIX] sap.ui.core.routing.HashChanger replaceHashChanger fixed
  - Does not throw an error if you did never call getInstance before

- [Feature] Routes can get the url for a context path
  - New context property in route config
  - getUrl now has variable param instead of object

- [Feature]sap.ui.core.routing.History class for navigation direction
- [Fix] routing + splitcontailer
  - history now determines backwards direction correctly
  - Route passes its config to the matched event
  - SplitContainer is also able to insert previous pages

Desktop

- I18N: Added new resource bundles from B0Y
- [FEATURE] sap.ui.UX3 DataSet: Enabled MultiSelect

- [FIX] Commons/Menu?: Refresh hover item after rerendering
- [FIX] Commons/MenuButton: Remove unnecessary close call
- [FIX] Consolidated UI5 Core & Runtime Demokit content
  Following topics have been touched:
  - Made API Documentation link consistent on all Demokit pages.
  - Added BlueCrystal theme to the list of supportedThemes for UX3 controls.
  - Fixed broken references

- [FIX] FileUploader Keyboard Navigation for Internet Explorer 9
- [FIX] InPlaceEdit on iPad
- [FIX] Label alignment of icon in RTL mode
- [FIX] Label alignment of icon in RTL mode
- [FIX] Link: the Link handles the enter by itself
- [FIX] MatrixLayout render tooltips
- [FIX] Menu: Made Menu Usable in Popup
  - If the menu was used in a popup - especially within the ToolBar overflow it wasn't usable. Therefore the menu registers itself to the root-popup as additional focusable area even if the menu closes via its autoclose mechanism this close event is bubbled to the corresponding popup created a new test page to demonstrate this rare scenario.

- [FIX] Table: Enter action mode not working with jQuery 1.8.1
- [FIX] ToolPopup: Merged Fixes for Intelligent Arrow
  - When the ToolPopup switched its position or changed it due to any occured collision the arrow needs to be repositioned correclty
  - Now the arrow width/height is taken in account as well to calculate the ToolPopup's width/height properly

[FIX] Toolbar: fix issue where overflow popup items would be duplicated when rerendering happens while popup is open
- [FIX] UX3/NavigationBar: Fix scroll item into view
- [FIX] UX3/Shell: PaneBar Overflow Menu
  - The overflow button in the pane-bar is now shown
- [FIX] sap.ui.commons Button: Fixed Lite Button disabled state

  This fix forces the lite button to use the default icon when the button is set to disabled, instead of using the icon depending on the state it was in when disabled was set.

Mobile

- I18N: Added new resource bundles from B0Y.
- MAKit bi-direction fix
- [FEATURE] Add numberState valueState property in Object Header
- [FEATURE] Added Emphasized Button to Explored
- [FEATURE] Adding numberState property in ObjectListItem
- [FEATURE] Explored documentation updates
  - TablePersoDialog / TablePersoController
  - TableSelectDialog
  - Icon (different sizes, colors, press)
  - Text (maxLines)
  - Table (mergeDuplicates)

- [FEATURE] Extended Explored docu
  - "Panel": extra toolbars
  - "Input - Assisted": new, with suggestion and valuehelp
  - "Input": is now "Input - Simple"
  - "Icon": new, various simple examples
  - "SelectDialog": new
  - More mock data for eventual consistency
  - Cleaned up whitespace; minor adjustments

- [FEATURE] ListBase: new growing events and methods are added
  - growingStarted and growingFinished events are added
  - getGrowingInfo method is added

    http://veui5infra.dhcp.wdf.sap.corp:8080/demokit/#docs/api/symbols/sap.m.!ListBase.html

- [FEATURE] Loosen / modify interface for PersoProvider

  type for persoService aggr sap.m.TablePersoProvider → Object get/set/del methods now have 'PersData' suffix added docu in abstract that a promise should be returned

- [FEATURE] NavContainer: add insertPreviousPage method
  - used to populate the history stack when deep-linking is used

- [FEATURE] Replace ObjectNumber control numberUnit property name with unit
- [FEATURE] Select: added open() method
- [FEATURE] sap.m.BusyDialog: Added Keyboard Navigation
- [FEATURE] sap.m.DateTimeInput: Added Keyboard Navigation
- [FEATURE] sap.m.DateTimeInput: Keyboard Handling Improvements
- [FEATURE] sap.m.Dialog: padding in content area is removed for 1.16 compatible applications by default.
  - Adding class sapUiPopupWithPadding to popups to enable additional padding in content area
  - ResponsivePopover now forwards the style change functions to inner popup instance to enable setting the additional padding

- [FEATURE] sap.m.FacetFilter: add single select property for FacetFilterList

[FEATURE] sap.m.Input: input now support suggestions.
  - suggestion can be added by listening to the suggest event which is fired when user types in the input. In the event listener, additional sap.ui.core.Item instance can be added to the suggestionItems
  - aggregation which will be filtered by the filter function
  - Input comes by default with a filter function which can be modified by calling setFilterFunction

- [FEATURE] sap.m.Input: suggestionItemSelected event will be fired when suggestionItem is selected in the suggestion popup
- [FEATURE] sap.m.ObjectListItme,sap.m.FeedListITem: Added RTL Support
- [FEATURE] sap.m.Panel: Added header toolbar and info toolbar, removed header level
- [FEATURE] sap.m.Shell design changes
- [FEATURE] sap.m.TileContainer, StandardTile: Added Keyboard Navigation
- [FEATURE] sap.ui.core.IconPool: IconPool is updated with icon version 2.3 and two new icons are added: multi-select and exit-full-screen.
- [FIX] IconTabBar: don't break when oSelectedItem is null
- [FIX] NavContainer: don't break when no page is added, don't rerender when same page is added repeatedly as first page
- [FIX] Page:page without header leads to wrong calculations of the scroll
  - Add qUnit-tests
- [FIX] SplitContainer now always render its nav containers
  - since the NavContainer does not throw an error if it is rendered without pages any longer, the SplitContainer should not be responsible to decide when to render

- [FIX] SplitContainer only adds a Page once
- [FIX] minor fixes to perso test/qunit
  - swap headerContent for headerToolbar
  - use design-official perso icon

- [FIX] sap.m.Bar: content is sometimes invisible
  - Check visibility of bar, if it's not visible, don't update position

- [FIX] sap.m.Dialog with a SearchField is too wide
  - Correction for an error in release 1.16 when a Dialog with a SearchField is 100% wide on the screen

- [FIX] sap.m.Dialog|Popover: fix the javascript error message when no content is given
  - Calling getContent instead of getAggregation("content") because getContent always returns an array not null

- [FIX] sap.m.Dialog|Popover: remove instance from instance manager when dialog or popover is destroyed
  - also fix a css issue of dialog in firefox

- [FIX] sap.m.FeedListItem: timestamp starting with a number fixed
  - There was a problem with the display of timestamps starting with a number such as 1.12.2013. This lead to an unreadable display of the timestamp

- [FIX] sap.m.Popover: autoclose doesn't work anymore when popover is rerendered while open.

  The try in fixing popup.js made the snippet list dialog broken in snippix application
  I have to go this way to render the content of popover manually when it's
  invalidated while open also added a check for checking if webkittransitionend is
  fired. This event can be broken when popover is rerendered before animation is
  finished. Now the check checks if this is fired. If not, the cleaning code after
  animation will be called again.

- [FIX] sap.m.Popover|Dialog: listen to resize event from sap.ui.Device instead of the native resize event
  - native resize event is fired when user types the on screen keyboard on real device which

leads to unwanted reposition

- [FIX] sap.m.Shell: stop setting parents' heights to 100% when marker is found
- [FIX] sap.m: incorrect vertical alignment of labels in a form

  fix: text and labels should be baseline aligned in a read only form fix: sap.m.SearchField: refresh button unexpectedly appears even if showRefreshButton is set to false (internal)

- [Fix] sap.m.Tilecontainer suppress master swipe
- [Fix] routing + splitcontailer

  history now determines backwards direction correctly Route passes its config to the matched event SplitContainer is also able to insert previous pages

Charts

- [FIX] Fix manifest id to idStr.

  Fix manifest id to idStr.

Inbox

- I18N: Added new resource bundles from B0Y
- [FIX] Inbox: ForwardButton disabled for completed tasks

  standard action buttons are not visible any more for completed tasks

Others

- Update dev-guide-doc from Trac
- [FEATURE] Blue Crystal: new footer colors on desktop
- [FEATURE] sap.m.Input sap.m.TextArea new design for Fiori 2
- [FEATURE] teched 2013 : grouping exercise
- [FIX] Commons/Tree?: Collapse/Expand? icons in Gold Reflection and Blue Crystal Theme
- [FIX] Form line-height and label alignment for editable mobile controls

  removed general line-height for editable mobile forms but adjust alignment of labels using paddings

- [FIX] GridLayout fix for IE10

  On IE10 the cells sometimes have the wrong size

- [FIX] Inline comment causes issue in theme engine parser
- [FIX] Remove connection to base framework

  Might cause incompatibility with existing themes

- [FIX] Unified/Shell?: HCB Theme for Shell control
- [FIX] databinding: epm products example
- [INTENAL] sap.m.Popover: fix the arrow right style in bluecrystal
- [INTERNAL) Mobile: Button CSS changed in header and bar

Version 1.14.3 (September 2013)

Core

- [FIX] Allow grouping to be defined declaratively
- [FIX] Allow whitespace around numbers in CSSColor type
- [FIX] Callout: Fix multiple closed-event firing

- [FIX] Cursor position in Internet Explorer 9+
- [FIX] Data should not be removed when calling refresh
- [FIX] DataBinding: refresh method of Bindings not working
- [FIX] Grouping: Handle named models
- [FIX] LocalBusyIndicator: Fix possible javascript errors
- [FIX] LocalBusyIndicator: New Implementation
- [FIX] OData Two-Way Binding suppressing Text Field Change Event
- [FIX] ODataModel: add getNavigationProperty undefined check
- [FIX] ODataModel: endless loop in error cases when request fails
- [FIX] ODataModel: fix successHandler when no data was retrieved
- [FIX] ODataModel: fix token handling when using BATCH
- [FIX] ProxyServlet: filter secure flag in set-cookie header
- [FIX] ProxyServlet: support to delegate multiple occurences of headers
- [FIX] Refresh model after batch with change requests
- [FIX] ResourceHandler: fixed the stream handling when caching
- [FIX] Scrollbar: Fix errors when UI5 is in touch simulation mode
- [FIX] Table: No action mode for cells without tabbable element
- [FIX] Theming: Runtime Resource Path Pattern is wrong
- [FIX] Views: only render height and width style when set
- [FIX] XMLModel: fix setProperty function
- [FIX] bindElement: Trigger new request if $expand supplied/changes
- [FIX] datajs: Edm.String type bug for POST requests in Atom/XML format
- [FIX] sap.ui.core.Popup.js: ShieldLayer isn't hidden when popup is closed twice within 400ms.
- [FIX] ODataModel: submitChanges now deletes all expand properties

Desktop

- [FEATURE] Table: Inform in sort event if column was added
- [FEATURE]Form: Bluecrystal adoption
- [FIX] Accordion Ü, Ä, Ö with Dots in Section Header
- [FIX] Callout: Fix multiple closed-event firing
- [FIX] Carousel: Better ACC Keyboard handling and ARIA
- [FIX] Carousel: setFirstVisibleIndex not working
- [FIX] Commons/Menu: Fix positioning for jQuery UI 1.9 ff.
- [FIX] Commons/Menubar: Focus highlighting in HCB theme
- [FIX] Commons/SearchField: Fix rendering of associations ariaLabelledBy and ariaDescribedBy
- [FIX] DataSet: Check if scrollArea is in DOM after rerendering
- [FIX] DataSet: Fix for pagination feature (removing items which where already loaded)
- [FIX] DataSet: Update existing items when template changed
- [FIX] FileUploader: Wrong width when using %-values
- [FIX] QuickView: navigate event by icon click
- [FIX] Repair ProgressIndicator Contrast for Text
- [FIX] ResponsiveFlowLayout rendering issue fixed
- [FIX] ResponsiveFlowLayout: Corrected the Row's IDs
- [FIX] Slider. Fire change only when value is changed
- [FIX] Table: Calculate correct height for non std. height header
- [FIX] Table: Column menu does not open using keyboard navigation
- [FIX] Table: No action mode for cells without tabbable element
- [FIX] Table: Removed timer which is not needed anymore
- [FIX] Table: Return this in setVisibleRowCountMode
- [FIX] Table: fixed the scrolling behavior of the Table
- [FIX] Table: prevent column visibility menu text truncation
- [FIX] Table: unbindRows not working
- [FIX] ToolPopup arrow pos., exception after addButton, QUnit Test
- [FIX] ToolPopup: Prevent Fix of Size When Opened at Bottom
- [FIX] ToolPopup: Recalc the ToolPopup's Height if Open and Content added
- [FIX] Toolbar: Properly handle incoming focus for invisible items
- [FIX] TreeTable: Added ARIA code for TreeTable
- [FIX] TreeTable: Added visible focus for node icons
- [FIX] TreeTable: Replaced expand/collapse icons for HCB theme
- [FIX] UX3/ExactBrowser: Fix 'cannot call method

- [FIX] UX3/NavigationBar: Scrolling when mobile content exists
- [FIX] UX3/Shell NavigationBar-Items did not write data into DOM
- [FIX] UX3/Shell: Minor BlueCrystal issues
- [FIX] UX3/Shell: Panebar width
- [FIX] UX3/Shell: Stop facetbar animation on theme change
- [FIX] UX3/ToolPopup: Crashed when not content set
- [FIX]ComboBox in ItemNavigation Click on F4-Button focus different item
- [FIX]ComboBox/DropdownBox wrong item selected if similar text
- [FIX]ComboBox/DropdownBox: empty value after destroying and recreating
- [FIX]DatePicker empty after rerendering if pattern is set by binding
- [FIX]Label: wrong name for requiredAtBegin property
- [FIX]TextArea with MaxLength: possible to enter spaces before end

Mobile

- [FIX] Growing list - experimental grouping flag removed
- [FIX] sap.m.Bar: label text does not truncate in IE
- [FIX] sap.m.Dialog: contentWidth is now not applicable on Message type dialog.
- [FIX] sap.m.IconTabBar/Filter: fix for CSN.
- [FIX] sap.m.IconTabBar: destroy methods for aggregation.
- [FIX] sap.m.Popover: Arrow goes out of the popover area when it's near the border
- [FIX] sap.m.Popover: contentWidth/contentHeight don't have effect on popover.
- [FIX] sap.m.Popover: focus is set to popover instead of the focusable content after open.
- [FIX] sap.m.Popover: set the initial focus to leftButton or rightButton only when it's visible property set to true.
- [FIX] sap.m.Popover|Dialog: Dialog can't be fully closed in IE9.
- [FIX] sap.m.ProgressIndicator: replaced "barColor" with "state"

Inbox

- [FIX] TaskDescription: Task Description in RR View.
- [FIX] TaskInitiator: Task Initiator Value dosen't appear in BPM Inbox.

Others

- [FIX] Calendar legend square sizes
- [FIX] Font Sizes for TextView in BlueCrystal
- [FIX] Form: make background half-transparent
- [FIX] Tab focus outline was hardly visible
- [FIX] Table Blue Crystal Fix for Table in Table
- [FIX] UX3/ExactList: Fix Colors in BlueCrystal
- [FIX] sap.m.RadioButton: incorrect display in a MessageBox
- [FIX]Adapted List Header Background Color for BlueCrystal

Version 1.14.2 (August 2013) ¶

Features

- [FEATURE] Table: Inform in sort event if column was added (fixes CSS 2933507)
- [FEATURE]Form: Bluecrystal adoption

Fixes

- [FIX] Allow grouping to be defined declaratively (fixes CSS 3219422)
- [FIX] Allow whitespace around numbers in CSSColor type (fixes CSS 2805046)
- [FIX] Callout: Fix multiple closed-event firing (fixes CSS 3034639)
- [FIX] Cursor position in Internet Explorer 9+ (fixes CSS 3023333)
- [FIX] DataBinding: refresh method of Bindings not working
- [FIX] LocalBusyIndicator: New Implementation (fixes CSS 2943939, CSS 2855178, CSS 2993255, CSS 2972289)

[FIX] ProxyServlet: filter secure flag in set-cookie header
- [FIX] ProxyServlet: support to delegate multiple occurences of headers
- [FIX] Table: No action mode for cells without tabbable element (fixes CSS 2952642)
- [FIX] datajs: Edm.String type bug for POST requests in Atom/XML format (fixes CSS 3013693)
- [FIX] sap.ui.core.Popup.js: ShieldLayer isn't hidden when popup is closed twice within 400ms.
- [FIX] Callout: Fix multiple closed-event firing (fixes CSS 3034639)
- [FIX] Carousel: Better ACC Keyboard handling and ARIA (fixes CSS 2951952)
- [FIX] Commons/Menu?: Fix positioning for jQuery UI 1.9 ff. (fixes CSS 3332235)
- [FIX] Commons/Menubar?: Focus highlighting in HCB theme (fixes CSS 2950906)
- [FIX] DataSet: Fix for pagination feature (removing items which where already loaded) (fixes CSS 2890281)
- [FIX] FileUploader: Wrong width when using %-values (fixes CSS 2969892)
- [FIX] Slider. Fire change only when value is changed
- [FIX] Table: Column menu does not open using keyboard navigation (fixes CSS 2964429)
- [FIX] Table: No action mode for cells without tabbable element (fixes CSS 2952642)
- [FIX] Table: Removed timer which is not needed anymore (fixes CSS 3109177)
- [FIX] Table: fixed the scrolling behavior of the Table (fixes CSS 3220829)
- [FIX] Table: prevent column visibility menu text truncation (fixes CSS 3248859)
- [FIX] Table: unbindRows not working (fixes CSS 2798198)
- [FIX] ToolPopup arrow pos., exception after addButton, QUnit Test (fixes CSS 3200187)
- [FIX] Toolbar: Properly handle incoming focus for invisible items (fixes CSS 2284040)
- [FIX] TreeTable: Added ARIA code for TreeTable (fixes CSS 2952716)
- [FIX] TreeTable: Added visible focus for node icons (fixes CSS 2951076)
- [FIX] TreeTable: Replaced expand/collapse icons for HCB theme (fixes CSS 2951782)
- [FIX] UX3/ExactBrowser: Fix 'cannot call method (fixes CSS 2289307)
- [FIX] UX3/Shell NavigationBar-Items did not write data into DOM (fixes CSS 3258099)
- [FIX] UX3/Shell: Minor BlueCrystal issues (fixes CSS 3069355)
- [FIX] UX3/Shell: Panebar width
- [FIX] UX3/ToolPopup: Crashed when not content set (fixes CSS 3263604, CSS 3034729)
- [FIX]ComboBox in ItemNavigation Click on F4-Button focus different item
- [FIX]ComboBox/DropdownBox: empty value after destroying and recreating (fixes CSS 2816100)
- [FIX]DatePicker empty after rerendering if pattern is set by binding (fixes CSS 2334212)
- [FIX]Label: wrong name for requiredAtBegin property
- [FIX]TextArea with MaxLength: possible to enter spaces before end (fixes CSS 2517367)
- [FIX] Growing list - experimental grouping flag removed
- [FIX] sap.m.Bar: label text does not truncate in IE
- [FIX] sap.m.Dialog: contentWidth is now not applicable on Message type dialog. (fixes CSS 3058671)
- [FIX] sap.m.IconTabBar/Filter: fix for CSN. (fixes CSS <0120061532 0003191078 2013>)
- [FIX] sap.m.Popover: Arrow goes out of the popover area when it's near the border (fixes CSS 3012718)
- [FIX] sap.m.Popover: contentWidth/contentHeight don't have effect on popover. (fixes CSS 3090456)
- [FIX] sap.m.Popover: focus is set to popover instead of the focusable content after open. (fixes CSS 2838150)
- [FIX] sap.m.Popover|Dialog: Dialog can't be fully closed in IE9. (fixes CSS 3223079)
- [FIX] TaskDescription: Task Description in RR View. (fixes CSS 2566005)
- [FIX] TaskInitiator: Task Initiator Value dosen't appear in BPM Inbox. (fixes CSS 2391584)
- [FIX] Calendar legend square sizes
- [FIX] Font Sizes for TextView in BlueCrystal
- [FIX] Tab focus outline was hardly visible
- [FIX] UX3/ExactList: Fix Colors in BlueCrystal (fixes CSS 3070020)
- [FIX] Adapted List Header Background Color for BlueCrystal

Version 1.14.1 (August 2013) ¶

Features

- [FEATURE] Application: support for model propagation from Core

Fixes

- [FIX] fix for combination column chart is rendered wrong
- [FIX] Select: position fixed do not work properly on mobile
- [FIX] AppCacheBuster: added infinite cache header for requests
- [FIX] Application: models can be initialized without root component
- [FIX] ComboBox: Avoid "oListBox is not a function" error
- [FIX] Commons/ApplicationHeader: Add tooltip to logoff button
- [FIX] Commons/MenuTextFieldItem: Fix positioning of TextField
- [FIX] Core: fixed attachInit to be aware of listeners when booting
- [FIX] Core: make sure that the init listeners are called
- [FIX] Cumulative downport of fixes for Table & Link
- [FIX] DataSet: SegmentedButton lost focus when switching views
- [FIX] ListItemBase: nav icon deactivated for 1.14
- [FIX] MessageToast: problems on orientation change in Samsung Galaxy N2
- [FIX] Models: fix setProperty function when path is unresolvable
- [FIX] NotificationBar: Ellipses for Inplace Message
- [FIX] NotificationBar: Text-Color and Timestamp-Color
- [FIX] ODataModel fix getPropertyMetadata which considers navigation
- [FIX] Popup.js: a shield layer is added to prevent the delayed mouse
- [FIX] ResourceServlet: fix for experimental caching & mime types
- [FIX] ResourceServlet: fixed the experimental server cache
- [FIX] ResponsiveFlowLayout: CSS-Class Handling
- [FIX] RichTextEditor: Allow setValue("") to clear content
- [FIX] Splitter: Enhance keyboard navigation
- [FIX] Splitter: fix height calculation in IE and FF
- [FIX] Splitter: fix special resizehandler
- [FIX] Table: Header increasing when using multi header feature
- [FIX] Table: added support for filter type during filter creation
- [FIX] Table: fixed the delegate support for columns
- [FIX] ThingInspector tabchain does not work in light shell
- [FIX] ThingInspector tabchain does not work in light shell
- [FIX] ThingInspector: ThingGroup tooltips are not rendered
- [FIX] ToolPopup: Check of Opener
- [FIX] ToolPopup: Make Control More Stable If no Opener Was Set
- [FIX] ToolPopup: Position of Arrow With fit-Collision
- [FIX] UX3/NavigationBar: NavigationItem add missing "data-sap-ui"
- [FIX] UX3/Shell: Alignments in BlueCrystal Theme
- [FIX] UX3/Shell: FacetBar height animation
- [FIX] UX3/Shell: Fix SideBar Blue Chrystal Theme
- [FIX] UX3/Shell: Fix tool active color in Blue Crystal theme
- [FIX] UX3/Shell: Fix tools and pane overflow
- [FIX] UX3/Shell: Fixed ToolbarOverflowButton appearance
- [FIX] UX3/Shell: Removed destroy call to wrong Aggregation
- [FIX] UX3/Shell: ToolPopup invalidation
- [FIX] commons.Dialog: Closing Via Autoclose
- [FIX] sap.m.App: BeforeShow and AfterShow aren't fired for the first
- [FIX] sap.m.BackgroundHelper: only write opacity if it has been modified
- [FIX] sap.m.BusyIndicator does not rotate
- [FIX] sap.m.Popover: Fix the issue where popover is immediately closed
- [FIX] sap.m.demokit.explored application: error in the Form view
- [FIX]ComboBox: Adapted ComboBox font
- [FIX]DatePicker: Adapted DatePicker font
- [FIX]DropdownBox: Adapted DropdownBox font
- [FIX]Support Monospace text Design in BlueCrystal
- [FIX] Inbox: TLT SP05_HE BPM inbox New rule popup location
- [SECURITY][FIX] Utils: usage of latest commons-fileupload library

Version 1.14.0 (July 2013)

Changes

Fixes

- [FIX] SegmentedButton: focus on wrong button when using it within a popup
- [FIX] ThingInspector tabchain does not work in standalone mode
- [FIX] NavContainer: fix case where it is initially rendered empty and pages are added later
- [FIX] Checkbox, RadioButton: work around IE10 bug to ensure focus outline is displayed
- [FIX] ActionBar: make it work in QuickView with bluecrystal theme
- [FIX] ActionBar: fixed business actions alignment/jQuery behavior
- [FIX] ActionBar: fixed 'Follow' menu nullpointer exception
- [FIX] ActionBar: 'removeBusinessAction' possible using Action IDs now
- [FIX] `ToolPopup`: Added `tabindex` to root element in Renderer.
- [FIX] `ResponsiveFlowLayout`: Added a missing CSS-class that got lost after overhaul of control
- [FIX] `ResponsiveFlowLayout`: Prevent unneeded DOM-calls
- [FIX] `NotificationBar`: Fixed the mouse-pointer over not selectable messages within Notifiers and inplace message
- [FIX] `UX3/Shell`: Overlays that show the side-bars are now correctly resized when the side-bars change
- [FIX] `UX3/Shell`: ToolPopup invalidation now works correctly when databinding is used
- [FIX] `UX3/ExactList`: Static menu positioning now works even when the combined width of the menus is larger that the screen/container.
- [FIX] `UX3/NavigationBar`: Upper case now themable
- [FIX] `Table`: Added an attribute for a min rowcount configuration for visibleRowCountmode=Auto
- [FIX] `RichTextEditor`: SHIFT + TAB in IE didn't change the focus correctly due to a bug in TinyMCE
- [FIX] `SearchField`: Disable type ahead functionality (inherited from Combobox) when focus is applied (e.g. after rendering).

Framework

- `BlueCrystal` Theme is now available for the desktop libraries as `GoldReflection` replacement
- `ODataModel` Introduced option `useBatch` in the constructor or via `setUseBatch` function to allow all requests except count, CSRF token refresh and metadata request to be sent in an encapsulating batch request.
- `ODataModel` Introduced option `maxDataServiceVersion` to set the max version the client can accept. Currently supported: "2.0" and "3.0".
- `datajs` Upgrade to version 1.1.0.
- Databinding: new !isRelative method on Binding
- Databinding: Remove unnecessary updates/renderings
- Models: `attachRequestCompleted` event now has an additional boolean parameter `success` which states if the request has been processed successfully or not.
- ODataModel: XSRF token: fetch only if necessary:
  - automatic token fetch before first change operation
  - setTokenEnable method for switching on/off the functionality
- ODataModel: set forceNoCache method to deprecated

Desktop Controls

- [FEATURE] New OverlayDialog control
- `ToolPopup`: Added `autoClose` property.
- `ToolPopup`: Added `maxHeight` property.
- `Splitter`: Changed behavior. Now the panes are not focusable any more but the splitter bar itself can be focused and moved via the Shift+Arrow keys. This was done to address accessibility problems. The splitter bar now has the aria separator role.
- [FEATURE] `UX3/Shell`: ToolPalette and PaneBar overflow

Mobile

- NEW: `sap.m.RatingIndicator`: Added control.
- NEW: `sap.m.ProgressIndicator`: Added control.

- NEW: `sap.m.Page`: Added solid and transparent background design.
- NEW: `sap.m.PopinDisplay`: type with *Block* and *Inline* options is added to control pop-in Header/Value? fields display from *sap.m.Column*. API
- FEATURE: `sap.m.Column`: *popinDisplay* property is added to control pop-in Header/Value? fields display. See API
- DEPRECATED: `sap.m.Column`: *popinHAlign* property is deprecated. Use *popinDisplay* instead. See API
- FEATURE: `sap.m.Text`: *maxLines* property is added to limit number of lines in Text control. See API
- FEATURE: `sap.m.Dialog`: stretch property is added to stretch the dialog to full screen size. The stretchOnPhone property has been deprecated and please use the new stretch property instead. See API
- FEATURE: `sap.m.ActionSheet`: Texts in ActionSheet control are left aligned instead of middle aligned in theme Blue Crystal.
- FIX: `sap.m.MessageBox`: MessageBox now accepts customized action name with space inside.

Tools

- NEW: Determine installed UI5 plugins dynamically so that all installed UI5 Lib plugins are added to the JavaBuild Path and JavaScript Include Path when creating a SAPUI5 Application Project
- NEW: UI5 Application Development Tool and Eclipse Outline View: Enhanced generated coding for controller and JavaScript view files by adding `@memberOf` in the Java Doc of the generated methods so that the Eclipse Outline View works

Contribution "sap.suite.ui.commons"

- NEW: `sap.suite.ui.commons.ViewRepeater`: The itemHeight and height properties were added to support the responsive layout.
- NEW: `sap.suite.ui.commons.RepeaterViewConfiguration`: The itemHeight property was added to support the responsive layout.
- NEW: Developed LaunchTile control (experimental). The LaunchTile control provides a press-able element that can be used to launch a link to an application or another location. The control allows definition of a descriptive title and an icon that can be used to provide a hint the user about the launched application.
- NEW: Developed FeedTile control (experimental). The FeedTile control provides a container for an aggregation of FeedItems. The control will cycle through the list of FeedItems and display the title, age of the article, source and the image associated with the article. The control provides a press event that can be used to navigate down to a second level. As an example, it may navigate to a list of FeedItemHeaders where all, or a subset, of feed articles are presented.
- NEW: Developed FeedItemHeader control (experimental). The FeedItemHeader control displays the summary information associated with the referenced feed item. The control will display the title, age of the article, source, description, and image associated with the article. The control provides a press event that can be used to open the URL directly to the feed article.
- NEW: `sap.suite.ui.commons.ThreePanelThingViewer` control extends the sap.ui.ux3.ThingViewer control and provides an ability to display data in three panels with a vertical navigation bar.
- NEW: `sap.suite.ui.commons.ThreePanelThingInspector` control extends the sap.ui.ux3.ThingInspector control and displays the sap.suite.ui.commons.ThreePanelThingViewer control in sap.ui.ux3.Overlay.
- NEW: `sap.suite.ui.commons.ViewRepeater` control extends the sap.ui.commons.RowRepeater control and provides an ability to change data representation by switching between different views.
- NEW: `sap.suite.ui.commons.ViewRepeater`: Added an ability to substitute original repeater rendering with any external control.
- NEW: `sap.suite.ui.commons.ViewRepeater`: The Search field is provided to fire the search event when a user chooses Search, no actual search occurs.
- NEW: `sap.suite.ui.commons.VerticalNavigationBar` control extends the sap.ui.ux3.NavigationBar and provides an ability to display navigation items vertically.
- NEW: `sap.suite.ui.commons.VerticalNavigationBar`: The CountingNavigationItem control was added to display the quantity of items on a corresponding content area.
- NEW: `sap.suite.ui.commons.ThingCollection` contains a collection of items and allows you to navigate through them.
- NEW: `sap.suite.ui.commons.ThingCollection`: Added an ability to delete items from the collection.

- NEW: `sap.suite.ui.commons.ThingCollection`: The addNextContent method was added to enhance the navigation of items in the collection. This method provides an ability to insert a new item into the next display position.
- NEW: `sap.suite.ui.commons.BusinessCard`: Added an ability to set any UI5 control as a title for application developers.
- NEW: `sap.suite.ui.commons.BusinessCard` control displays business card information.
- FIX: `NoteTaker` Edit card (with DB) updates the model.
- FIX: `sap.suite.ui.commons.NoteTaker`: NoteTaker AddCard event contains newly created card.

Contribution "sap.viz"

- New Charts
  - Treemap
  - Heatmap
  - Area
  - Horizontal Area
  - Area100
  - Horizontal Area100
- Chart Supported Features (New and Changes)
  - Below UI theme is supported
    - Golden Reflection
    - High Contrast Black
    - Platinum
  - New property of legend to switch on/off scrollable legend
  - Number format setting supported in MND charts
  - Keep Selection after chart resized
- API (New and Changes)
  - Selection: Get and Set selected info

Experimental Features:

Experimental features are not part of the officially delivered scope that SAP is going to guarantee for future releases – means experimental Features may be changed by SAP at any time for any reason without notice. The **Experimental features** are **"NOT FOR PRODUCTION USE"**. You may not demonstrate, test, examine, evaluate or otherwise use the Experimental Features in a live operating environment or with data that has not been sufficiently backed up.

The purpose of Experimental features is to get feedback at an early point of time allowing customers/partners to influence the future product accordingly. Please use the SCN Developer Center http://scn.sap.com/community/developer-center/front-end to provide feedback accepting that Intellectual Property rights of the contributions or derivative works shall remain the exclusive property of SAP.

For general information about **Experimental features**, please check the Compatibility Rules, for a detailed list check out the list below:

- `Mobile Tablet Support`: The controls of the UI libraries sap.ui.core, sap.ui.commons, sap.ui.ux3 and sap.ui.table are only partially optimized or adapted for mobile tablet usage.
- `OData Write Support`: First experiments for two way binding.
- `Module sap.ui.core.plugin.LessSupport`: Only for development purposes.
- `Module sap.ui.core.delegate.ScrollEnablement`: Current status is prototype.
- `Configuration Parameters` with "-xx-" and the corresponding features: e.g. sap-ui-xx-test-mobile
- `Weinre Server Configuration`: Might change or removed in future releases
- `Control sap.ui.table.Table`: Current status for Threshold and Column Grouping is prototype.
- `Deprecated Message Controls in sap.ui.commons`: MessageBar, Message, MessageList, MessageToast - Use `sap.ui.ux3.NotificationBar` instead
- `Control sap.ui.core.HTML`: Support of HTML with multiple root nodes
- `Control sap.uiext.inbox.Inbox,sap.uiext.inbox.SubstitutionRulesManager`: API is not yet finished and might change
- `Control sap.ui.suite.TaskCircle`: API is not yet finished and might change
- `Control sap.ui.suite.VerticalProgressIndicator`: API is not yet finished and might change

- `Control sap.ui.ux3.ThingViewer`: API is not yet finished and might change
- `Control sap.ui.ux3.Shell`: Personalization, Color Picker (`Control sap.ui.ux3.ShellColorPicker`) and "Inspect"-Tool
- `Controls / Types sap.ui.ux3.Feed*`: Open discusses might result in API changes and adaptation. Especially the text presentation (e.g. @-references and formatted text) is not yet clear. Also Feed Model topic still open.
- `Control sap.ui.ux3.Exact`: Open discussions might result in API changes and adaptation. Especially the Result Area is not final.
- `Control sap.ui.ux3.ExactArea`: Open discussions might result in API changes and adaptation.
- `Type sap.ui.ux3.ActionBarSocialActions`: Open discussions might result in API changes and adaptation.
- Improved performance for touch devices: Mouse events are now fired on their corresponding touch events
- Control `sap.makit.*`: API is not yet finished and might change
- Theme Parameters for `sap.m` and `sap.makit` are not yet finished and might change
- `Controls sap.viz.*`: API of the new charting library is not yet finished and might change
- `sap.ui.core.Component, sap.ui.core.ComponentContainer` API is not yet finished and might change
- `Control sap.m.GrowingList`. API is not yet finished and might change completely
- Performance recorder: API is not yet finished and might change
- `Control sap.m.Input` . API is not yet finished and might change completely
- `sap.ui.ux3.FacetFilterList` API is not yet finished and might change completely
- NEW: Property `type` added to `sap.m.Switch`. This makes possible to have different Switch types (e.g "Default", "AcceptReject?"), see API
- NEW: Property `navButtonType` added to `sap.m.Page`. This property is used to set the appearance of the NavButton?. By default, a back button is shown in iOS and up button in other platforms. In case you don't want platform dependent styling, you can set the value to "Default", see API
- New control library **sap.me** with extensions controls for mobile
- NEW: Custom theme support: checks for inclusion of custom css created by the Theme Designer dependending on the existence of a custom CSS rule.

Version 1.12.6

Fixes

- sap.m.bar: left button is cut off in IE
- Button alignment in IE for popup
- [FIX] `sap.m.RadioButton` : disabled button is always checked in MVI theme
- [FIX] `sap.m.RadioButton` : label is wrapped to the next line
- [FIX] ToggleButton : arrow is not showing to right in IE10 with RTL
- [FIX] ToggleButton and other Button types: focus visualization in HCB theme
- [FIX] CSSColor: allow "transparent", "inherit" and "" as legitimate values
- [FEATURE] ScrollEnablement: maintain scroll position across navigation and rerendering (additional fixes)
- [FIX] sap.ui.commons.ListBox: workaround for IE10 bug to fix hover effect
- [FIX] sap.m.Shell: avoid rerendering when "headerRightText" is set

Version 1.12.5.1 (Tools only)

Tools:

- [FIX] Compatibility with ABAP in Eclipse version 2.13.x
- NEW: UI5 Application Development Tool and Eclipse Outline View: Enhanced generated coding for controller and JavaScript view files by adding `@memberOf` in the Java Doc of the generated methods so that the Eclipse Outline View works

Version 1.12.5 (July 2013)

A patch for the rel-1.12 code line (Note 1878280). It contains the following fixes for the UI5 Core and

Controls:

Fixes

- sap.m.Link should mark events as handled
- Feeder - remove spaces between lines in IE
- TM91FIT10 Cargo Items lost when Reject
- TextField - no LiveChange event needed pressing Tab
- ManagedObject: fix CompositeBinding check
- [FIX] Sematic colors for accept/eject buttons in bar, header and footer
- Fix build - paginator in Inbox fix failed in FF
- ListBox styling: adapted padding for Internet Explorer 10
- [FEATURE] AppCacheBuster: enable batch support (experimental)
  To use this performance feature with a SAP ABAP Web Application Server backend pay attention to SAP Note 1878871.
- [FIX] Caching: adopted the cache headers for the resource servlet.
- [FIX] Make I18N Text formatting more compatible with JDK MessageFormat
- [FIX] ResizeHandler: enable timer based solution for all browsers
- [FIX] RowRepeater: fixed the threshold behavior
- [FIX] Tile Container correction of tap and click behavior
- [INTERNAL] Move 'credits page' into resources folder

Version 1.13.1 (June 2013)

Changes

Fixes

- NotificationBar: Fixed Inplace Message Visually Clickable

Framework

- `sap.ui.app.Application`: Added `config` property
- `sap.ui.app.Application`: Added `view` aggregation and protected `createView` factory method
- `sap.ui.model.odata.ODataModel`: Added option to load metadata asynchronously.
- Extended Diagnostics Tool (CTRL + SHIFT + ALT + S) with debugging features.

Desktop Controls

- `SimpleForm`: a new property `layout` is created to choose the FormLayout that's used to render the `SimpleForm`. Per default `ResponsiveLayout` is used. There is a new enum `SimpleFormLayout` that defines the available layouts.
- `ToolPopup`: Made the arrow smarter. It positions itself always accordingly to its opener.
- `ToolPopup`: Extended the test page with a FileUploader
- New library `sap.ui.unified`: Currently all contained controls (new Shell control and other dependent controls) have an **experimental state**. The Implementation, API and Behavior might change in future.

Mobile

Tools

Version 1.12.4 (June 2013)

A patch for the rel-1.12 code line (Note 1870994). It contains the following fixes for the UI5 Core and Controls:

Fixes

- Inbox control Custom action button adding

- Mismatch in paginator and tasks
- Adapt to incompatible Webkit change (https://bugs.webkit.org/show_bug.cgi?id=106460) in order to fix tablet/phone detection in new Chrome versions on Android
- ListItemBase: Duplicate id from navigation icon fixed
- sap.m.SearchField: fix occasional focus loss in IE
- sap.m - close soft keyboard by scroll start
- Inbox control Custom action button adding.
- Mismatch in paginator and tasks,
- fix accept-language header in metadata call
- fixed the flickering of the Dialog
- Table RowCountMode auto not working on vertical resize
- ToolPopup: Fixed Focus Handling
- Shell: Reposition Toolpopups after Rendering
- bindElement should not reset context immediately
- TextView - put CSS-alignment into style classes
- Dropdownbox in Table, Databinding Issue
- RangeSlider invalid behaviour in RTL mode
- NotificationBar: Removed BeforeRendering
- Shell: Do not stop animations on TI
- Panel positioning in AbsoluteLayout
- Fixed issue when changing number of rows
- Components: fix double ids
- fix MatrixLayout in IE9 and ColSpan in every row
- FormattedTextView: Tooltip
- Styling for Segmented Button
- Shell: Fix FacetBar Height Calculation
- Explored App: Fix for focus (which is needed for
- datajs: fix error handling in sync calls
- Splitter: fix height calc in FF and IE when resizing
- error displayed during request abort
- Fixed HCB Theme for TriStateCheckbox
- Click on tick not working correctly
- Custom attributes eager loading
- FIX ResponsiveFlowLayout: Fixed Check For Render Content
- Inbox: Fix for categories icon binding issue.
- LocalBusyIndicator: Fixed Animation Overflow
- Mobile - Bar control - Duplicate resize listener
- Mobile List: Duplicate ID navIcon
- Mobile: Focus handling for Input fields
- Mobile: Stand-alone month names are added into DateTimeInput
- Mobile: TextArea paste problem is fixed
- ODataModel: support for custom headers in $count request
- ResponsiveFlowLayout: Fix Flickering
- SimpleForm: fix initial flickering
- Theme Designer: Various Bugfixes, Selenium Tests
- TriStateCheckBox: Modified test page
- sap.m: fix not-escaped strings in renderers
- sap.ui.commons: Added missing types/escaping

Version 1.12.3 (June 2013)

A patch for the rel-1.12 code line (Note 1866304). It contains the following fixes for the UI5 Core and Controls:

Fixes

- Popover: fix for autoclose
- NavContainer: workaround for webkit bug
- SplitApp: workaround for webkit bug
- avoid onbeforeunload event on Dialog close-X

- sap.m.FeedListItem: FeedListItem on phone shows too
- Page shows header icon in Blue crystal theme
- image seems to be selectable
- fix backgroundOpacity in SplitApp,App,Shell
- Popup: Fixed Setting Focus When Focusing tinyMCE
- NotificationBar: Fixed Arrow Rotation for IE8
- NotificationBar: Fixed Arrow Position
- NotificationBar: Added Stable IDs for DOM-elements
- NotificationBar: Fixed Inplace Message Visually Clickable
- RichToolTip Added carriage feed replacement to br tag
- NotificationBar: Fixed non-transparent Icons in
- Table: Select-All selects row in Single Mode
- SearchField: Timing issue in type ahead functionality
- ResponsiveLayout only invisible container
- Table: setVisibleRowcount behaving wrong
- ResponsiveLayout destroy and create FormElement new
- ToolPopup: Enabled 'getEnabled' Correctly
- Shell - Tooltip on Title
- Dialog: fix selection of Text in Chrome/IE
- ODataModel: fix server paging with relative URLs
- sap.m.SplitApp/App/Shell: fix background opacity property
- Work around Webkit bug (http://code.google.com/p/chromium/issues/detail?id=246965) which causes PullToRefresh to briefly appear on top of the page header when sliding in
- Mobile Growing List
- INBOX Issue:Search Mismatch in List and Stream View
- INBOX: Issue with Inbox control API Documentation
- INBOX: Text:Issue with the sorting functionality.
- INBOX: Issue In rendering the Lane Task By Drafts.
- sap.me.TabContainer: add CSS class for selected tab
- sap.m.IconTabBar: add CSS class for selected tab
- sap.m.GrowingList: 'Load more'-trigger centered, text position for IE9 fixed
- sap.m.List: group list header color adjusted, custom list item selection in blue crystal fixed, list plain header paddings fixed for blue crystal
- sap.m.SplitApp: correctly pass on the event parameters internal from NavContainers
- Fix landscape detection in SplitApp when keyboard opens, fix Dialog Bar size calculations
- Appending Task Title as tool tip for the link
- Downport of: "Fix in jQuery.sap.mobile.js and sap.m.Dialog"
- Enable Mass Custom Actions
- Popup: Completely Overhauled UI-Area Check
- Inbox fix for GO button in Forward Pop up in RTL.
- Mobile: List: GroupListHeader color, customlistitem selection fix, minor
- Mobile: Switch minor fix for blackberry 10, wrong less variable name
- NotificationBar: modified test page
- Removing Settings button when Substitution is disabled.
- Table: Fixed wrong firstVisibleRow when no data present
- ThemeDesigner: Bugfixes

Version 1.13.0 (May 2013)

Changes

Fixes

- NotificationBar: Activated Firing of 'Reize'-event
- NotificationBar: Made Callout follow the Notifier when opened

Framework

- `sap.ui.core.Popup`: Added a public API to use the 'followOf'-functionality
- `sap.ui.app.MockServer`: Added new experimental mock server class

- Theming: Automatic switch from SAP_CORBU to SAP_GOLDREFLECTION
-

Tools

- FIX: UI5 App Tool adds templates also for com.sap.suite.suite-ui-commons UI lib

Desktop Controls

- `sap.ui.core.ScrollBar`: Support for high number of steps.
- `sap.ui.ux3.DataSet`: Paging support
- `sap.ui.ux3.Shell`: Visual Adaptions
- `sap.ui.table.Table`: Column header menu API

Mobile

- The "Blue Crystal" theme (which was previously contained in the "sap.m" control library) has now been moved out to its own library "themelib_sap_bluecrystal". So if your application uses this theme, you need to add this one to the dependencies.
- sap.m.RatingIndicator?: a rating control has been added to the library, this control is still experimental

Version 1.12.2 (May 2013)

A patch for the rel-1.12 code line. It contains the following fixes for the UI5 Core and Controls:

Fixes

- sap.m.List: Active feedback for SingleSelectMaster in MVI-Theme fixed
- sap.m.StandardListItem: Title is set to bold again
- sap.m.List: styles for inner separators added
- sap.m.Select: add separator in footer context in bluecrystal
- sap.m.Input: incorrect placeholder color
- sap.m.Button: theme dependent fixes, padding for icon fonts
- sap.m.SearchField: placeholder not visible in IE9
- sap.m.Switch: Android browser 2.3 does not support % value for border-radius. Use em instead of %
- Blue Crystal theme: use sans-serif as fallback, not serif
- sap.m.List: width fixed, StandardListItem active feedback for icon added
- sap.m.Page: navButtonTap event is deprecated, instead a new navButtonPress event is introduced as a replacement
- sap.m.Shell: fix live change of logo in Theme Designer
- Added setCancelButtonText in ActionSheet
- Added subHeader aggregation to sap.m.Dialog.
- AppCacheBuster: fixed the traversal of folders to search webctx files
- Blue Crystal: use sans-serif as fallback font
- sap.m.Switch: Fix webkit border-radius rendering issue (Blackberry 10). Background color of elements with border-radius shows around outer edge of border at corners
- Fixed ToggleButton? press event fired when scrolling
- sap.m.Shell: avoid initial rerendering
- ScrollEnablement: allow live change of horizontal/vertical scrolling mode in ScrollContainer
- Notifier: Set Collision of Callout to 'None'
- sap.m.Shell: avoid unnecessary initial rerendering
- Refresh Popup-List
- NotificationBar: Really Per Default at Bottom
- Table: PAginator not updated correctly when data
- SegmentedButton: no background-color on hover in IE8
- NotificationBar: Fixed the Callout's Position After
- NotificationBar: Removed Focus Outline
- NotificationBar: Fixed Contrast of Counter in HCB

Allow mailto in links when protocol is added to
- Avoid tooltip on MenuButton click
- ComboBox: recognize if item text or key changes
- DropdownBox error after focusout and item update
- Bindings - extended change detection not working
- FileUploader - custom parameter support
- NotificationBar: Activated Firing of 'Reize'-event
- Scrollbar: scaled scrolling in the table with millions
- SplitApp master appears outside app area in Shell
- Tooltipbase: multiple Quickviews and RichTooltips
- Change scatter's random data to fixed data for ST team in Charting.html
- Changed to use fn for function, and f for float type.
- Fix a style issue in Popover.
- Generation/Core?: no CSS or parameter loading for libs without CSS
- Hot fix: Document domain issue. Based on Viz 144 version.
- Inbox: Downport of Major Fixes into 1.12.2
- Inbox:- Tasks by Priority Search in expanded lane.
- Merge "ScrollEnablement: preserve scrolling position" into rel-1.12
- Merge changes I0484b17a,I7a8bd622,Idd39abbe into rel-1.12
- Mobile List: MVI active icon feedback STDlist, width list fix
- Mobile List: MVI singleSelectMaster, title bold fix
- Mobile List: Separators Inner added
- Mobile: Button control fixes (theme dependent)
- Mobile: Button fix for blue crystal in iOS
- Mobile: Fix webkit border-radius rendering issue
- Mobile: Column box-sizing workaround
- Mobile: Complex Binding issue for Support is fixed
- Mobile: Custom Select, remove workaround added for previous versions
- Mobile: Custom select add separator in footer context and arrows for IE9
- Mobile: Page control Remove navButtonText + fire navButtonTap
- Mobile: Page control replaces tap event by press event
- Mobile: Switch use writeAttributeEscaped instead of writeAttribute
- Mobile: remove page appIcon in bluecrystal theme
- Mobile:Switch use em instead of % for border-radius,fix Android2.3 issue
- Modified visual test page for Slider
- New test for NotificationBar visual tests
- ResourceHandler: added special language fallbacks
- ScrollEnablement: allow live changes of vScroll/hScroll enabling
- ScrollEnablement: preserve scrolling position
- ScrollEnablement: small refinements
- Theme-Designer: Bugfixing Tooltips
- ThemeCheck: Ignore non-existing styles
- ThemeDesigner: BusyIndicator for Images;
- ThemeDesigner: Downport of latest bug fixes
- Theming: enhance Theme Parameters API
- Theming: map sap_corbu to sap_goldreflection also in Core#applyTheme()
- ToggleButton: avoid firing press when not necessary.
- Update makit library in sap.makit
- initMobile: display warning when IE9+ runs MVI
- sap.m.BusyIndicator: cancelTimeout
- sap.m.Dialog: added contentWidth and contentHeight properties to give
- sap.m.Input and (IE9)SearchField: incorrect placeholder color
- sap.m.SearchField: placeholder for IE9
- sap.m.SearchField: search event on Esc and Enter keys in IE9/10
- sap.m.Shell: update logo on theme change

Tools

- FIX: UI5 App Tool adds templates also for com.sap.suite.suite-ui-commons UI lib

Version 1.12.1 (May 2013)

A patch for the rel-1.12 code line. It contains the following fixes for the UI5 Core and Controls:

Fixes

- AppCacheBuster: fix to ignore the Servlet 3.0 static resources
- ButtonRenderer: workaround for theme name check
- SegmentedButton: issue with focus in Toolbar
- Text color on Dialog close button hover
- [Kelley] Popover: gets too big in IE9
- TextField - Edit mode
- MatrixLayout in IE10
- Form Demokit page, error if apply changes
- [Kelley] Dialog: stretchOnPhone only stretches
- Master area in SplitApp is hidden by iframe in Detail
- AutoComplete: Fix destroy
- Popover(ActionSheet) after resize out of screen
- Form - keyboard navigation and not focusable element
- SegmentedButton/ItemNavigation error
- Fix updateAggregation for Treebinding
- ODataListBinding: fix getEntityType if sPath and
- fix bug in setContext
- Charting: Update VIZ library to REL-144
- Core Device API: Fix os flags
- DataBinding: fix for complex syntax; custom data in views
- Dialog/Popover doesn't react to content size change when change
- Fix a browser bug in IE for sap.m.ActionSheet.
- Fix on Dialog style:
- Fix: CSN: [UI][Desktop]Budget/spending desapears in Home and Detail
- FlexBox - fix for parent == null
- FlexBox - fix rerender issue
- FlexBox - sanitize fix
- List: fix mouse pointer for labels in active items
- Merge "ShellRenderer: require Parmeters before using them" into rel-1.12
- MergedModules: fix optimizer servlet + fix fortify findings
- Mobile Growing List: Hide loading trigger
- Mobile Growing List: Fix for table in growing list
- Mobile List: InputListItem fix for slider control in IE9
- Mobile List: InputListitem fix for IE10
- Mobile List: SeperatorsNone for all listitems
- Mobile List: Standardlistitem images for IE9 fixed
- Mobile Object Status: fixed icon rerender
- Mobile Tile Container drag drop fixes
- Mobile: Custom select restrict the max-width of the popup
- Mobile: DateTime arrows for IE9
- Mobile: DateTime dialog positioning problem is fixed
- Mobile: Growing List - hide loading trigger - fix for table
- Mobile: ListTable Pop-in event handling
- MessageToast? fix position in ie9
- Mobile: Select, downport adaptation when the fonts are not downloaded
- Mobile: TextArea border issue is fixed
- Mobile: Transition bug related to TextArea in IOS fixed
- Modified visual test page for Shell control
- New test page for visual tests.
- PullToRefresh + SearchField: fix alignment and font
- ResizeHandler: don't call listeners for inactive DOMNodes
- ShellRenderer: require Parmeters before using them
- ThemeDesigner: Master Theme, Various fixes
- ThemeDesigner: Theming Engine fixes

- Theming: Fix contents of .theming framework file
- Theming: map sap_corbu to sap_goldreflection
- TileContainer: position:relative
- TileContainerRenderer: do not close too many divs; adapt Page style
- TileContainerRenderer: move left/right arrows on top of left/right drag
- Tiles: some transparency when in edit mode
- [rel-1.12] Fix the content width calculation in popover for theme
- jQueryUI: Remove jQuery UI :active selector because of performance
- sap.m.ActionSheet Fix: style width: 100% is removed.
- sap.m.Dialog fix: bStretch only takes effect on phone
- sap.m.FeedListItem: CSS amends for display in IE9 and overflow in all
- sap.m.FeedListItem: Fixed Bug in exit
- sap.m.FlexBoxCssPropertyMap: declare
- sap.m.ObjectHeader: contents is jumpung
- sap.m.ObjectHeader: jumping contents, part 2
- sap.m.PullToRefresh: incorrect scrolling after refresh
- sap.me - tagged controls as experimental
- sap.me.Calendar - Fixes
- sap_bluecrystal: short blue line in the button separator
- sap_bluecrystal: search field left border in a dialog
- sap_bluecrystal: vertical alignment in a read only simple form

Version 1.10.5 (April 2013)

Fixes

- AppCacheBuster: fix to ignore the Servlet 3.0 static resources
- TextField - Edit mode
- AppCacheBuster - exclude the OSGI-INF folder
- DeclarativeSupport - added cleanup of root DIV
- NavigationBar: Fix overflow detection
- ComboBox/DropdownBox - SeparatorItem no error
- Form - keyboard navigation and not focusable element
- Send count requests after filtering
- TextField in Form and numPad ±
- fix extraction of entity type
- HTML/XML/JSON view: Avoid unnecessary resource model creation
- Initial Filter API for Inbox.
- Paginator: Fix for focus-problems when using keyboard navigation
- Toolbar: overflow button issue

Version 1.12.0 (April 2013)

Framework

- NEW: Extended binding syntax for declarative views (incl. calculated fields, formatters and data types)
- New: `sap.ui.Device`: API for device and feature detection and media query support (see API Documentation)
- NEW: Quick Theming for GoldReflection is integrated into the Theme Designer.
- NEW: The location of custom themes can be configured via an URL parameter. However, the set of allowed URLs is currently limited to themes in the Theming Repository on the same server.
- NEW: Declarative support and HTMLView now support multiple associations (see Docu)

Tools

- NEW: New UI5 libs com.sap.suite.suite-ui-commons and com.sap.ui5.mobile-ext also added to JavaBuild Path and JavaScript Include Path when creating a SAPUI5 Application Project
- CHANGE: due to the different semantics, the standard SAP URL parameter `sap-accessiblity` is no

longer automatically mapped to `sap-ui-accessibility`. The UI5 parameter indicates whether UI5 renders accessibility information (ARIA) into the page and it is active by default. The standard `sap-accessibility` switch indicates that the user or administrator requested additional measures to make the application accessible. Applications that want to react on the second scenario, should evaluate the URL parameteron its own.
- CHANGE: handling of language codes has been enhanced for one special scenario: if for the Chinese language no region but a script is specified (e.g. zh-Hans), a best-matching region is automatically added to map to the existing translated texts (zh_CN and zh_TW).

Desktop Controls

- NEW: `ResponsiveLayout` not longer experimental. It can be used now as `Layout` for the `Form`
- NEW: Property `visible` on `Form`, `FormContainer` and `FormElement`
- NEW: Property `designType` on `sap.ui.ux3.Shell` and corresponding new type `sap.ui.ux3.ShellDesignType` to switch the design of the Shell.
- NEW: Event `paneClosed` on `sap.ui.ux3.Shell` which is fired when the side panel is closed.
- NEW: `Table` supports fixed rows and columns.

Contribution: "sap.viz"

- NEW: The chart controls of the `sap.viz` library have been enhanced in several ways:
  - all charts support basic touch gestures on touch enabled devices
  - all charts allow to specify additional CSS styles according to the VIZ chart documentation (property `css`)
  - all charts all the specification of an initial selection. See method `setDefaultSelection`. Thise default selection is not bindable yet.
  - a few properties and their types have been renamed. Please consult the API reference for an overview of the current API.

Contribution "sap.suite.ui.commons"

- NEW: sap.suite.ui.commons.NoteTaker Users can edit the note title.
- NEW: sap.suite.ui.commons.NoteTaker Color indication was added to the notes containing positive or negative information.
- NEW: Developed DateRangeSlider control. This control provides the user with a Range Slider control that is optimized for use with Dates. The Date Range Slider control allows the user to slide grips on a bar to select a certain range of dates where the value set is predefined. The slider can contain steps, or not; if steps are available, the Slider control can have ticks (totalUnits property). Standard date ranges are supported: yearly, quarterly, monthly, weekly, and daily.
- NEW: Developed DateRangeScroller control. This control provides a method to scroll through a series of time periods, each of which is represented by a starting date and an ending date, known as the date range. The user may scroll to the previous or next date range. Several predefined ranges are supported such as day, week, month, year, and custom. The user may also set Date Format (to override the control's default formatting) by an API call.
- NEW: Developed SplitButton control. This is a composite control that consists of a default-action Button and a MenuButton control. The default-action button control represents a simple pushbutton. It is used for initiating actions, such as save or print. It can contain some text, an icon, or both; the order of the two can be configured.
- The menu button control is a button that opens a menu when a user chooses this button. MenuButton is a composition of the Menu control and the Button control and thus inheriting all features.

Experimental Features:

Experimental features are not part of the officially delivered scope that SAP is going to guarantee for future releases – means experimental Features may be changed by SAP at any time for any reason without notice. The **Experimental features** are **"NOT FOR PRODUCTION USE"**. You may not demonstrate, test, examine, evaluate or otherwise use the Experimental Features in a live operating environment or with data that has not been sufficiently backed up.

The purpose of Experimental features is to get feedback at an early point of time allowing

customers/partners to influence the future product accordingly. Please use the SCN Developer Center http://scn.sap.com/community/developer-center/front-end to provide feedback accepting that Intellectual Property rights of the contributions or derivative works shall remain the exclusive property of SAP.

For general information about **Experimental features**, please check the Compatibility Rules, for a detailed list check out the list below:

- `Mobile Tablet Support:` The controls of the UI libraries sap.ui.core, sap.ui.commons, sap.ui.ux3 and sap.ui.table are only partially optimized or adapted for mobile tablet usage.
- `OData Write Support:` First experiments for two way binding.
- `Module sap.ui.core.plugin.LessSupport:` Only for development purposes.
- `Module sap.ui.core.delegate.ScrollEnablement:` Current status is prototype.
- `Configuration Parameters` with "-xx-" and the corresponding features: e.g. sap-ui-xx-test-mobile
- `Weinre Server Configuration:` Might change or removed in future releases
- `Control sap.ui.table.Table:` Current status for Threshold and Column Grouping is prototype.
- `Deprecated Message Controls in sap.ui.commons:` MessageBar, Message, MessageList, MessageToast - Use `sap.ui.ux3.NotificationBar` instead
- `Control sap.ui.core.HTML:` Support of HTML with multiple root nodes
- `Control sap.uiext.inbox.Inbox,sap.uiext.inbox.SubstitutionRulesManager:` API is not yet finished and might change
- `Control sap.ui.suite.TaskCircle:` API is not yet finished and might change
- `Control sap.ui.suite.VerticalProgressIndicator:` API is not yet finished and might change
- `Control sap.ui.ux3.ThingViewer:` API is not yet finished and might change
- `Control sap.ui.ux3.Shell:` Personalization, Color Picker (`Control sap.ui.ux3.ShellColorPicker`) and "Inspect"-Tool
- `Controls / Types sap.ui.ux3.Feed*:` Open discusses might result in API changes and adaptation. Especially the text presentation (e.g. @-references and formatted text) is not yet clear. Also Feed Model topic still open.
- `Control sap.ui.ux3.Exact:` Open discussions might result in API changes and adaptation. Especially the Result Area is not final.
- `Control sap.ui.ux3.ExactArea:` Open discussions might result in API changes and adaptation.
- `Type sap.ui.ux3.ActionBarSocialActions:` Open discussions might result in API changes and adaptation.
- `Theme Editor:` This application allows for easy branding of SAPUI5 applications. Currently optimized for the use with Chrome and Firefox.
- Improved performance for touch devices: Mouse events are now fired on their corresponding touch events
- Control `sap.makit.*:` API is not yet finished and might change
- Theme Parameters for `sap.m` and `sap.makit` are not yet finished and might change
- `Controls sap.viz.*:` API of the new charting library is not yet finished and might change
- `sap.ui.core.Component, sap.ui.core.ComponentContainer` API is not yet finished and might change
- `Control sap.m.GrowingList`. API is not yet finished and might change completely
- Performance recorder: API is not yet finished and might change
- `Control sap.m.Input` . API is not yet finished and might change completely
- `sap.ui.ux3.FacetFilterList` API is not yet finished and might change completely
- NEW: Property `type` added to `sap.m.Switch`. This makes possible to have different Switch types (e.g "Default", "AcceptReject?"), see API
- NEW: Property `navButtonType` added to `sap.m.Page`. This property is used to set the appearance of the NavButton?. By default, a back button is shown in iOS and up button in other platforms. In case you don't want platform dependent styling, you can set the value to "Default", see API
- New control library **sap.me** with extensions controls for mobile
- New platform agnostic theme **SAP Blue Crystal** for mobile and mobile extensions controls
- Control library **sap.m** has now basic desktop enablement. In general focus was put on enabling better support of mobile first and responsive designs that allows to build applications that work on mobile devices as well as on desktop browsers with a single codeline. A preview using experimental APIs is possible but the real shipment can be expected with 1.14.

Fixes

- NEW: Function `getEnabled` added to `sap.ui.ux3.ToolPopup`: This returns the state of the Popup

whether the ToolPopup is currently enabled or not.
- FIX: `sap.ui.commons.RichTooltip`: Fixed some padding issues
- FIX: `sap.ui.ux3.ToolPopup`: Fixes for the HCB-theme
- FIX: `sap.ui.ux3.ToolPopup`: Is now able to be run under jQueryUI version 1.10.x

Version 1.10.4 (April 2013)

Mobile Controls

- NEW: Property `showBusyIndicator` added to `sap.m.Carousel`: Show or hide a busy indicator in the carousel when loading pages after swipe. These busy indicators are part of the carousel and can be swiped as well: they do not block the carousel. Up to now, these busy indicators have always been shown. Now, the application developer has a choice. By default, busy indicators are shown.
- NEW: Property `busyIndicatorSize` added to `sap.m.Carousel`: Size of the busy indicators which can be displayed in the carousel, when loading pages after swipe. The default size is rather big (6em) to make sure that users notice that they may swipe the busy indicator.
- CHANGED: Return value of method `removePage` of `sap.m.Carousel`: returns the page that has been removed instead of returning the carousel
- CHANGED: Return value of method `removeAllPages` of `sap.m.Carousel`: returns an array of removed pages instead of returning the carousel

Fixes

- `sap.m.Carousel`: carousel is now displayed even if it is embedded in a scrollable page
- `sap.m.Carousel`: width problem with view control pages is fixed
- `sap.m.Carousel`: carousel works properly now in loop mode with less than 3 pages
- `sap.m.Carousel`: Fixed hide busy indicator order issue
- AppCacheBuster: fixed the normalization of URLs (avoid search/hash)
- AutoComplete: fix exit calls on items
- Sync issue with Overlay and Shell
- ToolPopup: Attached ToolPopup on Resize-Handler
- Table: fixed text-align to only apply to table tds
- Table: fixed text-align to only apply to table tds
- Paginator not reset if number of rows changed
- ScrollBar: fixed that SB is visible on iPad in Shell
- DropdownBox - possible to enter invalid text using IME
- ComboBox, value check after rerendering
- NotificationBar: Fixed Focusbility of Inplace Message
- TextField setting Editable in TreeTable fails
- Table: fixed the boundary calc for Pagination
- Only fire ValidationSuccess, if type is used
- Paginator: IE8 throws an error when trying to switch to
- Make getBindingPath work again
- prevent the encoding of the URL path
- Column visibility menu shows scrollbar in IE
- CSN: 1362724/2013 Association in HTML View: added createid for
- DropdownBox - fix qUnit test for IE9
- Fix makit height issue when use placeAt immediately
- Fixed HTMLViews for canary Chrome which supports WebComponents
- ODataModel: Make filter BT including instead of excluding
- Proxy: fixed critical violation in Sonar - missing null check.
- ResourceHandler: make the ClasspathUtil more error safe.
- Table: fixed the multi-selection behavior with SHIFT.

Version 1.11.1 (April 2013)

Framework

Tools

- NEW: The ABAP server side implementation of the application cache buster now does accept requests to non JavaScript resources with an invalid eTag in the URL. This is necessary in case resources are specified with relative URLs and triggered from usual script or css files. These are not initiated from the UI5 core and hence the eTag is not updated properly in the URL request. As a consequence these files not covered by the application cache buster mechanism and the user may have to reset the browser cache manually from time to time.

  Furthermore the experimental URL parameter `sap-ui-xx-appcachebuster-verbose` may be used to detect requests without or with an invalid eTag: This usually indicates the need for the JavaScript application developer to register a SAPUI5 component for cache buster use or to make consequent use of the jquery.sap.includeScript() and jquery.sap.includeStylesheet() functions.

  Lastly a developer may use the experimental URL parameter `sap-ui-xx-devmode` to activate development mode: Then requests with no or an invalid eTag are handled with the usual http 304 status mechanism; this takes a little longer but up-to-date resources are passed to the browser, which especially supports the development.

  Delivery is via SPS04 of the UI add-on 1.0 for SAP NetWeaver and via correction instruction in SPS03. See SAP note 1832992.

Desktop Controls

- NEW: Enabled HTML-tags within text of `sap.ui.commons.RichTooltip`: It's possible to use the same HTML-tags to the RichTooltip's texts that are valid for the `sap.ui.commons.FormattedTextView`.
- NEW: Property `valueStateText` added to `sap.ui.commons.RichTooltip`: This is an optional text for the ValueState of the RichTooltip's parent element.
- NEW: Property `imageAltText` added to `sap.ui.commons.RichTooltip`: This is an optional text for the alt-attrubute of the image that can be set to the content.
- NEW: Property `inverted` added to `sap.ui.ux3.ToolPopup`: With this property the ToolPopup can be set to a dark or bright background (needed for Kelley).

Mobile Controls

- NEW: Property `selectedItemId` added to `sap.m.Select`. This makes possible to bind the selected item id as a property, see API
- NEW: Property `translucent` added to `sap.m.Bar`. This indicates whether the bar is partially translucent., see API

Experimental Features:

Fixes

- `sap.ui.core.Popup`: Extended check for when content is in UI-area (with corresponding warning if not)
- SAP Note 1845740 "SAPUI5 Cache Buster - Http Status Code 404, META-INF" resolves a problem with eTags containing a hyphen character. Furthermore the META-INF folder is kept private. The note is available via SPS04 of the UI add-on for SAP NetWeaver and a correction instruction for SPS03.
- SAP Note 1846902 "SAPUI5 Cache Buster for Applications - CX_BSP_INV_PARAM_LEN" resolves an exception problem for SAPUI5 applications with long names. The note is available via SPS04 of the UI add-on for SAP NetWeaver and a correction instruction for SPS03.

Version 1.10.3 (March 2013)

A patch for the rel-1.10 code line. It contains the following fixes for the UI5 Core and Controls:

Fixes

- RadioButtonGroup remove from tabchain if disabled
- TextField/ComboBox toggle enabled - announcement

- Element bindings are not cloned correctly
- Error in updateBindingContext
- TextField/ComboBox on ToolPopup - Escape
- DropdownBox wrong value after rerendering
- Feeder - no empty lines in IE
- TabStrip, wrong rendered if all Tabs are disabled
- disabled DatePicker on iPad with iOS6
- Image of toggle button not updated
- SearchField: Switch to disable Cache
- DropdownBox error by rerendering
- Mobile UI5 Applications Inside iFrames
- Fixed height issues in the RichTextEditor
- DatePicker tooltip read twice if no label
- [Makit]Problem with charts on Android JellyBean
- Fix for IE10 specific close button in text field and
- Toolbar Overflow Pop-up
- Change: fire change event with filter/sort parameter
- Fixed QUnit test
- Mobile: MessageToast control fix qUnit timing issue
- Mobile: Select removeItem() method return always consistent value

Version 1.11.0 (March 2013)

Framework

- NEW: `sap.ui.core.IntervalTrigger` added. This control handles triggering of controls that register themselves. Triggering happens within a set interval.
- NEW: `sap.ui.core.CSSSizeShortHand` added. This type checks if a valid short-hand value was set. This can be used for a padding/margin type definition.
- NEW: API for "hidden aggregations" for composite control support (see respective documentation). **NOTE:** The support for the unofficial `_mHiddenAggregations` declaration will be removed in future versions of UI5.

Tools

- NEW: SAPUI5 Application Development Tool: Added "HTML View" as forth view type.

Desktop Controls

- NEW: Events `opened` and `closed` added to `sap.ui.commons.CalloutBase`: Possibility to attach to these events of the corresponding Popup
- NEW: the VIZ charts now support an automated resize of their content when the surrounding layout changes. To achieve "full height" for a chart, applications still have to layout their page accordingly, but at least will the charts adapt to any size changes.

Mobile Controls

- NEW: Property `selectedKey` added to `sap.m.Select`. This makes possible to bind the selected item key as a property, see API
- NEW: Easter egg `sap.m.Support` is added for internal usage.
- NEW & Experimental: Infinite scrolling option `scrollToLoad` is added to `sap.m.GrowingList` See API

Experimental Features:

Fixes

- AppCacheBuster: fix for property validation (to explicitly only check URI properties - avoids exceptions)

Version 1.10.2 (March 2013)

A patch for the rel-1.10 code line. It contains the following fixes for the UI5 Core and Controls:

Fixes

- ComboBox/DropdownBox fix cloning/item update
- Translation error fixes
- SearchField: Fix ARIA and HCB Theme
- TriStateCheckBox ARIA Fix + QUnit test
- sap.m.SearchField - wrong keyboard type
- NotificationBar: Fixed Splitting the ID in 'onclick'
- NotificationBar: Fixed Removal of One Single Message
- RefixTable getContexts
- Swipe Direction for List is added
- SearchField: Make destroy more robust
- Space not working in cells
- NavigationBar: Fix scroll animation
- ToolPopup: Fixed Too High ToolPopup
- Avoid error in setProperty with invalid context
- MOBILE: Custom DateTimeInput checker is added for
- ComboBox change event if same item selected
- RichTooltip: Long word wrapping
- RichTooltip: word wrapping
- Scrollbar: Chrom with zoom factor 90%
- FIX: Dataset: Dirty state management in setSelectedView
- HCB related issues in Task Execution UI.
- ManagedObject: fix property propagation when parent changes
- RichTextEditor not working correctly in PopUp when opened twice
- Table Fixed column feature not working correctly in chrome 25
- Table control: modified test page for visual tests
- ToolPopup: Adapted CSS-Styles for the Shell

Version 1.10.1 (February 2013)

A patch for the 1.10 code line. It contains the following fixes for the UI5 Core and Controls:

Fixes

- Fix the landscape/portrait detection
- AutoComplete: Fix ARIA
- applyTheme crashes in 1.10
- AppCacheBuster - fix for property validation
- Rangesldier method was assigned to slider prototype
- Carousel and RowRepeater don't work together
- sap.m.PullToRefresh - translated texts are too long
- SearchField: Fix ARIA
- Bindings updated for wrong Models
- Ensure Preserve Area does not increase screen size
- DropdownBox typing brings error in IE9
- Launch task in new tab for inbox
- SP7 functionality issues
- Sort on Custom Attributes and select filter does not
- Popup: Changed Check of UIArea
- fix: Scrolling problem of TextArea for And2.3.4
- Core: Extend OS Detection for Desktop
- Fix cursor position in Webkit on Mac
- TextField/DropdownBox drop handling
- ExactBrowser: Fix vertical list collapse
- GridLayout fix alignment issues in IE9
- Carousel wrong index after rerendering

- OData E-Tag Fix reduced to oData Fix for update method
- AutoComplete: Provide special suggest event
- DatePicker, validation of blank values
- ToolPopup: Changed Default Value of Collision
- jQuery.sap.encodeURL: * is not encoded
- MenuButton/Menu: Use control ref instead of DOM on Menu
- remove of 100 accounts leads to wrong member display
- Fix paging: iLength should always be the Visible row
- FileUploader: rerendering with wrong size
- ODataModel: fix incasesensitive headers: security token
- Carousel: Better scrolling behaviour
- Carousel: Don't scroll window when using arrow keys
- Carousel: Make sure all visible items are displayed
- Carousel: Optimized rendering and animation
- DataBinding: avoid incomplete bindings, fix bUpdateAll case
- Documentation: replace link to internal server with SDK reference
- Fix encoding of several test pages
- Fortify: split code transformation to avoid out of memory
- sap.m disable tap events during scrolling
- Script: Added jquery.sap.each which ignores length property in objects
- Table: modified test page for visual tests
- Table: Fixed Columns: Posititon horizontal scrollbar correct for RTL
- Table: Fixed scrolling behavior and page scroll onclick
- VIZ Charts: avoid access to console in unit tests
- VIZ Charts: fix memory leak reg. VIZ instances
- VIZ Charts: fix typo in FlDataset filling + enable unit tests
- VIZ Charts: properly transform sparse data
- VIZ Charts: use own message bundles, support alt text
- sap.m.PullToRefresh: use standard busy indicator

Version 1.8.9 (February 2013)

A patch for the 1.8 code line. It contains the following fixes for the UI5 Core and Controls:

Fixes

- Table shift click selection not working correct
- SearchField: Set min-width to avoid button overflow
- Core: Extend OS Detection for Desktop
- Fix cursor position in Webkit on Mac
- ToolPopup: Fixed Opening of the Popup
- NotificationBar: Fixed Clickability of Inplace-Message
- Shell + NotificationBar: Enabled New Messaging Concept
- ODataModel: fix incasesensitive headers: security token
- Carousel: Don't scroll window when using arrow keys
- FIX: Encoding of several test pages
- Fix paging: iLength should always be the Visible row size. Otherwise we
- Mobile: content in footer is not being seen in Page with flip animation
- Shell control: Modified QTP test page
- ToolPopup: Fixed the Mix Up of Fixes of 1.8 And Features of 1.9

Version 1.10.0 (January 2013)

Framework

- NEW: `sap.ui.core.Component` A basic concept for components in UI5 has been defined and implemented. (see API)
- NEW: `jQuery.sap.storage`: Option to define a custom key prefix instead of a default one: e.g. `jQuery.sap.storage(jQuery.sap.storage.Type.session, "MyOwnPrefix")`
  `sap.ui.core.mvc.HTMLView`

NEW: An HTML based format for declaring MVC views.
- NEW: `sap.ui.model.odata.ODataModel` Added function `setHeaders` to allow to set custom headers which will be sent in each request against the server. This is now also possible in the constructor as a parameter. Function `getHeaders` provides access to all headers.
- NEW: Data Binding: Aggregation binding support for named models.

Tools

- FIX: Correction in UI5 Application Development Tool to have `com.sap.ui5.resource.DEV_MODE` switched off in generated web.xml

Desktop Controls

- NEW: Property *autoClose* added to `sap.ui.commons.Dialog` control. If set to true the Dialog will behave like a Popup on 'autoclose'.
- NEW: `sap.ui.commons.form.ResponsiveLayout` added. This layout is another Layout to be used in conjunction with the Form.
- NEW: `sap.ui.commons.AutoComplete` added: Textfield with suggestion list (see API)

Mobile Controls

- NEW: All controls in library sap.m now support **BlackBerry 10** devices. This does not apply to library sap.makit.
- NEW: `sap.m.SplitApp` control. SplitApp is another top level control in application, which has a master and a detail area. It manages the show/hide of the master area according to the device and orientation. See API
- NEW: `sap.m.InstanceManager` Utility Class. InstanceManager is a utility class which manages instance under given category. Currently all open sap.m.Dialog, sap.m.Popover, sap.m.ActionSheet, option picker for sap.m.Select when runs in Android 2.3 and date(time) picker when runs in Android 2.3 are managed inside. It provides the possibility to close all of them at the same time which is required for handling the tap of physical back button in Android device. See API
- NEW: `sap.m.MessageBox` Utility Class. MessageBox provides an easier way for showing short message to user than creating a sap.m.Dialog with type sap.m.DialogType.Message. See API
- NEW: Scroll bars in scroll areas, scrolling enabled pages and dialogs.
- NEW: Type `sap.m.DialogType.Message` added to `sap.m.Dialog`: Dialog with type Message aims to show message to end user which has a similar design as the "alert" dialog in Javascript.
- NEW: `sap.m.ActionListItem` control. ActionListItem is a list item, with a centered text for triggering actions and also by default uses the active type.
- NEW: No Data Handling for lists added. a text will be shown for empty list, which can be customized and also be deactivated.
- NEW: `sap.m.ListItemBase` feature. Through the ListItemBase all list items can now use a counter feature, which shows a bubble containing an integer.
- NEW: `sap.m.ListItemBase` feature. Through the ListItemBase all list items can now use an unread indicator in a form of a little blue bubble.
- NEW: `sap.m.StandardListItem` feature. The StandardListitem now provides an info field, which can be used to show information texts. These texts support semantic colors based on the usage e.g. 'Warning','Error', 'Info'.

Experimental Features:

Experimental features are not part of the officially delivered scope that SAP is going to guarantee for future releases – means experimental Features may be changed by SAP at any time for any reason without notice. The **Experimental features** are **"NOT FOR PRODUCTION USE"**. You may not demonstrate, test, examine, evaluate or otherwise use the Experimental Features in a live operating environment or with data that has not been sufficiently backed up.

The purpose of Experimental features is to get feedback at an early point of time allowing customers/partners to influence the future product accordingly. Please use the SCN Developer Center http://scn.sap.com/community/developer-center/front-end to provide feedback accepting that Intellectual Property rights of the contributions or derivative works shall remain the exclusive property of SAP.

For general information about **Experimental features**, please check the Compatibility Rules, for a detailed list check out the list below:

- `Mobile Tablet Support:` The controls of the UI libraries sap.ui.core, sap.ui.commons, sap.ui.ux3 and sap.ui.table are only partially optimized or adapted for mobile tablet usage.
- `OData Write Support:` First experiments for two way binding.
- `Module sap.ui.core.plugin.LessSupport:` Current status is prototype.
- `Module sap.ui.core.delegate.ScrollEnablement:` Current status is prototype.
- `Configuration Parameters` with "-xx-" and the corresponding features: e.g. sap-ui-xx-test-mobile
- `Weinre Server Configuration:` Might change or removed in future releases
- `Control sap.ui.table.Table:` Current status for Threshold and Column Grouping is prototype.
- `Deprecated Message Controls in sap.ui.commons:` MessageBar, Message, MessageList, MessageToast - Use `sap.ui.ux3.NotificationBar` instead
- `Control sap.ui.core.HTML:` Support of HTML with multiple root nodes
- `Control sap.uiext.inbox.Inbox,sap.uiext.inbox.SubstitutionRulesManager:` API is not yet finished and might change
- `Control sap.ui.suite.TaskCircle:` API is not yet finished and might change
- `Control sap.ui.suite.VerticalProgressIndicator:` API is not yet finished and might change
- `Control sap.ui.ux3.ThingViewer:` API is not yet finished and might change
- `Control sap.ui.ux3.Shell:` Personalization, Color Picker (`Control sap.ui.ux3.ShellColorPicker`) and "Inspect"-Tool
- `Controls / Types sap.ui.ux3.Feed*:` Open discusses might result in API changes and adaptation. Especially the text presentation (e.g. @-references and formatted text) is not yet clear. Also Feed Model topic still open.
- `Control sap.ui.ux3.Exact:` Open discussions might result in API changes and adaptation. Especially the Result Area is not final.
- `Control sap.ui.ux3.ExactArea:` Open discussions might result in API changes and adaptation.
- `Type sap.ui.ux3.ActionBarSocialActions:` Open discussions might result in API changes and adaptation.
- `Theme Editor:` This application allows for easy branding of SAPUI5 applications. Currently optimized for the use with Chrome and Firefox.
- Improved performance for touch devices: Mouse events are now fired on their corresponding touch events
- Control `sap.makit.*:` API is not yet finished and might change
- Theme Parameters for `sap.m` and `sap.makit` are not yet finished and might change
- `Controls sap.viz.*:` API of the new charting library is not yet finished and might change
- `View sap.ui.core.mvc.HTMLView` The HTML view concept is still under discussion so this will be changed in future.
- `sap.ui.core.Component,sap.ui.core.ComponentContainer` The Component concept is still under construction, so some implementation details can be changed in future.
- `Control sap.m.GrowingList.` API is not yet finished and might change completely
- Performance recorder: API is not yet finished and might change
- `Control sap.m.Input` . API is not yet finished and might change completely
- `Layout sap.ui.commons.form.ResponsiveLayout` This layout is not 100% finished yet.
- `sap.ui.ux3.FacetFilterList` API is not yet finished and might change completely

Fixes

- sap.m.Select: Back button does not close popup in Android 2.3
- sap.m.Bar: Search field is not shown on Page footer with flip animation
- sap.m.Bar: Binding of icons does not work in Page Header
- sap.m.Input: Displaced cursor by scroll
- sap.m.Input: Outside touch does not close the soft keyboard
- sap.m.Carousel: No resize during orientation change
- sap.m.Carousel: Carousel in split app: wrong width after orientation change
- sap.m.Carousel: Pages overlap when swiping
- sap.m.Carousel: 2 paged carousel bug
- sap.m.Carousel: Mobile Carousel not working after orientation change
- sap.m.Carousel: Carousel waiting indicator after last page
- sap.m.Dialog: Fix blinking issue when reposition dialog in iPhone.
- sap.m.Dialog: Fix "No feedback color when tap Dialog Button" issue.

- sap.m.ActionSheet: Fix "no active color for buttons in ActionSheet? for Android 2.3".
- sap.m.ActionSheet: Fix "cancel button hidden after orientation change" issue.

Version 1.8.8 (January 2013)

A patch for the 1.8 code line. It contains the following fixes for the UI5 Core and Controls:

Fixes

- Datajs: fix batch delete where response.body is empty string
- ResourceHandler: fix in the CacheControlFilter for nocache headers
- Mobile: Switch update qUnit-test
- Infra: fix encoding of jsunit test reports, fix chrome specific test
- SimpleProxyServlet: fixed the localhost check to consider 0.0.0.0 addr.
- RowRepeater: Rendering error on last page
- MenuBar: Overflow Menu does not close
- Table: Wrong return value in selectedIndices
- BorderLayout: Fixed Alignment in Content Areas
- ToolPopup: Arrow at Wrong Position When Scrolling
- Inverted Content Text Color
- FileUploader: disable + set a width not working
- Displaced active input in Android 4.0
- !ODataModel: Use substringof in $filter
- Select closes immediately
- Toolbar/ItemNavigation: first item not focusable
- ToolPopup: Fixed Setting Focus of First Focusable Item
- Data binding is not working in Page Header
- FileUploader: IE8 uploadOnChange bug

Tools

- FIX: Correction in UI5 Application Development Tool regarding view name and folder name validation
- FIX: Correction in UI5 Application Development Tool to have `com.sap.ui5.resource.DEV_MODE` switched off in generated web.xml

Version 1.9.2 (January 2013)

Framework

- NEW: Databinding: Calculated fields. It is possible to bind multiple path from different models to a control property. The values can be accessed via the formatter function. See the databinding documentation for more details.
- NEW: A first, still experimental implementation of Components has been added to the Core.
- NEW: `sap.ui.core.mvc.HTMLView` Added experimental HTMLView. You can now create views with declarative HTML.
- CHANGE: `sap.ui.core.DeclarativeSupport` Enhanced aggregation handling in declarative support: Added support for alternative types and added support for 0..n aggregation binding with templates.
- NEW: the `sap.ui.core.ApplicationCacheBuster` now has experimental support for multiple index locations and can manage more types of URL modifications
- NEW: all Controls and Elements (all objects derived from `sap.ui.base.ManagedObject`) now support setting models and the binding context already in the constructor (property names `models` and `bindingContexts`). The `models` property can be set either to a single model (by definition this will be the default model) or to a map of models (keyed by their name, use the value `undefined` (not the string 'undefined'!) for the default model). In 1.9.2, the `bindingContexts` may only be set to a single context, but in one of the next releases, UI5 will support one binding context per model (means: aggregation bindings will finally be supported for named models, too). The semantics of these properties is equal to calling the corresponding setters after construction. But note that using

the properties might result in an earlier resolution of property and aggregation bindings. The properties can be used in Views. This finally allows to access the models in the controller without setting them manually in onInit.
- NEW: Added Sinon.JS as third party library. Use spies, mocks, stubs, faked timers / XHR in QUnit tests.
- NEW: Added URI.js as third party library. Supports URI parsing, construction, normalization etc.

Tools

-
- FEATURE: Time needed to launch a SAPUI5 application served from a SAPUI5 ABAP Repository and with the application cache buster in place has been reduced (by .2 to 3 seconds depending on the use case). Report /UI5/RESET_CACHEBUSTER to reset related server side cookies after system- and language imports.
- FIX: Correction in UI5 Application Development Tool regarding view name and folder name validation

Desktop Controls

- NEW: Properties *responsive* and *itemMinWidth* added to `sap.ui.ux3.DataSetSimpleView` control. See this example for details.
- NEW: Property *enableSave* and event *save* added to `sap.ui.ux3.ExactBrowser` control. If activated a save button is shown in the toolbar of the ExactBrowser.
- NEW: Keyboard Handling added to `sap.ui.commons.form.Form` control and `sap.ui.commons.form.GridLayout`. Action Mode and Navigation Mode added (also affects `sap.ui.commons.Toolbar`).
- NEW: `sap.ui.core.VariantLayoutData` created. This ist to add multiple `LayoutData` to controls. Currently it's only used for form layouts.
- NEW: `sap.ui.table.Table` has a new property `visibleRowCountMode` which allows to activate auto height calculation (Table shows as many rows as possible) or manual height changes by the end user. See API documentation for details.
- NEW: `sap.ui.table.Column` has a new property `filterType` that allows to specify a data type for the filter field. The data type should match the type used for formatting/parsing data in the same column. This allows end users to use the same data format in the filter field as is used for the data in the table column.

Mobile Controls

- NEW: `sap.m.URLHelper` Added to trigger native applications(e.g Email, SMS, Telephone) easily over URI Schemes. See API and Documentation
- NEW: Swipe List for Action is added into `sap.m.List`. See Documentation
- NEW: `sap.m.PullToRefresh` control. The control adds the "Pull Down to Refresh" functionality to pages and scroll containers. See Pull to Refresh
- NEW: `sap.m.MessageToast` control. A message toast offers simple feedback about an operation in a small Pop-up. See API

Version 1.8.7 (January 2013)

A patch for the 1.8 code line. It contains the following fixes for the UI5 Core and Controls:

Fixes

- ToolPopup: Fixed Height of Focusable Dummy-Element
- ToolPopup: Fixed Arrows for HCB
- ToolPopup: Fixed Setting the Focus
- Tree: Enabled Tree On Black Background
- Notifier: Prevent Re-render If There Are No Messages
- DataSet Items rendered even if they are deleted
- ToolPopup: Can't Be Closed If Modal
- Shell: Fix HeaderType Switch

Shell: FacetBar not shown
- NavigationBar: Fix Scroll Animation
- Shell: FacetBar not shown
- DataJS: In Atom format DateTime seconds are optional
- Panel translate expander texts if toggled
- ToolPopup: Enabled Transparancy for IE9
- NavigationBar: Prevent IE from firing beforeunload event
- ExactBrowser: Fix timing issues
- Shell: Fix animations
- MessageBox: Removed Unneeded Icons
- Shell: Fix Position of Navigation items in HCB
- Fix Refresh security token endless loop
- ToolPopup: Fixed detach of open handler
- NotificationBar: Extended Demokit-Page
- ToolPopup: Made Arrows for HCB nicer
- SimpleProxyServlet: support for IPv6 addresses (in local check)
- Infra: fix test issues due to QUnit 1.10's global error handler
- sap.m.Carousel: XSS prevention: replaced 'write' calls by 'writeEscaped' when writing Ids in CarouselRenderer
- sap.m.Carousel: Fixed 'showBusyIndicator' issue in Renderer

Version 1.9.1 (December 2012)

Framework

- NEW: Many of the basic features of `Element` (support for declared properties, aggregations, associations, events ...) have been factored out in a new base class `sap.ui.base.ManagedObject`. This class is not intended for control development (it intentionally does not cover rendering aspects), but might be helpful when creating other classes that don't need rendering capabilities. The `extend` function is available on the new class and can be used to subclass it.
- CHANGE: Cloning of Views has been fixed. Instead of cloning the content **and** going through the MVC lifecycle, which resulted in duplicate content, Views only go through the lifecycle (`createContent()`) and just clone those aspects that aren ot covered by `createContent()`. See the MVC documentation for more details.
- CHANGE: The validation for aggregations and for property and aggregation bindings (`bindProperty`, `bindAggregation`) has been activated. Those methods now can only be called for existing properties/aggregations and only valid types must be used for the aggregated objects. The only important exception is the often discussed `Toolbar.items` aggregation, for which only a warning is raised when an invalid type is added to that aggregation.
- CHANGE: The databinding implementation has been enhanced wrt. performance. Until 1.9.0, controls/elements always had to use their parent hierarchy to find a model or a binding context. As this code was executed quite often, it could result in a performance penalty for more complex UIs. Starting with 1.9.0, the model/context information is pushed down the hierarchy when it changes.
  Note: This change has a significant impact on custom composite controls: composite controls now always should use aggregations for their parts (composite components). Just setting the parent relationship is no longer sufficient wrt. databinding.
- CHANGE: OData Write functionality now also supports the use of ETags for concurrency control.

Desktop Controls

- NEW: two new layouts have been added: `sap.ui.commons.form.GridLayout` (used in the `sap.ui.commons.form.Form` control) and `sap.ui.commons.layout.ResponsiveFlowLayout`.

Mobile Controls

- NEW: Control `sap.m.ActionSheet` is added. The main usage is to show some options and let the end user choose one from them. `sap.m.ActionSheet` behaves as a `sap.m.Popover` and as a standard `sap.m.Dialog` in the other devices.
- NEW: A new mode PopoverMode is added to Control `sap.m.SplitApp`. With PopoverMode the

Master Panel is shown inside a Popover.
- NEW: Properties *width* and *height* added to FlexBox control in sap.m library
- NEW: Control `sap.m.InputDateTime` is added. Let end users to interact with date and/or time pickers.
- NEW: event `liveChange` in `sap.m.SearchField`
- FIX `sap.m.SearchField`/`sap.m.Input`: on-screen keyboard remains open after page navigation
- NEW: `sap.m.Slider` support for floating point numbers
- NEW: `sap.m.Page` control has now a `headerContent` aggregation which can be used to add items (usually Buttons) to the right side of a Page header without creating a custom header
- NEW: the `sap.m.NavContainer` (and `sap.m.App`) control sets its direct children to display:block and height:100% to avoid additional effort in applications, in particular when using MVC Views
- DEPRECATED: `Date`, `Time`, `Datetime`, `DatetimeLocal`, `Week` and `Month` are deprecated in `sap.m.InputType`
- DEPRECATED: `dateFormat` is deprecated in `sap.m.Input`

Tools

- FEATURE: Report to synchronize UI5 Application in local file system with UI5 Repository located in a SAP NetWeaver Application Server 7.00 or higher. For details see SAP Note 1793771 "Up/Download SAPUI5 Application into/from UI5 Repository".
- FEATURE: When a project has been shared with the SAPUI5 ABAP Repository you can directly start a preview of the deployed application on the server from Eclipse
- FIX: Open External Browser button in application preview considers the Eclipse settings which browser to use

Version 1.8.6 (December 2012)

A patch for the 1.8 code line. It contains the following fixes for the UI5 Core and Controls:

Fixes

- Panel Header width in IE9
- FormattedTextView: Removed Reference in TestPage
- Panel: fix header width
- MessageBox: Fixed Icons to New Visual Design
- MessageBox: Fixed Mirrored Icons in RTL-mode
- ItemNavigation with DatePicker
- NotificationBar: Fix Open Callout on iPad (mobile devices)
- MessageBox: Fix MessageType Icons Fitting the New VD
- DateFormat: Fix date parsing for negative timezones
- DatePicker, not possible to enter + or -
- NotificationBar: Set the Callout To Fixed Position
- NotificationBar: Fixed Disappearing Notifier Icons
- Mobile - Reduce page app icon size
- Notifier: Fixed Used Value in 'removeMessage'
- SearchField: Update suggestions on cut or paste
- ActionBar: "ThingInspector.setFavoriteState() has no effect"
- TreeTable - JS error when model data comes later
- Rerendering not working
- RichTextEditor XSS issue
- Changed display none to visibility hidden in order to make select control get the right wid
- Clicking SVG elements inside the sap.ui.ux3.Shell may cause a JS error
- Demo apps: remove unnecessary 100% height
- Enhanced the touch to mouse check - seems like there are some browsers which provide touch events e
- Mobile: Slider, Bugfix properties may be outdated.
- Mobile: update MobiScroll to version 2.2
- NavContainer: set height of children to 100% by default
- Removed dev dependency from Range Slider Test Page
- Removed the internal URL from the messagebundles.

RichTextEditor: Content not set when setValue is called to early
- sap.m.Button: add hooks for inheriting controls to add their HTML
- SimpleProxyServlet: Fixed the URL validation (allow special chars)
- Upgraded to latest TinyMCE version (3.5.8)
- VIZ Charts: fix invalidation when model data becomes unavailable

Version 1.8.5 (November 2012)

A patch for the 1.8 code line. It contains the following fixes for the UI5 Core and Controls:

Fixes

- Demokit XSS vulnerability
- Menu: Prevent browser history navigation
- Table: apply the correct laf for multi selection
- MenuButton, MenuBar, Column: Change Menu open
- DatePicker validation via databinding
- Table: fixed the alignment of the filter/sort ind.
- Richtext Editor not working in ThingInspector Facet
- ODataListBinding: filter/sort fix
- NotificationBar: Fixed Setting Width of Inplace Message
- Arrow Key Behavior of TextFields with ItemNavigation
- DropdownBox selection in RTL mode
- Accordion - Bugfix for height calculation in IE8
- RangeSlider move right grip if both habe same position
- RangeSlider sometimes doesn't fire change event
- Databinding not working in ExactBrowser
- ToggleButton - remove mouseover event after click
- ComboBox/DropdownBox fix focus by opening in IE
- DataSet: Unnecessary rerendering/removeView fails
- Carousel: Allow elements without width/height property
- Demokit: SAPUI5 Tools related changes
- Mobile: Removed performance app from library
- Mobile: set up Mobiscroll and integrate with the select
- NumberFormat: Fix bug when parsing large numbers
- ODataModel: Avoid unnecessary code in $filter
- ODataModel: fix import data to support select
- sap.m.Carousel: Removed destroy call from 'exit' method
- sap.m.Carousel: replaced test-PNGs with JPGs

Version 1.9.0 (November 2012)

Framework

- FEATURE: CustomData attached to controls (with the `.data(...)` function) can now optionally be written to the HTML document. See the documentation.
- FEATURE: the new methods `addEventDelegate` and `removeEventDelegate` are now available on all Elements/Controls and can be used by applications to also react on all events which are happening to those Elements (this is different than listening to events fired *by* those elements, which was always possible). This is basically a public version of the private method `addDelegate`.
- FEATURE: `sap.ui.core.CSSSize` now also allows value `inherit`. However, there are some usages of this type where `inherit` doesn't make sense.

Desktop Controls

- FEATURE: `sap.ui.ux3.Shell`:
  - NEW: Property `fullHeightContent` to activate / deactivate 100% height of the content area
  - NEW: Property `applyContentPadding` to switch padding of the content on / off
  - NEW: Application, header and background image can be set via theme-editor

NEW: control `sap.ui.commons.FormattedTextView`
- NEW: control `sap.ui.ux3.CollectionInspector`
- NEW: Property `displaySecondaryValues` on control `sap.ui.ux3.FacetFilterList`
- `sap.ui.commons.RichTooltip`
  - NEW: support for {{addStyleClass}}}
- `sap.ui.commons.Toolbar`
  - NEW: keyboard support for right (end) items
- CHANGE: `sap.viz` library has been updated and refactored as documented below for [version 1.8.4](#).

Fixes

- ThingInspector: Add aria-labelledby to links on ACC testpage
- Page in Popover direcly causes wrong display.
- fix iScroll to ignore subsequent touches
- open multiple dialog can't hide block layer when close
- Issues fixed in Popover:
- Reason: fix Desc: Toolbar error handling
- Roadmap: Fix ARIA
- Shell: Fix ARIA
- Button: Fix Platinum Theme
- ExactBrowser: Fix QUnit test on FF10
- Reason: fix Desc: BC-WD-HTM - Fix Pegasus Environment test for Firefox ESR
- ThinkInspector doesn't show long Type properly
- TextField - remove hover effect for iPad
- FileUploader: tabbing backwards does not work
- MenuButton, MenuBar, Column: Change Menu open
- Table: FF only, Empty table has header only, no rows
- Accordion: last open section cannot be closed
- Dialog doesn't fit the screen in Android 2.3
- sap.m.Select and sap.m.Slider controls in the input list item are still not aligned horizontally to the right
- sap.m.Select dropdown does not even open in most cases, sometimes the wrong one opens in Android 2.3
- DropdownBox - visualization of wrong input
- OverlayContainer rendering issue in Shell
- Table - fixed the ff default selection behavior
- ODataModel: make methods parameters and doc consistent
- DropdownBox error if update via binding
- ComboBox setSelectedKey("") initialize control
- ODataModel: add language accept headers
- ComboBox change text of selected item
- VIZ Charts: use position:relative by default
- VIZ Charts: support 'controller' properties
- Fixed rendering for Toggle Button (when rendered in table control)
- Switch of History feature in DropdownBox and SearchField by default
- NavContainer: content looks blurry
- NotificationBar: Fixed width of inplace message
- DatePicker chinese day names
- VIZ Charts ignore settings via method calls
- VIZ Charts: null values in dataset lead to exception
- Support default values for array types
- Fix "NO Feedback color when tap Dialog Button"
- EPM Demo app: do not say "Northwind when loading data
- XMLView: Encode HTML properties and content

Mobile Controls

- FIX Select doesn't open every time

Tools

Version 1.8.4 (November 2012)

Desktop Controls

- VIZ Charts: update to newest VIZ Library (build no. 78) VIZ has streamlined its APIs to achieve better consistency within the BI suite. As the wrappers are still experimental and should reflect as close as possible the VIZ native library, it was decided to fully adapt to these changes without providing full backward compatiblity with 1.8.0 - 1.8.3.

  The most important changes are:

xaxis	xAxis	configuration property name
yaxis	yAxis	configuration property name
bar,combination, ...	plotArea	the main configuration object for a chart has been renamed from the specific *chart* to the generic *plotArea*
tooltipVisible	tooltip.visible	the flat boolean attribute for the tooltip visiblity used by some charts has been transformed to a more flexible configuration object for the tooltip with a dedicated property visible
*VerticalBar	*Column	all charts containg 'VerticalBar' have been renamed to use 'Column' instead.
Percentage*	*100	the term 'Percentage' has been replaced with '100'
events	*	event parameters have been renamed, moredetails to be provided by VIZ library

  With the same change, a better separation of the namespaces of the native VIZ library and the UI5 wrappers was introduced. This resulted in a generic renaming of all controls/elements in the sap.viz library:

sap.viz.core.*	sap.viz.ui5.core.*	for the chart and structure base classes
	sap.viz.ui5.data.*	for the dataset implementations
sap.viz.*	sap.viz.ui5.*	for the different charts
sap.viz.types.*	sap.viz.ui5.types.*	for the different helper types

  Early adopters of 1.8.0 might check the JSDoc for the sap.viz library for an overview about the 1.8.4 APIs or contact the UI5 team for a more detailed overview of the renamings.

- FIX: NotificationBar: Fixed width of inplace message
- FIX: VIZ Charts ignore settings via method calls
- FIX: Support default values for array types
- FIX: DatePicker chinese day names
- FIX: Switch off history feature in DropdownBox and SearchField by default
- Fixed toggle button eventing when same property is set twice
- FIX: VIZ Charts: null values in dataset lead to exception
- FIX: VIZ Charts: use position:relative by default

Mobile Controls

- FEATURE: `sap.m.Switch`:
  - NEW: Properties `customTextOn` and `customTextOff`, application defined text for the "ON" and "OFF" state.

Version 1.8.3 (November 2012)

Mobile Controls

- FIX: BusyDialog - Issue: calling open() immediately after close(), blocklayer can't be removed after calling close() again.
- FIX: `sap.m.Carousel`: Image sizing under Android 2.3
- FIX: `sap.m.Carousel`: Problems with orientation change under Android 2.3
- FIX: `sap.m.Popover`: Problems with Popover size and position in Android 2.3
- FIX: `sap.m.SearchField`: Reset button does not work in Android 2.3
- FIX: `sap.m.Select`: Select without items causes errors

Version 1.8.2 (November 2012)

Framework

- FIX: Table - fixed the ff default selection behavior
- FIX: ThingInspector doesn't show long Type properly
- FIX: Fixed rendering for Toggle Button (when rendered in table control)
- FIX: Element - Improve performance by reducing getParent() calls.
- FIX: BPM Inbox - Substitution Manager UI fixes and timezone fixes.

Version 1.8.1 (End of October 2012)

A patch for the 1.8 code line. It contains the following fixes for the UI5 Core and Controls:

Framework

- FIX: ODataModel: When using Context.getPath() the path starts now with a '/' like the path in JSONModel and XMLModel. The path can be used in the oDataModel.getProperty function to retrieve the object. Also Context.getObject() is now working with the ODataModel.
- FIX: ODataModel: documentation for refreshSecurityToken. Change parameter order for submitBatch function to (fnSuccess, fnError, bAsync).
- FIX: ODataModel: Batch to set the content-length for the batch parts and also remove odata.verbose for accept headers and content-type headers.
- FIX: sap.commons.Dialog: header tooltip added
- FIX: ODataModel: added accept-language header to use the current configured language

Mobile Controls

- FIX: `sap.m.Bar`: Ellipsis is shown when there are enough space
- FIX: `sap.m.Popover`: Fixed calling non-existing `_close` method
- FIX: `sap.m.Input`: Set default font for different input types
- FIX: `sap.m.SegmentedButton`: width calculation in Android 2.3
- FIX: `sap.m.Input`: Alert icon not present for `setValueState("Error")`
- FIX: `sap.m.Carousel`: `removeAllPages` does not work in XML Views
- FIX: `sap.m.Dialog`: Issue with close in Android
- FIX: `sap.m.Select`: Values are ellipsed though there is enough space
- FIX: `sap.m.!ListItemBase`: active state for navigation icon
- FIX: `sap.m.Page`: proper destroy of scroller
- CHANGE: `sap.m.List`: each list has an unique radiobutton group for the single selection feature

Tools

- FIX: New UI5 libs com.sap.ui5.makit and com.sap.ui5.viz now also added to JavaBuild Path and JavaScript Include Path when creating a SAPUI5 Application Project
- NEW: Compatibility of SAPUI5 ABAP Repository Provider with ABAP in Eclipse 2.0.x

Theme Designer

- FIX: Page title was not loaded in FF and IE due to bug in iFrame load event handling
- FIX: The removal of a preview page was not working properly
- FIX: Several bugfixes regarding parameter loading, updating and resetting

- FIX: Various stability fixes (related to custom CSS, invalid link elements)
- NEW: Support of theme parameter API and applyTheme event added

Version 1.8.0 (during development 1.7.2-SNAPSHOT) (October 2012)

Framework

- CHANGE: added jQuery-1.8.1, but the default version still is 1.7.1
- CHANGE: removed older versions of jQuery (1.4-1.6). They are no longer provided by SAPUI5 and SAPUI5 doesn't run on top of them. Minimal required version is 1.7.
- CHANGE: Updated QUnit to v1.10.0
- NEW: `ODataModel`: new `refreshSecurityToken` function to allow to fetch a new token by performing a GET request against the service root URL. In case of an invalid token error a new request will be performed automatically to fetch a new token.
- NEW: `ODataModel`: Batch functionality: new `createBatchOperation` function create single batch operations which can be added to the batch via `addBatchChangeOperations` and `addBatchReadOperations`. The batch request can be triggered via the `submitBatch` function. The data in the batch regardless if GET or Change requests won't be stored in the model. This batch function works standalone. If data was changed which is also bound and included in the model the data can be refreshed with the `refresh` function. Note: Currently this feature is **experimental**.
- CHANGE: `jQuery.sap.properties`:
  - NEW: Possibility to specify a map of additional header key/value pairs to send along with the request (see headers option of jQuery.ajax).
  - CHANGE: `jQuery.sap.util.Properties`: Function `getProperty` returns `null` if the given key is not defined in the properties file and no default value is given. It return an empty string if the key is available but no value is defined (e.g. "key=").
- CHANGE: `jQuery.sap.resources`:
  - CHANGE: `jQuery.sap.util.ResourceBundle`: Function `getText` only returns the given key when the key is not defined in the bundle at all. Otherwise the defined text in the bundle (also an empty string) is returned.
  - CHANGE: Changed support for HANA ressource bundles:
    - Supported file extension changed from `hdbtextproperties` to `hdbtextbundle`
    - Desired language code not longer part of the file name but `Accept-Language` header of the request.
- NEW: event bus feature
  - NEW: class `sap.ui.core.EventBus`
  - NEW: Function `getEventBus` of class `sap.ui.core.Core` to get the singleton instance of the event bus class.
- CHANGE: all controls now support "toggleStyleClass" in addition to "addStyleClass" and "removeStyleClass".

Desktop Controls

- NEW: control library `sap.viz` containing SVG based charts (using D3 library) for desktop applications. Requires a browser with full SVG support (not IE8, not FF ESR!)
- DEPRECATION: the "showInspectorTool" property of the `sap.ui.ux3.Shell` has been deprecated, the default is set to "false", and if set to "true", the Inspector Tool will still be not displayed. This Inspector Tool anyway only opened a "work in progress" popup and could not be put to any proper use. ThingInspectors are supposed to be launched from the respective element in the content area, not from the Tool Pane.
- NEW: **experimental** Control `sap.ui.ux3.ThingViewer`
- NEW: control `sap.ui.commons.RangeSlider`
- NEW: control `sap.ui.commons.Carousel`
- NEW: control `sap.ui.commons.InPlaceEdit`
- NEW: control `sap.ui.commons.TriStateCheckBox`
- CHANGE: control `sap.ui.richtexteditor.RichTextEditor`: The RichTextEditor of SAPUI5 contains a third party component TinyMCE provided by Moxiecode Systems AB. The SAP license agreement does not cover development of own applications with RichTextEditor of SAPUI5. To develop own

applications with RichTextEditor of SAPUI5 a customer/partner has to first obtain an appropriate license from Moxiecode Systems AB. To prevent accidental usage, the TinyMCE code cannot be used directly starting with SAPUI5 version 1.8.

Mobile Controls

- NEW: control library `sap.m` containing 30 new controls for mobile devices. The full list of controls is: `App, Bar, BusyDialog, BusyIndicator, Button, Carousel, CheckBox, CustomListItem, Dialog, DisplayListItem, FlexBox, HBox, Image, Input, InputListItem, Label, List, NavContainer, Page, Popover, RadioButton, ScrollContainer, SearchField, SegmentedButton, Select, Slider, StandardListItem, Switch, Text, VBox`
- NEW: control library `sap.makit` containing basic charts for mobile devices

Tools

- FEATURE: The SAPUI5 Application Tools are enhanced so that when creating a SAPUI5 Application Project the user can choose between Target Device 'Desktop' (default) and 'Mobile'. If Mobile was chosen and creating an initial view was selected, special coding instantiating sap.m.App and sap.m.Page is generated.

Experimental Features:

Experimental features are not part of the officially delivered scope that SAP is going to guarantee for future releases – means experimental Features may be changed by SAP at any time for any reason without notice. The **Experimental features** are **"NOT FOR PRODUCTION USE"**. You may not demonstrate, test, examine, evaluate or otherwise use the Experimental Features in a live operating environment or with data that has not been sufficiently backed up.

The purpose of Experimental features is to get feedback at an early point of time allowing customers/partners to influence the future product accordingly. Please use the SCN Developer Center http://scn.sap.com/community/developer-center/front-end to provide feedback accepting that Intellectual Property rights of the contributions or derivative works shall remain the exclusive property of SAP.

For general information about **Experimental features**, please check the Compatibility Rules, for a detailed list check out the list below:

- `Mobile Tablet Support:` The controls of the UI libraries sap.ui.core, sap.ui.commons, sap.ui.ux3 and sap.ui.table are not yet optimized or adapted for mobile tablet usage.
- `OData Write Support:` basic modificator functions for OData and first experiments for two way binding including batch functionality.
- `ODataModel`: Support of `batch` (see above)
- `Module sap.ui.core.plugin.LessSupport:` Current status is prototype.
- `Module sap.ui.core.delegate.ScrollEnablement:` Current status is prototype.
- `Configuration Parameters` with "-xx-" and the corresponding features: e.g. sap-ui-xx-test-mobile
- `Weinre Server Configuration:` Might change or removed in future releases
- `Control sap.ui.table.Table:` Current status for Threshold and Column Grouping is prototype.
- `Deprecated Message Controls in sap.ui.commons:` MessageBar, Message, MessageList, MessageToast - a new messaging concept is planned which will replace these controls.
- `Control sap.ui.core.HTML:` Support of HTML with multiple root nodes
- `Control sap.uiext.inbox.Inbox:` API is not yet finished and might change
- `Control sap.ui.suite.TaskCircle:` API is not yet finished and might change
- `Control sap.ui.suite.VerticalProgressIndicator:` API is not yet finished and might change
- `Control sap.ui.ux3.ThingViewer:` API is not yet finished and might change
- `Control sap.ui.ux3.Shell:` Personalization, Color Picker (`Control sap.ui.ux3.ShellColorPicker`) and "Inspect"-Tool
- `Controls / Types sap.ui.ux3.Feed*:` Open discusses might result in API changes and adaptation. Especially the text presentation (e.g. @-references and formatted text) is not yet clear. Also Feed Model topic still open.
- `Control sap.ui.ux3.Exact:` Open discussions might result in API changes and adaptation. Especially the Result Area is not final.
- `Control sap.ui.ux3.ExactArea:` Open discussions might result in API changes and adaptation.

- `Type sap.ui.ux3.ActionBarSocialActions:` Open discussions might result in API changes and adaptation.
- `Theme Editor:` This application allows for easy branding of SAPUI5 applications. Currently optimized for the use with Chrome and Firefox.
- Improved performance for touch devices: Mouse events are now fired on their corresponding touch events
- Control `sap.makit.Chart:` API is not yet finished and might change
- Theme Parameters for `sap.m` and `sap.makit` are not yet finished and might change
- Controls `sap.viz.*:` API of the new charting library is not yet finished and might change

Version 1.6.4 (September 2012)

A patch for the 1.6 code line. It contains the following fixes for the UI5 Core and Controls:

- fix dependencies in MenuButton, all-in-one
- Toolbar: focus on disabled button not visible
- DropdownBox: some special characters can be entered
- Table: Header should be truncated gracefully
- NumberFormat: int/float parsing accepts wrong chars
- Feeder: text copied from MS Word
- Table: Fixed the selection behavior.
- Feed: fix for default thumbnail not found
- RowRepeater: gotoPage crashes when the RR is hidden
- NavigationBar: Add aria-checked
- Core: Change event of the SelectionModel was fired even on no change!
- Table: fixed the selection behavior.
- Shell: Handle multiple Overlays
- Fix update bindings
- Callout/Quickview: fix corrupted tip
- ComboBox/DropdownBox: allow set key/value before update items
- Feeder: Copy and Paste from MS Word
- DatePicker: fix to use legacy date format if set
- Panel: Provide translatable tooltip texts
- TextArea: fix call of TextFields focus handling
- SimpleProxyServlet: Proxy content of arbitrary responses not only 200.
- datajs: fix Edm.Time with null values
- ODataModel: fix XSRF token refresh
- DropdownBox: fix model update brings error
- Add latest Core lib translations
- Fix Tab Chain in Shell/TI Scenario
- Table: fixed the re-rendering on row selection
- Table: prevent Table to grab focus on scrollbar click
- Shell: Fix paneWidth
- Table: fixed the rows announcement
- DatePicker: fix value if entered twice
- Model: override getInterface method
- Table: Grouping Labels are not rendered.
- Table: ACC improvements for rows/cols and row hdr.
- Button: Fix Focus on iPad
- ComboBox: cloning of bound "items" aggregation fails

Tools

- ENHANCEMENT: When submitting the deletion of a file with the SAPUI5 ABAP Repository Provider it is checked that the deletion can be performed and transported consistently, to avoid repository inconsistencies.
- FIX: When starting the SAPUI5 View Wizard in a workspace in which no SAPUI5 Application project is available, there is no exception anymore but a proper error message is displayed mentioning that first a SAPUI5 Application Project needs to be created.

Version 1.7.1 (September 2012)

Framework

- The deprecated class `sap.ui.util.Rect` has been deleted.
- `Numberformat`: Type validation for `Integer` and `Float` values works more restrictive. The format is aligned with the EDM type formats. Example for float: "1.3A" won't be parsed to "1.3" anymore, but will result in a ValidationException. Example for integer: "1.3" won't be parsed to "1" anymore, but will result in a ValidationException.
- `ODataModel`: additional `sap.ui.model.odata.Filter` enables to AND or OR filters for the same property.
- A new `Application Cache Buster` mechanism is introduced to allow caching for application files (like views) like in the runtimes `Cache Buster` mechanism

Controls

- INCOMPATIBLE CHANGE: ThingInspector: ThingAction IDs not unique. The IDs of the ThingAction instances that came as `action` parameter with the `actionSelected` event are now prefixed with the ThingInspector ID. This is necessary to deal with multiple ThingInspector instances.
- CHANGE: `sap.ui.commons.Slider` control has a new property `labels` to allow own labels instead of the numbers.
- CHANGE: `sap.ui.commons.Slider` control has a new property `vertical` and `height` to display the slider in vertical direction.
- CHANGE: `sap.ui.ux3.SHell`: Integration of control `sap.ui.ux3.NotificationBar` completed including collapse/expand feature.
- NEW: `sap.ui.ux3.NotificationBar` not longer flagged as experimental
- CHANGE: Control `sap.ui.ux3.ExactBrowser`:
  - New API to set the order of the lists (see JSDoc of ExactBrowser and ExactAttribute
  - Fixed keyboard support.

Tools

- FEATURE: The SAPUI5 Application Handler for SAPUI5 Repository on ABAP side implements the new `Application Cache Buster` mechanism which is introduced to allow caching for application files (like views) like in the runtimes `Cache Buster` mechanism.
- FEATURE: When sharing a project or submiting files the SAPUI5 ABAP Repository Provider checks whether the SAPUI5 runtime version on the server matches the one which is installed in Eclipse and used for code completion and application preview. If not a warning is raised.

Version 1.6.3 (September 2012)

A patch for the 1.6 code line. It contains the following fixes for the UI5 Core and Controls:

- FileUploader: reupload the same file again
- Fixed scroll bar on first mouse click
- Table - column header not rendered correctly
- Shell: Fix pane width
- Menu: Close Menu on TAB like on Escape
- Fix for Scrollbar in Webkit RTL mode.
- SearchField: Allow binding for attributes, esp. value
- Panel: Fix title length issue in IE
- Table - Fix for memleak in column for indicators.
- General: fix more occurrences of the 'ID with dot' issue
- Splitter: Ghost Overlay Fix when "." in ID.
- ComboBox/DropDown doesn't open with a . in the id.
- Callout/QuickView: Fix close issue
- Table, Accessibility: relationship between table cell and colHdr was missing
- Table: setSelectedIndex() does not select a row
- Fixed the scrollbar behavior for Webkit RTL.

QuickView.open() results in Table invalidate
- Theming: prameters API should not fail if a UI library lacks the json file
- JSON/XML Model: change default caching behaviour. No timestamp added anymore. Caching (default) now depends on browser or server configuration.
- DropdownBox: no Change event on typeAhead
- Table: support two way binding for property 'selectedIndex'
- FileUploader: Fix for Chrome with Mac OS.
- SegmentedButton: Keyboard navigation issue when used in Toolbar
- SearchField+DropdownBox: do not store history in localStorage
- SegmentedButton: fix clone
- DropdownBox: fix ARIA on keyboard navigation
- General: Fixes for Chrome21
- Configuration: locale fixes
- Callout/QuickView: corrections
- Configuration: Support of SAP Portal URL parameters sap-locale sap-accessibility sap-rtl sap-theme
- Button,MenuButton: Fix focus problem
- Table: added support for RowOnly selection behavior (no row selector)
- Column Chart: add missing 'experimental' flag
- I18N: Added new resource bundles from B0Y.

Version 1.7.0 (August 2012)

Framework

- FEATURE: Now the SAP portal URL parameters sap-locale, sap-accessibility, sap-rtl and sap-theme are supported. If the similar sap-ui-* parameter are used in addition, they will be used.
- FEATURE: `sap.ui.core.plugin.DeclarativeSupport` and `sap.ui.core.DeclarativeSupport`:
  - Declarative support is not experimental anymore
  - See deprecation warnings for future changes
- FEATURE: `sap.ui.model.odata.ODataModel` support for `$select` parameter.
- FEATURE: `sap.ui.model.odata.ODataModel` filter URL creation: now reads the metadata for the Edm type for the filter property and then creates the filter URL according to this type: e.g. `Edm.String` will result in the URL as e.g. `?$filter=CustNo eq '112233'` whereas `Edm.Int32` will simply result in the URL as `?$filter=CustNo eq 112233`.

Controls

- NEW: Control `sap.ui.ux3.NotificationBar`:
  - New controls NotificationBar and Notifier and new element Message introduced.
  - `sap.ui.ux3.NotificationBar` included into `sap.ui.ux3.Shell` (see JSDoc). **Limitation:** The change height feature of the NotificationBar is not yet available within the Shell.
- CHANGE: Control `sap.ui.ux3.ExactBrowser`:
  - Change collapse behavior of the lists.
  - Improved open animation for inital list rendering
  - New API to set the widths of the lists (see JSDoc of ExactBrowser and ExactAttribute)
  - ARIA Support
  - New API to hide the first list (see JSDoc)
  - **Limitation:** Due to the changes above the keyboard support does not work completely. This will be corrected with version 1.7.1.
- CHANGE: sap.ui.util.Rect was deprecated and will be removed. Use `jQuery(...).rectContains(x,y)` provided by jQuery.sap.dom.js to determine whether a point is contained in the rectangle defined by a DOM element.

Tools

- FEATURE: Tools also run on Eclipse 3.5, i.e. with NWDS.

Experimental Features

- Includes the full set of experimental features as documented for 1.6.x, with the exception of

DeclarativeSupport which is a productive feature now, see above
- new experimental feature: Control sap.uiext.inbox.SubstitutionRulesManager: API is not yet finished and might change

Tools Version 1.6.2 (August 2012)

Tools

- Minor bugfixes

Runtime Version 1.6.1 (July 2012)

Controls

- FIX: Shell: Labels and Texts not selectable
- FIX: Security Fix for XSS prevention.
- FIX: FacetFilter: Fix declare in Renderer
- FIX: Fix the columnHeaderHeight property of the Table
- FIX: Grouping not working in IE RTL mode.
- FIX: Fix for Paginator focus issues in IE8
- FIX: Factory function doesn't work as documented
- FIX: Due Date Filter not available Added support from TaskInitial Filter.
- FIX: Due Date Filter Not Available Added Due Date Filter capability in Inbox.
- FIX: QUnit tests are failing for BPMInbox. Qunits falling due to issue in bindTaskTable.
- FIX: Some dropDown filters not working in Inbox

Tools

- FEATURE: The `SimpleProxyServlet` (used for local testing) is now using the standard Java proxy settings (`http.proxyHost`, `http.proxyPort`, `http.noneProxyHosts`, `https.proxyHost`, `https.proxyPort`, `https.noneProxyHosts`) instead of special servlet parameters. This information by default comes from the Eclipse proxy settings. The base remote location is configured via the servlet context parameter `com.sap.ui5.proxy.REMOTE_LOCATION`.

Runtime Version 1.6.0 (during development 1.5.3-SNAPSHOT) (July 2012)

Framework

- CHANGE: One common package `sap/ui/thirdparty` (incl. sub packages) within the Core library which contains all open source third party libraries:
  - The following files were moved from their origin location without replacement: `iscroll.js`, `iscroll-lite.js`, `zyngascroll.js`, `qunit.css`, `jquery-mobile-custom.js`
  - The following files were moved from their origin location, but a stub for loading the new file was added at the origin location:
    - `datajs.js`, `qunit.js`, `jquery-ui-core.js`, `jquery-ui-position.js`, `jquery-ui-datepicker.js`
    - The stub uses the `jQuery.sap.require` functionality and therefore only works when this functionality is available.
    - Using the stub as fallback causes additional requests and is therefore only intended for an intermediate solution. Applications should adapt their dependencies as soon as possible.
  - The following files were newly added: `d3.js`, `jszip.js`, `less.js`, `caja-html-sanitizer.js`
  - jQuery UI was upadted to version 1.8.21 and addional jQuery UI plugins were added: `jquery-ui-widget.js`, `jquery-ui-mouse.js`, `jquery-ui-draggable.js`, `jquery-ui-droppable.js`, `jquery-ui-resizable.js`, `jquery-ui-selectable.js`, `jquery-ui-sortable.js`, `jquery-effects-*.js`
  - The available jQuery versions are kept at the origin location but a copy is added to the new

package. The Core uses by default the new location. Future versions of jQuery will only be available at the new location.
    - Please notice also the corresponding section in the Compatibility Rules.
- CHANGE: The theming/styling of controls has been changed to use LESS. Therefore nearly all color parameters had changed. So if custom styling uses the color parameters they must be changed to the new oned to still work.

- NEW: `sap.ui.model.json.JSONModel`, `sap.ui.model.xml.XMLModel`, `sap.ui.model.odata.ODataModel` introduce a new method `forceNoCache()`. If set to true it will force requested pages not to be read from the browser cache. For XMLModel and JSONModel the default is set to true, for the ODataModel to false.
- NEW: `sap.ui.model.json.JSONTreeBinding` Support for nested arrays.
- CHANGE: `sap.ui.model.odata.ODataModel.submitChanges(...)` Does a MERGE request instead of PUT. No deep update possible, therefore associated entries are removed if the data was retrieved via expand.
- CHANGE: `sap.ui.model.odata.ODataModel.update(...)` New option to do a MERGE request instead of a PUT request.
- CHANGE: `sap.ui.model.odata.ODataModel.getProperty(...)` New Flag `bIncludeExpandEntries`. If the data was retrieved with expand, the data will be returned as deep structure if `bIncludeExpandEntries` is set to true. If `bIncludeExpandEntries` is set to false the referenced data will not be included.
- DEPRECATION: `sap.ui.model.odata.ODataModel.getData(...)` was deprecated. Use `sap.ui.model.odata.ODataModel.getProperty(...)` instead

Controls

- NEW: Various header types for control `sap.ui.ux3.Shell`: Aggregation headerType of type ShellHeaderType
- NEW: Options to hide tool area and side panel in control `sap.ui.ux3.Shell`: Properties showTools and showPane
- CHANGE: Adapations of GoldRelection theme for `sap.ui.ux3.Shell` and `sap.ui.ux3.NavigationBar`
- NEW: Base class `sap.ui.core.search.SearchProvider` for search providers introduced.
- DEPRECATION: `sap.ui.commons.SearchProvider` was deprecated. Instead `sap.ui.core.search.OpenSearchProvider` (OpenSearchProvider API) should be used.
- NEW: New property enableFilterMode on control `sap.ui.commons.SearchField`: When this property is set the search event is also fired when the SearchField is empty or the Clear icon is pressed.
- NEW: Added the UI library `sap.ui.richtexteditor` containing the `RichTextEditor` control to provide a proper control to enter formatted text.
    - The RichTextEditor of SAPUI5 version 1.6 contains a third party component TinyMCE provided by Moxiecode Systems AB. The SAP license agreement does not cover development of own applications with RichTextEditor of SAPUI5 version 1.6. To develop own applications with RichTextEditor of SAPUI5 version 1.6 customer/partner has to first obtain an appropriate license from Moxiecode Systems AB.
    - Please be aware that the RichTextEditor control of SAPUI5 is going to be changed with the next SAPUI5 release 1.8: the reason is the embedded third party component TinyMCE from Moxiecode Systems AB, which potentially can be used by customers/partners without being aware of the specific license terms. To prevent this accidental usage, the TinyMCE code is going to be removed. More instructions how to manually add TinyMCE code will follow in the release notes of the SAPUI5 release 1.8.
- NEW: Popup pplacement in `sap.ui.commons.CalloutBase`: a callout control can be now positioned to the left or right of the parent control, in addition. A new method setPosition is added to simplify positioning. This functionality is available for `sap.ui.commons.Callout` and `sap.ui.ux3.QuickView` controls.
- NEW: `sap.ui.table.ColumnMenu` Added new control
- NEW: `sap.ui.table.ColumnMenu` Added visibility submenu
- NEW: `sap.ui.table.Table` Added new event "columnVisibility"
- NEW: `sap.ui.table.Table` Enabled zooming on touch devices
- NEW: `sap.ui.table.Table` Added new aggregation "menu"

Tools

- NEW: SAPUI5 Tools in Eclipse are new with release 1.6.0 and offer
- SAPUI5 Application Development Tool to create applications. It comes with
- wizards to create the application project
- and its views (with a JavaScript, XML or JSON layout)
- JavaScript coding completion and code templates/snippets and a
- build-in application preview.
- There is a Team Provider functionality to share/retrieve/submit application projects with an ABAP system (NetWeaver UI Extensions AddOn for NetWeaver 7.31) to support development within a team and handle conflicts.

Experimental Features:

Experimental features are not part of the officially delivered scope that SAP is going to guarantee for future releases – means experimental Features may be changed by SAP at any time for any reason without notice. The **Experimental features** are **"NOT FOR PRODUCTION USE"**. You may not demonstrate, test, examine, evaluate or otherwise use the Experimental Features in a live operating environment or with data that has not been sufficiently backed up.

The purpose of Experimental features is to get feedback at an early point of time allowing customers/partners to influence the future product accordingly. Please use the SCN Developer Center http://scn.sap.com/community/developer-center/front-end to provide feedback accepting that Intellectual Property rights of the contributions or derivative works shall remain the exclusive property of SAP.

For general information about **Experimental features**, please check the Compatibility Rules, for a detailed list check out the list below:

- `Mobile Tablet Support:` The controls of the UI libraries sap.ui.core, sap.ui.commons, sap.ui.ux3 and sap.ui.table are not yet optimized or adapted for mobile tablet usage.
- `OData Write Support:` basic modificator functions for OData and first experiments for two way binding
- `Module sap.ui.core.plugin.DeclarativeSupport:` Current status is prototype.
- `Module sap.ui.core.plugin.LessSupport:` Current status is prototype.
- `Module sap.ui.core.delegate.ScrollEnablement:` Current status is prototype.
- `Configuration Parameters` with "-xx-" and the corresponding features: e.g. sap-ui-xx-test-mobile
- `Weinre Server Configuration:` Might change or removed in future releases
- `Control sap.ui.table.Table:` Current status for Threshold and Column Grouping is prototype.
- `Deprecated Message Controls in sap.ui.commons:` MessageBar, Message, MessageList, MessageToast - a new messaging concept is planned which will replace these controls.
- `Control sap.ui.core.HTML:` Support of HTML with multiple root nodes
- `Control sap.uiext.inbox.Inbox:` API is not yet finished and might change
- `Control sap.ui.suite.TaskCircle:` API is not yet finished and might change
- `Control sap.ui.suite.VerticalProgressIndicator:` API is not yet finished and might change
- `Control sap.ui.ux3.Shell:` Personalization, Color Picker (`Control sap.ui.ux3.ShellColorPicker`) and "Inspect"-Tool
- `Controls / Types sap.ui.ux3.Feed*:` Open discusses might result in API changes and adaptation. Especially the text presentation (e.g. @-references and formatted text) is not yet clear. Also Feed Model topic still open.
- `Control sap.ui.ux3.Exact:` Open discussions might result in API changes and adaptation. Especially the Result Area is not final.
- `Control sap.ui.ux3.ExactArea:` Open discussions might result in API changes and adaptation.
- `Type sap.ui.ux3.ActionBarSocialActions:` Open discussions might result in API changes and adaptation.
- `Theme Editor:` This application allows for easy branding of SAPUI5 applications. Currently optimized for the use with Chrome and Firefox.
- Improved performance for touch devices: Mouse events are now fired on their corresponding touch events

Runtime Version 1.5.2 (June 2012)

Framework

- CHANGE: The `BeforeRendering` Event (optional function `onBeforeRendering` in controls) is now called before **every** (re-)rendering of the control. This affects also the corresponding function in view controllers.

Controls

- CHANGE: `sap.ui.commons.DatePicker` on mobile devices now a native DatePicker is used. Therefore some functionlaity is not available there:
  - it is not possible to open the DatePicker via F4. Only the native events open it
  - The date pattern is used from the device. No pattern set by the application is used. But on the value property the date is provided with the pattern of the defined locale or set by the binding.
  - Not all mobile devices already support the native DatePicker, on iPad it works. (Here input type=date is used, so the device/browser must support this.) Here we should think about feature detection...
  - no direct input of the date possible, only via DatePicker.
  - no copy & paste possible
- NEW: `sap.ui.commons.ColorPicker` control has been created
- NEW: `sap.ui.commons.SegmentedButton` control has been created
- CHANGE: `sap.ui.ux3.DataSet` Toolbar and FilterArea added. View switches are implemented with new SegmentedButton
  - there is no build in sorting button anymore. Sorting can be handled via own functions in the new toolbar.
  - there is no build in filter support anymore. Filtering will be handled e.g. by adding a FacetFilter into the new filter area.
- CHANGE: `sap.ui.commons.layout.BorderLayout` has been refactored.
  - The specific RTL support of the BorderLayout has been dropped in favor of the standard RTL handling of UI5. Applications that have set the RTL property matching the global RTL configuration don't need to change anything. Only applications that have set the RTL property to a value different from the central configuration might run into issues. UI5 strongly suggests to rely on the central configuration. The value of the RTL property is ignored.
  - the animated hide/show of areas is no longer a supported feature. It is still available but applications should consider to remove it or to animate the area on their own.
- CHANGE: `sap.ui.core.ScrollBar` Added touch support for mobile devices

Runtime Version 1.4.3 (2012-06-2?)

A patch for the 1.4 code line. It contains the following fixes:

- FIX Demokit: navigation and history fixed, spelling corrections
- FIX ODataModel: fix init problem for 1.4
- FIX ODataModel: fix refresh data after write
- FIX Databinding: setSizeLimit for JSON- and XMLModel

Runtime Version 1.5.1 (May 2012)

Framework

- CHANGE: the 'preload' feature that loads all JavaScript modules for a library in one request has been made a non-experimental feature and it is activate by default. See Bootstrap documentation for details.
- CHANGE: Databinding syntax changed, so that absolute bindings now must have a leading slash. For compatibility reasons, method setLegacySyntax() can be used to enable the old binding syntax.
- CHANGE: `sap.ui.core.Element` renamed method bindContext to bindElement for better understanding

CHANGE: `sap.ui.core.Element` added support for model specific parameters in bindProperty, bindAggregation and bindElement
- CHANGE: `sap.ui.core.Element` extended signature of bindProperty and bindAggregation to accept object literals for binding information
- NEW: `sap.ui.model.odata.ODataModel` added support for $expand in createBindingContext and bindList
- NEW: `sap.ui.core.Element` added support for factory method instead of template control in bindAggregation

Controls

- CHANGE: Some properties of the charting controls (`sap.service.visualization.*`) have been renamed for clearer APIs, please check the API Reference. Most important changes are: `measuresAsObjects` has been renamed to `tabularData`, all `''Some''NumberFormat` properties have been renamed to `''Some''FormatString`.
- CHANGE: `sap.ui.commons.DatePicker` add keyboard navigation for "+" and "-" keys on input field
- CHANGE: `sap.ui.commons.DatePicker` use CLDR texts and locale information to localize DatePicker. This might change some texts and some default patterns!
- CHANGE: `sap.ui.commons.DatePicker` if bould to a model using the date type, the format pattern defined in the binding is used
- Change: `sap.ui.commons.ComboBox`/`sap.ui.commons.DropdownBox` on mobile devices now a native ComboBox is used. Therefore some functionlaity is not available there:
  - it is not possible to open the dropdown list via F4. Only the native events open it
  - there is no hover or active effect on the expander button
  - there is no sapui5 specific typeahead function in the open list, only a native one if available on the device
  - if there is a value entered in the ComboBox that is not in the list this is added as entry to the list (and removed if changed/deleted).
  - by opening the list, the input field gets a short moment grey on the iPad
  - on the DropdownBox there is no copy&paste possible
  - the search help functionality of the DropdownBox is not available for mobile devices
  - the history function of the DropdownBox is not available on mobile devices
- Change: The *showListExpander* feature of `sap.ui.commons.SearchField` is not available on mobile devices.
- Deprecation: The `sap.ui.table.DataTable` control is deprecated and should ve replaced by the new table controls (see below).
- NEW: `sap.ui.table.Table` control
- NEW: `sap.ui.table.TreeTable` control
- NEW: Menu item `sap.ui.commons.MenuTextFieldItem`
- NEW: `sap.ui.commons.Callout` control
- NEW: `sap.ui.ux3.QuickView` control

Runtime Version 1.5.0 (April 2012)

Framework

- NEW: API for performance measurements: jQuery.sap.measure
- NEW: API for mobile device detection and page initialization: "jQuery.sap.mobile", defining jQuery.device.is, jQuery.device.os and extending jQuery.support.

Controls

- NEW: control `sap.ui.ux3.FacetFilter` has been created.
- NEW: a first version of the CVOM charting library has been added. It contains controls for Bar chart, Line chart, Pie chart and Combination chart. Code samples can be found in the testsuite.

Runtime Version 1.4.2 (2012-05-16)

A patch for the 1.4 code line. It contains the following fixes:

- FIX Logging: allow log level configuration via URL parameter
- FIX Fix preserve DOM
- FIX Demokit: remove unnecessary content
- FIX ApplicationHeader: fix support for custom style classes
- FIX ODataModel: Make collection size limit configurable
- FIX Demokit: fix navigation on NetWeaver Java
- FIX Configurator WebUI: fix issue with the generation of boostrap files (containing Core)
- FIX All-In-One files: fix issue with module order
- FIX DropdownBox: Backspace creates invalid entry
- FIX Scrollbar: fix for event propagation
- FIX ExactList: Fix Positioning of expanded lists in RTL
- FIX ExactList: Optimize performance
- FIX Table: Bugfix for typos exit function of Table control
- FIX EPM Demo: adaptations to modified oData Fields

Runtime Version 1.4.1 (2012-04-13)

A patch for the 1.4 code line. It contains the following fixes:

- FIX: ODataModel: Renamed getMetadata to getServiceMetadata, getMetadata is reserved for getting the object metadata (see sap.ui.core.Metadata)
- FIX: Control / Element: make deferred placeAt() robust against premature destroy()
- FIX: DropdownBox: fixes on history feature
  - add maxHistoryItems property to allow to disable history
  - fix on typeAhead, show all valid history items up to max number
  - refresh default value if listbox (items) are changed
- FIX ODataModel: Wrong date format in OData filter parameter
- FIX ThingInspector: Avoid assertions for internal aggregations

Runtime Version 1.4.0 (during development 1.3.3-SNAPSHOT) (April 2012)

Framework

- NEW: SAPUI5 Support Tool available (Technical Info, Console Log and Control Tree + Properties List) - the shortcut to open the tool is: `CTRL+SHIFT+ALT+S`
- NEW: in addition to the locale specific formatting, applications can define their own default formattings for date, time and numeric data. For details check the new FormatSettings API
- CHANGE: Function `jQuery.sap.log.setLogListener` was removed. Instead the 2 new functions `jQuery.sap.log.addLogListener` and `jQuery.sap.log.removeLogListener` are added.
- CHANGE: The proxy servlet `com.sap.ui5.proxy.SimpleProxyServlet` has been restricted for local usage only.
- DELETION: The proxy servlet `com.sap.ui5.proxy.ProxyServlet` is removed from distribution.
- DELETION: The deprecated context parameters of the Resource Servlet (`com.sap.phx.*`) have been removed.

Controls

- NEW: Property `title` added to `sap.ui.ux3.Feed` control. Title of the feed can be changed now.
- DELETION: The old feed controls `sap.ui.ux3.feed.BaseFeedEntry`, `sap.ui.ux3.feed.CommentEntry`, `sap.ui.ux3.feed.FeedComponent`, `sap.ui.ux3.feed.FeedEntry` and `sap.ui.ux3.feed.Feeder`. Use `sap.ui.ux3.Feed`, `sap.ui.ux3.FeedChunk` and `sap.ui.ux3.Feeder` instead.
- DELETION: The deprecated properties `accessibleDescription` and `accessibleName` are deleted from the `sap.ui.commons.TextField` control.
- DELETION: The deprecated method `setContent` was deleted from the `sap.ui.commons.layout.MatrixLayoutCell` control.
- DEPRECATED: The controls `sap.ui.commons.MessageToast`, `sap.ui.commons.MessageList`, `sap.ui.commons.MessageBar` and `sap.ui.commons.Message` are deprecated. A new messaging concept will be created in future. Therefore these controls might be removed in one of the next

versions.
- CHANGE: The Fontsize and color for header texts in `sap.ui.commons.TextView` control are adopted to the goldreflection specification.

Experimental Features:

- `Mobile Tablet Support:` The controls of the UI libraries sap.ui.core, sap.ui.commons, sap.ui.ux3 and sap.ui.table are not yet optimized or adapted for mobile tablet usage.
- `Theming:` In future LESS might be introduced. Therefore adaptations in custom themes might be necessary.
- `OData Write Support:` basic modificator functions for OData and first experiments for two way binding
- `Module sap.ui.core.plugin.DeclarativeSupport:` Current status is prototype.
- `Control sap.ui.ux3.Exact:` Open discussions might result in API changes and adaptation. Especially the Result Area is not final.
- `Controls sap.ui.ux3.Feed*:` Open discusses might result in API changes and adaptation. Especially the text presentation (e.g. @-references and formatted text) is not yet clear. Also Feed Model topic still open.
- `Control sap.ui.core.HTML:` Support of HTML with multiple root nodes
- `Control sap.ui.ux3.Shell:` Personalization, Color Picker and "Inspect"-Tool
- `Control sap.ui.table.DataTable:` Plan is to deprecate it and to replace with sap.ui.table.Table. The old APIs should be kept and be routed to the new APIs - seamless migration.
- `Deprecated Message Controls in sap.ui.commons:` MessageBar, Message, MessageList, MessageToast - a new messaging concept is planned which will replace these controls.
- `Configuration Parameters` with "-xx-" and the corresponding features: e.g. sap-ui-xx-preload
- `Weinre Server Configuration:` Might change or removed in future releases

Runtime Version 1.3.2 / 1.3.1 (March 2012)

Note: Version 1.3.2 completely replaces 1.3.1, as 1.3.1 contains an outdated documentation.

Framework

- NEW: a new API for creating controls (or classes in general) has been introduced. See the documentation for details.
- CHANGE: the generation tools for controls have been adapted to the above mentioned new API. Due to this, the **build results of 1.3.2 tools definitly require a 1.3.2 runtime**. The same dependency between tools and runtime existed in older versions, but the impact was not that high. This time, any control build with 1.3.2 will fail on an older runtime!
- CHANGE: the whole concept for supporting optimized and unoptimized JavaScript sources in parallel has been revised (see documentation). It no longer requires server side support, but is handled completely on the client. If an application uses optimized sources and the debug mode is activated (either by specifying the URL parameter `sap-ui-debug=true`, by executing `jQuery.sap.debug(true)` in the console or address field or by opening the technical info with CTRL-SHIFT-ALT-P and using the corresponding check box), then the page will use debug sources from that point in time on.
- CHANGE: the configuration option `debug` has completely changed its meaning. In former releases, it activated the display of a control tree and a list of properties (as in the test suite). Starting from 1.3.2 on, the `debug` option instead toggles between optimized and debug sources. The old behavior still is supported, but with the new `inspect` option. See the configuration documentation for a complete list of options.

Controls

- CHANGE: Feed:
  - The property `commentChunk` is deprecated because its not longer used. A FeedChunk will be automatically an comment if it's a child of a FeedChunk.
  - The properties `feederThumbnailSrc` and `feederSender` of the `FeedChunk` control are now optional if the feedChunk is part of a feed control. In this case the value it taken from the parent. So the properties must only be set once at the feed control

    The order of the comments has changed like defined in the UI specification. A new comment is now added at the end.
  - Now it is possible to show only the recent comments if all comments are shown before.
- CHANGE: the "`worksetItemSelected`" event of the `sap.ui.ux3.Shell` control is now fired **before** the workset item selection has changed. Therefore in the event handler `Shell.getSelectedWorksetItem()` now returns the *previous* selection! Instead, `oEvent.getParameter("selected...")` needs to be used to get the respective information about the target workset. This change was required to enable "preventDefault" functionality on this event. So the application can actually cancel /disable the switch to another workset item now.

Runtime Version 1.3.0 (February 2012)

Framework

- NEW: method `Element.data(...)` has been introduced to attach arbitrary data to UI5 controls. In the background this is an aggregation to `sap.ui.core.CustomData` which can also be used in Views and with data binding. See the respective documentation.
- NEW: Included locale support for the major languages and `sap.ui.core.Locale` to access locale specific data like date formats, number formats, currencies etc. The simple types like `Date`, `Time` `Integer` etc. have been extended to support the formatting and parsing of these locale data. See here how to use it: Testpage and Documentation.
- NEW: new API to find all child elements / controls of an element
- CHANGE: the **jQuery version included in the UI5 core has been upgraded to 1.7.1** (from 1.4.4). You can still also use the UI5 core without jQuery (sap-ui-core-nojQuery.js) and additionally load your jQuery version of choice (1.4.4, 1.5.3, 1.6.4 and 1.7.1 are included as standalone files and supported).
- CHANGE: the included parts of jQuery UI (in particular the DatePicker) have been updated to version 1.8.17 (only a minor update)
- CHANGE: the support for databinding in the XMLView has been improved: binding of properties that have not a string type now works and list bindings can be used now
- CHANGE: the Java resource handler has been improved so that the cachebuster can be used with the CDN version or proxy configurations as well
- CHANGE: the handling of resource bundles has been changed so that empty or invalid language configuration doesn't break the application
- CHANGE: omit request to (often missing) central configuration file during bootstrap
- CHANGE: fix several assertions reg. model,

Controls

- NEW: control `sap.ui.commons.ValueHelpField` has been created.
- NEW: control `sap.ui.commons.ToggleButton` is ready to be used.
- NEW: ExactBrowser: support a reset via API
- CHANGE: DataTable: ARIA and keyboard support has been improved
- CHANGE: VerticalLayout: some style attributes have been moved to CSS classes to facilitate modifications by using custom style classes
- CHANGE: SearchField: Fix updated of suggestion list on BACKSPACE or DEL
- CHANGE: Some minor fixes and enhancements in Shell (rerendering of FacetBars, exception during destroy), MenuButton (redraw when modified while invisible), FileUploader (rendering without IFrame), ListBox (ID of items), DataTable (XSS fix reg. header)

Packaging & Documentation

- NEW: Core and all Controls: texts added in more languages
- CHANGE: Control Documentation revised
- CHANGE: to avoid misunderstandings reg. the contained functionality, the UI5 deliverable for static web servers has been renamed from sapui5-light to sapui5-static.

Runtime Version 1.2.0 (during development 1.1.2-SNAPSHOT) (January 2012)

Framework

- CHANGE: `XMLModel`: `XMLTreeBinding` didn't work when using hierarchical models with nested nodes with the same name. This has now been fixed. If your existing application doesn't work anymore (also this might be the case when using `XMLListBinding`), specify an absolute binding path because this behavior has been changed. Instead of `getElementsByTagName` which finds all nodes independently of its positions in the hierarchical tree we now use a `getChildElementsByTagName` internally to find only the nodes of a given parent node. So you have to specify the path to these parent node to get your application working again. —> e.g. model: <root><members><member> x</member></members></root>. Old binding was: `oCombo.bindItems("member", oItemTemplate1);`. This needs to be changed to: `oCombo2.bindItems("/members/member", oItemTemplate1);`.
- CHANGE: the deprecated methods `sap.ui.core.RenderManager.writeCustomStyleClasses()` and `sap.ui.core.RenderManager.getEscaped()` have been removed
- CHANGE: the logging APIs have been revised, logging is now configurable and the default behavior differs for debug sources (verbose) and optimized sources (only errors are logged). See [Logging API documentation].
- CHANGE: out of the box support for the long deprecated old view names JsView, JsonView and XmlView have been removed. Please use the long established new names of JSView, JSONView and XMLView instead (all upper case letters for the technology acronyms) or create aliases on your own (oldName = newName)
- CHANGE: For associations within declarative Views (JSON, XML), the framework so far did not correctly manage the Ids (e.g. labelFor). As a result, such associations pointed to non-existent or (even worse) to wrong controls. This has been fixed, all associated Ids are prefixed now with the Id of the view. The view does not try to determine whether an id is part of the view or not. Instead it is assumed that views can declare associations reliably only to controls that they contain. Associations to controls outside the view must be established at runtime in controller code.

Controls

- NEW: control `sap.ui.commons.ToggleButton` has been created BUT is not ready to be used yet (only GoldReflection theme is supported and there is no accessibility support), so it has been marked as deprecated. This status will be removed in the next drop (sprint result).
- NEW: Splitter now has a new property `splitterBarVisible` which lets you hide the splitter bar.
- NEW: The `sap.ui.ux3.DataSet` control has been created.
- CHANGE: The enumeration values of the `Orientation` type (used by the Splitter) were renamed to upper case. Please adapt to these changes. The old versions still work though.
  - `sap.ui.commons.Orientation.vertical` —> `sap.ui.commons.Orientation.Vertical`
  - `sap.ui.commons.Orientation.horizontal` —> `sap.ui.commons.Orientation.Horizontal`
- NEW: `TreeBinding` now supports filtering so all model implementations can now implement the tree filtering functionality. Tree filtering is implemented for the `JSONModel` and `XMLModel`.
- NEW: the `Button` control was enhanced with the following properties:
  - `styled`: if set to `false` the button is rendered without any CSS styles. So applications can design their own styles more easily.
  - `lite`: if this is active, the button is rendered in a lite style (only specified in goldreflection, in other themes it looks like the toolbar button).
  - `style`: A button can now have different styles like emphasized, accept or reject. For the possible styles a new type `sap.ui.commons/ButtonStyle` was created.
- CHANGE: the `Button` control style for Gold Reflection was adopted to implement the visual specification.
- CHANGE: `Label` control logic to determine ID of labelled HTML element of labelled control changed. A new function `getIdForLabel` introduced for all controls. By default the control ID is returned. But if the label should point to an inner HTML element it must be overwritten to return the right ID. `Slider` control and `ListBox` control adopted for this.
- CHANGE: `ListBox` control change of FocusDomRef to UL element.
- CHANGE: the long-deprecated method `sap.ui.commons.Dialog.setContent()` has been deleted. Use `addContent()` instead.
- CHANGE: the long-deprecated **Web Table** has been removed from the sap.ui.table library. It was never ment to be a productive control, so products must not use it. However, it is still available in

our sap.ui.dev library with the name sap.ui.dev.webtable.Table. (remember: sap.ui.dev must not be delivered outside SAP).
- FIX: The tooltip for the 'select all' button in a DataTable? remained active, even when the selection mode had hidden the button itself. This has been fixed.
- FIX: The ApplicationHeader? failed to cleanup all its parts which might have led to duplicateId errors when an ApplicationHeader? was created again with an older Id. This has been fixed.
- CHANGE: Rework of `ExactBrowser` control:
  - NEW: Header bar with reset functionality
  - NEW: Keyboard support
  - CHANGE: Display mode for lists was changed from a flat to a hierarchical visualization.
  - DELETION: The internally used control `ExactBrowseAttribute` was removed and replaced by the new (private) control `ExactList`.

Runtime Version 1.1.1 (December 2011)

- DEPRECATED: The feed controls in `sap.ui.ux3/feed` folder are deprecated because there is a new set of feed controls direct in the `sap.ui.ux3` root folder. In this takt these new controls have a base functionality and will be enhanced in future.
- NEW: Touch event support in mobile environments for control development ("on<EventName>" functions):
  - The basic touch events **touchstart**, **touchmove**, **touchend** and **touchcancel** are added to the SAPUI5 Core event list (only if touch is supported).
  - To to extend this touch event support parts of the jQuery Mobile 1.0 framework are included into the SAPUI5 Core (only if touch is supported). In detail the following jQuery Mobile plugins are included:
    - *jquery.mobile.media.js* + *jquery.mobile.support.js* to enrich the feature detection (`jQuery.support` object).
    - *jquery.mobile.vmouse.js* which provides basic event functionality to unify the browser and mobile event world.
    - *jquery.mobile.event.js* which mainly provides additional touch events, which are derived from the basic touchstart/touchmove/touchend events.
  - Therefore in addition the following jQuery Mobile "touch" events are added to the SAPUI5 Core event list (only if touch is supported): **tap**, **swipe**, **swipeleft**, **swiperight**, **scrollstart**, **scrollstop**
  - To better support RTL scenarios the SAPUI5 Core provides on top of these jQuery Mobile "touch" events (swipeleft / swiperight) the semantic events **swipebegin** and **swipeend**.
- NEW: Validation hooks where the application can attach on parse/format/validation error events when these occur upon user input. An application can call `attachParseError`, `attachValidationError`, `attachFormatError`, `attachValidationSuccess` on the core and register a callback function to receive these events and then extract the affected element, its property and error messages out of the event object to react correspondingly. Currently these events are only fired if a binding with a type exists for the element property which input should be verified.

Runtime Version 1.1.0 (November 2011)

Framework

- NEW: Data Binding: Added new Events `requestSent` and `requestCompleted` which are fired by model implementations when they sent requests to the server to load data. The `requestCompleted` event is fired also when the request failed. In this case a `requestFailed` event is additionally fired. You can attach/detach to these events by calling the corresponding `attach/detach` functions on the model: e.g. `oModel.attachRequestCompleted(this,function(oEvent){alert('request completed')}).`
- CHANGE: datajs 1.2 integrated. This is mostly a maintenance release, so nearly everything should work as before.
- CHANGE: jQuery.sap.XML plugin method: `jQuery.sap.parseXML()` behavior changed. Now always a parseError object is appended as property of the returned XML document. To check if a parse error really occurred check if the errorCode property of the parseError object is != 0. This change was necessary because IE8 always has a parseError object attached even if no parse error occurred. In

this case the error code is 0.

Controls

- NEW: The `sap.ui.mobile` library has been created, but is not meant for productive use yet! It is supposed to be the home for mobile-specific controls.
  - CHANGE: the `MobileApp` and `MobilePage` control from the dev library have been copied into this new library and renamed to `sap.ui.mobile.App` and `sap.ui.mobile.Page`. Both are NOT ready for use. The controls in the dev library have been deprecated.
- NEW: new event `liveChange` on `TextField`, `TextArea`, `ComboBox` and `DropdownBox` controls. This is fires on keyup and returns the current content.
- NEW: new method `getLiveValue` on `TextField`, `TextArea`, `ComboBox` and `DropdownBox` controls. This returns the current content of the field, e.g. during typing.
- CHANGE: The `sap.ui.table.Table` has been reworked (deprecation has been removed and is re-implemented). The API has been changed. It is an alternative to the `sap.ui.table.DataTable`.
- CHANGE: The initialization of the `DropdownBox` has changed. If no valid value is set the first item is used as value.
- CHANGE: The properties `accessibleDescription` and `accessibleName` of the `TextField` control are deprecated because they have no functionality.

Runtime Version 1.0.0 (AKA 0.20.0 - October 2011)

Framework

- CHANGE: SAPUI5 has changed its versioning scheme: 0.20.x has been renamed to 1.0.x. In Nexus you will find both versions, but only the 1.0.0 line might benefit from patches.
- CHANGE: jQuery.sap.getUriParams now properly decodes '+' encodes spaces in URL parameters
- NEW: Data binding: New binding mode: One Time (read only from the model once).
- NEW: Data binding: New method on `Element`: `hasModel`: To check if a model is set to the element or one of its parents (including the Core).
- NEW: Data binding: Introduced SimpleTypes: `Integer`, `Float`, `String`,`Boolean` which can be set to the bindProperty method e.g.: `oText.bindProperty("/clients/0/name",new sap.ui.model.type.String())`. Restrictions, Input parsing and output formatting is then supported: `new sap.ui.model.type.String(null,{ minLength : 3, maxLength : 10}).`
- NEW: Data binding: Multi model support: It is now possible to set an additional model to an element/core by specifying a name for the model: e.g. `element.setModel(oModel, "modelname");`. When creating property bindings to this model you have to specify its name: `oText.bindValue("modelname>/clients/0/name")`. Note: these "secondary models" currently only support property bindings and not aggregation bindings.

Controls

- CHANGE: the "key" parameter of the "worksetItemSelected" event of the `sap.ui.ux3.Shell` control is now null instead of an empty string if the triggered Item does not have a key. The "paneBarItemSelected" event now contains the same parameters.
- CHANGE: the CSS classes written by the BorderLayout have been changed to fit our naming scheme (only relevant when referred to in a selector in some custom CSS used by an application).
- CHANGE: the `sap.ui.commons.Dialog` does no longer automatically choose a default Button. You explicitly have to define one if you want a Dialog to forward any "Enter" key press inside the Dialog to a certain Button.
- NEW: new properties `selectedKey` and `selectedItemID` on `ComboBox` and `DropdownBox` controls. The value can now be set using the item id or key.
- CHANGE: When an XMLView is rerendered, it now preserves its pure HTML (XHTML namespace), only controls are rerendered. Dynamic modifications to the DOM are also preserved.
  LIMITATION: HTML controls as direct children of an XMLView are currently not supported. In most cases you can use XHTML content instead, which should even be easier to use. HTML controls can be used as children of other UI5 controls within an XMLView, but even there using XHTML might be more convenient. Outside of XMLViews, the HTML control is still the only way to embed HTML content into a control hierarchy.
- NEW: the `sap.ui.commons.HorizontalLayout` control has been created.

- CHANGE: some colors of the "Gold Reflection" theme have changed according to the newest Ux guidelines. You may want to adapt if you hardcoded the old colors:
  - The general background from #F7F7F7 to #F2F2F2
  - The tool icons in the left-hand side of the Shell to a main color of #8899AA

Runtime Version 0.19.0 (September 2011)

Framework

- CHANGE: The default execution context `this` for control event handlers `attachXYZ` and browser event handlers (`attachBrowserEvent` has been made more consistent.

For control event handlers e.g. `Button.press` the context was `window` before, for browser events (e.g. keydown, mousedown) it was the corresponding `DOMref` before. Code samples and requests from our consumers have shown, that access to the control is needed most of the time. This often resulted in extra code, e.g. using `jQuery.proxy()` at registration time or using `oEvent.getSource()` or `jQuery(oEvent.target).control(0)` in the event handler.

NOTE: Therefore, as of now, all control event handlers are by default called with the corresponding control/element as their context (`this`).

This obviously is an incompatible change, but for applications that do not use `this` in event handlers, the existing code should continue to work. The three formentioned workarounds are to be used preferably. They might benefit from the change and remove the workarounds. Applications that do use `this` in the event handler need to modify their code. In control event handlers, they can access `window` directly. In browser event handlers, they can use `oEvent.target`, where `oEvent` is the first argument of the event handler (event object).

In addition, the `attachBrowserEvent` method now supports a third parameter (`oListener`) to define a concrete context for the event handler function.

- CHANGE: `jQuery.sap.includeScript` can only be used after the document has been initialized since `document.write` does not work for XHTML pages. We recommend to include initial scripts via the `SCRIPT` tag.
- CHANGE: Accessibility Mode is activated now by default and can be explicitly switched off using the configuration parameter `sap-ui-accessibility=false`.
- NEW: There are several new features available for the data binding:
  - New events have been added which are fired when data loading (`requestFailed`) or data parsing (`parseError`) fails.
  - New method `PropertyBinding.setValue` which updates the value on the binding and also in the model.
  - Two-Way data binding has been introduced. Per default, all models have the `defaultBindingMode` set to `sap.ui.model.BindingMode.TwoWay` and support the One-Way and Two-Way binding modes. A model implementation can now specify its supported binding modes (e.g. only One-Way). An application can specify the default binding mode to be used for the model instance in the case that the mode is supported by the model. In the `bindProperty` method you can also specify the binding mode which should be used only for this specific property binding. The model needs to support this binding mode. The available binding modes are defined in `sap.ui.model.BindingMode`.
  - Binding of the `tooltip` property finally also works in the constructor (e.g. {{{...tooltip: "{comment}", ...})
- CHANGE: Data Binding for JSON Model: The optional data merge does now behave like `jQuery.extend( [deep], target, object1, [objectN] ))` with `deep=true` (the merge becomes recursive (aka. deep copy).

Controls

- CHANGE: The default value of the HTML control property `preferDOM` has been changed from `false` to `true`. This was a development request, as all use cases we have seen so far used the HTML control to create and preserve some piece of HTML. Most of the users first had to learn about the

property `preferDOM` before they got their use cases working, so we decided to make it easier and to change the default (although this is an incompatible change). If an application really needs the old behavior, it simply can set `preferDOM=false`. Furthermore, the documentation of the contract and the restrictions of the HTML control have been detailed.
- NEW: The `sap.ui.commons.TabStrip` control has been enhanced with the tooltip function.
- CHANGE: Keyboard navigation improved for `sap.ui.commons.TabStrip` control
- CHANGE: `sap.ui.commons.layout.MatrixLayout` now has a property `widths` to set the widths of the colums. So the widths can now be read, not only set. As a result, the `widths` can be used in the XMLViews and JSONViews (MVC) as well.
- CHANGE: `sap.ui.commons.DropdownBox` now has the properties `searchHelpEnabled`, `searchHelpText`, `searchHelpAdditionalText` and `searchHelpIcon` to activate the search help functionality. In former releases, the same functionality was available via pure JavaScript APIs. With the current change, the features can be used more easily with the MVC concept, especially with XMLView and JSONView. For applications, this change should be transparent.
- DEPRECATION: Control `sap.ui.commons.RoadMap`: Substeps are no longer supported and will be removed in a future version. Therefore, the property `expanded` and the aggregation `substeps` of `sap.ui.commons.RoadMapStep` have been deprecated.
- CHANGE: The `sap.ui.commons.layout.AbsoluteLayout` has been reworked. The changes in detail are:
  - CHANGE: The aggregation `content` was split into two aggregations, to make it usable in XMLViews:
    - The AbsoluteLayout now has an aggregation `positions` which allows to aggregate `sap.ui.commons.layout.PositionContainer` elements. These containers then have an aggregation `control` to aggregate a child control which should be placed in the `!AbsoluteLayout.`
    - A `!PositionContainer` which provides attributes to set the position of the child control in the `!AbsoluteLayout.`
    - The former properties `wrap*` are not longer supported.
    - For compatibility reasons the accessor functions of the old aggregation `content` (e.g. `addContent`) are kept. The corresponding `!PositionContainers` are maintained automatically in this case.
  - The `!AbsoluteLayout` now allows to center a child control; see properties `centerHorizontally` and `centerVertically` of element `sap.ui.commons.layout.PositionContainer`
  - The render mechanism of the `!AbsoluteLayout` was optimized. No re-rendering should occur when e.g. changing a position of a child control or when adding or removing a child control.

Runtime Version 0.18.0 (August 2011)

- DEPRECATION: The `web.Table` control of library `sap.ui.table` and all its related controls, elements and types are only prototypic. They are now deprecated and might be removed in one of the following releases. In detail the following entities are deprecated:
  - `sap.ui.table.web.Table`
  - `sap.ui.table.web.Record`
  - `sap.ui.table.web.Cell`
  - `sap.ui.table.web.RecordCell`
  - `sap.ui.table.web.Toolbar`
  - `sap.ui.table.web.ToolbarItem`
  - `sap.ui.table.web.ToolbarItemType`
  - `sap.ui.table.web.ToolbarType`
- CHANGE: The three view control types that are part of the MVC Model View Controller support, have been renamed. The new names are more consistent with similar names in the SAPUI5 API (XMLModel, JSONModel, CSSSize, HTML(control)).

New name	*Old name*
sap.ui.core.mvc.JSView	sap.ui.core.mvc.JsView
sap.ui.core.mvc.JSONView	sap.ui.core.mvc.JsonView
sap.ui.core.mvc.XMLView	sap.ui.core.mvc.XmlView

For compatibility reasons, the old names still will be supported for one or two

minor releases.

- CHANGE: The default behavior of the JSON Model methods setData(oData, bMerge) and loadData(sURL, oParameters, async, sType, bMerge) have been changed. By default, they no longer merge the loaded/given data with data already existing in the model, but replace it. The old behavior still can be activated by setting the new, optional parameter `bMerge` to `true`. Applications that rely on the merge behavior need to set that parameter explicitly.
- NEW: Both, content and style of the Demo Kit, have been reworked.

Runtime Version 0.17.0 (July 2011)

- CHANGE: `TabStrip` control: Rework of renderer. If a new tab is selected only, the content area is rerendered, but not the complete control.
- CHANGE: `TabStrip` control: Change of standard behaviour by selecting a new tab. Now the new selected tab is selected automatically. No implementation of event `select` is needed anymore. Existing implementations should be adopted.
- NEW: `TabStrip` control: New method `closeTab`. This method should be called in the handler of the `close` event to close the tab. This cannot be done automatically because before the close application dependend logic must be performed. If the tab is closed, the next one is selected.
- CHANGE: `TabStrip` control: Once the control is rendered it is not possible to set the `selectedIndex` to a not existing, not visible or not enabled tab.
- DEPRECATION: The property `selected` of the `Tab` control is deprecated because it is not used. To identify the selected Tab of a TabStrip the property `selectedIndex` is used.
- CHANGE: `TextField` control: Some alignment issues fixed for internet explorer.
- CHANGE: `MatrixLayout` control: Fix of vertical alignment of cell content in case of `layoutFixed` and defined row height.
- CHANGE: The deprecated aggregation `controls` of the `AccordionSection` control has been removed.
- CHANGE: Data binding: Datajs 1.0 integrated in oData model. Everything should work as before.

Runtime Version 0.16.0 (June 2011)

- DEPRECATION/NEW: `sap.ui.table.Table` is deprecated now. Use `sap.ui.table.DataTable` instead.
- CHANGE: The default value of the SAPUI5 configuration option `noDuplicateIds` has been changed to `true`. That means whenever a control or element is created with an already existing ID, an error will be thrown. This change has been done to avoid hard to find issues with duplicate IDs ("fail early"). When you encounter such a duplicate ID error in your application, you preferably should fix the issue. Either destroy the old control instance, or use a different, unique ID for the new instance or completely omit the ID - SAPUI5 then will create a new, unique one for you. If none of these solutions is possible for some reason, and if you accept the risk of undiscovered errors, then you might set the `noDuplicateIds` option back to `false` in your application. See Configuration for details about the SAPUI5 configuration.
- NEW: Data Binding: Introduced new method `bindContext` and `unbindContext` on element to support setting/removing a binding against a specific binding path. This is useful for master/detail scenarios.
- NEW: Data Binding: Model, JSONModel, XMLModel, ODataModel: Definition and implementation of the new methods `createBindingContext` and `destroyBindingContext`. These methods are used for master/detail scenarios and are called internally by the element. Only new model implementations need to implement these methods. Applications shouldn't call these methods.
- CHANGE: Data Binding: oData Model: Removed `load` method. The loading of additional data is now handled internally.
- CHANGE: The deprecated class `sap.ui.ux3.WorksetItem` has been removed. Use `sap.ui.ux3.NavigationItem` instead.
- CHANGE: The method `Dialog.setText()` that has anyway not been in the declared API has been removed. Use `setTitle()` to set the Dialog title text.
- CHANGE: In the `DatePicker` control there are some corrections to fix some issues.
- NEW: A new `VerticalLayout` control was created. With this layout some controls can be aligned one below the other.

Runtime Version 0.15.0 (May/June 2011)

- CHANGE: The process of loading and initializing a UI Library on the client has been changed. In SAPUI5 up to version 0.14, a library has been loaded by
  1. loading the corresponding library.js file (e.g. `sap/ui/commons/library.js`),
  2. executing it, which in turn
  3. called back into the Core by calling the `sap.ui.core.Core.initLibrary()` method which
  4. loaded the `library.properties` file from the same library package (e.g. `sap/ui/commons/library.properties`) and
  5. created convenience placeholders for all controls (AKA 'lazy imports')
- Starting with version 0.15, the need to load a separate `library.properties` file has been eliminated. All information necessary to execute step 5) is now included in a JSON object provided by `library.js` in the call to `initLibrary()` in step 3. This change reduces the number of requests for each library by 1 and is a prerequisite for future enhancements in this area. To make the new JSON object available to `initLibrary()`, a UI Library must have been built with the build tools of version 0.15 (Maven or IDE).

- DEPRECATION: For compatibility reasons, older libraries are still supported (`library.properties` is then read as before) and for the same reason, the `library.properties` file still exists. Both features will be removed in one of the upcoming releases of SAPUI5.
  Note: If you have an urgent need for the `library.properties` file, please let us know.
- CHANGE: The escaping functions `jQuery.sap.escapeHTML` and `jQuery.sap.escapeJS` have been reimplemented according to the new XSS Secure Programming Guide. In accordance with that guide, methods for escaping parts of URLs or CSS content have been added as well. **For clarity, all escaping methods have been moved into a separate module `jquery.sap.escape.js`** (previously `jquery.sap.strings`). The new module is automatically available to all SAPUI5 applications that use `sap-ui-core.js` (99%). In those rare cases, where applications do not reference `sap-ui-core.js` nor `jquery-sap.js`, but want to use the escaping, they have to include the new module directly via `jQuery.sap.require`("jquery.sap.escape"); }}}.
- CHANGE: The type `sap.ui.ux3.WorksetItem` has been replaced by `sap.ui.ux3.NavigationItem` (offering the same capabilities and to be used instead)
- NEW: It is now possible to define 0..n-associations in control APIs
- NEW: Elements can now be removed from aggregations not only by index, but also by ID or instance
- NEW: New control Ux3 NavigationBar offering tab-like navigation capabilities (limited accessibility as of this sprint)
- NEW: Data Binding: New binding syntax to define the binding directly in the control constructor
- CHANGE: Tree Data Binding enabled for `sap.ui.commons.Tree` control (see also new test page in test suite)
- CHANGE: The deprecated property `Panel.showCollapseArrow` has been removed.
- CHANGE: The deprecated property `Panel.showMinimizeButton` has been removed.
- CHANGE: The deprecated event `Panel.scroll` has been removed.
- CHANGE: The deprecated aggregation `Panel.controls` has been removed.
- CHANGE: The deprecated property `Shell.helpButton` has been removed.
- CHANGE: The deprecated property `ComboBox.displaySecondaryValue` has been removed.
- CHANGE: Complete rework of RoadMap control
- CHANGE: The deprecated aggregation `AbsoluteLayout.children` has been removed.
- CHANGE: Rework of `Splitter` keyboard navigation and many bug fixes.
- CHANGE: Complete rework of `sap.ui.core.RenderManager`. The changes might affect control development. The changes in detail are:
  - CHANGE: Internal code cleanup and rework of rendering algorithm.
  - DEPRECATION / CHANGE: The `Core.getRenderManager` function always returns a new RenderManager instance (respectively its interface) which can be used exclusivly without influence to the core rerendering. Too provide a proper naming the function `getRenderManager` has been deprecated.
  - NEW: Function `Core.createRenderManager` is introduced as replacement for `getRenderManager`.
  - DEPRECATION: The following functions were already or are newly deprecated and might be removed in future versions:
    - `getHTML`: There will be no equivalent option available anymore in future because creating the HTML of a control and putting it into the DOM manually can result in

controls which do not work properly. Use the new function flush as alternative (see description below)
    - `getEscaped`: Not needed any more. Use `writeEscaped`, `writeAttributeEscaped` or directly `jQuery.sap.escapeHTML` instead.
    - `translate`: Not implemented
    - `writeAcceleratorKey`: Not implemented
    - `writeCustomStyleClasses`: Not needed any more. Use writeClasses instead.
    - `writeAccessibilityState`: This function is only an experimental feature.
    - `getResourceRoot`: Use the more generic `jQuery.sap.getModulePath` instead.

- CHANGE: Stack is used for writing styles and classes to ensure that nested rendering of controls does not lead to side effects.
- CHANGE: Removed unnecessary parameter `bReplaceExisting` from the render function.
- NEW: Function destroy to cleanup an exclusive RenderManager instance when it is not needed anymore.
- NEW: Function `flush` as an alternative for the deprecated `getHTML` function (see above):
  - Example usage:

```
  // Create a new instance of the RenderManager
  var rm = sap.ui.getCore().createRenderManager();
  // Use the writer API to fill the buffers (e.g. write controls and / or some plain
HTML)
  rm.write("Hello");
  rm.renderControl(oControl);
  rm.write("World");
  // ...
  //Finally flush the buffer into a provided DOM node. The current content is
removed.
  //After the flush the buffers are cleared and the Rendermanager instance can be
reused if needed.
  rm.flush(oDomNode);
  //If the instance is not needed anymore, destroy it
  rm.destroy();
```

  - This function avoids the problems occuring when putting some control HTML manually into the DOM, e.g. the currently focused control is stored, and the focus is reinitialized again after rendering or the functions `onBeforeRendering` and `onAfterRendering` are called on the rendered controls if needed.
- CHANGE: The functions `render`, `flush` and `destroy` must not be used within control renderer code. To ensure this, a new interface has been introduced which is provided to the control renderers (see `render` functions of the control renderers). The interface provides all functions of the RenderManager except of the three functions mentioned above.

- CHANGE: Control Ux3 !Exact Pattern. The changes are in detail:
  - NEW: `Close` icon on attribute list which has the same behavior as when the corresponding attribute would be deselected by the user.
  - CHANGE: Changed visual appearance of the search field
  - CHANGE: Changed visual appearance/behaviour of the attribute list expanding
  - INCOMPATIBLE CHANGE: Attribute `hasValuesIndicator` of `sap.ui.ux3.ExactAttribute` was renamed to `showSubAttributesIndicator`.
  - INCOMPATIBLE CHANGE: Aggregation `values` of `sap.ui.ux3.ExactAttribute` was renamed to `attributes`.
  - INCOMPATIBLE CHANGE: The supply function mechanism of sap.ui.ux3.ExactAttribute? was reworked and the following API changes were done:
    - DELETE: The attribute `valuesSupplyFunction` was removed.
    - NEW: New attribute `supplyActive`
    - NEW: New attribute `autoActivateSupply`
    - NEW: New event `supplyAttributes`
    - CHANGE: Changed meaning of the attribute `showSubAttributesIndicator`
    - The new supply function mechanism works as follows:
      - A supply function is now a handler which is attached to event `supplyAttributes`.
      - The event is fired when the corresponding `sap.ui.ux3.ExactAttribute` is selected, it was already selected when a handler is attached, or function `getAttributes()` is called.
      - The event is only fired if the attribute `supplyActive` is `true` (default). After firing the event, the attribute is automatically set to `false`. The idea is that a supply

function is only called once when the data is requested. To enable the event again it is possible to manually set the attribute back to `true`.
        - If you want the supply function to be called on every `select` you can set the attribute `autoActivateSupply` to `true`. In this case, the attribute `supplyActive` is automatically set to `true` on every deselect.
        - Whether the sub attribute indicator (the little arrow beside an attribute in a list) is necessary is computed automatically (see new function `getShowSubAttributesIndicator_Computed()` of `sap.ui.ux3.ExactAttribute`). Only in case a supply function is attached and the attribute `supplyActive` is `true`, the Exact Pattern needs a hint whether sub-attributes might be available. Only in this case the attribute `showSubAttributesIndicator` is considered and must be maintained. If it is not possible to maintain it correctly (e.g. the backend does not support count-calls) it should be set to `true` (default).
        - Bug fixing for asynchronous supply function. The corresponding list does not open before the supply function has finished its work.
    - NEW: The functionality of the attribute list browsing has been extracted into a separate control: ExactBrowser
    - **CAUTION:** This control is still not entirely finished and discussions with Ux are ongoing, so behavioral, visual and API changes may still happen after version 0.15.0. The pattern is also not yet compliant to all product standards like accessibility. The same is valid for controls which are build as part of the Exact control like ExactAttribute, ExactBrowseAttribute, ExactBrowser and ExactArea.
- CHANGE: The method `MatrixLayout.setWidths` now accepts real arrays.

Runtime Version 0.14.0 (May 2011)

- CHANGE: The deprecated property `selectionFollowsFocus` of the `ListBox` control has been removed.
- CHANGE: The deprecated property `displaySecondaryValue` of the `ListBox` control has been (renamed to `displaySecondaryValues`.
- CHANGE: The deprecated event `click` of the `ListBox` control has been removed since the `select` event does the same.
- CHANGE: The property `displaySecondaryValue` of the `ComboBox` and `DropdownBox` controls has been deprecated. Instead there is a new property `displaySecondaryValues`.
- CHANGE: The deprecated property `valueState` of the `TextView` control has been removed. For changing the text color the parameter `semanticColor` exists.
- CHANGE: The `Popup.isOpen()` function does now only return `false` after any closing animation has completed. This affects similar methods like `Dialog.isOpen()` as well.
- DEPRECATION: The aggregation `helpButton` of the `sap.ui.ux3.Shell` has been deprecated in favor of a more general `headerItems` aggregation.
- DEPRECATION: The `mode` property of the Paginator has been deprecated and will be removed. Only "Full" mode will be available in the future.
- NEW: A new event `liveChange` was added to the `Slider` control. This event fires changes of the slider value during the mousemove.
- NEW: A new property `enabled` was added to the `Slider` control. This is to be compatible to the other controls. If the `Slider` is not enabled it can not be changed. In all themes (except platinum) the visualization is like `readonly` because there is no specification for this from UX. This could be changed later if needed.

Runtime Version 0.13.0 (April 2011)

- NEW: New control library "sap.ui.ux3"
- NEW: New theme "sap_goldreflection"
- NEW: New control Ux3 "Gold Reflection" Shell - CAUTION: This control is not entirely finished and discussions with Ux are ongoing, so behavioral, visual and API changes may still happen after version 0.13.0. The Shell is also not yet compliant to all product standards like accessibility.
- NEW: New control Ux3 Exact Pattern; CAUTION: This control is not entirely finished and discussions with Ux are ongoing, so behavioral, visual and API changes may still happen after version 0.13.0. The pattern is also not yet compliant to all product standards like accessibility. The

same is valid for controls which are build as part of the Exact control like ExactAttribute], ExactBrowseAttribute and ExactArea
- NEW: New control Ux3 Feed Component; CAUTION: This control is not entirely finished and discussions with Ux are ongoing, so behavioral, visual and API changes may still happen after version 0.13.0. The components also not yet compliant to all product standards like accessibility. The same is valid for controls which are build as part of the Feed Component like Feeder, FeedEntry?, and CommentEntry?.
- NEW: `!ListBox.scrollToIndex()` now also works when called before rendering
- NEW: The BusyIndicator control does now inherit from EventProvider and fires `Open` and `Close` events
- CHANGE: Data Binding for the Hierarchical Table supported.
- CHANGE: Keyboard navigation in the ListBox control now also accesses disabled items
- CHANGE: As started in version 0.12.0 for better overall consistency of the API, all container-like controls (controls with a general purpose aggregation accepting any child control(s)) now use the same name `content` and same cardinality (0..n) for their main aggregation.
  - COMPATIBLE: Whenever possible, the old aggregation name and cardinality have been preserved, but deprecated. This is the case now for AccordionSection.controls
  - INCOMPATIBLE: Splitter.first/secondPaneControls is now named `Splitter.first/secondPaneContent`
- CHANGE: A UIArea can now have several root controls. Therefore the UIArea has an aggregation `content` like all container-like controls which can be accesssed via the standard aggregation functions (e.g. getContent). In order of this change the following sub-changes were necessary:
  - DEPRECATION: The function `UIArea.setRootControl` is deprecated. Use the accessor functions for the aggregation `content` instead.
  - CHANGE: The function `Control.placeAt` was extended by a position parameter.
  - DEPRECATION: The function Core.setRoot is deprecated. Use the function `Control.placeAt` instead.
- CHANGE: TextArea? calculation of cursor position changed because of performance issues on long texts.

Runtime Version 0.12.0 (March 2011)

- NEW: New controls RadioButtonGroup?, PasswordField?, RowRepeater?

- NEW: When registering a listener for any of the control events or for events from any EventProvider, an additional payload object can be specified (`oData` parameter). When the corresponding event is fired, the payload will be provided to the registered listener function as a second parameter, right after the event object. For details see the generic implementation `EventProvider.attachEvent` or one of the concrete events `Button.attachPress`.

- CHANGE: For better overall consistency of the API, all container-like controls (controls with a general purpose aggregation accepting any child control(s)) now use the same name `content` and same cardinality (0..n) for their main aggregation.
  - COMPATIBLE: Whenever possible, the old aggregation name and cardinality have been preserved, but deprecated. This is the case for `Panel.controls`, `Tab.controls`, `AbsoluteLayout.children`
  - In two cases, the aggregation name `content` was already used, but with cardinality 0..1. For these controls, the cardinality of the aggregation also has been changed to 0..n, but with the following implementation peculiarities:
    - The old setter function `setContent(Control)` has been kept, but deprecated. In future, the new function `addContent(Control)` should be used. `setContent()` has been implemented as `removeAllContent() + addContent()`
    - The getter function `getContent()` now returns an array instead of `null` or a single control. All code that used the getContent() method so far to retrieve a single control, now must be adapted either to handle multiple controls or to access the first entry of the returned array:

      ```
      var oCtrl = oLayout.getContent()[0]
      ```

    - This affects MatrixLayoutCell (setContent, getContent) and Dialog (setContent, getContent).

- CHANGE: Some deprecated methods and internal features have been finally removed:
  - `sap.ui.require` - The method has been removed, use `jQuery.sap.require` instead.
  - `sap.ui.getClass` - The method has been removed as it was only a special case of the more consistent jQuery.sap.getObject
  - Parameter `bBypassSetters` of the `Element.applySettings` method has been removed. This parameter was intended for internal use only. As it caused issues with the core functionality of several controls, and with data binding and generic validation of settings in general, it has been discarded now.
  - Generic support for a 3rd parameter (beside Id and Settings) in the constructors of Elements or Controls has been removed since 0.9.0 support for such a 3rd parameter (aka `oSomething`) had been removed from all SAPUI5 controls already. However, the Element base class still wrote a (deprecation) log entry and called an `init3rd(oSomething)` method when it existed. This functionality has been removed completely.
  - Support for light controls (without runtime metadata) has been removed. Data binding and other features heavily rely on such runtime metadata. Separating the metadata independent features from the rest would have been too confusing for consumers.

- CHANGE: Added keyboard navigation feature to the Table Control.
- CHANGE: Enhanced vertical scrolling performance of the Table.
- CHANGE: Enhanced display of *details* messages from the Message Area/Bar?.

Runtime Version 0.11.0 (February 2011)

- CHANGE: The default width of the Panel control is no longer defined by the Panel contents, but always "100%".
- CHANGE: The High Contrast Black theme was changed to have a smaller font size, which fits the specification and is also the same font size used in the other themes.
- CHANGE: The ListBox HTML was completely changed (which is transparent as long as no code or CSS relies on it)
- CHANGE: Enhancements to the Table filtering functions.
- CHANGE: Enabled Web Toolkit consumption of the Table.
- DEPRECATION: The Panel properties `showCollapseArrow` and `showMinimizeButton` are now deprecated and will be removed in favor of the new property showCollapseIcon (the theme now defines how the icon is visualized)
- NEW: Every control has the new methods attachBrowserEvent and detachBrowserEvent which allow handling native browser events in this control
- NEW: The ListBox has new features like an optional icon column, additional event parameters in the `select` event, new properties like `minWidth`, `maxWidth`, text alignment and the new `scrollToIndex` method.
- NEW: New controls MessageBar, Tree and ApplicationHeader.
- NEW: Added drag and drop as well as resizing table columns.

Runtime Version 0.10.0 (December 2010)

- CHANGE: MatrixLayout has got a new rendering mode using the CSS property `layout:fixed`. To allow applications to use the old behavior, a new property `!layoutFixed` has been introduced. To enforce the usage of the more performant `layout:fixed` mode, the default of the property is `true`. So applications must explicitly set this property to `false` to get the old behavior.
- CHANGE: Due to the previous change, width and height are no longer supported for a `!MatrixLayoutCell`.
- CHANGE: Continous improvements to the Table control and its components
- CHANGE: Rework of the ComboBox control
- NEW: `!AbsoluteLayout`, `!DataBinding`, `!RichTextEditor`, `!ProgressIndicator`, `!HTMLContainer`, `DatePicker`, `MenuButton`, and more

Runtime Version 0.9.0 (September 2010)

- CHANGE: Introduction of jQuery

- CHANGE: A lot of APIs have been replaced by jQuery or are now part of our jQuery plugin
- CHANGE: `3rd` parameter in constructor of some controls has been removed in favor of more general object literal `mSettings`
- CHANGE: File system layout of control libraries has been changed, both at runtime and at design time
- CHANGE: Hooks for controls have been changed
- CHANGE: Eventing has been changed
- CHANGE: Property values are validated, invalid values are reported as exceptions
- CHANGE: SAPUI5-cdn contains public libraries (`core`, `commons` and `table`)
- CHANGE: Old table control has been moved from `sap.ui.table` to `sap.ui.dev`
- NEW: Introduction of new controls: Table, Accordion, Splitter, RoadMap?, TextArea?, Slider, and more
- NEW: New way of configuring SAPUI5 in addition to attributes of bootstrap tag
- NEW: License texts of contributing components added *

Version 0.8.11 (End of June 2010)

SAPUI5 Runtime

- CHANGE: The name of the common underlying theme which is used as base for all specific themes is no longer `default` but has been changed to `base`. For any existing libraries outside the SAPUI5 Perforce source tree the respective theme folder must be renamed from `default` to `base`, otherwise specific themes will fail to pick up this theme as a base.

Version 0.8.10 (End of May 2010)

SAPUI5 Runtime

- CHANGE: It is now possible to set a `tooltip` for almost any element/control, a description how to implement it can be found in the corresponding control documentation. A tooltip can be a simple text tooltip which is displayed via the HTML `title` attribute and via any subclass of `Tooltip`, for example `RichTooltip`. All previous implementations of tooltips in the different controls have been replaced by this new generic feature. For applications this should be a fully compatible change. Control developers must no longer define properties or aggregations named `tooltip`. Existing properties with that name should either be removed completely if the generic implementation in Element is sufficient or should be renamed. If a control should support simple text tooltips, it is still the responsibility of the control renderer to render the title attribute as the best suitable DOM element can not be determined by the framework. See `Element` for more details, e.g. methods `setTooltip`, `getTooltip_Text`, `getTooltip_AsString`.
- CHANGE: Several events of existing SAPUI5 controls contained a parameter `id` that identified the source control firing the event. This parameter has been removed from all event definitions as it was redundant to and less convenient than the always available `event.getSource()`. To be compatible with older releases, the event object delivered to the application still contains the source id, but the parameter is no longer documented and will be removed in one of the next releases.
- CHANGE: For some controls, the `id` event parameter did not identify the event source but a different control. For clarity, these parameters have been renamed, see `Toolbar.subItemSelected(itemId)`.
- CHANGE: The "High Contrast Black" theme has received a number of changes, including a larger font size and the replacement of styling images by simple characters and CSS borders. This was done to make the theme usable in Windows High Contrast mode.

Version 0.8.9 (May 2010)

SAPUI5 Runtime

- NEW: New controls in sap.ui.commons: `Toolbar`, `FileUploader`
- NEW: Declared library dependencies are propagated to the runtime and checked/resolved by the

framework
- CHANGE: The methods `sap.ui.core.Core.attachAnyEvent()` and `sap.ui.core.Core.detachAnyEvent` have been deprecated and will be removed in the next drop. These methods are now available on `sap.ui.core.BrowserEventManager` which can be retrieved using `sap.ui.core.Core.getBrowserEventManager`.

Version 0.8.8 (April 2010)

SAPUI5 Runtime

- NEW: New controls in sap.ui.commons: Dialog, MessageBox available
- NEW: New controls in sap.ui.table: Web Table available
- CHANGE: Improved following controls in sap.ui.commons: RichTooltip, ComboBox
- CHANGE: Internal facility BlockLayer was renamed to BlindLayer to match wording / naming known from Lightspeed implementation.
- REMOVE: Internal control PopupListBox was removed. This was an internal entity used in an earlier development phase and became obsolete with new Popup and ComboBox implementation from 0.8.7.

Version 0.8.7 (March 2010)

SAPUI5 Runtime

- NEW: Two new controls have been added to the sap.ui.commons UI library: A Panel and a RichTooltip.
- NEW: A plugin concept has been introduced for the SAPUI5 core.
- ENHANCE: The testsuite has been slightly enhanced and now supports log filtering and less initialization issues.
- CHANGE: As the new Panel supports all features of the TitlePanel?, the latter has been abandoned.
- CHANGE: Moved the themes "Tradeshow Plus" and "sap_system" out of the core and the control libraries into a separate theme library for each (below src/test). Dependencies to those are now required in order to use these themes. Also moved the `sap_ux` theme library from src/test to src/libraries.
- CHANGE: The API of the SAPUI5 core has been revised and several unnecessary methods have been deprecated, or their visibility has been reduced. The changes should not affect applications or control development.
- CHANGE: The support for JSP Renderers has been moved out of the core and must be included as a separate script: jsprenderer.js. This leads to a minimized size of the core.
- CHANGE: Support for transformed lightspeed controls, and lightspeed facade controls has been moved out of the SAPUI5 core. As the corresponding libraries (sap.ui.legacy and (sap.ui.classic) compensate this change, applications should not notice a difference.

Version 0.8.6 and Prior

SAPUI5 Runtime

- In order to make the SAPUI5 runtime API cleaner and easier to remember, some global methods have been renamed. To avoid waste in this early development state, the older method names have not been deprecated but have been removed. After release to customer, such incompatible renamings will no longer occur.

  List of renamings:

Old Names	New Names
`sap.ui._package`	`sap.ui.namespace`
`sap.ui._import`	`sap.ui.require`
`sap.ui._lazyImport`	`sap.ui.lazyRequire`
`sap.ui._includeScript`	`sap.ui.includeScript`
`sap.ui. includeStyleSheet`	`sap.ui.includeStyleSheet`

```
sap.ui._assert            sap.ui.assert
sap.ui._fatal             sap.ui.fatal
sap.ui._measure           sap.ui.measure
sap.ui._trace             sap.ui.trace
```

- For the same reasons as above, some of the generated information for UI Controls and other classes have been modified.

```
<SomeClass>.metadata              <SomeClass>.getMetadata()
<SomeClass>.metadata.properties   <SomeClass>.getMetadata().getProperties()
<SomeClass>.metadata.aggregations <SomeClass>.getMetadata().getAggregations()
<SomeClass>.metadata.associations <SomeClass>.getMetadata().getAssociations()
<SomeControl>.aPublicMethods      <SomeControl>.getMetadata().getAllPublicMethods()
<SomeControl>.getElementName()    <SomeControl>.getMetadata().getName()
<SomeControl>.getLibraryName()    <SomeControl>.getMetadata().getLibraryName()
```

- The HTML created by several renderers has been changed. Usually this should not be noticable

- DOM attributes and IDs created and managed by SAPUI5 have been cleaned up to reduce potential naming conflicts with other frameworks as well as application code.
  - Changes in detail
    - Clean up of the suffixes separator for controls (now uses - instead of _).
    - Switched to ID suffixes with "-" separator
      - **Note:** Due to interpretation of `<someControlId>-r` as Root-Dom-Node the use of this id is prohibited for inner structures of controls.
    - Adopted the generated IDs to "id-<timestamp>-<counter>" (Script.js)
    - Moved logger to `sap.ui.Logger`
    - Switched sapdata to `data-sap-ui`
    - Prefixed !Test Suite ids, classes and frames with "sap-ui"
    - Shadow uses custom attribute `data-sap-ui-shadow` now
- New UI controls in sap.ui.commons UI library: TabStrip, ComboBox, Link, Image, RadioButton, ListBox
- New UI controls in sap.ui.dev UI library: CvomChart, DatePicker, GoogleMap, PacMan
- Extensibility concept of controls: How can existing controls be extended to enrich them with additional functionality
- ARIA accessibility attributes rendered for TabStrip, Button, CheckBox, RadioButton.
- UI Controls in sap.ui.commons UI library: Button, TextField, TextView, Label, Menu, MatrixLayout, CheckBox
- Theming infrastructure for UI controls based on CSS and parametrization
- Theme support for sap_platinum, sap_tradeshow_plus, sap_hcb (and some experimental others)
- GWT POC: Wrapping SAPUI5 UI controls with GWT widgets
- Server-side rendering POC: Providing server-side renderers for SAPUI5 UI controls
- Data binding POC
- Well-defined SAPUI5 core API and UI control API, provided for JavaScript
- SAPUI5 core features like
  - UI control library plugin capabilities
  - Utilities for AJAX requests
  - Property files and language resource bundles
  - OpenAjax support
  - Lazy loading of UI control renderers
- Unit test environment based on JsUnit

Terminology

UI development toolkit for HTML5 (SAPUI5)

General Terms

Term	Description
application cache buster	A cache buster allows the application to notify the browser to refresh the resources only when the application resources have been changed. Otherwise the resources can always be fetched from the browser's cache. The application cache buster is a special mechanism to extend this function to SAPUI5 application resources
base type	A control property and UI text in UI5. The term is also used for the generic type description for UI5 entities.
control editor	A design-time tool for developing UI controls.
element	A basic unit in the framework.
item	A control base type
library editor	A design-time tool for creating libraries.
render manager	A core API that represents a writer for UI controls and provides some additional convenience methods.
SAPUI5 ABAP repository	A repository for SAPUI5 applications and SAPUI5 applications projects located on the SAP NetWeaver Application Server ABAP. A SAPUI5 application stored in the SAPUI5 application repository can be deployed and executed in a browser directly. It is connected to the ABAP transport system.
SAPUI5 ABAP repository team provider	is a team provider in eclipse to upload and share SAPUI5 applications and projects to the SAPUI5 ABAP repository.
simple type editor	A design-time tool for defining and editing simple types.
simpleType	The framework provides an abstract base class for defining simple types; the class is part of the sap.ui.model package.
tooltipBase	A base class for any extended tooltip that provides the open/close behavior and a text property.
user interface development toolkit for HTML5 (SAPUI5)	The official name is "UI development toolkit for HTML5". A framework providing UI controls for developing Web applications for the SAP ABAP or SAP Java server, or for Open Source servers such as Jetty. The framework also provides tools for developing applications and artifacts.

Programming Model

bi-directional data binding	Allowed synonym for Two-Way-Binding
controller	An application unit containing the active part of the application. It is the logical interface between a model and a view, and corresponds to the model view controller (MVC) concept.
data binding	A technique that binds two data sources together in order to keep them in sync. All changes in one data source are automatically reflected in the other; the involved layers are the view and the model. The framework supports various binding modes, which are defined in sap.ui.model.BindingMode.
default binding	A binding mode that is defined by the model usage.
JSONModel	A model implementation for the JS Object Notation (JSON) format.
JSONView	A view flavor that allows you to use JavaScript Object Notation (JSON) data in a view.

JSView	A view flavor that allows you to use JavaScript (JS) data in a view. JSView is the default view type when Web applications are created with wizard support.
list binding	A binding concept available for lists in a model. List binding can be used to populate tables or item lists.
model	Data provider for the application where the model instance is assigned to the UI and the controls are bound to the model. Various model types are available; the model type used depends on the data format available on the server side.
Model View Controller (MVC) concept	A UI programming model that separates the layout (view) from the content (model) and the behavior (controller). The MVC concept is used by the framework to model the architecture of the applications.
ODataModel	A model implementation for the Open Data (OData) Web Protocol format.
one-time binding	A binding mode where data is fetched exactly once from the model backend.
one-way binding	A binding mode where the data is read from the model and transferred to the view.
property binding	Is used to access single data values in a model.
resourceModel	A model implementation for resource bundles. It allows you to bind texts of a control to language-dependent resource bundle properties.
tree binding	A specific binding manner for trees in a model that can be used to populate trees.
two-way binding	A binding mode where data transfer takes place from the model to the view, and vice versa.
view	An application unit containing the control definitions for the user interface layer in the application. It is an application part analogous to the common model view controller (MVC) concept. The framework provides the view types JSView, XMLView, and JSONView.
XMLModel	A model implementation for the Extensible Markup Language (XML) format.
XMLView	A view type that allows you to use Extensible Markup Language (XML) data in a view.

Themes

theme	Defined by a style sheet with the file extension .css and provides the colors and styles for applications.
GoldReflection	The default theme that can be used for all UI controls provided by the framework.
High Contrast Black	A theme that can be used for any UI control provided by the framework.
Platinum	A theme providing a standardized CSS parameter set regarding colors and font sizes. The theme can be used for any UI control provided by the framework.

Controls

Term	Description
AbsoluteLayout	A UI control that positions the embedded controls absolutely.
Accordion	A UI control that allows you to provide texts or actions in a categorized manner. Accordion acts as a container control and provides sections into which other controls can be embedded. At application start, one section is opened; at runtime, another section can be opened, and the previously opened one is then closed. The subcontrol is AccordionSection.
AccordionSection	A UI control that is to be used in an AccordionControl.
ApplicationHeader	A UI control that provides a generic header that can be used by other controls. It provides areas for a title text, an image or a logo, and a welcome text, as well as a logoff button.
Area	A UI control that defines areas for an ImageMap. The user can trigger a link or an action from an area.

AreaDesign	A static enumeration class that provides the values for designing the background of an area.
BackgroundDesign	A static class provided with the sap.ui.commons.layout package for defining the background of cells.
BarColor	Static class providing configuration options for the colors of the ProgressBar control.
BorderDesign	A static enumeration class that provides the design options for an area.
BorderLayout	A UI control that divides the user interface area into five predefined subareas. Other controls can be embedded into these areas.
BorderLayoutArea	Represents one area of a BorderLayout.
BorderLayoutAreaTypes	A static class that provides the values for defining the area types in a BorderLayout.
Button	A static enumeration class that provides the design options for an area.
ButtonStyle	A static class that provides the available options for Button design.
CheckBox	A UI control that provides a labeled box, which can be flagged. A checkbox can either stand alone, or be provided as a CheckBox group with multiple instances.
Collaboration control	A control based on the Web standard for supporting social media by facilitating participatory information sharing, interoperability, user-centered design, and collaboration.
ComboBox	A UI control that provides a field that allows users to choose an item from a list of predefined items. The items can be provided in the form of a complete ListBox, single ListBoxItems, or text strings defined for the current application. ComboBox also allows you to enter items that are not part of the predefined list.
Control	Provides methods and parameters for defining the layout and user interaction. The framework provides basic controls such as Button and Label, complex controls such as DataTable and TabStrip, layout controls such as MatrixLayout and AbsoluteLayout, as well as comprehensive layout patterns such as Shell and ExAct.
DataSet	A UI control that provides a frame for displaying unstructured textual or graphical content. DataSet provides the convenience functionalities filtering and sorting. Its subcontrols are DataSetSimpleView and DataSetItem. DataSet can be used as an integral part for other controls, especially Web 2.0 controls; or alternatively standalone.
DataSetItem	An element to be used for the DataTable control. The content to be displayed is bound to DataSetItem. All items are bound to the DataSet using a model.
DataSetSimpleView	To be used as a child control of the DataSet control, where DataSetSimpleView itself is a container control for views, layout controls, and basic controls to be embedded. DataSetSimpleView is responsible for content display in floating or nested mode.
DataTable	An interactive table control that allows you to integrate any control type to be the cell content. The table has a header section, and you can preconfigure the number of table rows that are to be displayed initially at runtime.
DatePicker	A UI control that provides an interactive calendar.
Dialog	A UI control that provides an interactive window that appears on request and displays information to the user. The window is displayed in front of the application in a separate layer. The control provides a header section, an area for some text, buttons for OK and Cancel (which can be disabled), and a close button. Other controls can be embedded into the Dialog.
DropdownBox	A UI control that provides a field that allows users to choose an item from a list of predefined items. The items can be provided in the form of a complete ListBox, single ListBoxItems, or text strings defined for the current application. DropdownBox provides a history for the five previously selected items and lists them at the top of the selection list.
ExAct	A design pattern approach. Abbreviation for Explore and Act. See also Exact, ExactArea, ExactAttribute, ExactBrowser, ExactList.
Exact	A UI control for searching data, exploring data, and acting on the data. Subcontrols are ExactBrowser and ExactList.

ExactArea	A UI control that consists of a toolbar and a content area where other controls can be embedded. ExactArea is intended to be used for the ExAct design approach, stand-alone usage is also possible.
ExactBrowser	A UI control representing an attribute browse area. The control is to be used with the ExAct control.
ExactList	A UI control that is to be used as a subcontrol of ExactBrowser.
Facet	The content area of a Thing Inspector contains a Facet Navigation Bar. Each item of this navigation bar is called facet.
Feed	A container control representing a full feed page with a property field for image source. Feed is the parent control for FeedChunk, FeedComponent, FeedEntry, and Feeder.
FeedChunk	A UI control that provides a large set for creating and handling feeds. FeedChunk would generally be embedded into the Feed control.
FeedComponent	A UI control that acts as a container for Feed and FeedEntry.
FeedEntry	A UI control for defining feed entries.
Feeder	A UI control for defining feeders.
FileUploader	An interactive UI control for implementing file uploads. The control supports various upload modes.
HorizontalDivider	A UI control that visually separates the Web UI into two areas or pages.
HorizontalDividerHeight	A static enumeration class that provides the values to specify the height of the HorizontalDivider control.
HorizontalDividerType	A static enumeration class that provides the values to specify the type of the HorizontalDivider control.
HorizontalLayout	A layout that allows you to align UI controls horizontally.
HTML	A control the framework provides with the sap.ui.core package.
Image	A UI control that allows you to include graphics for Web page design.
ImageMap	A UI control that allows you to provide complex graphics on the UI Web screen, from where the user can click an area to trigger a link or another action. The subcontrol is Area.
Label	A UI control that can be used standalone for text display, or in combination with another control to label it.
LabelDesign	A static class providing the formatting options for the Label control.
Library	Contains and provides the controls in the framework. The controls are bundled by category. Users also have the option to create libraries with tool support.
Link	A UI control that provides an absolute or relative reference to an internal or external URL. A link can also trigger an action.
ListBox	A UI control that is used to provide a list of items. The subcontrol is ListItem.
ListItem	Inherits from sap.ui.core.Item and is intended to be used for the ListBox control, but can also be used for other controls such as DropdownBox.
MatrixLayout	A UI control that arranges controls in a grid structure using rows and cells.
MatrixLayoutCell	An element used as part of the inner structure of a MatrixLayout control. A cell embeds the content for the MatrixLayout.
MatrixLayoutRow	The element is used as part of the inner structure of a MatrixLayout control and is responsible for the cell handling.
Menu	A UI control representing a popup menu, which opens in front of the application and allows the user to select one of the actions. A menu can have one or more levels. The subcontrol is MenuItem.
MenuBar	A UI control that is used to offer a set of actions that shall be provided in a structured way.
MenuBarDesign	The static class determines the visual design of MenuBar.
MenuButton	A UI control that extends sap.ui.commons.button and is used to provide a button control with a menu, which is opened by user interaction.
MenuItem	A class representing the smallest unit in menu hierarchies. Can be a direct part of

	MenuBar or of Menu, or of a submenu.
MenuItemBase	An element providing the standard properties for MenuItem.
Message	A UI control that creates the messages that shall be displayed in the MessageBar Control. See also MessageBar, MessageBox, MessageList, MessageToast.
MessageBar	A UI control that presents the messages to the user. The control is configurable as regards the display mode for the messages.
MessageList	A subcontrol of the MessageBar control. It is created on demand and instantiated by the MessageBar control for displaying the messages as a list.
MessageToast	A UI control that displays new incoming messages (one at a time) to the user, on top of the MessageBar.
MessageType	A static class providing the type for Message.
NavigationBar	The NavigationBar provides enhanced navigation capabilities and is the parent control of NavigationItems. It is displayed in the form of a horizontal line with switching markers depending on the currently selected item.
NavigationItem	NavigationItems are used as children of the NavigationBar or of worksetItems of the Shell. Similar to tabs they are used to display different content areas.
Overlay	A UI container control that is displayed in front of the rest of the screen, hiding the content. The content is only shown on user request, for example by clicking on a button.
OverlayContainer	A UI control to be embedded into the Overlay control as a content container.
Padding	A static enumeration class that is used in a MatrixLayout control. It defines where the MatrixLayoutCell content shall be displayed, in relation to the cell border.
Paginator	A UI control that provides graphical support for browsing through the pages of a given page set.
PaginatorEvent	A static class providing the event types for the Paginator control.
pane	A part of the window.
Panel	A container control providing a content area for an arbitrary number of embedded controls, and a header area.
PasswordField	A UI control representing a TextField with masked characters that allows hidden input.
Popup	A helper class that is used for UI controls that are displayed on the screen in a separate mode.
PositionContainer	A subcontrol of the AbsoluteLayout control that is used to position the embedded controls.
ProgressIndicator	A UI control that is used to display the progress of a process in the form of a bar. The bar can be colored and/or show percentage rates.
RadioButton	A UI control that represents a round element and a descriptive text.
RadioButtonGroup	Represents a group of RadioButtons that allow the user to make an interactive choice from a set of options.
RatingIndicator	A UI control that allows the user to rate a certain topic.
RatingIndicatorVisualMode	A static class providing the possible values of float values in a RatingIndicator control.
RichTooltip	A UI control providing an area for displaying field-related long text, an image, and a title.
RoadMap	A UI control that is used to graphically display step-by-step workflows of a clearly defined work process. The subcontrol is RoadMapStep.
RoadMapStep	An element used within the RoadMap control to define the single instances for the steps.
RowRepeater	A control that is used to repeat complex controls. The repeated items are displayed in a stacked list format. The control provides support for sorting, filtering, and paging.
RowRepeaterDesign	Static class determining the visual appearance of the RowRepeater control.
RowRepeaterFilter	An element used by the RowRepeater control to define filters.

RowRepeaterSorter	An element used by the RowRepeater control to define sorters.
ScrollBar	A UI control that allows you to vertically and horizontally navigate large areas in step mode or pixel mode. It has harmonized behavior in different browser versions.
SearchField	A UI control for implementing searches according to keywords. It is displayed in the form of a text field with a search icon.
SearchProvider	A UI control designed to be used in combination with the SearchField control. The SearchProvider provides suggestion lists.
SeparatorItem	A basic class of the core package that represents a visual horizontal separator to be used in complex controls, for example ListBoxes.
Shell	A comprehensive control which provides a user-centered design and enhanced navigation capabilities. It is to be included directly into the body tag of an HTML page and acts as a container for other collaboration controls as well as for controls from packages such as sap.ui.commons or sap.ui.layout, for example. The Shell layout consists of a header area, a work set area, a canvas area, a tool palette, and a pane bar.
Slider	An interactive UI control that is displayed as a horizontal line with a pointer and units of measurement. User interaction fires a change event, this allows you to use the control for various scenarios.
Splitter	A layout control that allows you to design the screen in such a way that it is divided into areas, called panes. The Splitter allows users to change pane sizes interactively.
Tab	A UI control representing a single page in a TabStrip control. A Tab is a container for content and/or for other controls.
TabStrip	A UI control representing a container for Tabs and providing the functionality for switching between the Tabs.
TextArea	A flexible UI control used to display multiple row text; the area can also be interactive, allowing the user to enter text.
TextField	A flexible UI control where the status can be Editable = true or false. Predefined value states can be displayed, including coloring.
TextView	A basic text control for continuous texts, providing various formatting options, designs, and semantic colors for text display.
TextViewColor	Static class providing the semantic color for the text in a TextView control.
TextViewDesign	Static class providing the designs for texts in a TextView control.
Thing	A representation of one or several entities in the SAP system that is intended to match the corresponding entity in a user's working environment.
ThingInspector	A UI control that acts as a container for Things to be displayed, where a Thing is a representation of one or several entities in a SAP system that are intended to best match the corresponding entity in a user's working environment. The ThingInspector frame consists of a header area, a toolbar area, a facet navigation area, and a canvas area. Subcontrols are ThingGroup and ThingAction.
Title	An element that can be used optionally for aggregation with UI controls.
ToggleButton	A control representing a button that can have the state "pressed yes" or "pressed no" (boolean property). A typical usage scenario is to allow the user to switch a certain setting on or off.
Toolbar	A UI control that is used to display a group of items in a single horizontal row. Controls that are typically embedded are Buttons (with text or icon), DropdownBox, or ComboBox. The subcontrol is ToolbarItem.
ToolbarDesign	Static class that provides the design options for the Toolbar control.
ToolPopup	A subcontrol of the Shell control. It represents a popup that can be opened from the Shell’s tool pane. A ToolPopup can have any content.
Tree	A UI control that allows you to display selectable options as nodes, subnodes, and items in a structure hierarchy.
TreeNode	An element that is used with a Tree control to display, select, and toggle single options.
UIArea	A class that represents a placeholder for the controls to be rendered. It is

	responsible for positioning the whole area on the screen. A UIArea can have several root controls.
VerticalLayout	A UI control that allows you to position the embedded elements one below the other.
worksetItem	worksetItems are used in the Shell to provide the main content navigation.

The SAPUI5 Test Suite

The Test Suite is an integral part of the SAPUI5 core library and is thus automatically available for applications using SAPUI5 . To start the Test Suite, just add the URL path 'resources/testsuite' to your application URL, like for instance ' http://localhost:8080/demokit/resources/testsuite' for the 'demokit' application.

Content of the Test Suite

The Test Suite consists of several areas providing functionality related to inspecting and trying all aspects of SAPUI5 controls:

- Left Top - the tree of test pages: different test pages for controls etc. can be selected here
- Center Top - the preview area: the page selected on the left is rendered here and can be used and tested
- Right Top - the control tree: the hierarchy of SAPUI5 controls is displayed here; you can select controls in the tree and they will be highlighted in the preview area
- Right Bottom - the property sheet: once you have selected a control in the tree above, its properties can be inspected here; they can also be modified! Click "Apply Changes" after modifying.
- Center Bottom - the event log: trace output as well as all control events go here
- Left Bottom - preview settings: here you can adjust how the page is rendered, you can e.g. select a theme from the delivered ones, but also enter the name of a theme you created.

Using the SAPUI5 DiscoveryServlet to Automatically Find Test Cases

SAPUI5 is able to detect test cases for controls automatically via its *DiscoveryServlet*. To use this service, the application must configure the servlet in its web.xml like this:

```
<!-- ============================================================= -->
<!-- SAPUI5 discovery servlet used to find available UI test cases -->
<!-- ============================================================= -->

<servlet>
    <display-name>DiscoveryServlet</display-name>
    <servlet-name>DiscoveryServlet</servlet-name>
    <servlet-class>com.sap.ui5.discovery.DiscoveryServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
    <servlet-name>DiscoveryServlet</servlet-name>
    <url-pattern>/discovery/*</url-pattern>
</servlet-mapping>
```

The servlet discovers all HTML files which are located inside the *test-resources/<ui-library-name>* folder of the web application and all resources inside the modules (JAR files) where the location is *META-INF/test-resources/<ui-library-name>*.

The discovery servlet is contained in the following artefact, which needs to be added to the dependencies:

```
<dependency>
      <groupId>com.sap.ui5</groupId>
      <artifactId>utils</artifactId>
      <version>1.11.0</version>
</dependency>
```

Naming Conventions for Control and Application Development

Control API

We recommend to use following naming conventions for control artifacts:

- A control name should start with an upper case letter and use camel case for concatenated words (example: *MatrixLayout*).
- A control property name should start with a lower case letter and use camel case to for concatenated words (example: *colSpan*).
- An control event name should start with a lower case letter and use camel case to for concatenated words (example: *press*).
- A control method should start with a lower case letter and use camel case to for concatenated words (example: *doSomething*).
- A control aggregation should start with a lower case letter and use camel case to for concatenated words.
- A control association should start with a lower case letter use camel case to for concatenated words.
- For a control, properties, events and relations should have unique names to avoid name clashes for generated classes.
- Parameters of control events should start with a lower case letter.

The SAPUI5 tools will check and enforce some of these rules.

For control development, the generated setter/getter-methods for the properties, as well as the generated methods for the events (*attach<EventName>*), for the aggregations and associations will use camel case notation.

SAPUI5 controls have constructors which accept a set of properties and events as JSON string. The framework expects that the names used as keys in a JSON string exactly match the case of the defined control properties and events.

If SAPUI5 controls are exposed as tags, the tag name and its attributes should fit to following convention:

- Tag names should start with a lower case letter, and camel case should be used for concatenated words. Tag names should match the names of the control except tag names start in lower case. This establishes a natural relation between the tag and its control (example: *matrixLayout*).
- Tag attributes should start with lower case letters, and camel case should be used for concatenated words (example: *colSpan*).
- Tag attributes for event handlers should be named *on<event-name>*, all in lower case letters (example: *onpress*).

HTML/DOM related (Renderers, Application Development)

Reserved Characters for Control IDs

The SAPUI5 framework regards the dash '-' as a forbidden character for control IDs. Applications must not use this character in their control IDs.

Control Renderers however can and should use the dash to derive synthetic IDs from control Ids (e.g. by appending a suffix like "-mysuffix" to the control ID). By reserving the '-', naming conflicts with the application can be avoided.

Character Description

-	The minus is reserved as a separator for synthetic IDs

Reserved IDs for DOM nodes/elements

The following table contains a list with reserved IDs by SAPUI5 Core:

ID	Description	
`sap-ui-bootstrap`	ID of the bootstrap script tag	
`sap.ui.core-script`	ID of the bootstrap script tag	*deprecated*
`sap-ui-*`	Prefix for SAPUI5 Core internal created IDs	
`*-r`	Suffix for former UR LightSpeed root areas (similar to `UIArea` of SAPUI5)	

Within the `sap-ui-` namespace the following IDs are currently used:

ID	Description
`sap-ui-library-*`	Prefix for UI libraries script tags
`sap-ui-theme-*`	Prefix for theme stylesheets link tags
`sap-ui-highlightrect`	ID of the highlight rect for controls in TestSuite
`sap-ui-blindlayer-*`	ID for `BlockLayer`
`sap-ui-static`	ID of the static area of SAPUI5
`sap-ui-TraceWindowRoot`	ID of the `TraceWindowRoot`
`sap-ui-xmldata`	ID of the `XML Data Island`

Reserved DOM Attributes

The following table contains a list of reserved DOM element attributes names:

Attribute	Description
data-sap-ui-*	Namespace for custom attibutes of SAPUI5

In general all custom attibutes should be prefixed with `data-`.

Reserved URL Parameters

The following table contains a list of reserved URL parameter names that should be avoided by applications (Applications can use these SAPUI5 parameters. However, they should not use these to name their own parameters):

Attribute	Description
sap-*	Namespace for URL parameters introduced by SAP AG
sap-ui-*	Namespace for URL parameters introduced by SAPUI5

Control CSS

- To avoid collisions, all CSS classes written to the HTML must start with "sap" if developed inside SAP, and with "sapUi" or "sapM" if developed inside the UI5 development teams. Outside the respective groups this prefixes should not be used.
- CSS classes for a certain control should add the control name or an abbreviation of it to the above prefix. Controls outside the "commons" library may benefit from adding also the library name in between, especially if developed outside the core UI5 development teams.
    - The HTML root element of every control should have this CSS class name written, any inner HTML elements requiring a class name should use this class name and add a suffix.

All CSS selectors written by UI5 control developers must refer to at least one such CSS class, pure HTML elements should not be styled (to avoid affecting non-owned HTML)
- Inline CSS should only be used for control instance specific style (like the button width)

Styling Contexts

When the visuals of certain controls are different depending on the context/container where they are used, the suggested pattern to achieve this is using CSS cascades:

- The area/container shall write a certain marker CSS class to the HTML and document this CSS class in its JSDoc. The documentation should mention the purpose and contract/meaning of this class, e.g. that it is meant to modify the appearance of children in a way that better fits table cells, toolbars or headers.
- This CSS class may not have any CSS styles attached, it is a pure marker (such styles would also need to stay stable and might cause trouble for other containers where these styles are not desired)
- The naming convention for these classes is to have the suffix "-CTX", e.g. "sapUiTable-CTX" or "sapMList-CTX" or "sapUiBorderless-CTX". This makes them easily discernible from "normal" CSS class names.
- Controls which want to modify their appearance in such an area shall have CSS where this marker class is used in a cascade and provides the suitable style (e.g. .sapUiTable-CTX .myCompanyInputField { border: none; })

Loading / Initializing SAPUI5 within an HTML Page

To make use of the SAPUI5 framework or its controls, applications must include SAPUI5 into their HTML pages.

Overview

There is a broad range of alternatives, how the SAPUI5 runtime and controls can be integrated into an HTML page. Common to all alternatives is the need to load and execute at least one UI5 specific JavaScript resource. The following table lists the most important resources and describes the use case that they are intended for. The names of the resources are meant relative to the `resources/`folder of an UI5 installation. The concrete base URL depends on the platform and administrator decisions.

Resource	Description
sap-ui-core.js	**standard bootstrap file** that already contains jQuery, jquery-ui-position and a minimal UI5 core. Required files are loaded dynamically using XMLHttpRequests (XHRs)
sap-ui-core-nojQuery.js	intended for applications that bring their own jQuery version with them. It contains the same parts of UI5, but **anything from jQuery is left out** (jQuery & jquery-ui-position)
sap-ui-core-all.js	**single js file containing (nearly) all resources from the sap.ui.core library**. Only a few duplicates (multiple jQuery versions), testing stuff etc. are omitted. When this file is used, for all other libraries the *-all.js file is loaded as well. This reduces the number of JS request to the number of libraries (typically 1..4). Note: for proper encapsulation reasons, the *-all.js files will be renamed in future versions of SAPUI5. The sap-ui-core-all.js will remain as is, but for other libraries the file will use a name relative to the libary, e.g. `sap/ui/commons/library-all.js`. **Applications never should address these files on their own, but let the UI5 runtime load them**. Only sap-ui-core-all.js might be referenced in the bootstrap tag.
sap/ui/core/library-preload.js	similar to the *-all.js file but modules are not parsed and executed immmediately, only on demand. **Applications never must reference this file themselves, it is automatically loaded by UI5** when the configuration option 'preload' is given.
sap-ui-core-lean.js	same outcome and mechanisms as with sap-ui-core.js, but only jQuery and one UI5 file are immediately loaded, the rest is loaded dynamically. Usually not used by most applications, might be removed in the future.
sap-ui5.js	all-in-one file that contains all JS modules from the sap.ui.core, sap.ui.commons, sap.ui.table and sap.ui.ux3 libraries. Using this file as bootstrap file might require a longer load time as the other techniques but requires a single request only. Another drawback is the fixed set of libraries. This file is not available on all platforms, only on those based on the `sapui5.war / sapui5-static.zip` artifacts. E.g. the OSGI/Eclipse versions (`com.sap.ui5.core.jar`) don't contain it.
sap-ui-custom*.js	names reserved for custom merged files created by the application. Note: the proposed naming scheme for these files needs to be adapted in future versions for the same encapsulation reasons as mentioned above.

Standard Variant

The most common variant to load SAPUI5 is to include the sap-ui-core.js file. It is self contained in the sense that it already brings jQuery and some jQuery plugins with it. Applications only need to specify

this single resource in their page, the rest is done automatically by the UI5 runtime.

```
<script
        id="sap-ui-bootstrap"
        src="resources/sap-ui-core.js"
        data-sap-ui-libs="sap.ui.commons"
        data-sap-ui-theme="sap_goldreflection" >
</script>
```

If UI libraries are specified, UI5 loads them all synchronously (see below for an explanation what loading a library means). If a libary requires another libary that has not been mentioned in the configuration, then that required library is automatically loaded as well. The theme configuration is used to load the corresponding stylesheets for all libraries.

Immediately after the above script tag, applications can call most of the UI5 APIs. Especially, they can access the Core APIs or instantiate, configure and place controls. Only access to the DOM must be delayed until the controls have been rendered. This happens at the earliest immediately after the document became ready. Applications can use the attachInitEvent method to be notified about that point in time.

Note: As a further optimization to the standard variant, UI5 in its default configuration automatically activates the `preload=sync` mode (explained below) when running from optimized sources.

Further configuration options can be found in the [runtime configuration]() documentation.

noJQuery Variant

When applications already include jQuery and/or want to use a different version than the one integrated into UI5, they can do so with the `resources/sap-ui-core-noJQuery.js` module. It requires that jQuery and jquery-ui-position have been loaded already.

```
<!-- include some jQuery version -->
<script src="my-jQuery-min.js" ></script>

<!-- application doesn't have its own jquery-ui-position, so it might use the one from UI5
-->
<script src="resources/jquery-ui-position.js" ></script>

<!-- now booting UI5 -->
<script
        id="sap-ui-bootstrap"
        src="resources/sap-ui-core-nojQuery.js"
        data-sap-ui-libs="sap.ui.commons"
        data-sap-ui-theme="sap_goldreflection" >
</script>
```

Preload Variant

With the preload feature, all JavaScript modules for a library are loaded in adavance with a single request. But the code for the modules is not executed in the browser. Only when the module is required by the application, then the runtime executes the code once. Using this preload mechanism helps to significantly reduce the number of roundtrips. To use the preload feature, the 'preload' configuration option can be used. It can have three values:

- **`sync`** loads the modules for all declared libraries in a sychronous manner. After the bootstrap tag has been processed, all preload files for all libraries have been loaded and the libraries have been initialized as usual. Using the 'preload=sync' mode should be transparent to most applications.
- **`async`** loads the modules for all declared libraries in an asynchronous manner. So any code after the SAPUI5 bootstrap tag cannot be sure that the UI5 classes are available already. Applications that want to use the 'preload=async' mode must delay access to UI5 APIs with the help of the Core.attachInitEvent method. Only then it is save to access UI5 APIs. The asynchronous loading is only supported for the declared libraries (those loaded by the UI5 core). Libraries loaded dynamically (using the `sap.ui.getCore().loadLibrary()` API) will be preloaded in `sync` mode.
- **`auto`** this is the default preload mode, if nothing else is configured. It checks whether the UI5 runtime uses optimized sources. If so, it enables the 'sync' mode to further optimize the runtime. When normal or debug sources are used, the preload is deactivated.

Note: Preload sources are always optimized, so using preload in the debug mode of UI5 would be counter productive. Therefore the debug mode always overrides the preload mode.

The easiest way to check the preload feature with an existing application is to specify the `sap-ui-preload=`*mode* parameter in the start URL or to add the `data-sap-ui-preload` attribute to the bootstrap tag:

```
<script
        id="sap-ui-bootstrap"
        src="resources/sap-ui-core.js"
        data-sap-ui-libs="sap.ui.commons"
        data-sap-ui-theme="sap_goldreflection"
        data-sap-ui-preload="sync" >
</script>
```

Note: as the `async` mode requires active cooperation of the application, it is not activated automatically, only by configuration.

Note: the preload mode can be combined with other bootstrap modules as well (e.g. `sap-ui-core-noJQuery`).

all-in-one per Library Variant

The all-in-one variant has a similar request behavior as the preload=sync, but it immediately executes all code for a library. Historically, it is a predecessor of the preload feature and should no longer be used.

sap-ui5

Another predecessor of the 'preload' variant. It loads all classes and controls for the 4 standard UI5 libraries `sap.ui.core`, `sap.ui.commons`, `sap.ui.table`, `sap.ui.ux3` with one single request. While this sounds good at a first glance, it has several disadvantages:

- set of libraries is not extensible. Custom libs don't benefit from this approach. They are loaded using the all-in.one variant before.
- all contained code is executed, which might increase the initial startup time of the application. Depends on other factors like browser, client computer / device etc.
- file is quite huge

Current suggestion is to use the preload variant, if any (remember that the preload is automatically chosen for optimized sources).

Runtime Behavior of Bootstrap

When the SAPUI5 runtime is initialized, the following important steps are executed

- jQuery is enriched with a few plugins (most of them in the namespace `jQuery.sap`). These plugins provide fundamental functionality used by the rest of the UI5 core, namely the modularization concept, a small logging framework, performance measurement etc.
- the global object `sap` is created if it doesn't exist already
- the SAPUI5 Core class (`sap.ui.core.Core`) and all its dependencies are executed.
- the runtime configuration is determined from different sources
- all libraries and modules declared in the configuration as well as their dependencies are loaded
- for each loaded library, the stylesheet for the configured theme is added to the page
- when loading the libraries has finished and when the document becomes ready, the initEvent of the core is fired and all registered handlers are executed

Steps during Library Loading

For each control library that has to be loaded during bootstrap, the runtime loads and interprets the

following additional resources:

- a library bootstrap file `/''context-path''/resources/''library-name''/library.js`

  A JavaScript file that contains the JavaScript code for all enumeration types provided by the library as well as library specific initialization code that is independent from any controls in the library. The file must call the sap.ui.getCore().initLibrary method with an object that describes the content of the library (list of contained controls, elements etc.). For libraries that are developed with the SAPUI5 Toolset or build with the SAPUI5 offline build tools, this file is automatically generated during build.

- a library stylesheet file `/''context-path''/resources/''library-name''/themes/''theme-name''/library.css`

  A standard CSS file that contains all styles relevant for this library. For libraries that are developed with the SAPUI5 Toolset, this file is automatically generated during build.

Dynamic Loading of UI Libraries

Declaring UI libraries in the runtime configuration is just one way to use a library at runtime. Another way is to use the `sap.ui.getCore().loadLibrary()` at runtime to instruct UI5 to load an additional library. After loading the library, all controls from that library can be used as well, but the same restriction as for declared libaries (DOM access only after document.ready and rendering) apply.

Configuration of the UI5 Runtime

When the SAPUI5 bootstrap script is included in a page, the UI5 runtime will automatically be initialized as soon as the script is loaded and executed by the browser. For simple use cases and for a default UI5 installation, this should already be sufficient to build and run UIs. The only additional information that usually is specified, is the set of used UI5 libraries as well as the used theme.

So a typical bootstrap script tag might look like this:

```
<script id="sap-ui-bootstrap"
    type="text/javascript"
    src="resources/sap-ui-core.js"
    data-sap-ui-theme="sap_goldreflection"
    data-sap-ui-libs="sap.ui.commons"
    >
</script>
```

The attributes `data-sap-ui-theme="sap_goldreflection"` and `data-sap-ui-libs="sap.ui.commons"` already provide examples for how the UI5 runtime can be configured to the needs of an application. This page summarizes the different configuration options as well as the different ways to configure the runtime.

Ways to Provide Configuration

Before discussing the different configuration options in detail, the different ways *how* to provide these options to the runtime, need to be explained:

Runtime Defaults

The easiest way to specify a configuration value is *not to specify it*. UI5 runtime defines a default value for each configuration option. When you don't need to change it, you don't need to specify it. See the list of configuration options for the default values.

Individual Script Tag Attributes

As mentioned in the introduction above, you most likely have already used the next way of specifying UI5 configuration options: one or more attributes of the bootstrap script tag. For each configuration option, UI5 accepts a correspondingly named attribute in the bootstrap script tag.

Name of the Attribute

The name of the attribute is derived from the name of the configuration option by prefixing it with `data-sap-ui-`. The first part (`data-`) is necessary to comply with the W3C recommendations for custom attributes in HTML. The second part (`-sap-ui-`) helps to separate UI5 attributes from custom attributes defined by any other framework.

As attribute names in HTML are case-insensitive, the configuration attribute names are case insensitive as well. As you can see in the list of configuration options, UI5 defines some camel case names for configuration options. To use them in attributes, you don't have to convert their names to lowercase. UI5 does this internally when accessing the configuration.

Value

Element attributes in HTML by definition have a *string* value. For configuration options that are of type string, the attribute value is just the value of the option (Note: when the value contains special HTML characters like < or > or when it contains the same quote character that is used to wrap the attribute value, then the usual HTML escaping mechanisms must be used. Either use entities (e.g. < instead of <) or in the case of quotes, just switch the type of quotes (from single to double or vice versa)).

For configuration options that are **not** of type string, the format of the allowed values needs to be defined:

Type	Notation / Values
string	just a string, but escaped according to the HTML conventions
boolean	**true** and **x** are both accepted as true value (case insensitive), all others are false. Nevertheless **false** is prefered for false values
int	any integer value
Array of string	comma separated list of values, comma is not supported in the values (no escaping)
Map from String to String	a JavaScript object literal (preferably JSON syntax)

The list of configuration options also lists the type for each configuration option.

Single, Complex Configuration Attribute

Instead of attaching individual options with individual configuration attributes to the script tag, one can use a single attribute with a more complex structure. The name of the attribute for the complex configuration is `data-sap-ui-config`. It's content is similar to the global configuration object, but without the enclosing parenthesis. In other words: it is a comma separated list of key/value pairs.

Note: again, the usual HTML escaping mechanisms must be used when the value contains characters special to HTML (<,>,&) or the quote character that is used to enclose the attribute value.

```
<script id="sap-ui-bootstrap"
    type="text/javascript"
    src="resources/sap-ui-core.js"
    data-sap-ui-config="theme:'sap_platinum',libs:'sap.ui.commons'"
```

```
    >
</script>
```

Global Configuration Object

Another way of specifying the configuration is to create a property in the global `window` object with property name `sap-ui-config`. The property must be a simple object, where each property represents the configuration option of the corresponding name.

Note: the name of the window property intentionally is not a valid JavaScript identifier. This helps to avoid conflicts with typical JavaScript coding and the structured nature of the name should avoid conflicts with SAP objects. To define the object, quotes must be used.

Note: when a configuration option has a name that is not a valid JavaScript identifier or that is a reserved token in JavaScript, then the property name in the configuration object must be quoted. Currently, there is no such configuration option.

Note: As configuration is evalued during the bootstrap of UI5, the configuration object must be created **before** UI5 is booted. Otherwise the contained configuration could not be evaluated. As a consequenece, using the global configuration object requires just another script tag in front of the bootstrap tag. It is up to the application whether it uses an inline script tag or whether it uses a separate JavaScript file for that purpose which is loaded via a script tag. Using a dedicated file initially requires a bit more work, but has at least two advanteges:

1. the file can be shared by multiple pages that then by design use the same configuration
2. using a file also works with the new Content Security Policy (CSP) mechanism as introduced e.g. by FireFox 4.0 and others.

An example usage with an inline script could look like this:

```
<script type="text/javascript">
    window["sap-ui-config"] = {
        theme : "sap_platinum",
        libs : "sap.ui.commons",
    };
</script>
<script id="sap-ui-bootstrap"
    type="text/javascript"
    src="resources/sap-ui-core.js"
    >
</script>
```

One might wonder why one would use this approach at all. The overhead of an additional script/script tag makes this option less appealing. But as already mentioned, it opens the possibility to share configuration between pages, it can be used in environments where the script tag can not be influenced (e.g. because it is created out of some configuration, like in some meshup frameworks) and it allows to provide configuration before the core boots.

URL Parameters

Another way of specifying configuration options is to add theme as parameters to the start URL of an application. The name of the URL parameters is again derived from the name of the configuration option by prefixing it with `sap-ui-` (Note: the W3C proposed `data-` prefix is not needed and even not allowed here as all URL parameters are kind of custom parameters).

Values of URL parameters are strings as well, so the same type mapping applies as for the HTML attributes. But please be aware of the fact, that URLs require a different encoding than HTML does (use % encoding instead of entity encoding).

For security reasons, only some configuration options can be set via URL parameters. And by setting the `ignoreUrlParameters` option to true, applications can even disable URL configuration parameters completely.

Runtime Configuration Object

All ways explained so far have one thing in common: they are evaluated while the UI5 runtime is booting up. After that, changes to these parameters are ignored. The final configuration result can be read via the sap.ui.getCore().getConfiguration() method.

The same object also provides set methods for a very limited set of configuration options. Such options can be modified at runtime and the runtime and / or the controls can react on the configuration changes. The most prominent (and so far only) example for such a configuration option is the theme.

Order of Significance

Attributes of the DOM reference override the system defaults, URL parameters override the DOM attributes (where empty URL parameters set the parameter back to its system default).

Calling setters at runtime will override any previous settings calculated during object creation.

List of Configuration Options

The following list is a snapshot of the configuration options as they were available at the time when this page has been created. For an always up-to-date list, please check the API documentation for sap.ui.core.Configuration.

Option	Type	Default Value	URL Modifiable at Runtime	Description
				The theme to be used for the current page. Can be changed at runtime by calling sap.ui.getCore().applyTheme(). **Theme Root:** When the theme string contains an at-sign ('@'), anything before the '@' is assumed to denote the

theme	string	*'base'*			id of the theme while anything after the '@' is assumed to represent the URL location of the theme. To defend against XSS attacks, only URLs that point into the SAP Theming Repository on the same server as the application are accepted. If the URL is not valid, the theme id is also ignored.
language	string	*user language*			the language to be used for localized texts, formattings etc. The default value is not static but determined from the browser or user language (`navigator.language`, `navigator.userLanguage`, `navigator.browserLanguage` - in that order).
formatLocale	string	`undefined`			locale to use for formatting purposes, defaults to the locale derived from the language
accessibility	boolean	`true`			Whether the UI5 controls are rendered in accessibility mode. This only affects the addition of ARIA information to the HTML, it does not indicate whether the current user needs accessibility mode or not.
animation	boolean	`true`			whether animations are generally allowed in UI5 controls
rtl	boolean	`false`			whether all controls should be rendered in Right-to-Left (RTL) mode. Not yet determined automatically
debug	boolean	`false`			when set to true, debug sources (unoptimized sources) are loaded. If the bootstrap code is loaded from an optimized source, the bootstrap will be aborted and start anew from a debug source
inspect	boolean	`false`			when set to true, the `sap-ui-debug.js` module is included and provides some supportability features
originInfo	boolean	`false`			when set to true, additional information for text resources is maintained that should allow to trace where a translated text in the UI came from
noConflict	boolean	`false`			whether the UI5 core should already force jQuery into noConflict mode
noDuplicateIds	boolean	`true`			enforces a check in the UI5 runtime that doesn't allows mutliple controls to have the same ID. Highly recommended, as duplicate IDs might cause unforseeable issues and side effects!
trace	boolean	`false`			Activates an overlay div that contains a trace
logLevel	0|1|2|3|4|5|6| NONE|FATAL|ERROR| WARNING|INFO|DEBUG|ALL	`ERROR`			Sets the log level to the given value. For minified (productive) sources, the default level is ERROR, for debug sources it is DEBUG. At runtime, the log level can be modified with jQuery.sap.log.setLevel
modules	string[]	[]			List of JavaScript modules to load after the core has been initialized
areas	string[]	`null`			UIAreas to create in advance. At runtime, new UIAreas can be created with sap.ui.getCore().createUIArea, existing ones can be deleted by sap.ui.getCore().getUIArea(id).destroy()
libs	string[]	`[]`			List of libraries to be loaded initially. Further libs can be loaded via the loadLibary() method of the UI5 runtime.
resourceRoots	object	`undefined`		jQuery.sap.registerModulePath(), sap.ui.localResources()	Locations of resources (Javascript modules, ...). The value can be a map of URL locations keyed by a resource name prefix. All resources whose name starts with such a prefix will be loaded from the corresponding location. The empty string can be used to define a location for all resources.

				See API documentation for jQuery.sap.registerModulePath for details about the underyling mechanism.
themeRoots	object	`undefined`	sap.ui.getCore().setThemeRoot()	Locations of themes, for details see the Theming FAQ
onInit	code	`undefined`		Code to be executed after the core has been initialized
uidPrefix	string	`'__'`		Prefix to be used for automatically generated control IDs. Must be choosen carefully to avoid conflicts with application defined IDs or DOM IDs
ignoreUrlParams	boolean	`false`		Security relevant parameter that allows applications to disable configuration modifications via URL parameters.
weinreServer	string			URL to a WEINRE server to be used for debugging purposes. If set, UI5 will automatically include the WEINRE target module
weinreId	string			
xx-loadAllMode	boolean	`false`		**This is an experimental feature which might be modified or removed in future versions**. Used internally by the runtime. To activate it, load sap-ui-core-all.js instead of sap.ui.core.js
preload	not specified, auto, sync or async	`'auto'`		After loading the UI5 Core runtime, but before loading any further libraries, so called 'preload' files are loaded that contain all modules of a library. The modules are only loaded but not executed which should reduce initialization time. When set to 'auto', the runtime automatically uses 'sync' when running from optimized sources. When set to `sync`, the preload files for the declared libraries are loaded synchronously, when seto to `async` they are loaded asynchronously. for any other value (e.g. blank), the preload feature is deactivated and modules are loaded on demand.
xx-test-mobile	boolean	`false`		**This is an experimental feature which might be modified or removed in future versions**. Activates support for mobile device specific events (like touch events) so that standard UI5 controls can be tested on such devices
xx-fakeOS	string	`undefined`		**This is an experimental feature which might be modified or removed in future versions**. Used to simulate iOS/Android/BlackBerry/Windows Phone on desktop PCs for easier development of mobile apps and controls. Supported values are: "ios", "android" "blackberry", and "winphone". The user-agent will be modified and the complete runtime and theming of the UI5 Mobile library will act like on the selected mobile platform. This includes browser detection mechanisms like `jQuery.browser`. Touch events will also be simulated from the respective mouse events (see xx-test-mobile) **Only works on desktop Webkit browsers (Chrome and Safari) and Internet Explorer 10**, in other browsers and on mobile devices this switch has no effect. Note that IE10 only supports the Windows Phone simulation and the Webkit browsers only support iOS/Android/BlackBerry simulation. This configuration parameter is not handled by the UI5 configuration object and only available as URL parameter and as attribute in the bootstrap script

			tag.
xx-bindingSyntax	string	`'simple'`	**This is an experimental feature which might be modified or removed in future versions**. Whether to use the simple binding syntax or the complex one. Only affects bindings that are defined as strings (e.g. in the constructor of a control or when specifying a binding in a declarative view, like XMLView or HTMLView. See the Databinding Documentation for details.
xx-supportedLanguages	string[]	`[]`	**This is an experimental feature which might be modified or removed in future versions**. With this option the client can be instructed to limit its backend requests for translatable texts to the configured set of languages. An empty value or the value `*` allows any language, the value `default` limits the requests to the set of languages that are delivered with SAPUI5.
xx-preloadLibCss	string[]	`[]`	**This is an experimental feature which might be modified or removed in future versions**. Specifies a list of UI libraries (same syntax as for the `libs` property) for which the UI5 core does **not** include the `library.css` stylesheet in the head of the page. If the list starts with an exclamation mark (`!`), no stylesheet is loaded at all for the specified libs (it is assumed that the application takes care of loading CSS, e.g. a manually merged, single CSS file). Otherwise, the Core instructs the backend to create a merged CSS for the specified libs. In both cases, if the first libraries name is an asterisk (`*`), it will be expanded to the list of already configured libraries. **Note:** The merge feature is currently only available for Java and only for apps that include the additional backend component `resource-ext`. Without the merge, applications can include their own merged CSS file and suppress the loading of the standard library.css

UI Control API: Using JSON Data in UI Control Constructors

This example shows the full power of SAPUI5 UI control constructors.

You can specify a unique identifier as the first argument to the constructor. If you omit it, an identifier is computed automatically by the SAPUI5 framework. Specifying an identifier is optional and allows your application to later on find the control again, e.g. for retrieving the current user input from it. Alternatively, you might keep a reference to the control in a variable.

As second value of the constructor, you can specify a JSON string with all control properties that should be set.

```
var oTextField = new sap.ui.commons.TextField("testTextField",
     {value : "Hello SAPUI5", tooltip: "This is an example tooltip", width: "100px"});
```

The constructor call above is an abbreviation for the following list of statements which are alternatively supported:

```
var oTextField = new sap.ui.commons.TextField("testTextField");
oTextField.setValue("Hello SAPUI5");
oTextField.setTooltip("This is an example tooltip");
oTextField.setWidth("100px");
```

In summary, SAPUI5 control constructors accept the following arguments (in the order specified below):

1. An optional ID of type string, which must be the first argument or omitted altogether.
2. One JSON-like object (object literal) as parameter `mSettings` that can define values for any property, aggregation, association or event
    - If for a control a specific name is ambiguous (a property has the same name as an event), then the framework assumes property, aggregation, association, event in that order.
    - To resolve ambiguities, add an "aggregation:", "association:" or "event:" prefix to the key in the JSON object.

Have a look into the JsDoc of the control to see which parameters are supported.

UI Control API: Event Handling

For controls which can raise events, JS event handlers can be registered. For this, the control API provides *attach<EventName>* methods like shown in the following example:

```
var oButton = new sap.ui.commons.Button("b1", {text:"Go", width:"80px"});
oButton.attachPress(
    function (oControlEvent) {
        alert("You clicked on Button " + oControlEvent.getSource().getId());
    }
);
```

Note that for the Button control the event handler could also be registered via the Button constructor directly.

```
function buttonPressed(oControlEvent) {
        alert("You clicked on Button " + oControlEvent.getSource().getId());
};
var oButton = new sap.ui.commons.Button("b1", {text:"Go", width:"80px", press:buttonPressed});
```

See JSON Constructors for details on control constructors.

SAPUI5 calls events like the Button "press" event semantic events. They do not exist as native HTML browser events. The UI control implementation is responsible for mapping native browser events to semantic events. In case of the Button control, the Button implementation must, for instance, define a mapping of the "onclick" event to the "press" event. This can be done in the Button behavior file by following coding:

```
/**
 * Function is called when button is clicked.
 */
sap.ui.commons.Button.prototype.onclick = function(oBrowserEvent) {
  if (this.getEnabled()) {
    this.firePress({id:this.getId()});
  }
};
```

Introduction to Data Binding

Definition

Data binding is a technique that binds two data/information sources together in order to keep them in sync. When data binding is properly implemented and defined, all changes in one data source are automatically reflected in the other. In the UI, data binding is usually used to bind the UI controls to a data source that holds the application data, so that the controls are updated automatically whenever the application data is changed.

The term "data binding" is also used when changes in the control, such as data being entered by a user, cause updates in the underlying application data. This is called bidirectional or two-way binding.

Purpose

The purpose of data binding in the UI is to separate the definition of the user interface (view), the data that is going to be visualized by the application (model), and the code for the business logic that is processing the data (controller). This is widely known as the MVC (model, view, controller) paradigm. There are several positive aspects of this separation of concerns: In addition to better readability, maintainability, and extensibility, it allows you to change the view without touching the underlying business logic and to define several views of the same data.

Design

While there are many possible data sources in a Web application (JSON, XML, Atom, or OData) that should be suppported, the way in which the binding is defined and the implementation of data binding within the UI controls should be independent of the specific data source. In addition, it should be possible to create a custom model implementation for data sources that are not yet covered by the framework or are domain-specific.

To achieve this, there is a `Model` and a `Binding`. The `Model` instance holds the data and provides methods to set the data or to retrieve data from a server. It also provides a method for creating bindings to the data. When this method is called, a `Binding` instance is created, which contains the binding information and provides an event, which is fired whenever the bound data is changed. An Element can listen to this event and update its visualization according to the new data.

Getting Started

This chapter gives an overview of how to use data binding in conjunction with SAPUI5 controls in a simple application. It provides information about binding data to a property of a control (property binding) as well as binding a collection of values (aggregation/list binding).
It is very simple to use data binding in your application; just perform the the following steps:

1. Decide which model you want to use. Depending on the service or backend type, you may want to choose a different model that supports your backend. Currently, there are model implementations for JSON, XML format, and OData services. It is also possible to implement your own implementation if there is no suitable model for your backend.
2. Create a model instance.
3. Create a control instance.
4. Bind properties or lists of the control to your model.
5. Unbind properties if you wish.

In the following example, we make use of the already available JSON model, which means that the data must be provided in JSON format. This model is not specific to a particular backend type or implementation. The only requirement is that the data for the model is provided in JSON format.

The JSON model supports two-way/bidirectional data binding by default, which means that the model will automatically reflect changes to the view and vice versa.

Defining the Data

You start by creating a simple HTML page and loading the SAPUI5 runtime. The next step is to create the data you want to bind to a control property later on. As we are using the JSON model, you need to provide this data in JSON format. Place the code into your sample page:

```
// JSON sample data
var data = {
    firstName: "John",
    lastName: "Doe",
    birthday: {day:01,month:05,year:1982},
    address:[{city:"Heidelberg"}],
    enabled: true
};
```

Creating a Data Binding Model Instance

Now you create a new JSON data model and add the data you have created to the model, so that the data is stored there and can be used for binding. Finally, you attach the model to the SAPUI5 core so that it can be used by various controls. It is also possible to attach the model to a specific control by calling `oElement.setModel(oModel)`.

```
// create JSON model instance
var oModel = new sap.ui.model.json.JSONModel();
// set the data for the model
oModel.setData(data);
// set the model to the core
sap.ui.getCore().setModel(oModel);
```

Creating controls and property binding

At this point you can create the SAPUI5 controls and define the binding to the properties. In the sample, you first define a TextView control and a TextField control. Both controls should display the `firstName` property of the model. The corresponding control properties have to be bound to the model property. You do this directly in the control constructor by using {} braces and specifying the path to the property in the model. You do this for both controls.

```
// create your controls
```

```
var oTextView = new sap.ui.commons.TextView("textView", {
    // bind text property of textview to firstName property in the model
    text: "{/firstName}",
    tooltip: "First Name"
});
var oTxt = new sap.ui.commons.TextField("txtField", {
    // bind text property of textfield to firstName property in the model
    value: "{/firstName}"
});
```

Next you create a CheckBox control and bind its `checked` property to the `enabled` property in the model. You also do this for the previously created TextField by using an alternative binding notation:

```
// create your controls
var oChkBx = new sap.ui.commons.CheckBox("box", {
    // bind checked property against enabled property in the model
    checked: "{/enabled}",
    change: handleCheckBoxChange
});
// generic bind method bindProperty(control property, model property)
oTxt.bindProperty("enabled", "/enabled");

function handleCheckBoxChange(oEvent){
    var bEnabled = oEvent.getParameter("checked");
    // modify the enabled property value in the model to reflect changes
    oModel.setData({enabled: bEnabled}, true); // true means merge the data instead of
replacing
};
```

The `handleCheckBoxChange` method sets the `enabled` property in the model, depending on the current checked state of the CheckBox.

Finally, you create a Button and define the `update` method for updating the `firstName` value in the model with the value of the TextField when the button is clicked.

```
// create your controls
var oBtn = new sap.ui.commons.Button({
    id: "btn",
    text: "Trigger Change",
    press: update
});

function update(){
    // modify the firstName property value in the model to reflect changes
    oModel.setData({firstName: sap.ui.getCore().byId("txtField").getValue()}, true);
};
```

Now, when you open the sample application in the Web browser, you can see that the value of the `firstName` property is already displayed in the TextView and TextField. When you select the CheckBox, the enabled value is modified in the model (by the `handleCheckBoxChange` method), and the changes are immediately reflected in all control properties that are bound to this property. In this case, the TextField is enabled or disabled.

As described above, clicking the button modifies the `firstName` value in the model with the value of the TextView. When this happens, all control properties bound to this model field are updated immediately to reflect the changes (direction Model → View).

Additionally, since the JSON model supports two-way binding, entering a text value into the TextField will also update the corresponding value in the model, and the TextView will display the changes because it is bound to the same property (direction Model ← View).

The page should look like this:

Aggregation Data Binding in a Simple Application

The above sample covers the binding of single properties to a control property. Now we want to focus on binding a collection of values, for example, binding multiple rows into a table, the values for a ListBox, or the items in a ComboBox.

In this example we use the RowRepeater control and bind it to a collection of data. Again, you start with the definition of data and set it to the model and the model to the core.

```
//create test data
var dataObject = { data : [{index:0, level: "Warning", description: "HAL: I'm sorry, Dave. I'm afraid I can't do that."},
        {index:1, level: "Warning", description: "Windows Boot Manager has encountered a problem."},
        {index:2, level: "Error", description: "Failwhale: Twitter is over capacity"},
        {index:3, level: "Success", description: "Jun 25 12:20:47 pc1h kernel: lp0 on fire"},
        {index:4, level: "Error", description: "Software failure. Press left mouse button to continue. Guru Meditation #00000004,#0000AACB."},
        {index:5, level: "Error", description: "[root@localhost root]# Kernel Panic"},
        {index:6, level: "Error", description: "That does not compute."},
        {index:7, level: "Warning", description: "404 File not found. Stop messing with the URL."},
        {index:8, level: "Success", description: "Blue Screen of Death."},
};
```

```
//create JSON model
var oModel = new sap.ui.model.json.JSONModel();
oModel.setData(dataObject);
sap.ui.getCore().setModel(oModel);
```

The next step covers the creation of the controls and the definition of a single `Message` control, which will be used as a template for all items in the RowRepeater. The Message control will hold the corresponding `level` and `description` values from the model so that these properties are bound to the model:

```
//create the template control that will be repeated and will display the data
var oRowTemplate = new sap.ui.commons.Message("rowTemplate", {
    text : "{description}",
    type : "{level}"
});
```

Once the template is defined, you create the RowRepeater control. The RowRepeater has an aggregation `rows`, which can be any SAPUI5 control. In our case, this is the abovementioned Message control. You create an aggregation or list binding for this `rows` property. To do this, you specify the name of the array in the JSON model that contains the actual data (`path`) and also specify the Message row template:

```
//create the RowRepeater control
var oRowRepeater = new sap.ui.commons.RowRepeater("rowRepeater", {
    design : "Standard",
    numberOfRows : 5,
    currentPage : 1,
    title : oTitle,
    // bind row aggregation
    rows : {path : "/data", template : oRowTemplate}
});
```

The SAPUI5 runtime clones the row template for each entry in the JSON array for the `data` property and also binds the `description` and `level` properties for each message item.

If you start the sample page it will look similar to this:

For more information, see Usage of Data Binding in Applications, which tells you how to enable data binding in controls.

Data Binding in Applications

This page describes how data binding can be used in SAPUI5 Web applications. It describes how to create a model instance, how to assign it to the UI, and how the controls on the UI can be bound to the model.

Creating a Model Instance

The first step when using data binding is to create a model instance and to provide the data for that model. Depending on the requirements and the availability of data formats on the server side, you can either choose one of the existing model implementations or, if none of them suits your needs, create a custom model implementation.

The predefined models available in SAPUI5 are the JSONModel for JSON data, the XMLModel for XML data, the ResourceModel for resource bundle data, and the ODataModel for retrieving data from OData services. The JSONModel, XMLModel, and the ResourceModel are client-side models, so the data of the model is loaded completely and is available on the client, and operations such as sorting and filtering are executed on the client side without further server requests. The ODataModel, on the other hand, is a server-side model, so only the data that is requested by the UI is loaded from the server; any change of binding or list operations require a new request to the server.

The JSONModel and the XMLModel support one-way, two-way (default), and one-time binding modes, whereas the ODataModel currently supports the one-way binding (default) and one-time binding modes. The ResourceModel only supports the one-time binding mode because it deals with static texts only. For more information, see the subchapter Binding Modes.

Assigning the Model to the UI

Before you can bind your controls to your model instance, you have to make it known to the framework. For flexibility and modularization, you can not only define one model for your applications, but different areas in your application can have different models and even single controls can be assigned a model. It is possible to have nested models, for example, a JSONModel defined for the application and an ODataModel for a table control contained in the application.

The mechanism by which a control finds the model it should bind to is to look up the parent hierarchy until it finds a control or UI area that has an assigned model. A control can only bind to one model, so if you have a container control which has a model assigned, all controls contained in this container can only see the local model of the container and are no longer able to bind to the global model.

Global Model

It is possible to define a global model, which can be accessed by all controls from all UI areas, by using the `setModel` method on the SAPUI5 core object. This is useful for simple form applications or demo applications.

```
sap.ui.getCore().setModel(oModel);
```

Control-Specific Model

It may be useful to have a special model for sections within a UI area, for example, inside a panel or for a Table control. In this case, you can use the `setModel` method available on any control:

```
var oTable = sap.ui.getCore().byId("table");
oTable.setModel(oModel);
```

Defining a Binding Path

To address the different properties and lists contained in a model, you have to use a binding path. The binding path defines how a specific node in the hierarchical data tree can be found. A binding path contains a number of name tokens, which are separated using a separator char. In all models provided by the framework, the separator char is the slash "/".

A binding path can either be absolute (starting with a slash), or relative (starting with a name token). A relative binding path is resolved relative to the context of the control that is bound. A context exists either when aggregation binding is used, for each entry of the aggregation, or when the context is set explicitly on a control by using the `setBindingContext` method. Relative bindings that do not have a defined context are resolved relative to the model root.

The syntax is specific to the model used, as explained in the specific model documentation page.

Binding Controls to the Model

To retrieve data from the model for visualization in the UI, the controls have to be bound to the model, using a binding path as explained above. In general, the control itself, all properties of controls as well as all multiple aggregations can be bound to a model.

The element binding allows to bind Elements to a specific object in the model data, which will create a binding context and allow relative binding within the control and all of its children. This is especially helpful in master/detail scenarios.

The property binding allows properties of the control to get automatically initialized and updated from model data. You can only bind control properties to model properties of a matching type, or you use a formatter function or datatype to convert the data as needed.

Finally the aggregation binding can be used to automatically create child-controls according to model data. This can be done either by cloning a template control or by using a factory function. Aggregations can only be bound to lists defined in the model, that is, to arrays in a JSONModel or a collection in the ODataModel.

Binding Modes

A model implementation supports different binding modes. The available binding modes are:

- `One Way` → The model implementation supports one-way binding, which means from the model to the view (value changes in the model will update all corresponding bindings and the view).
- `Two Way` → The model implementation supports two-way binding, which means from the model to the view and from the view to the model (value changes in the model and in the view will update all corresponding bindings and the view and model, respectively).
- `One Time` → The model implementation supports one-time binding, which means from model to view once.

The JSONModel and the XMLModel support `One Way`, `Two Way`, and `One Time`, and the ODataModel supports `One Way` and `One Time` binding modes:

Model	One Way	Two Way	One Time
Resource Model	-	-	X
JSON Model	X	X	X
XML Model	X	X	X
OData Model	X	X	X

The available binding modes are defined in `sap.ui.model.BindingMode`:

JS Doc: Binding Modes

Default Binding Mode of Models

When a model instance is created, the instance has a default binding mode. This means that all bindings bound to this model instance will have this binding mode as their default.

For the supported binding modes and the default binding mode for each model implementation, see the summary below:

Model	Default binding mode
Resource Model	One Time
JSON Model	Two Way
XML Model	Two Way
OData Model	One Way

The default binding mode can be changed directly after model creation by calling `setDefaultBindingMode` on the model:

```
var oModel = new sap.ui.model.json.JSONModel();
oModel.setDefaultBindingMode(sap.ui.model.BindingMode.OneWay);
```

In the example above, all new bindings for the model will have the one-way binding mode by default. Of course, you can only set supported binding modes as the default binding mode. It is possible to check if a binding mode is supported:

```
var oModel = new sap.ui.model.json.JSONModel();
if (oModel.isBindingModeSupported(sap.ui.model.BindingMode.OneTime)) { // true
    oModel.setDefaultBindingMode(sap.ui.model.BindingMode.OneTime);
}
```

Multimodel Support

It is possible to set an additional model for an element or the core by specifying a name for the model, which identifies it:

```
var oModel = new sap.ui.model.json.JSONModel();
// define data for that model
oModel.setData({
            test: "Test",
            enabled: true
});
// define a name for that model
sap.ui.getCore().setModel(oModel, "myModel");
```

You can then create property bindings for this model by calling:

```
oText.bindValue("myModel>/test").
```

You also can create aggregation bindings for this model by calling:

```
oListItem.bindProperty("text", "myModel>text");
oComboBox.bindItems("myModel>/items", oListItem);
```

First you specify the model name, and after the `>` sign you specify the binding path to the values in the model that is to be bound.

Note:

- It is possible to create a model with an identifying name without first creating a model without a name.

An example can be found here:

Multimodel example with two models

ResourceModel

The ResourceModel is used as a wrapper for resource bundles. You can, for example, bind texts of a control to language-dependent resource bundle properties.

Creating the Model Instance

A ResourceModel can be instantiated with a `bundleName`, which is the name of a resource bundle and equals a SAPUI5 module name within the require/declare concept (see modularization concept), or a `bundleUrl`, which points to a resource bundle. When using the bundle name, the file must have the `.properties` suffix. If no `locale` is given, the login language will be used. For more information about localization and resource bundles, see here.

```
var oModel = new sap.ui.model.resource.ResourceModel({bundleName:"myBundle",locale:"en"});
```

Note: In this ResourceModel implementation you cannot pass parameters to your texts within the resource bundle. If you have to pass parameters, you must do this on your own. Therefore, you can load the bundle yourself or retrieve it from the model.

```
var myBundle = oModel.getResourceBundle();
```

After the instance has been created, you have a model containing the resource bundle texts as data.

For a complete overview of the available methods and parameters, see the API reference.

Binding Path Syntax

The ResourceModel has a simple binding path syntax, as it contains only a flat list of properties. Here is a simple ResourceModel that illustrates the possible binding paths:

Resource bundle content:

```
CLOSE_BUTTON_TEXT=Close
OPEN_BUTTON_TEXT=Open
CANCEL_BUTTON_TEXT=Cancel
...
```

Binding paths within the model:

```
CLOSE_BUTTON_TEXT
OPEN_BUTTON_TEXT
CANCEL_BUTTON_TEXT
```

JSONModel

The JSONModel can be used to bind controls to JavaScript object data (usually serialized in the JSON format). It is a clientside model, so it is meant for small datasets, which are completely available on the client, it does not contain any mechanisum for server based paging or loading of deltas. It does support TwoWay-Binding.

Creating the Model Instance

The JSONModel is used for JSON data. You can create a model instance like this:

```
var oModel = new sap.ui.model.json.JSONModel();
```

After the instance has been created, there are different ways to get your data into the model. The easiest way is to set data by using the `setData` method:

```
oModel.setData({
    "firstName": "Peter",
    "lastName": "Pan"
});
```

Please note the correct JSON notation which uses double quotes for the keys and string values.

Usually, you do not define your data inline in the application, you load it from a server-side service using an XHR request. For convenience, the JSONModel also has a `loadData` method, which loads the JSON data from the specified URL asynchronously and applies it to the model:

```
oModel.loadData("data.json");
```

For a complete overview of the available methods and parameters, see the API reference.

Binding Path Syntax

The JSONModel has a simple binding path syntax, as it consists only of named objects, which are either properties, arrays, or nested objects. Here is a simple JSONModel that illustrates the different binding paths:

```
{
    "company": {
        "name": "Treefish Inc",
        "info": {
            "employees": 3
        },
        "contacts": [
            {
                "name": "Barbara",
                "phone": "873"
            },
            {
                "name": "Gerry",
                "phone": "734"
            },
            {
                "name": "Susan",
                "phone": "275"
            }
        ]
    }
}
```

Absolute binding paths within this model:

```
/company/name
/company/info/employees
/company/contacts
```

Relative binding paths within the "/company" context:

```
name
```

```
info/employees
contacts
```

Relative binding paths within an aggregation binding of "/company/contacts":

```
name
phone
```

Custom sorting and filtering

As all data is available on the client, sorting and filtering are also implemented in JavaScript. This allows the usage of custom sort- and filter-methods in the JSONModel. To define custom methods, set the `fnCompare` method on the Sorter object or the `fnFilter` method on the Filter object after creating it.

The `fnFilter`-method of the Filter gets the value to test as the only parameter and returns, whether the row with the given value should be filtered or not.

```
var oFilter = new sap.ui.model.Filter("property");
oFilter.fnFilter = function(value) {
    return (value > 100);
};
```

The `fnCompare`-method of the Sorter gets the two values to compare as parameters and returns -1, 0 or 1, dependent which of the two values should be ordered before the other.

```
var oSorter = new sap.ui.model.Sorter("property");
oSorter.fnCompare = function(value1, value2) {
    if (value1 < value2) return -1;
    if (value1 == value2) return 0;
    if (value1 > value2) return 1;
};
```

XMLModel

The XMLModel allows to bind controls to XML data. It is a client-side model, so it is meant for small datasets, which are completely available on the client, it does not contain any mechanism for server based paging or loading of deltas. It does support TwoWay-Binding.

Creating the Model Instance

The XMLModel can be used for all kinds of XML data. To create a model instance, you have call the constructor:

```
var oModel = new sap.ui.model.xml.XMLModel();
```

The XMLModel also has a `setData` method, but because XML documents cannot be embedded inline in JavaScript?, it takes an XML document reference as a parameter.

```
oModel.setData(oXMLDocument);
```

If you want to create inline XML data, or for some reason you get XML data as a string, the XMLModel provides a `setXML` method, which takes XML in text format, and uses the XML parser in the browser to create a document from it.

```
oModel.setXML("<?xml version=\"1.0\"?><some><xml>data</xml></some>");
```

Usually, you load your data from the server using an HTTP-based service, so the `loadData` method provides an easy way to load XML data from the given URL:

```
oModel.loadData("data.xml");
```

For a complete listing of the available methods and parameters, see the API reference.

Binding Path Syntax

In the XMLModel, binding gets a little more complicated. XML differentiates between attributes and content, and XML does not have arrays, but lists are defined as multiple elements with the same name. So, for attributes, there is a special selector using the "@" character, and "text()" can be used to reference the content text of an element, and lists are referenced using the path **to the multiple element**. Please note that for the XML model **the root must not be included in the path**.

```
<companies>
  <company name="Treefish Inc">
    <info>
         <employees>3</employees>
    </info>
    <contact phone="873">Barbara</contact>
    <contact phone="734">Gerry</contact>
    <contact phone="275">Susan</contact>
    <locations>
      <location>New York</location>
      <location>Moscow</location>
    </locations>
  </company>
</companies>
```

Absolute binding paths within this model:

```
/company/@name
/company/info/employees
```

Relative binding paths within the "/company" context:

```
@name
info/employees/text()
```

Relative binding paths within an aggregation binding of "/company/contact":

```
text()
@phone
```

Important: in a similar JSONModel one would use `/companies/company/locations` as binding path for the "locations" collection. In an XMLModel the respective collection binding path is: `/company/locations/location`

XML Namespace support

The XMLModel does support documents which are using XML namespaces. For this purpose namespaces must be declared using the "setNameSpace"-method. The namespace prefixes do not necessarily need to be the same as in the XML document, they only used in the binding paths which are used to address nodes in the document.

Assumed this sample XML document:

```
<data xmlns="http://tempuri.org/base" xmlns:ext="http://tempuri.org/ext">
    <ext:entry id="1" value="foo" />
    <ext:entry id="1" value="foo" />
</data>
```

The Namespaces must be declared in the JavaScript like this, to be able to bind to them:

```
var oModel = new sap.ui.model.xml.XMLModel(oXMLDoc);
oModel.setNameSpace("http://tempuri.org/base");
oModel.setNameSpace("http://tempuri.org/ext", "e");
[...]
oTable.bindRows("/e:entry");
```

Custom sorting and filtering

As all data is available on the client, sorting and filtering are also implemented in JavaScript. This allows the usage of custom sort- and filter-methods in the XMLModel. To define custom methods, set the "fnCompare" method on the Sorter object or the "fnTest" method on the Filter object after creating it.

The "fnTest"-method of the Filter gets the value to test as the only parameter and returns, whether the row with the given value should be filtered or not.

```
var oFilter = new sap.ui.model.Filter("property");
oFilter.fnFilter = function(value) {
    return (value > 100);
};
```

The "fnCompare"-method of the Sorter gets the two values to compare as parameters and returns -1, 0 or 1, dependent which of the two values should be ordered before the other.

```
var oSorter = new sap.ui.model.Sorter("property");
oSorter.fnCompare = function(value1, value2) {
    if (value1 < value2) return -1;
    if (value1 == value2) return 0;
    if (value1 > value2) return 1;
};
```

ODataModel

The ODataModel enables binding of controls to data from OData services. The ODataModel is a server-side model, so the whole dataset is only available on the server, the client only nows the currently visible rows and fields. This also means sorting and filtering on the client is not possible, the client has to send a request to the server to accomplish these tasks.

The default binding mode of the ODataModel is OneWay, but there is experimental write-support implemented, so TwoWay binding can be enabeld.

Note: Please be aware of the Same-Origin-Policy security concept which prevents access to backends on different domains/sites.

Creating the Model Instance

The constructor has the service URL as its first parameter. One ODataModel instance can only cover one OData service; if you need access to multiple services, you have to create multiple instances of the ODataModel.

```
var oModel = new
sap.ui.model.odata.ODataModel("http://services.odata.org/Northwind/Northwind.svc/");
```

Requests to the service to fetch data are made automatically according to the data bindings defined in the controls.

For additional methods and parameters, see the API reference.

Binding Path Syntax

The data provided by the ODataModel can be accessed according to the structure of the OData service as defined in the metadata of a service. The syntax of the binding path matches the URL path relative to the service URL used in OData to access specific entries or collections. URL parameters, like filters, cannot be added to a binding path. The following samples of bindings within the ODataModel are taken from the well-known Northwind demo service.

Absolute binding paths:

```
/Customers
/Customers('ALFKI')/Orders
```

Relative binding paths within a Customer context

```
CompanyName
Address
Orders
```

Relative binding paths within an aggregation binding of "/Customer('ALFKI')/Orders:

```
OrderID
ShipName
ShipAdress
Employee/LastName
Order_Details/Discount
```

Adding additional parameters

When using OData services, a lot of things can be configured using URL parameters. SAPUI5 does set most needed URL parameters automatically, according to the binding used. There are two possibilities to add custom parameters to the request: URL parameters can be appended to the service URL and a map of parameters can be passed when using bindElement or bindAggregation.

Additional parameters, wich are added to the OData service URL, will be included with every request

sent to the OData server. This may useful for authentication tokens or general configuration options.

```
var oModel = new sap.ui.model.odata.ODataModel("http://myserver/MyService.svc/?
custom1=value1&custom2=value2");
```

There are parameters, which do not make sense to be included with every request, but should only be added to specific aggregation or element bindings, e.g. $expand or $select. For this purpose the binding methods do have a possibility to pass a map of parameters, which are then included to all requests for this specific binding. Currently the ODataModel supports $expand and $select only.

Expand Parameter

To use the $expand parameter see the following snippet below:

```
oControl.bindElement("/Category(1)", {expand: "Products"});

oTable.bindRows({
    path: "/Products",
    parameters: {expand: "Category"}
});
```

In the first example all products of `Category(1)` will be embedded inline in the server response and loaded in one request. In the second example the Category for all Products is embedded inline the response for each Product.

Select Parameter

The $select parameter returns only the specified properties of requested entries.

```
oControl.bindElement("/Category(1)", {expand: "Products", select: "Name,ID,Products"});

oTable.bindRows({
    path: "/Products",
    parameters: {select: "Name,Category"}
});
```

In the first example the properties `Name` and `ID` of`Category(1)` and all properties of the embedded Products are returned. In the second example the properties `Name` and `Category` are included for each Product. The `Category` property will contain a link to the related Category entry.

Custom Headers

It is possible to add custom headers which should be sent with each request. This can be done by providing a map of headers in the `ODataModel` constructor or by using the `setHeaders` function:

```
// option 1
new sap.ui.model.odata.ODataModel(sServiceUrl, bJSON, sUser, sPassword, {"myHeader1" :
"value1", "myHeader2" : "value2"});

// option2
oModel.setHeaders({"myHeader1" : "value1", "myHeader2" : "value2"});
```

Please note that when adding custom headers all previous custom headers will be removed if not specified again in the headers map.

Custom Query Options

Since version 1.14 it is possible to add custom query options which are placed as a query string in the URL. The name of a query string does not begin with a $ character.

http://services.odata.org/OData/OData.svc/Products?x=y

Custom query options can also be used as input parameters for service operations.
When creating the aggregation binding you can specify these custom parameter as follows:

```
oTable.bindRows({path: "/Products",
```

```
                parameters: {
                      custom: {
                          param1: "value1",
                          param2: "value2"
                      }
                },
                template: rowTemplate
            });
```

When using bindElement you can also specify custom parameters as follows:

```
oTextField.bindElement("/GetProducts", {
                      custom: {
                         "price" : "500"
                      }
            });
```

Binding Properties

There are two ways to define a property binding on a control: either within the settings object in the constructor of a control, or later on using the `bindProperty` method of a control. Once a property binding is defined, the property will be updated automatically whenever the bound model property value is changed.

The most convenient way to define a property binding, which is sufficient in most cases, is to include the binding path within curly brackets as a string literal in the `settings` object:

```
var oTextField = new sap.ui.commons.TextField({
    value: "{/company/name}"
});
```

Alternatively, there is an extended syntax for property bindings. This allows you to define additional binding information, such as a formatter function to be contained in the `settings` object. In this case, a JS object is used instead of a string literal. This must contain a path property containing the binding path and can contain additional properties:

```
var oTextField = new sap.ui.commons.TextField({
    value: {
        path: "/company/name",
        mode: sap.ui.model.BindingMode.OneWay
    }
});
```

Depending on the use case, it can be useful to define the binding at a later time, using the `bindProperty` method:

```
oTextField.bindProperty("value", "/company/name");
```

It does also allow the usage of the same object-literal as in the constructor to define the binding

```
oTextField.bindProperty("value", {
    path: "value",
    type: new sap.ui.model.type.Integer()
});
```

Some controls offer convenience methods for the main properties of a control that are most likely to be bound by an application. This can be done as described in Data Binding in Custom Controls.

```
oTextField.bindValue("/company/name");
```

To remove a property binding, you can use the `unbindProperty` method. This is done automatically whenever a control is destroyed.

```
oTextField.unbindProperty("value");
```

For a complete list of supported parameters, see API reference.

Formatting Property Values

Often the values in the data are represented in some internal format and need to be converted to an external format for visual representation. This is especially necessary for numbers, dates and times, which have locale dependent external formats. SAPUI5 does provide two different means for doing these type of conversion: Formatter functions allow one-way conversion only, while usage of data types can also be used for TwoWay-Binding as they cannot only format, but also parse user input. You can chose freely for each binding, depending on your scenario, the one or the other might be the better match.

The formatter function can either be passed as a third parameter to the bindProperty method or contained in the binding info with the key "formatter". It has a single parameter `value`, which is the value which needs to be formatted to an external representation, and is executed as a member of the control, so can access additional control properties or model data.

```
oTextField.bindProperty("value", "/company/title", function(sValue) {
```

```
    return sValue && sValue.toUpperCase();
});
```

```
oControl = new sap.ui.commons.TextField({
    value: {
        path:"/company/revenue",
        formatter: function(fValue) {
            if (fValue) {
                return "> " + Math.floor(fValue/1000000) + "M";
            }
            return "0";
        }
    }
})
```

The formatter function, as it can contain any JavaScript, can not only be used for formatting a value, but can also do type conversion or calculate results from a given value, e.g. to show a special traffic light image depending on a boolean value:

```
oImage.bindProperty("src", "/company/trusted", function(bValue) {
    return bValue ? "green.png" : "red.png";
});
```

Using Data-Types

The type system provides the possibility to format and parse data, as well as validating if the entered data is within defined constraints. You can define a type to be used for a property binding, by passing it as a third parameter in `bindProperty` or by adding it to the binding information with the key "type".

```
oTextField.bindProperty("value", "/company/title", new sap.ui.model.type.String());
```

```
oControl = new sap.ui.commons.TextField({
    value: {
        path:"/company/revenue",
        type: new sap.ui.model.type.Float({
            minFractionDigits: 2,
            maxFractionDigits: 2
        })
    }
})
```

You can define custom types by inheriting from `sap.ui.model.SimpleType` and implementing the three methods `formatValue`, `parseValue` and `validateValue`. `formatValue` will be called whenever the value in the model is changed to convert it to the type of the control property it is bound to and may throw a `FormatException`. `parseValue` is called, whenever the user modified a value in the UI and the change is transported back into the model. It may throw a `ParseException` in case the value cannot be converted. In case of successful parsing `validateValue` is called to check additional constraints, like minimum or maximum value, and throws a `ValidateException` in case constraints are violated.

```
sap.ui.model.SimpleType.extend("sap.test.PLZ", {
    formatValue: function(oValue) {
        return oValue;
    },
    parseValue: function(oValue) {
        return oValue;
    },
    validateValue: function(oValue) {
       if (!/^(\d{5})?$/.test(oValue)) {
            throw new sap.ui.model.ValidateException("PLZ must have 5 digits!");
        }
     }
});
```

Binding Mode

By default, all bindings of a model instance have the default binding mode of the model. You can change this behavior. When creating a `PropertyBinding`, you can specify a different binding mode, which is then used exclusively for this specific binding. Of course, a binding can only have a supported binding mode of the model.

```
var oModel = new sap.ui.model.json.JSONModel();
// default binding mode is Two Way
oModel.setData(myData);
```

```
sap.ui.getCore().setModel(oModel);
var oText = new sap.ui.commons.TextField();
// bind value property one way only
// propertyname, formatter function, binding mode
oText.bindValue("/firstName", null, sap.ui.model.BindingMode.OneWay);
oText.placeAt("target1");
oText = new sap.ui.commons.TextField();
// bind value property two way which is the default
oText.bindValue("/firstName");
oText.placeAt("target2");
}
```

In the example above, two TextFields are created and their `value` property is bound to the same property in the model. The first TextField binding has a one-way binding mode, whereas the second TextField has the default binding mode of the model instance, which is two-way. So, when text is entered in the first TextField, the value will not be changed in the model. This happens only if text is entered in the second TextField. Then, of course, the value of the first TextField will be updated as it has a one-way binding, which means from model to view.

See an example here:

Two Way data binding example with JSON Model

A more complex example with two tables can be found here:

Two Way data binding example with JSON Model and two tables

Binding Aggregations

The aggregation binding can likewise be defined either in the settings object in the constructor, or with the separate `bindAggregation` method call. The aggregation binding requires a template to be defined, which is cloned for each bound entry of the list. For each clone that is created, the binding context is set to the respective list entry, so that all bindings of the template are resolved relative to the entry. The aggregated elements will be destroyed and recreated whenever the bound list in the data model is changed.

Controls that are able to visualize large data sets, like the Table or the RowRepeater, do not create clones for all entries of the list, but just for the currently visible entries. This is done using a custom `updateAggregation` method as described in Data Binding in Custom Controls.

To bind an aggregation, you first have to create a template or provide a factory function, which is then passed when defining the aggregation binding itself. In the `settings` object, it looks like this:

```
var oItemTemplate = new sap.ui.core.ListItem({text:"{name}"});
var oComboBox = new sap.ui.commons.ComboBox({
    items: {
        path: "/company/contacts",
        template: oItemTemplate
    }
});
```

The aggregation binding can also be defined using the `bindAggregation` method of a control, as follows:

```
oComboBox.bindAggregation("items", "/company/contacts", new
sap.ui.core.ListItem({text:"{name}"}));
```

Some controls have a typed binding method for aggregations that are likely to be bound by the application:

```
oComboBox.bindItems("/company/contacts", oItemTemplate);
```

To remove an aggregation binding, you can use the `unbindProperty` method. This is done automatically whenever a control is destroyed.

```
oComboBox.unbindAggregation("items");
```

For more information, see API reference.

Using Template Controls

Using template controls within the aggregation binding is usually the right choice for structured data, where you have lists of entries with the same properties. A template is not necessarily a single control, as in the items above, but can be a tree of controls. For each entry in the list, a deep clone of the template will be created and added to the bound aggregation.

```
var oMatrixLayout = new sap.ui.commons.layout.MatrixLayout();

var oRowTemplate = new sap.ui.commons.layout.MatrixLayoutRow({
    cells: [
        new sap.ui.commons.layout.MatrixLayoutCell({
            content: new sap.ui.commons.Label({text:"Name:"})
        }),
        new sap.ui.commons.layout.MatrixLayoutCell({
            content: new sap.ui.commons.TextView({text:"{name}"})
        })
    ]
});

oMatrixLayout.bindAggregation("rows", "/company/contacts", oRowTemplate);
```

Using Factory Functions

The factory function is the more powerful approach to creating controls from model data. Here the factory function is called for every entry in the list to create the controls necessary to represent the

current entry. The developer can decide freely, whether it is the same control with different properties set or even a completely different control for each entry.

The factory function gets the parameters `sId`, which must be used as they ID for the created control and the context-object `oContext`, which allows accessing data from the list entry. It must return an instance of `sap.ui.core.Element`.

```
oContainer.bindAggregation("content", "/company/properties", function(sId, oContext) {
    var value = oContext.getProperty("value");
    switch(typeof value) {
        case "string":
            return new sap.ui.commons.TextField(sId, {
                value: {
                    path: "value",
                    type: new sap.ui.model.type.String();
                }
            });
        case "number":
            return new sap.ui.commons.TextField(sId, {
                value: {
                    path: "value",
                    type: new sap.ui.model.type.Float();
                }
            });
        case "boolean":
            return new sap.ui.commons.CheckBox(sId, {
                checked: {
                    path: "value"
                }
            });
    }
});
```

Sorting and Filtering for Aggregation Binding

You can provide initial sorting and filtering when specifying the aggregation binding. Proceed as follows:

- Define a sorter and/or filters:

```
var oSorter = new sap.ui.model.Sorter("name", true); // sort descending
var oFilter1 = new sap.ui.model.Filter("name", sap.ui.model.FilterOperator.StartsWith, "M");
var oFilter2 = new sap.ui.model.Filter("name", sap.ui.model.FilterOperator.Contains, "Paz");
var oFilter3 = new sap.ui.model.Filter("name", sap.ui.model.FilterOperator.BT, "A","G"); //
name between A and G
```

- Hand the sorter and/or filters to the aggregation binding:

```
var oComboBox = new sap.ui.commons.ComboBox({
    items: {path:"/company/contacts", template:oItemTemplate, sorter:oSorter,
filters:[oFilter1,oFilter2,oFilter3]}
});
```

You can also use the other aggregation binding possibilities (for example, `bindAggregation`, `bindItems`) and provide the sorter and filters as parameters.

To sort/filter data manually after the aggregation binding is complete you can do so by getting the corresponding binding and calling the sort/filter function:

```
var oSorter = new sap.ui.model.Sorter("name", true); // sort descending
var oFilter1 = new sap.ui.model.Filter("name", sap.ui.model.FilterOperator.StartsWith, "M");
var oFilter2 = new sap.ui.model.Filter("name", sap.ui.model.FilterOperator.Contains, "Paz");
var oFilter3 = new sap.ui.model.Filter("name", sap.ui.model.FilterOperator.BT, "A","G"); //
name between A and G

// manual sorting
oTable.getBinding("rows").sort(oSorter);

// manual filtering
oTable.getBinding("rows").filter([oFilter1,oFilter2,oFilter3],
sap.ui.model.FilterType.Application);
oComboBox.getBinding("items").filter([oFilter1,oFilter2,oFilter3],
sap.ui.model.FilterType.Application);
```

For manual filtering you should always pass the FilterType. There are two Types of FilterTypes. Application filters should only be used by the application to modify filtering. Control filters are changed

by controls (for example the Table) to modify the displayed data. Control filters should not be modified by the application.

To get the binding of a control you have to know the name of the aggregation which is for example 'rows' for the table control.

For more information about the various sorting and filter methods and operators, see the JS Doc API:

Sorter

Filter

Filter operators

Binding Elements

Binding an element allows to set the binding context of the element to the object referenced by the given binding path. So all relative bindings within the control and all of its children are then resolved relative to this object. For server side models the element binding is triggering the request to the server to load the referenced object in case it is not yet available on the client. Element bindings can also be defined relatively. In this case it will update its binding context whenever the parent binding context is changed.

To define an element binding use the method `bindElement` on any control.

```
oControl.bindElement("/company");
oControl.bindProperty("value", "name");
```

This is interesting especially for containers or layouts, where a lot of controls are contained, which are all visualizing properties of the same model object.

```
var oMatrixLayout = new sap.ui.commons.layout.MatrixLayout();
oMatrixLayout.bindElement("/company");
oMatrixLayout.createRow(
    new sap.ui.commons.Label({text: "Name:"}),
    new sap.ui.commons.TextField({value: "{name}"})
);
oMatrixLayout.createRow(
    new sap.ui.commons.Label({text: "Revenue:"}),
    new sap.ui.commons.TextField({value: "{revenue}"})
);
oMatrixLayout.createRow(
    new sap.ui.commons.Label({text: "Employees:"}),
    new sap.ui.commons.TextField({value: "{employees}"})
);
```

Setting a New Context for the Binding (Master Detail)

You can create a new binding context for an element that is used to resolve bound properties or aggregations relative to the given path. This method is useful if the existing binding path changes or was not provided in the first place (for example, master detail scenarios).

Below is a simple example of how to use it:

```
var data = {clients:[{firstName:"Donald", lastName:"Duck"}]};
...// create and set model here
var oLabel = new sap.ui.commons.Label("myLabel");
oLabel.bindProperty("text", "firstName");
oLabel.bindElement("/clients/0");
```

At first, the `bindProperty` method binds the text value of the label to the `firstName` property in the model. This will not resolve because the path to this property in the model is not set. To do this, you use the `bindElement` method, which creates a new context from the specified relative path so that the binding can be resolved.

The current binding context can be removed by calling the `unbindElement` method on the label. All bindings will now resolve relative to the parent context again.

You can find the complete working example here

It is also possible to use the `bindElement` method in conjunction with the aggregation binding:

```
var data = {clients:[
                        {firstName:"Donald", lastName:"Duck"},
                           items: [ {name: "iPhone"}, {name:"iPad"} ]
                        }
            ]};
...// create and set model here
var oLB = new sap.ui.commons.ListBox("myLb", {height:"100px"});
var oItemTemplate = new sap.ui.core.ListItem();
oItemTemplate.bindProperty("text", "name");
oLB.bindAggregation("items", "items", oItemTemplate);
oLB.bindElement("/clients/0");
```

In the example above, the ListBox is used to display the detailed data (items data) for a specified client. Another control can now be used to show the master data (client data) as was done in the first example with the Label. The example only works if the additional detailed data in the model - which should be displayed in the ListBox (items in the example) - is nested in the corresponding parent data (the client in the example). As you can see in the model, the items are contained in the client data.

The full master/detail sample can be found here.

It is also possible to use a filtering function, for example in a table, to display detailed data for selected master data. In this case, nested data is not necessary in the model. Here is another example using two tables showing clients and their corresponding orders:

Master/Detail example with two tables

Usage of the data binding type system

Data binding supports the definition of types which can be handed over when binding properties. These types define the data type for properties in the model and the values are then automatically formatted for displaying them in the UI. Additionally input values in UI controls are parsed and converted back to the defined type in the model. If an error occurs during formatting/parsing a `sap.ui.model.FormatException`/`sap.ui.model.ParseException` occurs.

All types inherit from the abstract `sap.ui.model.Type` class. A subclass of this class is `sap.ui.model.SimpleType` (see JS Doc). Currently available types inherit from class `SimpleType`.

For each `SimpleType` you can in general define the following parameters in the constructor:

- **format options**: Format options define how a value is formatted and displayed in the UI.
- **constraints**: This is optional. Constraints define how an input value entered in the UI should look like. When parsing the value will be validated against these constraints. For example a `String` type has a constraint for `maxLength` and `minLength` which are automatically validated when parsing the input values.

See the following documentation for available constraints and format options for each type.

Usage of data types

Here is a simple example of how to use data types:

A `sap.ui.model.type.Float` type is created with the formatting options minimum/maximum fraction digits equals 2 and a maximum value constraint of 10:

```
// creating of a float type with 2 format options and one constraint
var oFloat = new sap.ui.model.type.Float(
  {
     minFractionDigits: 2,
     maxFractionDigits: 2
  },
  {
     maximum: 10
  }
);
```

To use the defined data type for the binding you will do the following:

```
// specify the binding and the type directly in the control constructor
var oText = new sap.ui.commons.TextField({value: {path: "/sliderValue", type: oFloat}});
// or alternatively do it afterwards
oText.bindValue("/sliderValue", oFloat);
```

Input parsing and output formatting will now be carried out automatically. So let's say that the float value in the model is 2.3345. The text field will then display the value 2.33. Valid values entered by the user must be lower than 10 and are validated after user entry.

A full example can be found here and here.

How to handle wrong user input

You can register a handler to validation, parse and format issues by using the following functions of the SAPUI5 Core which can be accessed via `sap.ui.getCore()`:

- attachFormatError
- attachParseError
- attachValidationError
- attachValidationSuccess

A full example can be found here.

Simple Types

This section gives an overview on the currently available simple types.

sap.ui.model.type.Integer

The `Integer` type (JS Doc) represents an integer value. The value in the model ("source value") must be given as number and is transformed into the type of the bound control property:

- **float**: the value is rounded using `Math.floor`
- **integer**: no transformation needed
- **string**: the value is formatted / parsed according to the given output pattern

Here are some examples how a `Integer` type can be initialized:

```
// The source value is given as Javascript number. Output is transformed into the type of the
bound control property.
// If this type is "string" (e.g. the value property of the TextField control) the used
default output pattern parameters depend on locale and fixed settings.
var oType = new sap.ui.model.type.Integer();
// The source value is given as Javascript number. Output is transformed into the type of the
bound control property.
// If this type is "string" (e.g. the value property of the TextField control) the given
output pattern is used (parameters which are not specified are taken from the default pattern)
oType = new sap.ui.model.type.Integer({
  minIntegerDigits: 1, // minimal number of non-fraction digits
  maxIntegerDigits: 99, // maximal number of non-fraction digits
  minFractionDigits: 0, // minimal number of fraction digits
  maxFractionDigits: 0, // maximal number of fraction digits
  groupingEnabled: false, // enable grouping (show the grouping separators)
  groupingSeparator: ",", // the used grouping separator
  decimalSeparator: "." // the used decimal separator
});
```

The `Integer` type supports the following validation constraints:

- maximum
- minimum

sap.ui.model.type.Float

The `Float` type (JS Doc) represents a float value. The value in the model ("source value") must be given as number and is transformed into the type of the bound control property:

- **float**: no transformation needed
- **integer**: the value is rounded using `Math.floor`
- **string**: the value is formatted / parsed according to the given output pattern

Here are some examples how a `Float` type can be initialized:

```
// The source value is given as Javascript number. Output is transformed into the type of the
bound control property.
// If this type is "string" (e.g. the value property of the TextField control) the used
default output pattern parameters depend on locale and fixed settings.
var oType = new sap.ui.model.type.Float();
// The source value is given as Javascript number. Output is transformed into the type of the
bound control property.
// If this type is "string" (e.g. the value property of the TextField control) the given
output pattern is used (parameters which are not specified are taken from the default pattern)
oType = new sap.ui.model.type.Float({
  minIntegerDigits: 1, // minimal number of non-fraction digits
  maxIntegerDigits: 99, // maximal number of non-fraction digits
  minFractionDigits: 0, // minimal number of fraction digits
  maxFractionDigits: 99, // maximal number of fraction digits
  groupingEnabled: true, // enable grouping (show the grouping separators)
  groupingSeparator: ",", // the used grouping separator
  decimalSeparator: "." // the used decimal separator
});
```

The `Float` type supports the following validation constraints:

- maximum
- minimum

sap.ui.model.type.String

The `String` type (JS Doc) represents a string. The value in the model ("source value") must be given as string and is transformed into the type of the bound control property:

- **string**: no transformation needed
- **integer** / **float**: the string is parsed accordingly
- **boolean**: "true" or "X" are interpreted as true, "false" and "" as false

The `String` type does not have any format options.

Here is an example how a `String` type can be initialized:

```
// The source value is given as string. The length of the string must not greater than 5.
var oType = new sap.ui.model.type.String(null, {maxLength: 5});
```

The `String` type supports the following validation constraints:

- maxLength (expects an integer number)
- minLength (expects an integer number)
- startsWith (expects a string)
- startsWithIgnoreCase (expects a string)
- endsWith (expects a string)
- endsWithIgnoreCase (expects a string)
- contains (expects a string)
- equals (expects a string)
- search (expects a regular expression)

sap.ui.model.type.Boolean

The `Boolean` type (JS Doc) represents a string. The value in the model ("source value") must be given as boolean and is transformed into the type of the bound control property:

- **boolean**: no transformation needed
- **string**: "true" or "X" are interpreted as true, "false" and "" as false

The `Boolean` type does not have any format or validation constraint options.

Here is an example how a `Boolean` type can be initialized:

```
// The source value is given as boolean.
var oType = new sap.ui.model.type.Boolean();
```

sap.ui.model.type.Date

The `Date` type (JS Doc) represents a date (without time). It transforms a given value in the model ("source value") into a formatted date string and the other way round. The format patterns must be defined in LDML Date Format notation. For the output, the usage of a style ("short, "medium", "long" or "full") instead of a pattern is preferred, as it will automatically use a locale dependent date pattern.

Here are some examples how a `Date` type can be initialized:

```
// The source value is given as Javascript Date object. The used output pattern depends on the
locale settings (default).
var oType = new sap.ui.model.type.Date();
// The source value is given as Javascript Date object. The used output pattern is "yy-MM-dd":
e.g. 09-11-27
```

```
oType = new sap.ui.model.type.Date({pattern: "yy-MM-dd"});
// The source value is given as string in "yyyy/MM/dd" format. The used output style is
"long". The styles are language dependent.
// The following styles are possible: short, medium (default), long, full
// This usecase might be the common one.
oType = new sap.ui.model.type.Date({source: {pattern: "yyyy/MM/dd"}, style: "long"});
// The source value is given as string in "yyyy/MM/dd" format. The used output pattern is
"EEEE, MMMM d, yyyy": e.g. Saturday, August 22, 2043
oType = new sap.ui.model.type.Date({source: {pattern: "yyyy/MM/dd"}, pattern: "EEEE, MMMM d,
yyyy"});
// The source value is given as timestamp. The used output pattern is "dd.MM.yyyy": e.g.
22.12.2010
oType = new sap.ui.model.type.Date({source: {pattern: "timestamp"}, pattern: "dd.MM.yyyy"});
// The source value is given as string. The used input pattern depends on the locale settings
(default). The used output pattern is "dd '|' MM '|' yyyy": e.g. 22 | 12 | 2010
oType = new sap.ui.model.type.Date({source: {}, pattern: "dd.MM.yyyy"});
```

The `Date` type supports the following validation constraints:

- maximum (expects an date presented in the source-pattern format)
- minimum (expects an date presented in the source-pattern format)

sap.ui.model.type.Time

The `Time` type (JS Doc) represents a time (without date). It transforms a given value in the model ("source value") into a formatted time string and the other way round. The format patterns must be defined in LDML Date Format notation. For the output, the usage of a style ("short, "medium", "long" or "full") instead of a pattern is preferred, as it will automatically use a locale dependent time pattern.

Here are some examples how a `Time` type can be initialized:

```
// The source value is given as Javascript Date object. The used output pattern depends on the
locale settings (default).
var oType = new sap.ui.model.type.Time();
// The source value is given as Javascript Date object. The used output pattern is "hh-mm-ss":
e.g. 09-11-27
oType = new sap.ui.model.type.Time({pattern: "hh-mm-ss"});
// The source value is given as string in "hh-mm-ss" format. The used output style is "short".
The styles are language dependent.
// The following styles are possible: short, medium (default), long, full
// This usecase might be the common one.
oType = new sap.ui.model.type.Date({source: {pattern: "hh-mm-ss"}, style: "short"});
// The source value is given as string in "hh/mm/ss/SSS" format. The used output pattern is
"HH:mm:ss '+' SSS 'ms'": e.g. 18:48:48 + 374 ms
oType = new sap.ui.model.type.Time({source: {pattern: "hh/mm/ss/SSS"}, pattern: "HH:mm:ss '+'
SSS 'ms'"});
// The source value is given as timestamp. The used output pattern is "HH 'Hours' mm
'Minutes'": e.g. 18 Hours 48 Minutes
oType = new sap.ui.model.type.Time({source: {pattern: "timestamp"}, pattern: "HH 'Hours' mm
'Minutes'"});
// The source value is given as string. The used input pattern depends on the locale settings
(default). The used output pattern is "hh:mm a": e.g. 06:48 PM
oType = new sap.ui.model.type.Time({source: {}, pattern: "hh:mm a"});
```

The `Time` type supports the following validation constraints:

- maximum (expects an time presented in the source-pattern format)
- minimum (expects an time presented in the source-pattern format)

sap.ui.model.type.DateTime

The `DateTime` type (JS Doc) represents an exact point of time (date + time). It transforms a given value in the model ("source value") into a formatted date+time string and the other way round. The format patterns must be defined in LDML Date Format notation. For the output, the usage of a style ("short, "medium", "long" or "full") instead of a pattern is preferred, as it will automatically use a locale dependent date and time pattern.

Attention: When we talk about exact points of time the timezones becomes imported. The formatted output of the `DateTime` type is currently always shows the "local" time (time settings of the maschine on which the browser runs on). When the source value is given as Javascript Date object or as timestamp the exact moment is sufficiently defined. In case of a string source value this value is

interpreted in "local" time if it does not explicitly a timezone. Currently all accepted timezone notations must be based on GMT/UTC.

Here are some examples how a `DateTime` type can be initialized:

```
// The source value is given as Javascript Date object. The used output pattern depends on the
locale settings (default).
var oType = new sap.ui.model.type.DateTime();
// The source value is given as Javascript Date object. The used output pattern is "yyyy/MM/dd
HH:mm:ss": e.g. 2011/04/11 09:11:27
oType = new sap.ui.model.type.DateTime({pattern: "yyyy/MM/dd HH:mm:ss"});
// The source value is given as string in "yyyy/MM/dd HH:mm:ss" format. The used output style
is "full". The styles are language dependent.
// The following styles are possible: short, medium (default), long, full
// This usecase might be the common one.
oType = new sap.ui.model.type.Date({source: {pattern: "yyyy/MM/dd HH:mm:ss"}, style: "full"});
// The source value is given as string in "dd.MM.yyyy HH:mm:ss" format (no timezone given).
The used output pattern is "MMMM d, yyyy, HH:mm:ss.SSS": e.g. August 22, 2043, 18:48:48.374
oType = new sap.ui.model.type.DateTime({source: {pattern: "dd.MM.yyyy HH:mm:ss"}, pattern:
"MMMM d, yyyy, HH:mm:ss.SSS"});
// The source value is given as timestamp. The used output pattern is "dd.MM.yyyy HH:mm": e.g.
22.12.2010 13:15
oType = new sap.ui.model.type.DateTime({source: {pattern: "timestamp"}, pattern: "dd.MMM.yyyy
HH:mm"});
// The source value is given as string. The used input pattern depends on the locale settings
(default). The used output pattern is "hh-mm-ss '/' yy-MM-dd": e.g. 06-48-48 / 43-08-22
oType = new sap.ui.model.type.DateTime({source: {}, pattern: "hh-mm-ss '/' yy-MM-dd"});
// The source value is given as string in "dd.MM.yyyy HH:mm:ss X" format (timezone is defined
in ISO8601 format, e.g. "+02:00"). The used output pattern depends on the locale settings
(default).
oType = new sap.ui.model.type.DateTime({source: {pattern: "dd.MM.yyyy HH:mm:ss X"}});
// The source value is given as string in "dd.MM.yyyy HH:mm:ss Z" format (timezone is defined
in RFC822 format, e.g. "+0200"). The used output pattern depends on the locale settings
(default).
oType = new sap.ui.model.type.DateTime({source: {pattern: "dd.MM.yyyy HH:mm:ss Z"}});
// The source value is given as string in "dd.MM.yyyy HH:mm:ss z" format (timezone is
currently defined as e.g. "GMT+02:00", "UTC+02:00", "UT+02:00" or "Z" (shortcut for
"UTC+00:00")).
// The used output pattern depends on the locale settings (default).
oType = new sap.ui.model.type.DateTime({source: {pattern: "dd.MM.yyyy HH:mm:ss z"}});
```

The `DateTime` type supports the following validation constraints:

- maximum (expects an dateTime presented in the source-pattern format)
- minimum (expects an dateTime presented in the source-pattern format)

Is it possible to use the data types without databinding?

The data types above are **not** intended to be used without databinding. But there are formatter classes available which are also used by the types above. These formatters can be used also standalone to format e.g. dates. Validation is not provided by the formatter classes.

sap.ui.core.format.DateFormat

The `DateFormat` class (JS Doc) can be used to parse a string representing a date, time or datetime into a Javascript Date object and vice versa (a.k.a. format). The format pattern must be defined in LDML Date Format notation. The following format options are available:

- style: can be "short", "medium", "long" or "full" and will use a locale dependent pattern
- pattern: a date pattern in LDML Date Format notation

In case both style and pattern are defined, the pattern will be used and the style is ignored.

Here is an example how the `DateFormat` class can be used:

```
jQuery.sap.require("sap.ui.core.format.DateFormat");
var oDateFormat = sap.ui.core.format.DateFormat.getDateTimeInstance({pattern: "dd/MM/yyyy
HH:mm"}); //Returns a DateFormat instance for date and time
//var oDateFormat = sap.ui.core.format.DateFormat.getInstance({pattern: "dd/MM/yyyy"});
//Returns a DateFormat instance for date
//var oDateFormat = sap.ui.core.format.DateFormat.getTimeInstance({pattern: "HH:mm"});
//Returns a DateFormat instance for time
```

```
var oField = new sap.ui.commons.TextField();
oField.setValue(oDateFormat.format(oDate)); // Set the formatted value on the text field
oField.attachChange(function(){
  //Parse the user input
  oDate = oDateFormat.parse(oField.getValue());
});
```

sap.ui.core.format.NumberFormat

The `NumberFormat` class (JS Doc) can be used to parse a string representing a number (float or integer) into a Javascript number and vice versa (a.k.a. format). The following format options are possible (for parameters which are not specified default values according to the type are used):

- minIntegerDigits: minimal number of non-fraction digits
- maxIntegerDigits: maximal number of non-fraction digits
- minFractionDigits: minimal number of fraction digits
- maxFractionDigits: maximal number of fraction digits
- groupingEnabled: enable grouping (show the grouping separators)
- groupingSeparator: the used grouping separator
- decimalSeparator: the used decimal separator

Here is an example how the `NumberFormat` class can be used:

```
var fValue = 20001000.563;

jQuery.sap.require("sap.ui.core.format.NumberFormat");
var oNumberFormat = sap.ui.core.format.NumberFormat.getFloatInstance({
  maxFractionDigits: 2,
  groupingEnabled: true,
  groupingSeparator: ",",
  decimalSeparator: "."
}); //Returns a NumberFormat instance for float
/*var oNumberFormat = sap.ui.core.format.NumberFormat.getIntegerInstance({
  maxFractionDigits: 0,
  groupingEnabled: false
}); //Returns a NumberFormat instance for integer*/

var oField = new sap.ui.commons.TextField();
oField.setValue(oNumberFormat.format(fValue)); // Set the formatted value on the text field
oField.attachChange(function(){
  //Parse the user input
  fValue = oNumberFormat.parse(oField.getValue());
    alert(fValue);
});
```

Implementing a Custom Model

To create a custom model implementation, you have to perform several steps:

1. Extend the `Model` class and specify which binding modes the model should support (for example, two-way, one-way, one-time).
2. Extend the `Binding` class to suit your specific binding or reuse the existing specific binding implementations `PropertyBinding`, `ListBinding`, and/or `TreeBinding`.
3. To enable filtering functionality, use the `Filter` class with `FilterOperator` enum in your binding implementation.
4. To enable sorting functionality, use the `Sorter` class in your binding implementation.

All the necessary classes can be found in the `sap.ui.model` namespace.

For more details, see the data binding API.

As a starting point, take a look at the `JSONModel` implementation, which can be found in `sap.ui.model.json.JSONModel`.

OData Write support

Overview

In version 1.4 of SAPUI5 simple Write support was introduced for the ODataModel. The following new operations and features are supported:

- create
- remove
- update
- read
- get metadata service document
- Cross-Site Request Forgery XSRF token support for the REST library
- refresh

As the above mentioned features are experimental there might be changes in future versions of SAPUI5. Currently all write operations have to be performed by the application and are triggered synchronously.

The default binding mode is still `sap.ui.model.BindingMode.OneWay` but can be set to `sap.ui.model.BindingMode.TwoWay`.

Create Request

The `create` function triggers a POST request to an OData service which was specified when creating the OData model.

The application has to specify the collection where to create the entry and the entry data payload. To get the result of the request the application can hand over callback handler functions. A refresh is triggered automatically after successful creation to recreate and update the bindings.

Here is a sample code which triggeres a new entry in the `Products` collection:

```
var oEntry = {};
oEntry.Name = "IPad";
oEntry.Price = "499$";
oModel.create('/Products', oEntry, null, function(){
        alert("Create successful");
    },function(){
        alert("Create failed");});
```

Delete Request

The `remove` function triggers a DELETE request to an OData service which was specified when creating the OData model. The application has to specify the path to the entry which should be deleted. A refresh is triggered automatically after successful deletion to update the bindings in the model.

As of SAPUI5 1.9.1 parameters for the function changed (while preserving compatibility to older implementations). A single parameter `oParameters` has been introduced which can be passed into the function and can carry all optional attributes such as for success and error handling functions as well as for an ETag. ETags can be used for concurrency control if the OData service is configured to provide them. See more about ETags here:
Concurrency Control and ETags
If an ETag is specified in the `oParameters` parameter, it will always be used in the If-Match-Header instead of any ETag found in the metadata of an entry. If not, the ETag will be retrieved from the entry's metadata. If no ETag is found, no If-Match-Header will be used at all.

```
var oParams = {};
oParams.fnSuccess = function(){alert("Delete successful");};
oParams.fnError = function(){alert("Delete failed");};
oModel.remove('/Products(1)', oParams);
```

The above sample deletes the product entry with the ID = 1 from the `Products` collection of the data service and alerts if it was successful.

Update Request

The `update` function triggers a PUT request to an OData service which was specified when creating the OData model.

Like for the delete request, the parameters that can be passed into the update function have been changed in 1.9.1, while preserving the compatibility with older versions. Please see the ODataModel API for usage of the attributes.

The application has to specify the path to the entry which should be updated with the specified updated entry. A refresh is triggered automatically after a successful request to update the bindings in the model.

```
var oEntry = {};
oEntry.Price = "599$";
var oParams = {};
oParams.fnSuccess = function(){ alert("Update successful");};
oParams.fnError = function(){alert("Update failed");};
oParams.bMerge = true;
oModel.update('/Products(1)', oEntry, oParams);
```

IN SAPUI5 1.6 the flag `bMerge` was introduced to also allow a MERGE request which performs a differential update, it has been included in the `oParameters` from 1.9.1 on.

Read Request

The `read` function triggeres a GET request to a specified path which should be retrieved from the OData service which was specified when creating the OData model. The retrieved data is returned in the success callback handler function.

```
oModel.read('/Products(1)', null, null, true, function(oData, oResponse){
        alert("Read successful: " + JSON.stringify(oData));
    },function(){
        alert("Read failed");});
```

The `read` function returns an object containing a function `abort` which can be used to abort a running read request. In case the request is aborted the error handler will be called, so you can be sure for every request either the success or the error handler will be executed.

Refresh

The `Refresh` function triggeres a refresh for each binding to check if a value has been updated or not. In case of a list binding a new request is triggered to reload the data from the server. Additionally if the XSRF token is not valid any more a read request is triggered to fetch a new XSRF token from the server.

XSRF Token

To address the challenge of Cross Site Request Forgery an OData service might require XSRF tokens for change requests by the client application for security reasons. In this case the client has to fetch a token from the server and send it with each change request to the server.

The OData model fetches the XSRF token when reading the metadata and then automatically sends it in each write request header. If the token is not valid any more a new token can be fetched by calling the `refresh` function on the OData model.

Metadata

The `getServiceMetadata` function returns the parsed metadata document as a JavaScript object.

Experimental Two Way Binding

Note: This feature might change in future versions of SAP UI5!

The Two Way binding mode enables the application to update values of a single collection entry without triggering an immediate change request. Create and delete operations are not collected can be called by the application as described above.

If the Two Way binding mode is enabled the `setProperty` function of the OData model is called automatically to update the value in the specified entry upon user input. If an update to a property of another entry is performed and there is already an update to an existing entry pending a `rejectChange` event is fired and no change to the property value will be stored in the model. The application can register to this event. So before updating a different entry the existing changes of the current entry have to be submitted or resetted by the application. To reset the current changes the application can call the `resetChanges` function. To trigger the final update request for the modified entry the application should call the `submitChanges` function. Callback handlers for these functions can be handed over as parameters.

To enable the Two Way Binding for the OData model the default binding mode has to be changed:

```
oModel.setDefaultBindingMode(sap.ui.model.BindingMode.TwoWay);
```

After the binding is done the application should attach to the `rejectChange` event to handle the user modifications to different entries accordingly and e.g. allow user input or not on a specific entry.

```
oModel.attachRejectChange(this,function(oEvent){
    alert("You are already editing another Entry! Please submit or reject your pending
changes!");
});
```

The application can also determine if there is a pending change with the `hasPendingChanges` function.

Now the application can decide (or let the user decide) when to submit or reject the changes:

```
var oUpdateBtn = new sap.ui.commons.Button({
        text : "Update",
        style : sap.ui.commons.ButtonStyle.Emph,
        press : function(){
                        var oParameters = {};
            oModel.submitChanges(function(){
                    alert("Update successful"););
                },function(){
                    alert("Update failed");});
    }
});


var oCancelBtn = new sap.ui.commons.Button({
        text : "Cancel",
        style : sap.ui.commons.ButtonStyle.Reject,
        press : function(){
            oModel.resetChanges();
    }
});
```

The `submitChanges` function also has an optional parameter `oParameters` which currently can have an attribute for an ETag.

How to write a Core Plugin

For some purposes like the Debug it is necessary to implement a Core Plugin. A Core Plugin listens to the lifecycle of the Core and will be notfied when the Core bootstraps. A Core Plugin receives the internal reference to the full Core class (right now - but this will be limited in future to the Plugins needs). This feature enables to decouple additonal Core features out of the Core library.

Interface: `CorePlugin`

The interface `CorePlugin` defines the basic API of a Core Plugin. In general two methods can be implemented by a Core Plugin:

- `sap.ui.core.CorePlugin.prototype.startPlugin`
  Called by the Core after it has been initialized. If a plugin is added to the core after its initialization, then this method is called during registration of the plugin. This method provides two parameters - the reference to the Core and a flag, whether the Plugin is inited during `sap.ui.Core.init()` or later.

- `sap.ui.core.CorePlugin.prototype.stopPlugin`
  Called by the Core when it is shutdown or when a plugin is deregistered from the core.

Registration of a `CorePlugin`

The registration of the `CorePlugin` is done within the class `sap.ui.Core`. The Core provides two methods for registration and unregistration:

- `sap.ui.core.Core.prototype.registerPlugin`
- `sap.ui.core.Core.prototype.unregisterPlugin`

After the registration of the `CorePlugin` to Core manages the lifecycle and notfies the Core Plugin when the Plugin should start up and shut down.

Sample of a `CorePlugin`

```
/**
 * Provides the MyCorePlugin
 */
jQuery.sap.declare("sap.ui.mypackage.MyCorePlugin");

// any core plugin requires the Core
jQuery.sap.require("sap.ui.core.Core");

/**
 * Creates an instance of the class <code>sap.ui.mypackage.MyCorePlugin</code>
 *
 * @class Central Class for the My Core Plugin
 *
 * @author <author>
 * @version @version@
 * @constructor
 * @private
 */
sap.ui.mypackage.MyCorePlugin = function() {
};

/**
 * Will be invoked by <code>sap.ui.core.Core</code> to notify the plugin to start.
 *
 * @param {sap.ui.core.Core} oCore reference to the Core
 * @param {boolean} [bOnInit] whether the hook is called during core initialization
 * @public
 */
sap.ui.mypackage.MyCorePlugin.prototype.startPlugin = function(oCore, bOnInit) {
  this.oCore = oCore;
  jQuery.sap.log.info("Starting MyCorePlugin plugin.");
};
```

```
/**
 * Will be invoked by <code>sap.ui.core.Core</code> to notify the plugin to start
 * @param {sap.ui.core.Core} oCore reference to the Core
 * @public
 */
sap.ui.mypackage.MyCorePlugin.prototype.stopPlugin = function() {
  jQuery.sap.log.info("Stopping MyCorePlugin plugin.");
  this.oCore = null;
};

/**
 * Create the <code>sap.ui.mypackage.MyCorePlugin</code> plugin and register
 * it within the <code>sap.ui.core.Core</code>.
 */
(function(){
  var oThis = new sap.ui.mypackage.MyCorePlugin();
  sap.ui.getCore().registerPlugin(oThis);
})();
```

The Model View Controller (MVC) Approach in SAPUI5

This page describes the concepts and the usage of MVC in SAPUI5 and especially the role of the artifacts View and Controller. For detailed information on the Model approach and the available model flavors, goto Data Binding in Applications

Introduction

SAPUI5 offers Views and Controllers in the form of single files, these are

- `sap.ui.core.mvc.Controller`
- `sap.ui.core.mvc.XMLView`
- `sap.ui.core.mvc.JSView` and
- `sap.ui.core.mvc.JSONView`
- `sap.ui.core.mvc.HTMLView`

The base class for defining other or custom view types is

- `sap.ui.core.mvc.View`

Objectives

- Provide support for MVC paradigm
- Support development in distributed teams with different source locations
- Suggest file structure, naming, and usage patterns
- Add capability of UI declaration (in comparison to a programmatic construction)

Usage

According to the MVC paradigm, the View is responsible for defining and rendering the UI, the Model manages the application data, and the Controller reacts to View events and user interaction by modifying View and Model. This pattern defines a useful separation of concerns which helps developing or changing parts independently.

Views and Controllers often form a 1:1 relationship; alternatively it is also possible to have UI-less Controllers which are also named application controllers; creating Views without Controllers is also possible. Since a View is a SAPUI5 Control from the technical point of view, it can have or inherit a SAPUI5 Model.
Note that View and Controller represent reusable units, and distributed development is highly supported.

File Names and Locations (View and Controller)

Controllers and Views are defined and instantiated via a name which equals a SAPUI5 module name within the require/declare concept. This means that by default all files have to be located in a subfolder of the `resources` folder in the Web application. If this location is not appropriate, deviations can be configured as follows:

Example: We assume that your Views and Controllers are located on your local machine where the SAPUI5 runtime is loaded from another machine. When you now try to instantiate a View or Controller, the SAPUI5 runtime tries to load them relatively to the `resources` folder of the machine where the SAPUI5 runtime was loaded. Therefore you have to inform the runtime that your Views and Controllers are located on your local machine:

```
jQuery.sap.registerModulePath(sModuleNamePrefix, sURL);
```

or

```
sap.ui.localResources(sModuleNamePrefix);
```

In most cases it will be sufficient to use `sap.ui.localResources` which internally registers `sModuleNamePrefix` to the URL `"./" + sTransformedModuleNamePrefix` where the transformed name has all dots replaced by slashes. All files starting with `sModuleNamePrefix` will then be loaded relatively to the location of the HTML page that was calling `sap.ui.localResources`.

If your files are located at " http://<localhost:8080>/<myapp>/", for example, you can use `registerModulePath` as follows:

```
jQuery.sap.registerModulePath("myapp", "http://<localhost:8080>/<myapp>/");
```

or

```
sap.ui.localResources("myapp");
```

All Views and Controllers having a name starting with `myapp`, for example `myapp.MyView`, will now be loaded from your local machine.

Controller Definition

You define a simple Controller, having no function yet, as following:

```
sap.ui.controller("sap.hcm.Address", {
   // controller logic goes here
});
```

The string in quotes is the Controller name, which is described in the chapter above. The Controller file is then named `Address.controller.js`.
Note that each Controller must have the suffix `.controller.js`.

Controller Functions

There are predefined lifecycle hooks you can implement. Typically you also add event handlers or other functions. Controllers can fire events that other Controllers or entities can register for.

Lifecycle Hooks

The lifecycle hooks are as following:

- `onInit()` - Called when a View is instantiated and its controls (if available) are already created. Can be used to modify the View before it is displayed to bind event handlers and do other one-time initialization.
- `onExit()` - Called when the View is destroyed. Use this one to free resources and finalize activities.
- `onAfterRendering()` - Called when the View has been rendered (therefore its HTML is part of the document). Post-rendering manipulations of the HTML can be done here. This hook is the same one that SAPUI5 controls get after being rendered.
- `onBeforeRendering()` - Is invoked before the Controller's View is re-rendered. You would use `onInit()` in the case that the hook shall be invoked only before the first rendering.

For Controllers without a View, no lifecycle hooks will be called.

Example:

```
sap.ui.controller("sap.hcm.Address", {

   onInit: function() {
      this.counter = 0;
   }

});
```

Event Handlers and Other Functionality

In addition to the lifecycle methods, a Controller can define any additional methods that can serve as event handlers, or as some functionality offered by the Controller.

Example:

```
sap.ui.controller("sap.hcm.Address", {

   increaseCounter: function() {
      this.counter++;
   }

});
```

View Types

While the XML and JSON notation for SAPUI5 UI controls has been introduced, the MVC pattern shall also be supported in the case that a traditional programmatic UI construction is preferred.

- XMLView
  - UI is defined in an XML file/string
- JSONView
  - UI is defined via JSON in a file/string
- JSView
  - UI is constructed in a traditional manner
- HTMLView
  - UI is defined in an HTML file/string

Note that the XMLView type is constructed in a way that a mix with plain HTML is possible.

While the View types mentioned above are predefined and offered to be chosen as View option in the application creation wizard - in the case that the tools are used for Web application development - others can also be plugged in.

JSView Definition

A JavaScript View is created like a Controller. The suffix for such a file is `.view.js`. There are two default methods that can be implemented and that would usually always be used:

- `getControllerName()` - Specifies the Controller belonging to this View. In the case that it is not implemented, or that "null" is returned, this View does not have a Controller.
- `createContent()` - Is initially called once after the Controller has been instantiated. It is the place where the UI is constructed. Since the Controller is given to this method, its event handlers can be attached right away.

Example:

```
sap.ui.jsview("sap.hcm.Address", {  // this View file is called Address.view.js

   getControllerName: function() {
      return "sap.hcm.Address";     // the Controller lives in Address.controller.js
   },

   createContent: function(oController) {
      var oButton = new sap.ui.commons.Button({text:"Hello JS View"});
      oButton.attachPress(oController.handleButtonClicked);
      return oButton;
   }

});
```

The string in quotes is the View name, which again equals the SAPUI5 module name within the require/declare concept.

XMLView Definition

This type of View is defined in an XML file, and the file name has to end with `.view.xml`, or as an XML string. The View name along with the folder structure above is the name of the View. This name again equals the SAPUI5 module name within the require/declare concept.
Example: In the case of `resources/sap/hcm/Address.view.xml`, the View name would be `sap.hcm.Address`. This View name is used when the application displays an instance of this View. If you define the XMLView via an XML string, no file or require/declare is needed.

The file looks like this:

```
<core:View controllerName="sap.hcm.Address" xmlns="sap.ui.commons" xmlns:core="sap.ui.core">
   <Panel>
      <Image src="http://www.sap.com/global/ui/images/global/sap-logo.png"/>
      <Button text="Press Me!"/>
   </Panel>
<core:View>
```

Therefore you simply nest the XML tags (analogous to a nesting sequence of the SAPUI5 controls), and add the property values as attributes. (Some details on more complex Views are described below.)

Namespaces in XMLViews

Analoguous to generic XML mapping features, the names of the SAPUI5 control libraries are mapped to XML namespaces. You can choose any of the required namespaces to be the default namespace, therefore the respective control tags do not need a prefix.
The surrounding <View> tag is always required and in this case has the `core` namespace defined in the first line. Of course you can give any name; to keep the tag names shorter it might be useful for example to use "c" instead of "core".

Note: If controls are located in a subpackage of a control library, as it is the case for the `sap.ui.commons.layout.MatrixLayout`, they need to have their own XML namespace:

```
<core:View controllerName="sap.hcm.Address" xmlns="sap.ui.commons" xmlns:core="sap.ui.core"
xmlns:layout="sap.ui.commons.layout">
   <layout:MatrixLayout>
      ...
```

Aggregation Handling in XMLViews

Child controls are simply added as child tags, as described in the example above where a <Button> tag was added into a <Panel> tag.

Some controls have more than one content area, for example the Shell control which has main content, a pane bar, a headerItems aggregation, worksetItems and so on. This is the reason why in general an aggregation tag serves as direct child of a container, and inside are the children. Directly adding children as seen above is only possible if the container control has marked one child aggregation as "default". Some containers may not have "default" content like the Splitter, which has two equally important content areas. Note that the framework leads you as a user by providing an error message if you made a mistake here.

In general, aggregations are filled as follows. Note: The namespace of the parent control tag and the aggregation tag must be the same.

```
<core:View controllerName="sap.hcm.Address" xmlns="sap.ui.commons" xmlns:core="sap.ui.core">
   <Panel>
      <content>  <!-- this is the general way of adding children: use the aggregation name -->
         <Image src="http://www.sap.com/global/ui/images/global/sap-logo.png"/>
         <Button text="Press Me"/>
      </content>
   </Panel>
</core:View>
```

Usage of Event Handlers in XMLViews

Event handlers are bound as attributes, where the attribute name is the event name, like "press" in case of a Button, and the attribute value is the event handler name. This event handler is expected to be defined as a function in the view's controller. The function will be executed with the controller as the context object (`this`).
Therefore, the following declaration is equivalent to a call of `controller.doSomething()` when the Button is pressed:

```
...
        <Button text="Press Me" press="doSomething"/>
...
```

Data Binding in XMLViews

You can bind data in XMLViews: If you want to bind texts of a control to a language-dependent resource bundle, simply define the resource bundle via a name (`resourceBundleName` attribute) or a URL (`resourceBundleUrl` attribute) and assign an alias (`resourceBundleAlias` attribute) for the bundle within the View definition. The binding path is the same as for every other SAPUI5 databinding:

Resource bundle content:

```
MY_TEXT=Hello World
```

Example:

```
<core:View resourceBundleName="myBundle"
           resourceBundleAlias="i18n"
           controllerName="sap.hcm.Address" xmlns="sap.ui.commons" xmlns:core="sap.ui.core"
           xmlns:html="http://www.w3.org/1999/xhtml">
   <Panel>
      <Button text="{i18n>MY_TEXT}"/>
   </Panel>
<core:View>
```

The necessary ResourceModel for binding these texts is created during View instantiation. The Model will be set as secondary Model with the given alias to the View instance. If you want to bind other properties to another Model you have to create the Model on your own within the corresponding Controller, or HTML page, and attach it to the View with another alias. The binding itself behaves like every SAPUI5 databinding and like described above.

Usage of Native HTML in XMLViews

The usage of Native HTML depends on the XHTML feature set. When mixing XHTML and SAPUI5 controls, some additional rules have to be obeyed:

1. XHTML elements can be used whenever the SAPUI5 type "Control" could be used, for example in the root of an XMLView, or in the content aggregation of a layout container
2. When embedding XHTML in an aggregation of a SAPUI5 control, the XHTML must not consist of a single text node. The topmost node of an embedded XHTML tree must be an XHTML element. Embedding pure text into an aggregation is not supported.
3. The XHTML nodes are converted 1:1 to HTML, the XMLView does not deal with any differences between XHTML and HTML (for example re-writing auto-closing tags)
4. The created HTML DOM nodes are preserved during re-rendering of an XMLView: Modifications to the DOM are not lost.

Further restrictions are currently not known.

Note: As an alternative to embedding XHTML, you can use the `sap.ui.core.HTML` control. Note that it is less convenient since encoding of the content is required.

To mix SAPUI5 controls freely with native XHTML, you simply need another namespace - the XHTML one - and then you can use (X)HTML:

```
<core:View controllerName="sap.hcm.Address" xmlns="sap.ui.commons" xmlns:core="sap.ui.core"
           xmlns:html="http://www.w3.org/1999/xhtml">
```

```
   <Panel>
      <Button text="Press Me. I am a SAPUI5 Button"/>
      <html:button>No, press me. I am native HTML Button.</html:button>
   </Panel>
<core:View>
```

Usage of CSS Style Sheets in XMLViews

You include the style sheets like plain HTML. You can also add further CSS classes to SAPUI5 controls by using the `class` attribute. The effect is the same as it is when calling `myButton.addStyleClass(...)`.

```
<core:View controllerName="sap.hcm.Address" xmlns="sap.ui.commons" xmlns:core="sap.ui.core"
           xmlns:html="http://www.w3.org/1999/xhtml">
   <html:style>
      .mySuperRedButton {
         color: red;
      }
   </html:style>
   <Panel>
      <Button class="mySuperRedButton" text="Press Me"/>
   </Panel>
<core:View>
```

It is recommended to carefully choose the elements that you style since the CSS always affects the whole page and is NOT restricted to the View.

JSONView Definition

This type of View is defined in a file and that file's name must end with `.view.json`, or as a JSON string. The View name along with the folder structure above is the name of the View. This name again equals the SAPUI5 module name within the require/declare concept.
For example, the file could be `resources/sap/hcm/Address.view.json`, then the View name would be `sap.hcm.Address`.
This View name is used when the application wants to display an instance of this View.

The file looks like this:

```
{
   "Type":"sap.ui.core.mvc.JSONView",
   "controllerName":"sap.hcm.Address",
   "content": [{
      "Type":"sap.ui.commons.Image",
      "id":"MyImage",
      "src":"http://www.sap.com/global/ui/images/global/sap-logo.png"
   },
   {
      "Type":"sap.ui.commons.Button",
      "id":"MyButton",
      "text":"Press Me"
   }]
}
```

So you simply nest the JSON Objects like you want to nest the SAPUI5 controls and add the property values as attributes. The syntax is exactly the same as you can use a JSON constructor for any control.

Note: You can only use strings in your JSONView.

Aggregation Handling in JSONViews

As seen above, child controls are simply added as arrays, like the above example added an Image and a Button into the View content aggregation.

Usage of Event Handlers in JSONViews

Event handlers are bound as attributes, where the attribute name is the event name, like "press" in case of a Button, and the attribute value is the event handler name. This event handler is expected to be defined as function in the View's Controller.
So this declaration will cause `controller.doSomething()` to be executed when the Button is pressed:

```
...
   {
      "Type":"sap.ui.commons.Button",
      "id":"MyButton",
      "text":"Press Me",
      "press":"doSomething"
   }
...
```

Data Binding in JSONViews

You can bind data in JSONViews. You bind the texts of a control to a language-dependent resource bundle by defining the resource bundle via name (`resourceBundleName` property) or a URL (`resourceBundleUrl` property) and assigning an alias (`resourceBundleAlias` property) for the bundle within the View definition. The binding path is the same as for every other SAPUI5 databinding:

Resource bundle content:

```
MY_TEXT=Hello World
```

Example:

```
{   "Type": "sap.ui.core.JSONView",
    "controllerName":"my.own.views.test",
    "resourceBundleName":"myBundle",
    "resourceBundleAlias":"i18n",
    "content": [{
        "Type":"sap.ui.commons.Panel",
        "id":"myPanel",
        "content":[{
            "Type":"sap.ui.commons.Button",
            "id":"Button1",
            "text":"{i18n>MY_TEXT}",
            "press": "doIt"
        }]
    }]
}
```

The necessary ResourceModel for binding this texts is created during View instantiation. The Model will be set as secondary Model with the given alias to the View instance. If you want to bind other properties to another Model you have to create the Model on your own within the corresponding Controller, or HTML page, and attach it to the View with another alias. The binding itself behaves like every SAPUI5 databinding and like described above.

HTMLView Definition

A HTML View is defined by declarative HTML. Please see DeclarativeSupport for more information. Like the declarative support the HTML view supports embedded HTML.

The filename of an HTML View by convention ends with `.view.html`, e.g. `myview.view.html`.

Example:

```
<template data-controller-name="example.mvc.test">
   Hello
  <h1>Title</h1>
  <div>Embedded HTML</div>
  <div class="test test2 test3" data-sap-ui-type="sap.ui.commons.Panel" id="myPanel">
    <div class="test test2 test3" data-sap-ui-type="sap.ui.commons.Button" id="Button1" data-
text="Hello World" data-press="doIt"></div>
    <div data-sap-ui-type="sap.ui.commons.Button" id="Button2" data-text="Hello"></div>
    <div data-sap-ui-type="sap.ui.core.mvc.HTMLView" id="MyHTMLView" data-view-
name="example.mvc.test2"></div>
    <div data-sap-ui-type="sap.ui.core.mvc.JSView" id="MyJSView" data-view-
name="example.mvc.test2"></div>
    <div data-sap-ui-type="sap.ui.core.mvc.JSONView" id="MyJSONView" data-view-
name="example.mvc.test2"></div>
    <div data-sap-ui-type="sap.ui.core.mvc.XMLView" id="MyXMLView" data-view-
name="example.mvc.test2"></div>
  </div>

</template>
```

All view specific properties can be added to the `<template>` tag as *data-** attributes.

View Instantiation

Views can be instantiated with the factory method `sap.ui.view`. Necessary information for the instantiation can be passed via an object. This object can have the following properties:

- `type`: Type can be `JSON`, `JS`, `XML`. All possible types are declared in the enumeration `sap.ui.core.mvc.ViewType`
- `viewName`: The View name, corresponding to the module concept
- `viewContent`: XMLViews/JSONViews only - XML/JSON string representation of the View definition. If `viewName` and `viewContent` are given, the View definition will be loaded with the `viewName`
- `Controller`: Any Controller instance. The given Controller instance overrides the Controller defined in the View definition
- `viewData`: `JSViews` only - can hold user specific data. This data is available during the whole lifecycle of the View and the Controller

Example:

```
...
   var myController = sap.ui.controller("my.own.controller");

   var myView = sap.ui.view({ type: sap.ui.core.mvc.ViewType.XML,
                              viewName: "my.own.view",
                              controller: myController
                            });
...
```

All regular properties of a View (control) can be passed to the object as usual.

Typed Views and Controllers

The above ways of defining Views and Controllers are light-weight and easy to write. However, for more complex use-cases a more formal way to define Views and Controllers might be preferred, even if this means a bit more complexity.

This section shows how you can create new classes, using prototype inheritance, and declare their API.

Typed Views

Typed Controllers

If you want to create a controller that is a new type on its own, you can do it like this. There is some boilerplate code on top that needs to be written and the functions need to be declared on the new prototype. But apart from that it's a normal controller:

```
/* boilerplate code for typed Controller */
jQuery.sap.declare({modName:"sap.hcm.AddressController", type:"controller"}); // declaring a
special type of module
sap.hcm.AddressController = function () { // the constructor
   sap.ui.core.mvc.Controller.apply(this, arguments);
};
jQuery.sap.require("sap.ui.core.mvc.Controller"); // this is currently required, as the
Controller is not loaded by default
sap.hcm.AddressController.prototype =
jQuery.sap.newObject(sap.ui.core.mvc.Controller.prototype); // chain the prototypes
/* end of boilerplate code for typed Controller */

// to avoid the above we could in the future offer it behind a simple call to:
// sap.ui.defineController("sap.hcm.Address");

sap.hcm.AddressController.prototype.onInit = function() {
   // modify control tree - this is the regular lifecycle hook
};

// implement an event handler in the Controller
sap.hcm.AddressController.prototype.doSomething = function() {
```

```
   alert("Hello World");
};
```

A live example can be found here.

Nesting Views

All Views can be nested independent of the View type. Each View type behaves like any other SAPUI5 control. The `viewName` property defines which View to embed.

For XMLViews it can look like this:

```
<core:View controllerName="sap.hcm.Address" xmlns="sap.ui.commons" xmlns:core="sap.ui.core"
           xmlns:html="http://www.w3.org/1999/xhtml">
   <Panel>
      <core:JSView id="myJSView" viewName="sap.hcm.Bankaccount" />
   </Panel>
<core:View>
```

For HTMLViews it can look like this:

```
<template data-controller-name="example.mvc.test">

  <div data-sap-ui-type="sap.ui.core.mvc.HTMLView" id="MyHTMLView" data-view-
name="example.mvc.test2"></div>
  <div data-sap-ui-type="sap.ui.core.mvc.JSView" id="MyJSView" data-view-
name="example.mvc.test2"></div>
  <div data-sap-ui-type="sap.ui.core.mvc.JSONView" id="MyJSONView" data-view-
name="example.mvc.test2"></div>
  <div data-sap-ui-type="sap.ui.core.mvc.XMLView" id="MyXMLView" data-view-
name="example.mvc.test2"></div>

</template>
```

Support for Unique IDs

Normally, each View and its content defines static IDs. Via these IDs you can identify and modify the controls within your Controller during runtime. If you are reusing or nesting these Views, the IDs would be not unique any more. Therefore each View will add its own ID as prefix to all its child controls. This happens only if a static ID is given. If the ID is created during instantiation of the control it is unique per default. If you create further controls during runtime, the Controller can create a unique ID for you calling `oController.createId("ID")`. These methods add the necessary prefix to the ID. If you want to modify the control with the ID <ID> you can call the method `byId(<ID>)` on your view to get the right control directly. You don't have to handle all the prefix stuff by your own.

Important: only in the declarative View types (XMLView, JSONView, HTMLView) the IDs are prefixed automatically. In JSViews the controls are created directly (in the createContent() method) and it is the task of the JSView developer to create unique IDs by calling `this.createId("ID")`. Only when this is done, `View.byId(<ID>)` will return the control because this method will always add the prefix.

ButtonView:

```
<core:View viewName="sap.hcm.ButtonView" cont
```

Data Binding

This chapter explains concept, design and usage of data binding within SAPUI5 applications. The "Introduction" gives an overview about the purpose of data binding and how it is implemented. In "Getting Started" you find a tutorial to get a first impression of how the data binding is used. The following chapters provide detailed information about the various topics, up to creating a custom model implementation.

- Introduction to Data Binding
- Getting Started
- Usage of Data Binding in Applications
- Models provided by SAPUI5
  - The ResourceModel
  - The JSONModel
  - The XMLModel
  - The ODataModel
- The different binding types
  - Binding Properties
  - Binding Aggregations
  - Binding Elements
- The Data Binding Type System
- Data Binding in Custom Controls
- Creating a Custom Model Implementation
- Experimental Write support for OData Model
- Calculated Fields for databinding (Binding multiple model properties to a single control property)

Localized Texts

Concepts

The framework concepts for text localization are aligned with the concepts of the Java platform. The main components of the concept are:

* A way to **identify languages/locales** and
* A way to **store and access texts in multiple languages (resource bundles)**.

Both are explained now in more detail.

Language Code / Locale

For the identification of languages, the framework uses a **language code** of type `string`. Currently, two syntax forms are accepted:

* The older **Java Locale syntax** which combines a lower case ISO 639 alpha-2 or alpha-3 language code with an ISO 3166 alpha-2 country code. Both codes are combined with an underscore. As a third component, an arbitrary sequence of variant identifiers (also separated by underscores) can be appended.

- Examples: `en_US, zh_TN_Traditional`

* Language codes according to the defacto standard **BCP 47** (see IETF BCP-47). Most modern browsers use such codes for language identification. Since JDK 1.7 they are supported by the Java Locale class as well.

- Example: `bs-Cyrl-BA, i-klingon`

Current Language Code / Locale

UI5 has the notion of a *current language*. It is determined from the following sources of information, ordered by increasing priority, so the last one available wins:

1. A hard-coded UI5 default locale ('en')
2. A potentially configured user language (window.navigator.userLanguage)
3. A potentially configured browser language (window.navigator.browserLanguage)
4. A general language information from the browser (window.navigator.language)
5. A locale configured in the application coding (see jsdoc:symbols/sap.ui.core.Configuration.html)
6. A locale configured via URL parameters

The resulting **current language** can be retrieved via the Configuration API:

```
var sCurrentLocale = sap.ui.getCore().getConfiguration().getLanguage();
```

Note: The syntax of the returned value unfortunately depends on the syntax used for configuration. If the information source was one of the browser language properties, the returned language most likely will be in BCP-47 format. If it was configured as an URL parameter, the user might have choosen the JDK Locale syntax.

Note: In Internet Explorer, none of the window.navigator.* properties does reflect the settings of the "Language Preference" dialog that Internet Explorer provides. Instead, IE returns the language of the OS installation as `browserLanguage` and the language from the OS regional settings as `userLanguage`. As a result, the settings from the "Language Preference" dialog cannot be taken into account for the **current language** of UI5. This is often confusing for developers and a known shortcoming in IE. The only way to circumvent this is an additional server request where IE provides the corresponding setting in the `Accept-Language` header. As this technique requires a backend component and as UI5 must be

able to run without any server component, such a request is not implemented yet.

Resource Bundles

Resource bundles in the framework are a **collection of *.properties** files. All files are named with the **same base name (prefix which identifies the resource bundle) + an optional suffix that identifies the language contained in each file + plus the fixed .properties extension**. The language suffixes are formed according to the older JDK locale syntax. By convention, a file without a language suffix should exist and contain the raw untranslated texts in the developer's language. It will be used as a last resort if no more suitable language can be found.

Internally, a resource bundle file is a Java properties file (as described in the Javadoc of class java.util.Properties). It contains key/value pairs where the values are the language dependent texts and the keys are language independent and used by the application to identify and access the corresponding values.

When a localized text is needed (or any other localized data that can be represented as a string, e.g. a date formatter string), then the application can use the UI5 APIs to load the properties file that best matches the current language. It then can retrieve a text from it by using the (language independent) key. If the text for a key cannot be found in a file, the next best matching file is loaded and checked for the text. As a last resort, the 'raw' file will be loaded and checked.

- Example: Assume the resource bundle name is `sap.ui.commons.message_bundle`, then it might consist of the following individual files:
  - `sap.ui.commons.message_bundle.properties` - contains 'raw' texts form the developer, determines the set of keys
  - `sap.ui.commons.message_bundle_en.properties` - contains English texts (without a specifc country)
  - `sap.ui.commons.message_bundle_en_US.properties` - contains texts in American english
  - `sap.ui.commons.message_bundle_en_UK.properties` - contains texts in British english

Using Localized Texts in Applications

jQuery.sap.resources

The first way of accessing localized texts is by using the JavaScript module `jQuery.sap.resources`. It contains APIs to load a resource bundle file from a given URL and for a given locale.

To make sure that the `jQuery.sap.resources` module is loaded, you have to require it:

```
jQuery.sap.require("jquery.sap.resources");
```

Afterwards you can use the equally named function `jQuery.sap.resources` to load a resource bundle from a given Url (= bundle name) and for a given locale. When no locale is given, a default locale (`en`) will be used. The following code snippet shows the usage of the `jQuery.sap.resources` function which returns a `jQuery.sap.util.ResourceBundle`:

```
var oBundle = jQuery.sap.resources({url : sUrl, locale: sLocale});
```

See also: jQuery.sap.resources API documentation

The returned `jQuery.sap.util.ResourceBundle` provides access to the contained localized texts. When the bundle is loaded you can access the texts by their key using the `getText(sKey)` method. The following code snippet shows the usage of a bundle to access texts and messages:

```
var sText = oBundle.getText(sKey);
```

See also: JSDoc for jQuery.sap.util.ResourceBundle

Localization Test Page

A test page showing the use of localized texts is available in the Test Suite. This section will give a short overview how the module `jQuery.sap.resources` is used there.

What you would need for a localized Web page is the .html page iself and the .properties files of the required languages, in this example English and German.

The resource bundle `i18n.properties` is the English fallback version which is the default version.

```
welcome=Welcome {0}. Please enter a new contact:
lastname=Last Name:
firstname=First Name:
street=Street:
zip=ZIP:
city=City:
```

The resource bundle `i18n_de.properties` contains the texts in German. The following code snippet shows the content of this file:

```
welcome=Willkommen {0}. Bitte geben Sie einen neuen Kontakt ein:
lastname=Nachname:
firstname=Vorname:
street=Straße:
zip=PLZ:
city=Ort:
```

The Localization test page makes use of those texts to display a welcome message and a form to enter the address of a person. The coding of the test page looks like the following:

```
01:     jQuery.sap.require("jquery.sap.resources");
02:
03:     var sLocale = sap.ui.getCore().getConfiguration().getLanguage();
04:     var oBundle = jQuery.sap.resources({url : "res/i18n.properties", locale: sLocale});
05:
06:     var oMatrixLayout = new sap.ui.commons.layout.MatrixLayout();
07:     oMatrixLayout.setLayoutFixed(false);
08:     oMatrixLayout.createRow(
09:                     new sap.ui.commons.TextView({text: oBundle.getText("welcome",
["Administrator"])})
10:     );
11:     oMatrixLayout.getRows()[0].getCells()[0].setColSpan(2);
12:     oMatrixLayout.createRow(
13:                     new sap.ui.commons.Label({text: oBundle.getText("lastname")}),
14:                     new sap.ui.commons.TextField()
15:     );
16:     oMatrixLayout.createRow(
17:                     new sap.ui.commons.Label({text: oBundle.getText("firstname")}),
18:                     new sap.ui.commons.TextField()
19:     );
20:     oMatrixLayout.createRow(
21:                     new sap.ui.commons.Label({text: oBundle.getText("street")}),
22:                     new sap.ui.commons.TextField()
23:     );
24:     oMatrixLayout.createRow(
25:                     new sap.ui.commons.Label({text: oBundle.getText("zip")}),
26:                     new sap.ui.commons.TextField()
27:     );
28:     oMatrixLayout.createRow(
29:                     new sap.ui.commons.Label({text: oBundle.getText("city")}),
30:                     new sap.ui.commons.TextField()
31:     );
32:     oMatrixLayout.placeAt("userForm");
```

The code above is doing the following with regards to localization:

1. Require the `jQuery.sap.resources` module (!Line: 01)
2. Determine the language (!Line: 03)
3. Load the resource bundle (!Line: 04)
4. Access the text using the `welcome` key and pass the value for the placeholder (`{0}`) via an array (!Line: 09)
5. Access the text using the `lastname` key and set it as text for the `Label` (!Line: 13)

Data Binding

A second way of accessing localized texts is to use data binding. With the ResourceModel, a wrapper for resource bundles exists that exposes the localized texts as a model for data binding. With the help of the ResourceModel, one can bind texts for Control properties to language dependent resource bundle properties. A ResourceModel can be instantiated with a `bundleName` which is the name of a resource bundle and equals a SAPUI5 module name within the require/declare concept, or a `bundleUrl` which points to a resource bundle. When using the bundle name the file must have the .properties suffix. If no `locale` is given the current language will be used.

Example:

```
var oModel = new
sap.ui.model.resource.ResourceModel({bundleName:"myBundle",bundleLocale:"en"});

var oControl = new sap.ui.commons.Button( {
    id : "myButton",
    text : "{i18n>MY_BUTTON_TEXT}"
});
// attach the resource model with the symbolic name "i18n"
oControl.setModel(oModel, "i18n");
```

Note: with the current data binding implementation, you can't pass parameters to your texts within the resource bundle. If you have to pass parameters you must do this on your own. As a convenience, you can access the resource bundle directly from the model instead of loading it:

```
var myBundle = oModel.getResourceBundle();
```

After the instance has been created, you have a model containing the resource bundle texts as data.

For a complete overview of available methods and parameters check the API reference.

CustomData - Attaching data objects to Controls

Sometimes it is desired to attach data to UI5 controls, similarly to what jQuery.data() offers. To support this, the method `data()` has been added to the base class `sap.ui.core.Element`.

The method can be used to set and get data, the API is equal to jQuery's data(): http://api.jquery.com/data/. Data can also be attached declaratively, in XMLViews and JSONViews. Data binding can also be used in all these scenarios.

Optionally, this data can also be written to the HTML DOM as "data-*" attribute - but only when it is a string.

Setting and retrieving data

```
myButton.data("myData", "Hello");  // attach some data to the Button

alert(myButton.data("myData"));     // alerts "Hello"

var dataObject = myButton.data();  // a JS object containing ALL data
alert(dataObject.myData);          // alerts "Hello"
```

Binding data: usage in a list binding

```
// create a JSONModel, fill in the data and bind the ListBox to this model
var oModel = new sap.ui.model.json.JSONModel();
oModel.setData(aData);                                    // aData is a data array
consisting of elements like {question:"Some question?",answer:"Some answer!"}
var ctrl = new sap.ui.commons.ListBox({select:giveAnswer}); // method giveAnswer() retrieves
the custom data from the selected ListItem
ctrl.setModel(oModel);

// create an item template and bind the question data to the "text" property
var oItemTemplate = new sap.ui.core.ListItem();
oItemTemplate.bindProperty("text", "question");

// create a CustomData template, set its key to "answer" and bind its value to the answer data
var oDataTemplate = new sap.ui.core.CustomData({key:"answer"});
oDataTemplate.bindProperty("value", "answer");

// add the CustomData template to the item template
oItemTemplate.addCustomData(oDataTemplate);

// bind the items to the "questions" (which is the name of the data array)
ctrl.bindAggregation("items", "questions", oItemTemplate);
```

A running example can be found in the UI5 testsuite (look for "CustomData?" in sap.ui.core).

Usage in XMLViews

In XMLViews, CustomData objects could be written as the normal aggregated objects they are. But to reduce the amount of code and improve readability massively, a shortcut notation has been introduced: you can directly write the data attributes into the control tags. You just need to use a special namespace for these attributes:
`myNamespace="http://schemas.sap.com/sapui5/extension/sap.ui.core.CustomData/1".`
This namespace looks different and more formal than the existing MVC namespaces, but that's by intention. The other namespaces will also be adapted to this more formal naming scheme soon.

Without data binding

The following example shows how the string "just great" is attached to a Button:

```
<core:View xmlns:core="sap.ui.core" xmlns:mvc="sap.ui.core.mvc" xmlns="sap.ui.commons"
```

```
controllerName="my.own.controller"
            xmlns:app="http://schemas.sap.com/sapui5/extension/sap.ui.core.CustomData/1">
     <Button id="myBtn" text="Click to show stored coordinates data" app:mySuperExtraData="just
great" press="alertCoordinates"></Button>
</core:View>
```

This string would be returned at runtime by calling `button.data("mySuperExtraData")`.

With data binding

Data binding can as well be used with the usual notation:

```
<core:View xmlns:core="sap.ui.core" xmlns:mvc="sap.ui.core.mvc" xmlns="sap.ui.commons"
controllerName="my.own.controller"
            xmlns:app="http://schemas.sap.com/sapui5/extension/sap.ui.core.CustomData/1">
     <Button id="myBtn" text="Click to show stored coordinates data" app:coords="{data}"
press="alertCoordinates"></Button>
</core:View>
```

Usage in JSONViews

To add custom data to any Element in a JSONView (in these examples again depicted with data binding), you can add the following to the Element properties:

```
customData: {
  Type:"sap.ui.core.CustomData",
    key:"coords",
    value:"{data}" // bind custom data
  }
```

For multiple data elements to be added, use an array:

```
customData: [{
  Type:"sap.ui.core.CustomData",
    key:"coords",
    value:"{data}" // bind custom data
  },
  {
  Type:"sap.ui.core.CustomData",
    key:"coords",
    value:"{data}" // bind custom data
  }]
```

In context, it looks as follows:

```
        var json =
            {
                Type: "sap.ui.core.JSONView",
                controllerName:"my.own.controller",
                content: [{
                    Type: "sap.ui.commons.Panel",
                    content:[{
                        Type: "sap.ui.commons.Button",
                        text:"{actionName}",
                        press: "doSomething",
                        customData: {
                            Type:"sap.ui.core.CustomData",
                            key:"coords",
                            value:"{data}" // bind custom data
                        }
                    }]
                }]
            };
```

Usage in HTMLViews

In HTMLViews , CustomData objects could be added to any control or element with a specific HTML attribute. The syntax looks like this: `data-custom-data:my-key="myValue"`. A custom data attribute begins with `data-custom-data:` and is followed by the name of the key. The dashes converts the following character into an upper case character. The value can be either a string or a binding expression:

```
  <div data-sap-ui-type="sap.ui.commons.Button" data-text="This button is added dynamically"
```

```
data-custom-data:my-key="myValue" data-custom-data:my-bound-key="{/mypath}"></div>
```

Writing data to the HTML DOM as data-* attribute

It can be useful to write the custom data to the HTML DOM, e.g. to generate markers in the HTML from data-binding which then can be used for data-dependent styling, or to create stable anchors in the HTML which can be used for automated tests.
A "data-" prefix is added to the key and the result is then written as attribute into the root HTML element of the control. The CustomData value is written as attribute value.

This only works when the key is a valid HTML ID and the value is a string (otherwise an error is logged). Note that HTML attribute names are case-insensitive and the key may be converted to lowercase by browsers!

Be careful not to write too much data into the DOM!

In JavaScript the flag can be set like this:

```
myButton.data("mydata", "Hello", true);  // attach some data to the Button and mark it as
"write to HTML"
```

In XMLViews the aggregation has currently to be written in expanded notation to set the writeToDom flag:

```
<Button ... >
  <customData>
    <core:CustomData key="mydata" value="Hello" writeToDom="true" />
  </customData>
</Button>
```

Resulting HTML:

```
<button ...  data-myData="Hello"  ... >
```

Now CSS can use attribute selectors to check presence or the value of the custom data attribute:

```
 button[data-mydata="Hello"] {
       border: 3px solid red !important;
 }
```

How to Develop UI5 Controls Within JavaScript

Overview

This page explains how to create new Controls in JavaScript on the fly (so-called "Notepad Controls"). For this basic development approach, it is not necessary to define libraries or to run generation steps. It is possible to extend existing Controls and to create completely new Controls.

Technically, this functionality is not restricted to Controls. Arbitrary objects derived from sap.ui.base.Object can be created or extended.

When to use this instead of Eclipse-based Control Development?

Technically, the resulting Controls are equal to Controls developed using the UI5 Eclipse tools, so when to use which approach?

These on-the-fly Controls described here have the advantage of being created very quickly without any tooling or build or dependency overhead, e.g. just inline in the middle of the application. All JavaScript can be just in one file, the application CSS, which may anyway be present, can contain the required styles. So it is a very light-weight approach, but still offering all the features of a real UI5 Control.
A weakness is that the Control cannot be re-used easily from very different places and applications: there is no formal way to address it, there is no package which you can reference to import the control. Also, if the control should support multiple themes and should be styled using UI5 theme parameters which are editable in the Theme Editor, if the right-to-left version of the styles should be generated automatically, the Eclipse-based tools offer everything with their integrated build. Checks are not run on a on-the-fly Control, no JSDoc documentation can be created.

So the general rule of thumb is that if a Control is supposed to be re-used in different applications or by people who do not closely communicate with the Control developer and if it is going to be a bit more complex than a very basic Control, one should increasingly consider formally creating a Control Library and using the UI5 Eclipse to do so.

Basic Concept: Extend Method

The `extend()` method is available on all Controls (and the base classes) and is used to define a new subclass. The following code would create a new Control from scratch:

```
sap.ui.core.Control.extend(...);
```

The following code would create a new Control which inherits from Button:

```
sap.ui.commons.Button.extend(...);
```

The parameters to this function are the **name** and the **definition** of the new Control type. The definition part contains information about the Control API this means which properties, aggregations, events etc. it has and the implementation of the Control methods. Additionally, it provides the function that creates the Control's HTML structure.

Note that some methods such as the getters and setters for the properties and aggregations or the methods for attaching/detaching event handlers, for example, are automatically created by UI5. Indeed, you can develop a non-default implementation that provides more or other functionality.

A First Basic Example

The following code creates a simple Control which has a `name` property and which has the purpose to render the text "Hello <name>":

```
sap.ui.core.Control.extend("my.Hello", {      // call the new Control type "my.Hello"
                                              // and let it inherit from sap.ui.core.Control
    metadata : {                              // the Control API
        properties : {
            "name" : "string"                 // setter and getter are created behind the
scenes,
                                              // including data binding and type validation
        }
    },

    renderer : function(oRm, oControl) {      // the part creating the HTML
        oRm.write("<span>Hello ");
        oRm.writeEscaped(oControl.getName()); // write the Control property 'name', with XSS
protection
        oRm.write("</span>");
    }
});
```

The new Control is ready for use now. It can be instantiated and displayed as usual:

```
new my.Hello({name:"UI5"}).placeAt("content");
```

We will focus on some more realistic examples lateron; before, let's have a glance at some further basic concepts.

Further Technical Background

What is a Control?

- A Control is an object that defines the **appearance** and **behavior** of a screen area.
- A Control typically has **properties** like "text" or "width" that modify the appearance or relate to the data displayed by the Control.
- It can **aggregate** other Controls. This means it is a sort of *container* or *layout* Control when the application can add the child controls to use - or a *composite* Control if the Control itself decides what the child Controls are and just re-uses these available components.
- It can also have **associated** Controls that are not part or children of this Control, but rather loosely related.
- A Control can fire well-defined **events**. Typically, these have a meaning that relates to the Control's purpose and functionality and is on a semantically higher level than "click" or other browser events. Examples would be `triggerSearch` in a SearchField or `collapse` in a Panel.

All this information is defined in the Control meta data which on the one hand is the public API of the Control (and can be queried at runtime) and on the other hand contains useful information for runtime features like data binding and type validation checks.

Control Base Classes

`sap.ui.core.Control` is the base class of all UI5 Controls. Specific Controls can either inherit from this base class or from another Control, in order to inherit and extend its functionality. **`sap.ui.core.Element`** is the base class of `sap.ui.core.Control`. Elements can be like parts of Controls or rather configuration packages for parts of Controls, but in general they are NOT meant for standalone use and they do not have their own renderer. The Controls that use these elements do the rendering (for example a ListItem Element is rendered by the ListBox Control). Whatever this page documents about Controls, it usually also applies to Elements (but not to the renderer).

Control Name

The Control name is a string that consists of Library name and Control name. For both names, you use letters and dots for separation where you should avoid dublicates regarding other JS entity namings within the same application. It is also possible that you omit the library name and call the Control `Square`. You would generally do this if you there is no need for assigning the control to a library which

shall also be available for developing other applications. When you write a Control to be reused by others, a unique Library name is recommended: `sap.byd.Square`.

Control Metadata

For the creation of new Controls, the metadata block defines the properties, aggregations, events etc. of the Control. Depending on the Control, not only the number of entries, but also their amount of information can be small or quite extensive.

For the use case class extension - where the classes do not inherit from Control or Element but from a more generic class - these Control-specific settings are not available. For more information, read chapter "Object Metadata and Implementation" below. Note that all the object metadata described there is also applicable for extending Controls.

Properties

A property is defined by at least its name and its type. Additionally, the default value of the property can be defined. So the available settings are:

- `type`: The data type of the Control property. Automatic type validation is done in the UI5 core. Examples for valid types are:
  - `string` for a string property (default)
  - `int` or `float` for number properties
  - `int[]` etc. for arrays
  - `sap.ui.core.CSSSize` for a custom-defined type
- `defaultValue`: The default value this property should have when the application does not set a value. If no default value is given, the property initially has value `undefined`.

So property definitions can look like this:

```
properties: {
   "title" : "string",                          // a simple string property, default value is
{{{undefined}}}
   "buttonText" : {defaultValue: "Search"},    // when no type is given, the type is
{{{string}}}
   "showLogoutButton" : {type : "boolean", defaultValue : true},   // a boolean property where
a default value is given
   "width" : {type : "sap.ui.core.CSSSize", defaultValue : "50px"} // a CSS size property
where a default value is given
}
```

From the technical point of view, you could also define the "group" (the category into which this property falls), but this has no impact outside of development tools which may want to display and group the properties of your Control.

Once such a property is defined, the Control automatically has the methods `setShowLogoutButton` and `getShowLogoutButton` which are responsible for storing the data. You can give your own custom definition for either of them, you have to call the generic property setter/getter `setProperty`/`getProperty` then.

Events

Events are specified by the event name only. In many cases there is nothing to configure about them, so just give an empty object:

```
events: {
   "logout": {}
}
```

As an advanced feature, Controls can enable events to be interrupted by the application, when usually the Control would immediately execute an action. For example, if a Tab Control fires a "close" event and wants to enable the application to cancel the closing. To do so, the Control needs to set the `allowPreventDefault` property of the Event to "true" (and check the return value after firing the event):

```
events: {
   "close": {allowPreventDefault : true}
}
```

For every defined Event, methods for registering, de-registering and firing the event are created, in this case: `attachLogout`, `detachLogout`, `fireLogout`

Aggregations / Associations

Aggregations and associations are again defined by their name, along with an object that can have the following information:

- `type`: This should be a type which is subclass of Element or Control (default is `sap.ui.core.Control`)
- `multiple`: Whether it is a 0..1 aggregation or a 0..n aggregation (default is "true", which means 0..n, for aggregations and "false" for associations)
- `singularName`: For 0..n aggregations, the aggregation name typically is plural, but certain methods are created where the singular form is required (for example, `addWorksetItem`} for the "worksetItem**s**" aggregation).
- `visibility`: The visibility of this aggregation (default is "public"). See further details below in section "Hidden Aggregations"

If only the type needs to be set, you can just give it as a string instead of the configuration object.

One example:

```
aggregations: {
   "acceptButton" : "sap.ui.commons.Button", // if only type is given, no object is required
   "content" : {singularName: "content"},     // default type is "sap.ui.core.Control",
                                              // which is appropriate for generic containers
   "worksetItems" : {type : "sap.ui.ux3.NavigationItem", multiple : true, singularName :
"worksetItem"}
                                              // a fully specified aggregation
}
```

Multiple methods are created, depending on the multiplicity, for example: `getWorksetItems`, `insertWorksetItem`, `addWorksetItem`, `removeWorksetItem`, `removeAllWorksetItems`, `indexOfWorksetItem`, `destroyWorksetItems`

When you want to mark one aggregation as default aggregation (in order to be able to omit the aggregation tag in XMLViews), you can do it by setting the defaultAggregation property to the name of the aggregation like this:

```
aggregations: {
   "content": {singularName: "content"} // default type is "sap.ui.core.Control", multiple is
"true"
},
defaultAggregation: "content"
```

Methods

You can add any method to a new Control by providing the implementation, without adding it to the metadata. By convention, all methods are public, exception here is if their name starts with an underscore, or if it is one of the special method types listed below. Other Controls and the application may only call public methods and the Control needs to ensure they remain compatible. Though, there are no technical rules that prevent a call of private methods. Public methods are the generated getter/setter methods for properties etc. .

The Control Implementation

After the metadata is defined, you can add any method implementations to your new Control. In general, the method names can be chosen freely, but note that some method names must be avoided:

- Names of methods that are provided by a super class (or will be provided...). Your implementation

overwrites their's, that's the deal with inheritance.
- set.../get.../insert.../add.../remove.../indexOf... may collide with setters/getters for properties or aggregations you defined
- attach.../detach.../fire may collide with methods created for events

As long as you do not introduce the respective property or so, you are of course safe to use a certain "set..." method name.

There are some method names you may use but which have a special meaning:

- `on...`: Methods starting with "on" are event handlers that are automatically bound to browser events
- `init`: Is the name of the initialization function called right after Control instantiation
- `renderer`: Is a special name that holds either
  - the function that creates the Control's HTML or
  - a complete structure that contains this function and more (for more information see below)

Normal Methods

Any method, either public or private, is just appended to the implementation object. The convention is that private methods that may not be called from outside a Control start with an underscore. All other methods are considered public as long as they do not belong to the special method types listed below.

```
divide: function(x, y) {     // a public method of the Control
   if (this._checkForZero(y)) {
      throw new Error("Second parameter may not be zero");
   }
   return x / y;
},

_checkForZero: function(y) { // private helper method
   if (y === 0) {
      return true;
   }
   return false;
}
```

`init()` Method

The `init()` method is invoked by the UI5 core for each Control instance right after the constructor. Use this to set up things like internal variables or sub-Controls of a composite. Note that any values given in the Control constructor are NOT yet available! This is intentional to prevent the typical error where Controls would only work fine when values are set initially, but not when values are changed later. This method is considered private and only to be called by the UI5 core.

If you inherit from another control that has (or might get) an `init()` method, you need to explicitly call it, otherwise it will not be executed, which may leave the superclass not properly initialized.

```
   init: function() {
      this._bSearchHasBeenTriggered = false;
      this._oSearchButton = new sap.ui.commons.Button(this.getId() + "-searchBtn", {text:
"Search"});
   }
```

And with `init()` call of the superclass, assuming the custom control inherits from DatePicker:

```
  init: function() {
    if (sap.ui.commons.DatePicker.prototype.init) {   // check whether superclass has an
init() method
      sap.ui.commons.DatePicker.prototype.init.apply(this, arguments);  // call super.init()
    }
    // ...continue initialization...
  }
```

Event Handler Methods

Methods that have a name starting with "on" are reserved for event handlers. For common events such

as "click" or "keydown", browser event handlers for these methods are registered automatically by the UI5 core. So it is sufficient to simple add a handler method, and it will automatically be called. Additionally, the UI5 core fires events with a richer semantic meaning, so Control developers do not need to check so many keycodes etc. The name of these events starts with "sap", they are defined in `jquery.sap.events.js`. One example for such an event is `sapnext` which is triggered by "arrow down" or "arrow right" (or "arrow left" in right-to-left mode). Therefore, multiple checks would be required to check whether the user wants to navigate to the next item. The `sapnext` event takes over all these checks. The "evt" object given to the handler method contains more information. These methods are considered to be private and may only be called by the UI5 core.

```
  onclick: function(evt) {
     alert("Control " + this.getId() + " was clicked.");
  },

  onsapnext: function(evt) {
     // navigate to next item, an arrow key was pressed
     ...
  }
```

The Renderer Method/Object

This method is responsible for creating the HTML structure that makes up the Control. It is different from the other methods, as it is a static one, so the "this" keyword is not available. Instead, a Control instance and a RenderManager instance are given to the method. The RenderManager is used as a sort of "writer" - it is a collector for string fragments and takes care of efficiently concatenating them and placing them into the appropriate DOM position.

```
renderer: function(oRenderManager, oControl) {
   oRenderManager.write("<div>", oControl.getText(), "</div>");
}
```

All the features and rules of writing the Control Renderer in normal Control development apply here as well. E.g. the Control must have only ONE HTML element as root node and inside this node `oRenderManager.writeControlData(oControl);` must be called, so this root element can be marked as UI5 control root and the ID of the Control will be written.

The newly created renderer type will inherit from the renderer of the parent Control. So if your new Control extends TextField, the given function will be added to a class that inherits from `sap.ui.commons.TextFieldRenderer` (and will have access to that one's other functions). However, your renderer method will override the parent renderer method, so no HTML will be written as long as your method is not taking care of it.

If an existing renderer should be used without modification, you can give the name of this renderer class:

```
renderer: "sap.ui.commons.ButtonRenderer"
```

However, a normal Control Renderer can also override or implement methods from its Renderer superclass. Or just separate out some helper functions.
This is possible as well here, but in this case these methods need to be packed together into an object, so the `extend` method knows they should all go into the Control Renderer. The main rendering method is called "render" - just like in a normal Control Renderer. The keyword "this" actually refers to the Control Renderer type here, so it is used to access the other methods:

```
renderer: {
   render: function(rm, oControl) {
      rm.write("<div>");
      rm.writeEscaped(this.square(oControl.getValue()));
      rm.write("</div>");
   },
   square: function(value) {
      return value * value;
   }
}
```

Hidden Aggregations

Hidden Aggregations are aggregations for which the metadata property `visibility` is set to `hidden` (in contrast to public aggregations which are the default).

Aggregations are used to define a parent-child relationship between a parent control and its children (controls or elements). The knowledge about this relationship is for example relevant for the SAPUI5 Core to dispactch events properly or to cleanup the children when the parent is destroyed.

The purpose of hidden aggregations is to support the development of composite controls. Composite controls internally reuse already existing controls and combine those controls to something new (e.g. a SearchField control which internally uses a TextField and a Button). These inner controls are not intended to be accessed directly from the user of the composite control, so these controls should not be contained within a normal public aggregation. In this case a hidden aggregation can be defined to register the child controls within the control hierarchy without make them public available. You can find more information about composite controls here.

Because hidden aggregations are only used internally within a composite control for hidden aggregations no typed accessor functions are generated, they are not cloned and databinding is not enabled.

Example:'

```
sap.ui.core.Control.extend("my.SearchField", {
    metadata : {
        properties : {
            "value" : "string"
        },
        aggregations: {
            "_tf" : {type : "sap.ui.commons.TextField", multiple : false, visibility:
"hidden"},
            "_btn" : {type : "sap.ui.commons.Button", multiple : false, visibility: "hidden"}
        },

        events: {
            "search" : {}
        }
    },

    init: function() {
        var that = this;
        this.setAggregation("_tf", new sap.ui.commons.TextField({
          change: function(oEvent){
            that.setProperty("value", oEvent.getParameter("newValue"), true);
          }
        }));
        this.setAggregation("_btn", new sap.ui.commons.Button({
          text: "Search",
          press: function(){
            that.fireSearch();
          }
        }));
    },

    setValue: function(sValue){
        this.setProperty("value", sValue, true);
        this.getAggregation("_tf").setValue(sValue);
        return this;
    },

    renderer : function(oRm, oControl) {
        oRm.write("<div");
        oRm.writeControlData(oControl);
        oRm.addClass("mySearchField");
        oRm.writeClasses();
        oRm.write(">");
        oRm.renderControl(oControl.getAggregation("_tf"));
        oRm.renderControl(oControl.getAggregation("_btn"));
        oRm.write("</div>");
    }
});
```

Object Metadata and Implementation

For extending plain objects that are not Elements or Controls, only the following metadata is available (it

is *also* available for extending Controls, though!):

- "interfaces": an optional array of strings that denotes the implemented interfaces
- "publicMethods": an optional list of methods that should be part of the public API. By default all methods that do not start with an underscore are public.
- "abstract": an optional flag to mark the type as abstract
- "final": an optional flag to mark the type as final

Regarding the implementation, all methods given outside the metadata are attached to the new type. There is one reserved method name: "constructor". The function given under this name will be the constructor of the new class. While technically you can also define a Constructor for new Elements and Controls, you should **not** do it. Your Control may otherwise break in certain scenarios like with list bindings, or may break later when UI5 extends the constructor signature.

Examples for Creating New Controls

For creating an entirely new Control type, you extend the `sap.ui.core.Control` class. You define the Control API and the implementation from scratch.

A Simple "Square" Control

A "Square" Control that is rendered as a red square (surprise!) and has some text written inside and pops up an alert when clicked, looks like this:

Control Implementation

```
  sap.ui.core.Control.extend("Square", { // call the new Control type "Square" and let it inherit
                                         // from sap.ui.core.Control

      // the Control API:
      metadata : {
          properties : {           // setter and getter are created behind the scenes,
                                   // incl. data binding and type validation
              "text" : "string",   // in simple cases, just define the type
              "size" : {type: "sap.ui.core.CSSSize", defaultValue: "200px"}
                                   // you can also give a default value and more
          }
      },

      // the part creating the HTML:
      renderer : function(oRm, oControl) { // static function, so use the given "oControl" instance
                                           // instead of "this" in the renderer function
          oRm.write("<div");
          oRm.writeControlData(oControl);  // writes the Control ID and enables event handling - important!
          oRm.addStyle("width", oControl.getSize());  // write the Control property size; the Control has validated it
                                                      // to be a CSS size
          oRm.addStyle("height", oControl.getSize());
          oRm.writeStyles();
          oRm.addClass("mySquare");        // add a CSS class for styles common to all Control instances
          oRm.writeClasses();              // this call writes the above class plus enables support
                                           // for Square.addStyleClass(...)
          oRm.write(">");
          oRm.writeEscaped(oControl.getText()); // write another Control property, with protection
                                                // against cross-site-scripting
          oRm.write("</div>");
      },

      // an event handler:
      onclick : function(evt) {   // is called when the Control's area is clicked - no event registration required
          alert("Control clicked! Text of the Control is:\n" + this.getText());
      }
  });
```

The Control definition and implementation is finished now, but some styling remains to be done.
The visual appearance could have been written to the Control HTML in the renderer method, just like the instance-specific width and height was written. But it is better to define style that is common to all Control instances in a CSS file, or at least in a <style> tag, so it only needs to be written once and it can be easily modified by the application.
So we add a grey background, a red border and some alignment stuff:

```
<style>
    .mySquare {                  /* style the CSS class that has been written by the renderer
method */
        display: inline-block;   /* enable squares to appear next to each other within one
line */
        border: 1px solid red;   /* add some border, so the square can actually be seen */
        background-color: #ddd;
        padding: 8px;
        text-align: center;
        -moz-box-sizing: border-box; /* consider padding+border part of the width/height */
        box-sizing: border-box;
    }
</style>
```

Control Usage

This custom Control can now be used like any UI5 Control:

```
var myControl = new my.Square({text:"Hello", size: "100px"});
myControl.placeAt("content");
```

A Simple Container Control

A container Control that can hold arbitrary child Controls and renders them in a row, with a colored box around each child, looks like this:

Control Implementation

```
  sap.ui.core.Control.extend("ColorBoxContainer", { // call the new Control type
"ColorBoxContainer"
                                                     // and let it inherit from
sap.ui.core.Control

      // the Control API:
      metadata : {
         properties : {            // setter and getter are created behind the scenes,
                                   // incl. data binding and type validation
            "boxColor" : "string"  // the color to use for the frame around each child Control
         },
         aggregations: {
            content: {singularName: "content"} // default type is "sap.ui.core.Control",
multiple is "true"
         }
      },

      // the part creating the HTML:
      renderer : function(oRm, oControl) { // static function, so use the given "oControl"
instance
                                           // instead of "this" in the renderer function
          oRm.write("<div");
          oRm.writeControlData(oControl);  // writes the Control ID and enables event handling
- important!
          oRm.writeClasses();              // there is no class to write, but this enables
                                           // support for ColorBoxContainer.addStyleClass(...)
          oRm.write(">");

          var aChildren = oControl.getContent();
          for (var i = 0; i < aChildren.length; i++) { // loop over all child Controls,
                                                       // render the colored box around them
              oRm.write("<div");
              oRm.addStyle("display", "inline-block");
              oRm.addStyle("border", "3px solid " + oControl.getBoxColor()); // specify the
border around the child
              oRm.writeStyles();
              oRm.write(">");
```

```
            oRm.renderControl(aChildren[i]);   // render the child Control
                                                // (could even be a big Control tree, but you
don't need to care)

            oRm.write("</div>"); // end of the box around the respective child
        }

        oRm.write("</div>"); // end of the complete Control
    }
});
```

There is no additional CSS required, as the Control has no appearance on its own, but basically consists of its children and the boxes around them.

Control Usage

This strange container Control can now be used like any UI5 container:

```
var btn = new sap.ui.commons.Button({text:'Hello World'});
var tf = new sap.ui.commons.TextField({value:'edit text here'});

var container = new ColorBoxContainer({
    boxColor: "#ff7700",
    content:[
        btn,
        tf
    ]});
container.placeAt('content');
```

Examples for Extending Existing Controls

Extending the Button with an additional Event

Let's build a new Control, named "HoverButton" that is just like the normal Button, but fires a "hover" event in addition when the mouse enters its area.

Control Implementation

```
sap.ui.commons.Button.extend("HoverButton", { // call the new Control type "HoverButton"
                                               // and let it inherit from
sap.ui.commons.Button
      metadata: {
          events: {
              "hover" : {}  // this Button has also a "hover" event, in addition to "press" of
the normal Button
          }
      },

      // the hover event handler:
      onmouseover : function(evt) {   // is called when the Button is hovered - no event
registration required
          this.fireHover();
      },

      renderer: {} // add nothing, just inherit the ButtonRenderer as is;
                   // In this case (since the renderer is not changed) you can also specify
this explicitly with:  renderer:"sap.ui.commons.ButtonRenderer"
                  // (means you reuse the ButtonRenderer instead of create a new renderer)
  });
```

Control Usage

Use this ToggleButton in an application just like a regular Button - but you can now attach a handler to the "hover" event.

```
var myControl = new HoverButton("myBtn", {
    text: "Hover Me",
    hover: function(evt) {
        alert("Button " + evt.getSource().getId() + " was hovered.");
    }
```

```
  });
  myControl.placeAt("content");
```

Extending the TextField Rendering

Let's build a new Control type that inherits from TextField and has all its features, but changes the rendering to be very highlighted with yellow background.

Control Implementation

All of the Control API and even the "render" method can be inherited as is. We only overwrite the renderInnerAttributes method of the TextFieldRenderer:

```
  sap.ui.commons.TextField.extend("HighlightField", { // call the new Control type
"HighlightField"
                                                      // and let it inherit from
sap.ui.commons.TextField

      renderer: {
          // note that NO render() function is given here! The TextField's render() function
is used.
          // But one function is overwritten:

          renderInnerAttributes : function(oRm, oTextField) {
              oRm.addStyle('background-color', '#ffff00');  // this change could also be done
with plain CSS!!
                                                             // But you get the idea...
          }
      }
  });
```

Control Usage

Use this HighlightField in an application just like a regular TextField. It has all the normal features and behavior. But it has the modified rendering.

```
var myControl = new HighlightField({value:"Highlighted editing"});
myControl.placeAt("content");
```

Handling Events in Controls

There are two types of events in UI5 applications:

1. events **fired by the browser** (these are the well-known ones like "click", "blur" etc.) and
2. events **fired by UI5 Controls** (these are usually semantically richer and related to the Control functionality; e.g. a browser-level "click" on the respective icon in a Panel header could trigger a "maximize" or "minimize" event)

Applications can listen to both types of events.

This page describes how Control implementations can use browser events (type 1) to implement their behavior and to eventually fire Control events (type 2).

Registering for Browser Events

When a Control needs to react on browser events, there are two ways to register for the events, the "standard" way and an "optimized UI5" way:

1. **Explicitly register for browser events** on certain DOM elements, typically using `jQuery.bind()` (using the respective browser methods like `addEventListener` would also be possible).
   The event registration must be done in the onAfterRendering method of the Control, so the event binding is always repeated after the control is re-rendered (new DOM elements are created and old ones are discarded). Furthermore, to prevent memory leaks, the event binding must be removed (with jQuery.unbind()) in the "onBeforeRendering" method and in the "exit" method (which is called before the Control is destroyed).

- Pro:
  - ANY type of browser event can be handled
  - Works exactly like in any web page or jQuery-based web application
- Con:
  - Quite some code required to do the binding and unbinding of the event handler
  - Registering many event handlers can affect performance

  Example:

```
MyControl.prototype.onAfterRendering = function() {
   this.$().bind("click", jQuery.proxy(this.handleClick, this));  //
could also be:  jQuery.sap.byId(this.getId).bind("click",
jQuery.proxy(this.handleClick, this));
}

MyControl.prototype.onBeforeRendering = function() {
   this.$().unbind("click", this.handleClick);
}

MyControl.prototype.exit = function() {
   this.$().unbind("click", this.handleClick);
}

MyControl.prototype.handleClick = function(oEvent) {
   // do something...
}
```

2. **Just implement the event handler** for certain "common" event types, using a **name convention** for the handler method.
   UI5 automatically registers event handlers for a list of commonly used event types on the root element of a complete tree of UI5 controls. If the respective event occurs anywhere in the tree and the respective Control implements the "on<eventName>" method, this method is invoked as if it had been registered with jQuery.bind().

- Pro:
  - Saves Control code
  - Saves number of event handler registrations in the DOM

    - Saves evet handler registrations and deregistrations executed on every re-rendering
- Con:
    - Only works for a certain (comprehensive) list of events

      Example:

```
MyControl.prototype.onclick = function(oEvent) {
   // do something...
}
```

You see it saves quite some coding to use this UI5 event handling functionality.

List of supported Browser Events

The following events are available to be handled by just implementing an "on<eventName>" method (list is also available via API, see JSDoc of jQuery.sap.ControlEvents):

- click
- dblclick
- dragend
- dragenter
- dragleave
- dragover
- dragstart
- drop
- focusin
- focusout
- keydown
- keypress
- keyup
- mousedown
- mouseout
- mouseover
- mouseup
- paste
- select
- selectstart

Pseudo Events

In addition to the native browser events listed above, UI5 also creates so-called "pseudo events" which are semantically enriched, but can also be handled implementing an "on<eventName>" method. So they "feel like" browser events (but they cannot be used with jQuery.bind()).

These events help avoiding additional checks for modifier keys in the event handler or checking for certain keycodes.

The complete documentation of these events can be found in the JSDoc of jQuery.sap.PseudoEvents. The list is as follows:

- sapbackspace
- sapbackspacemodifiers
- sapbottom
- sapcollapse
- sapcollapseall
- sapcollapsemodifiers
- sapdecrease
- sapdecreasemodifiers
- sapdelayeddoubleclick
- sapdelete

sapdeletemodifiers
- sapdown
- sapdownmodifiers
- sapend
- sapendmodifiers
- sapenter
- sapentermodifiers
- sapescape
- sapexpand
- sapexpandmodifiers
- saphide
- saphome
- saphomemodifiers
- sapincrease
- sapincreasemodifiers
- sapleft
- sapleftmodifiers
- sapnext
- sapnextmodifiers
- sappagedown
- sappagedownmodifiers
- sappageup
- sappageupmodifiers
- sapprevious
- sappreviousmodifiers
- sapright
- saprightmodifiers
- sapselect
- sapselectmodifiers
- sapshow
- sapskipback
- sapskipforward
- sapspace
- sapspacemodifiers
- saptabnext
- saptabprevious
- saptop
- sapup
- sapupmodifiers

How to Write a Control Renderer

For control rendering purposes, the following three classes are relevant:

- The Control class - the control that is to be rendered
- The RenderManager class - responsible for injecting the generated markup into the DOM and offering helper functionality
- The Renderer class - the base class of all control renderers

Control Class `sap.ui.core.Control`

In general, a control is made up of its properties, events, aggregations, associations, and methods. Together, these parts define the **behavior** of a `Control`. The following information is relevant for the **appearance** and **data** of the control: properties, associations, and aggregations. This information can be accessed directly with `get` and `set` methods of the control during the `render()` method.

Property Access:

```
// var oValue = oControl.get<Property>();
// for example for the 'text'-property
var oValue = oControl.getText();
```

1..1 Aggregation Access:

```
// var oAggregation = oControl.get<Aggregation>();
// for example for content-aggregation
var oAggregation = oControl.getContent();
```

1..n Aggregration Access:

```
// var aAggregations = oControl.get<Aggregation>s();
// for example for rows-aggregation
var aAggregations = oControl.getRows();
```

Association Access:

```
// var sAssociatedControlId = oControl.get<Association>();
// for example labelFor-association
var sAssociatedControlId = oControl.getLabelFor();
```

RenderManager Class `sap.ui.core.RenderManager`

The `RenderManager` is responsible for injecting the generated markup into the DOM. It takes a control, determines and loads the corresponding renderer, and finally delegates the rendering of the control to the renderer. It also offers helper functionality for rendering purposes as follows:

Method	Description
`write()`	Writes string information to the HTML
`writeControlData()`	Writes the ID and the recognition data of the control to the HTML
`renderControl()`	Converts the specified control into HTML representation and adds it to the HTML. Use this for rendering child controls.

For further details, see the JSDoc of the `RenderManager`.

Renderer Class `sap.ui.core.Renderer`

The `Renderer` implements the static `render` method that is called when a control is added to the `DOM`. The `RenderManager` executes the `render` method on the corresponding `Renderer` for the control to be rendered, and passes the reference to itself and the control.

The following code snippet shows the implementation of a simple renderer:

```
/**
 * This module provides the renderer for the MyControl control
 */
jQuery.sap.declare("sap.ui.myuilib.MyControlRenderer");

/**
 * @class MyControl renderer.
 *
 * @author SAP - TD Core UI&AM UI Infra
 * @version @version@
 * @static
 */
sap.ui.myuilib.MyControlRenderer = {
};

/**
 * Renders the HTML for the given control, using the provided {@link
sap.ui.core.RenderManager}.
 *
 * @param {sap.ui.core.RenderManager} oRenderManager The RenderManager that can be used for
writing to the RenderOutputBuffer.
 * @param {sap.ui.core.Control} oControl An object representation of the control that should be
rendered
 */
sap.ui.myuilib.MyControlRenderer.render = function(oRenderManager, oControl) {

  // write the HTML into the render manager
  oRenderManager.write("<span ");
  oRenderManager.writeControlData(oControl);
  oRenderManager.write(" class=\"sap-ui-myuilib-MyControlRenderer\" ");
  oRenderManager.write("></span>");

};
```

How to Prevent Cross-Site Scripting (XSS)

You can find some basic facts about Cross-Site Scripting (XSS) here.

XSS in SAPUI5 Controls

You must ensure that an attacker cannot inject script code into an application page as it runs in a user's browser. From a controls perspective, this means prohibiting controls from writing any scripts to the page that comes from the application, or might have come from business data saved by a different user. This is achieved by a combination of two measures:

- **Validation**
  - The validation of typed control properties means that as soon as a control property has a type other than `string`, the Core validates the value against this type when the application sets it. In this way, an `int` is always guaranteed to be an `int`, and a `sap.ui.core/CSSSize` is a string representing a CSS size (and not containing a script tag). This is also the case for enumerations and control IDs. The control renderer can rely on this check when writing the HTML.
- **Escaping**
  - Escaping of `string` control properties and any other values coming from the application. This is the responsibility of the control developer when creating the renderer. The RenderManager and the Core offer helper methods for this (see below).

What You Can Do to Avoid XSS for Your New Renderer

To ensure maximum security for your new renderer, follow the steps described below:

- Use the type `sap.ui.core/CSSSize` instead of `string` (or `sap.ui.core/string`) for control properties that refer to a CSS size.
  - The more general rule is to use the most specific type for control properties that is available.
- If the value of a string property is written to the HTML, it must be escaped using one of the helper methods:
  - For writing plainly to the HTML, use `writeEscaped(oControl.getSomeStringProperty())` instead of just `write(...)`.

For writing attributes, use `writeAttributeEscaped("someHtmlProperty", oControl.getSomeStringProperty())` instead of just `writeAttribute(...)`.
    - For any usages of string properties where the above is not possible, use `jQuery.sap.escapeHTML(oControl.getSomeStringProperty())` to escape the string and then process it further.
- Carefully check the HTML coding you are writing and consider whether any application value might make its way to the HTML.
    - Check where the variable values come from (can the application set its value directly, or only decide which of certain hardcoded values are used?).
    - Parameters in method calls of controls are (currently) not validated by the Core, so values given there are candidates for extra escaping.
    - Always keep in mind that XSS can happen anywhere and anytime in CSS classes, or in styles, for example.

Focus Handling

Overview

Each of the controls provided by the SAPUI5 framework has its own behavior for focus handling, depending on the functionality that is provided by the control. The highest level of complexity is reached in the case of complex controls and their embedded content.

For custom development of controls, the core of the framework provides mechanisms for observing the moving focus in an application page and preserves this information for refocusing elements after (re-)rendering. Furthermore, the focus triggers event firing. Due to the high degree of flexibility in control rendering, it may be necessary to implement functionality tailored to the control's inner workings. The following sections provide some insights into how to tailor this functionality.

The framework provides helper functions for implementating focus handling.

Implementing Focus Handling When Developing Controls

The base class for all elements `(Element.js)` provides four methods to help you implement focus handling:

- `Element.getFocusDomRef()`
- `Element.focus()`
- `Element.getFocusInfo()`
- `Element.applyFocusInfo(oFocusInfo)`

`Element.getFocusDomRef()`

Once a visible element is rendered, it has a Document Object Model (DOM) representation. The root DOM node can be accessed by using the method `getDomRef()` on the element. The root DOM node is the default focused DOM node. After rendering, the framework asks the control for its focus DOM node by using the `getFocusDomRef()` method. If the root DOM node does NOT represent the element that should have the focus, you have to return another DOM node by overriding the `getFocusDomRef()` method.

`Element.focus()`

The `focus()` method sets the focus on the element. This is done using the focus DOM node.

`Element.getFocusInfo()`

With some controls, it is even more difficult to apply the focus once the control has been re-rendered. Controls such as lists have their own internal focus handling, which sets the focus on the different items. A DataTable? moves the focus over a matrix of cells. The requirement is that a control can apply the focus to its exact previous position after re-rendering. To provide this functionality, you override the `getFocusInfo()` method, and serialize the focus state into a JSON object and return it. Before rendering, the render manager calls this method for the element instance and stores this information for future use. After rendering, it calls the method `applyFocusInfo()` (see next method) and passes back this serialized object. The cursor position of a TextField control, for example, can also be stored in such an object.

`Element.applyFocusInfo(oFocusInfo)`

The `applyFocusInfo()` method applies the focus to the element after re-rendering. You use this method if different behavior is expected for the element. Note that the default implementation sets the focus as it is implemented in the `focus()` method (see above).

Example

In our example, a control usually sets the focus on the second child node of its root node. In this case, you would simply override the `getFocusDomRef()` method.

```
sap.ui.commons.<SampleControl>.getFocusDomRef = function() {
   return this.getDomRef().firstChild.nextSibling;
}
```

Another control generally sets the focus back to the element that previously had the focus. Therefore, it overrides the methods `getFocusInfo` and `applyFocusInfo`.

```
sap.ui.commons.<SampleControl>.getFocusInfo = function() {
   return {id:this.getId(),idx:this.<myFocusElementIndex>};
}
sap.ui.commons.<SampleControl>.applyFocusInfo = function(oFocusInfo) {
   var oDomRef = this.getDomRef();
   if (oDomRef) {
      this.<myFocusElementIndex> = oFocusInfo.idx;
      this.focus();
   }
}
```

Convenience Functionality

In addition to the automatic focus handling provided by the interaction between the render manager and the element instance, further focus-related functionality is available in the jQuery plugin `jquery.sap.dom` (see jQuery.sap.focus) and in the `sap.ui.core.Core` (see Core.getCurrentFocusedControlId).

ItemNavigation: A Helper for Keyboard Handling in List-like Controls

One common pattern for keyboard navigation is the item navigation in lists. SAPUI5 therefore provides a helper class sap.ui.core.delegate.ItemNavigation that implements this functionality. It is intended to be used as a **delegate** for the keyboard events occurring inside the using control.

Each control that needs the ability to navigate with arrow keys over a one dimensional list of DOM nodes can use this delegate. The delegate hooks into the browser events for arrow up/down/left/right, page up/down and home/end keys. With a list of DOM nodes provided by the control it sets the focus to the right DOM node in the list while handling the events.

In order to use this item navigation handling, the control needs to provide a DOM node that surrounds all DOM nodes of the items as well as a list of all the DOM nodes of the items themselves. The surrounding DOM node should initially get the focus when the control is entered to ensure the ItemNavigation delegate can do proper focus handling.

Via the methods `setCycling` the developer can choose whether the focus automatically moves to the top after the end of the list was reached. The page up/down keys will only work if the control developer sets a page size via `setPageSize` method on the delegate.

Some controls also have one currently selected item in the list that initially should get the focus when the Control is entered (again). With the `setSelectedIndex` method the control can specify such a preselected item for the delegate. If no selected index is given the first item will get the focus when entering the control.

If the control needs to be triggered before it will be focused by the item navigation, it can handle the events `BeforeFocus` respectively `AfterFocus`, to do e.g. preparation tasks for the controls visibility etc.

Using this delegate will save your control about 100 lines of JavaScript code for keyboard handling. If you like you can still react on the events handled by the delegate in your control.

How to Integrate the ItemNavigation Helper in Your Control

A delegate should be applied in the `onAfterRendering` hook of a control and besides that should be applied only once:

```
sap.ui.commons.ListBox.prototype.onAfterRendering = function () {
        //Collect the dom references of the items
        var oFocusRef = this.getDomRef(),
                aRows = oFocusRef.getElementsByTagName("TR"),
                aDomRefs = [];
        for (var i=0;i<aRows.length;i++) {
                aDomRefs.push(aRows[i].firstChild);
        }
        //initialize the delegate add apply it to the control (only once)
        if (!this.oItemNavigation) {
                this.oItemNavigation = new sap.ui.core.delegate.ItemNavigation();
                this.addDelegate(this.oItemNavigation);
        }

        // After each rendering the delegate needs to be initialized as well.

        //set the root dom node that surrounds the items
        this.oItemNavigation.setRootDomRef(oFocusRef);

        //set the array of dom nodes representing the items.
        this.oItemNavigation.setItemDomRefs(aDomRefs);

        //turn of the cycling
        this.oItemNavigation.setCycling(false);

        //set the selected index
        this.oItemNavigation.setSelectedIndex(this.getSelectedIndex());

        //set the page size
        this.oItemNavigation.setPageSize(this.getVisibleItems());
};
```

Please ensure that the delegate is also correctly removed after the control is destroyed. Otherwise memory will leak because DOM nodes are still referenced by the delegate.

```
sap.ui.commons.ListBox.prototype.destroy = function() {
        if (this.oItemNavigation) {
                this.removeDelegate(this.oItemNavigation);
                this.oItemNavigation.destroy();
        }
};
```

Right-to-Left (RTL) Support

Mini Introduction to RTL

What do the specs say?

- Unicode defines the directionality of characters which the browser must take into account when arranging characters as words.
- The HTML "dir" attribute overrides the overall directionality of blocks (and influences the text alignment if not explicitly set). In detail there are many more effects, e.g. parentheses are mirrored and viewed as a "word" on their own, which changes their position.
- the HTML "lang" attribute does NOT influence text directionality
- document.dir can be used (browser support is there, it can be set in the bootstrap already, but webkit seems to reset this to LTR, if set too early)
- There is a <BDO> HTML tag which turns off the bidirectional algorithm, i.e. the character order is not reversed if RTL and LTR words are mixed.
- CSS 2.1 has also a "direction" property

What does this mean in a nutshell?

Read this to (hopefully) understand in 1-2 minutes how RTL works:

- Each character inherently belongs to a RTL or LTR script (defined by Unicode). Some characters like parentheses and dots have no inherent directionality. Browsers know all of this.
- For single "words" (characters sequences with same directionality) the browser knows the text direction, and does it "right" automatically, handling them as sort of **blocks** that get their internal text direction ONLY from the used characters.
- These words/sequences/"runs" are separated by the direction-neutral characters like parentheses and dots - and obviously when character directionality changes.
- The direction in which these blocks are put *next to each other* depends on the **base direction**.
- The default base direction of HTML is left-to-right, but can be inverted by setting the attribute "dir='rtl'", either on the <html> tag or on any subregion which should have a different base direction.
- This base direction also determines the default text alignment, the order of columns in tables and the presentation of some direction-neutral characters: opening parentheses are still opening parentheses when RTL mode is switched! This means they render horizontally flipped!
- The algorithm for ordering words/runs/blocks according to the base direction only covers one level of mixed directionality. To achieve deeper nesting, spans with dir attribute can be used o define a subcontext with different base direction.
- While (as said above) words/runs internally always have the text direction implied by the used characters, this behavior can also be overridden (using the <bdo> tag or via CSS `unicode-bidi:bidi-override`) when the order of characters must follow the base direction regardless of the inherent character direction. E.g. because the characters do not form a word but are *really* just a list of characters, like e.g. in a part number.
- The "lang" attribute defining the document language does not have any influence on directionality.

UI5 Theming Concept - RTL Support

The UI5 theming concept defines RTL support to be achieved as follows:

- All controls are styled for the LTR case in their "normal" CSS files
  - Images are put into/below the "img" subfolder
- The CSS generator includes an RTL flipping algorithm (inherited from previous SAP UI technologies, proven over many years, similarly also used in the Open Source community, see CSSJanus)
  - `border-left:` is converted to `border-right:`, `padding: 1px 2px 3px 4px;` is converted to `padding: 1px 4px 3px 2px;`, `float:left;` is converted to `float:right;` etc. The assumption

and experience is that this automated conversion does almost the whole job.
    - Most CSS properties, including CSS3 ones are supported
- CSS generation also mirrors all images inside the "img" folder, as this has been found to be the right thing in most cases
    - Some images should not be mirrored for RTL. The RTL version of those images must then be put into the "img-RTL" folder with the same location and name. In this case the image reference in the CSS is modified to point to this image.
- A control should NOT write any special CSS classes to "notify" the CSS that RTL is turned on
    - In CSS the <HTML> tag's dir attribute can be checked to provide specific RTL style - but beware: the written style is still mirrored in the actual RTL case! Example:

      ```
      html[dir=rtl] .sapUiBtn {
         color: red;
      }
      ```

      However: this should only be required in rare cases! If you find yourself writing this kind of style more than 1-3% of the time, you should have another look at what UI5's automatic CSS mirroring offers.

Programmatic Access to RTL Mode

Some controls need to provide specific coding in case of RTL mode, e.g. because they are positioning or animating elements programmatically, not via CSS.

The UI5 RTL configuration can be read by calling

```
var bRtl = sap.ui.getCore().getConfiguration().getRTL();
```

What do applications need to know and do?

This wiki page will not supply the general knowledge to build RTL-aware applications, but with respect to UI5: the text direction is LTR by default but can be set using the various known configuration switches:

- URL parameter `sap-ui-rtl=true`
- Configuration object: `window["sap-ui-config"].rtl = true;`
- Script tag configuration: `data-sap-ui-rtl="true"`

UI5 will take care of setting the overall page direction to "RTL". All UI5 content will then be displayed in RTL mode. For self-written controls and content you need to test. If manual style adoptions are required, you need to provide style specifically for the RTL case by using `html[dir=rtl]`

Steps

If UI5 is configured to RTL mode, the core:

1. sets dir="rtl" on the HTML tag
    - The W3C officially recommends using the HTML attribute instead of the CSS properties, e.g. directionality is a matter of content, not of presentation, and because the CSS properties may even be ignored.
    - Using the <HTML> tag (not the BODY tag) is recommended as well
2. loads the respective library-RTL.css files

Resources

Standards:

- W3C bidi tutorial: http://www.w3.org/International/tutorials/bidi-xhtml/
- HTML 4.01 "dir" attribute/text direction: http://www.w3.org/TR/html401/struct/dirlang.html
- HTML 5 "dir" attribute: http://dev.w3.org/html5/spec/Overview.html#the-dir-attribute

- CSS 2.1 text direction: http://www.w3.org/TR/CSS21/visuren.html#direction
- CSS3 has its own "writing modes" module: http://dev.w3.org/csswg/css3-writing-modes/

Other Resources:

- A page with Hebrew RTL text: http://www.i18nguy.com/unicode/shma.html
- Good introduction: http://www.i18nguy.com/markup/right-to-left.html
- List of languages and their directionality: http://www.w3.org/International/questions/qa-scripts.en.php
    - Of the scripts commonly used in computer application localization, only Arabic and Hebrew are written right-to-left.
    - Scripts used in China, Hong Kong, Japan and Korea are written left-to-right or top-to-bottom. We happily choose the former...

Modularization and Dependency Management

Overview

The UI5 framework has built-in support for **modularizing** comprehensive JavaScript applications. That means, instead of defining and loading one large bundle of JavaScript code, an application can be split into smaller parts which then can be loaded at runtime at the time when they are needed. These smaller individual files are called **Modules**.

To load a module, the `jQuery.sap.require` function must be used. Assume the following page:

```
    <!-- UI5 bootstrap tag -->
    <script id="sap-ui-bootstrap" src="./resources/sap-ui-core.js"
data-sap-ui-libraries="sap.ui.commons"></script>

    <script>
        jQuery.sap.require("sap.ui.commons.MessageBox");

        function onPressButton() {
            sap.ui.commons.MessageBox.alert("Hello World!");
        }

    </script>
```

At first, the UI5 framework initializes and then loads the 'sap.ui.commons.MessageBox' module. Internally, the framework analyzes the module name and derives the module URL from it:

```
./resources/sap/ui/commons/MessageBox.js
```

The module script is then loaded from that URL and executed.

There is much more behind the UI5 module concept, but this is all you need to know to start working with it. If you want to know more, just read on!

What is a Module?

A module simply is a JavaScript file that can be loaded and executed in a browser. There are no rules or definitions what code belongs to a module and what code doesn't. It is totally up to the developer what content to bundle into a single module. But typically, the content of a module has some topic in common. Either it forms a JavaScript class or namespace or the contained functions address a specific topic, e.g. client to server communication, mathematical functions, etc .

There is also no special syntax or structure defined for modules. However, there are some features that module developers should be aware of and that they can use:

Name: As we have seen already, a module is loaded by calling `jQuery.sap.require` with the name of the module. So, all modules are identified by a name. Human readers often associate a module with the main JavaScript object declared in it. Therefore the module names by convention are a hierarchical sequence of dot-separated identifiers (like 'sap.ui.core.Core'). It is best practice to use all but the last identifier to group modules in a logical and/or organizational way (much like packages in Java) and to use the last identifier to give the module a meaningful, semantical name (e.g. the 'topic' common to the code in the module).

Declaration: Modules can declare themselves by calling the static `jQuery.sap.declare` function with their name. This helps UI5 to check at runtime whether a loaded module really contains the expected content (compare the required name against the declared name). As a side effect, `jQuery.sap.declare` will ensure that the parent namespace of the module name exists in the current global namespace (window). More details can be found in the API documentation of declare. If a module does not contain a declaration, the framework assumes that the module has the expected content and automatically

declares it with the name used for loading it. In some rare cases - which are explained below - a module declaration is mandatory.

Description: Furthermore, modules can contain a description which helps others to decide whether a module is useful for them or not. By convention, any JavaScript comment preceeding a module's declaration (`jQuery.sap.declare` statement) is interpreted as its description. The configuration UI (LINK) will display such descriptions next to the module name.

Dependencies: Last but not least, modules can use the `jQuery.sap.require` method to load other modules that they depend on. While `jQuery.sap.require` internally has the effect of a "loadModule" call, it can also be regarded as a dependency declaration (therefore its name 'require'). These dependency declarations can be evaluated at runtime (as explained above), but they can also be analyzed at built time or at runtime on the server.

Example:
A typical module that uses all of the above features might look like this (the module name is my.useful.SampleModule);

```
   /*
    * A short documentation of the module. This documentation is not evaluated at runtime,
only during build time
    */
   jQuery.sap.declare("my.useful.SampleModule"); // declaration of the module. Will ensure that
the containing namespace 'my.useful' exists.

   // list of dependencies of this module
   jQuery.sap.require("sap.ui.core.Core");
   jQuery.sap.require("some.other.Module");
   jQuery.sap.require("you.can.Also", "list.multiple.Modules", "if.you.Want");
   ...

   // create the 'main' object of the module
   my.useful.SampleModule = {};
```

Module Loading

As mentioned already, modules are loaded by calling function `jQuery.sap.require` with the name of a required module. The framework then checks whether the named module is loaded already. If so, the function simply returns. Otherwise it tries to load and execute the module synchronously. If any of these two steps fails, an exception is thrown and execution of the calling module thereby is disrupted. To summarize it:

```
A call to jQuery.sap.require ensures that the required module is loaded and has been executed
before execution of the caller continues (*).

(*) this is only true as long as no cyclic dependencies occur.
```

When loading a module, its dot-separated name must be transformed to an URL. This is done by replacing all dots ('.') with slashes ('/') and appending the standard suffix '.js' to it. This transformed name is then appended to the UI5 resource root URL (a prefix of the URL where UI5 has been loaded from, see explanation of bootstrap). The resulting URL is then used to load the module as a text. If loading succeeds, the module is first declared with the original module name and then executed in a global context (window). If either loading the module or executing it fails, the module name is internally marked as "failed" and an exception is thrown indicating the cause for the failure. Any further tries to load the same module will fail immediately, reproducing the same error message.

Multiple Module Locations

It is a common use case for web apps, that different modules are located in different locations (servers, web apps). Imagine for example that your web application is deployed as an individual web app and that it contains some very important modules to be loaded at runtime. But for administrative reasons, SAP UI5 and its provided modules have to be loaded from a content delivery network (CDN) or from a centrally deployed web app. As explained above, UI5 by default will try to load any required modules

from its resource root URL, namely from the centrally deployed web application. This would fail for the modules contained in your web application.

Such mixed location scenarios are supported with the `jQuery.sap.registerModulePath` function:

```
jQuery.sap.registerModulePath = function(sModuleNamePrefix, sURL);
```

It associates a module name prefix with an URL prefix. Any module whose name starts with the module name prefix will be loaded from the registered URL instead of the standard resource root URL. In the scenario prethought above, this can be used to redirect the request for the application-specific modules to the corresponding web application:

```
<!-- bootstrap tag which implicitly defines the resource root as
'http://www.sap.com/sapui5/1.0/resources/' -->
<script src="http://www.sap.com/sapui5/1.0/resources/sap-ui-core.js" ></script>

<script>
   // request will be mapped to http://www.sap.com/sapui5/1.0/resources/sap/ui/core/Core.js
   jQuery.sap.require('sap.ui.core.Core');

   // redirect the 'my.webapp' package to the local web app
   jQuery.sap.registerModulePath('my.webapp', '/my-webapp/resources/my/webapp/');

   // loads /my-webapp/resources/my/webapp/MyModule01.js
   jQuery.sap.require('my.webapp.MyModule01');
</script>
```

Note that the registered URL above contains the transformed module name prefix 'my/webapp/'. While this seems to be an unnecessary redundancy in the registration, it allows a more flexible packaging of the modules. For example, a company might decide to deploy all its specific modules named 'my.company.*' to the central URL 'http://my.company/shared/' without packaging them into a two level hierarchy of subfolders:

```
jQuery.sap.registerModulePath('my.company', 'http://my.company/shared/');
```

However, when the standard build tools of the UI5 framework are used, the full package name will be part of the runtime file hierarchy and the registration must contain the transformed package hierarchy as above.

Dependency Resolution Tool

The previous section contained some explanations how dependencies between modules are resolved on the client at runtime. During development, this is the typical use case. Modules can be modified in the development environment and can be deployed as individual entities to some runtime. When the client then is refreshed - and if caching is configured properly - it will reload only the modified modules.

In productive systems however, it might be desirable to bundle again several modules into one single file. This helps reducing the number of necessary roundtrips and can thereby help to reduce the impact of network latency. However, one doesn't want to loose the flexibility and transparency of the dependency management.

The UI5 framework supports this with a dependency resolution tool. It analyzes a module file and all its dependencies and creates a new file containing the original module content, as well as any required modules. It automatically avoids double inclusion of modules. The tool can be used in two ways: Either via an Ant task at build time to create a merged super module which then can be referenced in any HTML page instead of the original file or at runtime, then using a servlet on server side.

How to Avoid Duplicates?

When the runtime dependency resolution is used, the runtime maintains a list of the loaded modules. Before a new module is loaded and executed, the list is searched for it and if found, the module is not loaded again. But when the server or build-time tool is used, it creates a bigger file potentially containing multiple modules. The runtime then can only check in advance whether that bigger module has been loaded already. It does not know about the contained modules and therefore can not avoid

double-loading of them. To compensate this, the dependency resolution tool wraps any embedded module with a few lines of additional coding. These additional checks will be evaluated during execution of the merged module and will have the same effect as the original runtime checks in an unmerged scenario:

```
    ...
    // code of enclosing module
    ...

    // location of a jQuery.sap.require('xyz');

    if ( !jQuery.sap.isDeclared('xyz') ) { // check whether module 'xyz' has been loaded
already

        jQuery.sap.declare('xyz'); // will only be added if the module 'xyz' doesn't contain a
declaration

        // original text of module 'xyz'
        ...
    }

    ...
    // further code of enclosing module
    ...
```

Note that the generated wrapper coding will also contain a module declaration if the module doesn't contain one. The wrapper avoids double loading no matter whether a module has been loaded as an individual file or as part of a bigger, merged module. It is even possible to recursively merge files (merged module A includes a merged module .

Why Not Simply Concatenating Modules?

One might wonder why multiple modules can not simply be concatenated into a bigger module. Why have the modules to be parsed and to be nested according to the original `jQuery.sap.require` calls? The answer simply is that this makes the runtime behavior of the merged file more predictable. As soon as you look at concrete modules with complex (or even cyclic) dependencies, order of module execution becomes significant. The main promise of `jQuery.sap.require` that the caller continues only after the required module has been successfully loaded and executed can be hold only if the required module is embedded exactly at the place where the `jQuery.sap.require` call was located.

In cases where a use of the dependency resolution tool is not possible at all, one might indeed simply concatenate modules. But then the following two criteria must be ensured 'manually' by the developer:

- the order of the files must obey the dependencies. A module must not 'require' another module that is only merged lateron
- all merged modules must do a declaration with `jQuery.sap.declare`, otherwise the framework will not know that the modules have been loaded and potentially load them again.

Configurator Servlet

As mentioned already, one way of executing the dependency resolution tool is to call it via a servlet. Such a servlet has been included in the `sapui5-internal` application that is part of our drop. By default, the servlet is configured to react on the URL `http://` *host:port* `/sapui5/download/configurator`. It accepts several parameters

URL Parameter	Default	Description
modules	*none*	*Mandatory*: a comma separated, ordered list of module names that should be included in the resulting module. If the parameter occurs multiple times, the values will be concatenated
out	"sap-ui-custom.js"	Name of the resulting module. The resulting module will contain a declaration with that name. When the servlet is used from the configurator Web UI, then the name will also be suggested in the download dialog.

minimize	false	Activates the JavaScript minimization for the output (experimental feature)

The Configurator WebUI which is part of `sapui5-internal` as well, uses the servlet to create a downloadable JavaScript file containing all selected modules.

But it is also possible to use the servlet directly from within an application page and to load UI5 and the required controls etc. in one step. The following HTML fragment shows an example (line breaks added for better readability):

```
        <script id="sap-ui-bootstrap"
                data-sap-ui-theme="sap_goldreflection"
                data-sap-ui-libs="sap.ui.commons"
                src="/sapui5/download/configurator?modules=jquery-
1.7.1,jquery.sap.global,sap.ui.core.Core,
sap.ui.commons.Button,sap.ui.commons.ButtonRenderer,
sap.ui.commons.layout.MatrixLayout,sap.ui.commons.layout.MatrixLayoutRow,sap.ui.commons.layout.Mat

sap.ui.commons.layout.MatrixLayoutRenderer"
        >
        </script>
```

Special Cases

Chicken & Egg, or: How to Load jquery.sap.require?

Obviously, modules can only be loaded as soon as the jQuery.sap.require function is available. The implementation of this function is located in module 'jquery.sap.global' which in turn requires jQuery itself (located in module 'jquery-1.7.1'). At runtime, these two modules can not be loaded with 'jquery.sap.require' but must be loaded by some other mean. The UI5 framework includes both modules in its bootstrap files 'sap-ui-core.js' and 'sap-ui-core-lean.js'. The first one also embeds the UI5 core functionality and needs no further modules. The second one only contains the two bootstrap modules and a require statement for the core. It is better suited for the development scenario described above (loading the modules separately).

If you create a new bootstrap file with the configuration UI and decide to include the jquery.sap.global or jquery modules, they always will be the first modules in the created file, and they will always be embedded. This ensures the availability of 'jQuery.sap.require'.

Cyclic Dependencies

Sometimes, the functionality in two modules A and B might be interdependent. That means module A requires module B to execute and module B requires module A. We stated above that jQuery.sap.require ensures that the execution of a calling module doesn't continue until the required module has been loaded and executed. Taking this serious, cyclic dependencies could not be resolved but would lead to an endless series of requests (A→B→A→B→....).

This situation can be avoided by a workaround: As soon as a module A has been loaded and is about to execute, it is regarded as *declared*. So, when this module A embeds another module B which has not been loaded, module B will be loaded and executed. If B now again requires A, then the dependency resolution runtime will find that A has been declared already (despite the fact that its execution has not been finished yet) and simply returns. This workaround helps to break the endless loop, but it doesn't re-ensure the original promise of `jQuery.sap.require`.

Cyclic modules have to deal with that gap on their own. There are several ways/best practices how to do this:

Variant 1: Merge A and B

If the conflicts can not be resolved easily, or if the modules are so highly related that they will be used together most of the time, then merging them into one module is the most simple solution.

Variant 2: Interlaced Execution of A and B

This variant makes use of the fact that the module loading takes place exactly at the source location where the jQuery.sap.require function is executed. Let's assume that the content of modules A and B can be structured as follows:

```
    // Module A, Part A.1

    // Module Section A.1, does not depend on Module B and provides all code that module B
depends on.

    jQuery.sap.require("B");

    // Module Section A.2, might depend on code in Module Section B.1

    // Module B, Part B.1

    // Module Section B.1, does not depend on Module A and provides all code that module A
depends on.

    jQuery.sap.require("A");

    // Module Section B.2, might depend on code in Module Section A.1
```

Further assume (WLOG) that module A is loaded first. Then section A.1 will be executed and will be available to the outside world before the `require('B')` is executed. During the require, the framework will detect that B is not available yet, will load and execute it. The execution starts with section B.1 which succeeds as it does not depend on A. When the execution of B reaches the `require('A')` statement, the framework detects that A has been loaded already and continues without loading A again. But remember, that the code from section A.2 is not available yet. The execution of B however continues and succeeds as - by assumption - B.2 does not depend on A.2. Now, the first `require('B')` succeeds and returns and section A.2 will be executed. And it might use the code from section B.1.

Procedure and result are similar in the case that B is loaded first.

Resource Handling

The Resource Handling of SAPUI5 is separated in two parts - a client-side and a server-side resource handling. Both are not dependent on each other. Furthermore they are complementary.

The server-side mechanism is not required. This is an optional component which improves the client-server interaction (server-side locale fallback instead of client-side with multiple requests) and especially is used for the Eclipse IDE development to support modularized development of SAPUI5 application and libraries.

Client-Side Mechanism

On the client-side SAPUI5 provides the following mechanism for resources:

- Modularization Concept (Require/Declare for JavaScript files)
- Localization Concept (Resource Bundles)

Both concepts are loading additional resources from a server where this server might be any kind of web server (simple, Java, ABAP, ...). It does not depend on any server side technology.

Server-Side Mechanism

For the Java server and also the integration into the Eclipse IDE SAPUI5 provides a Resource Handler. This Resource Handler is aligned with the concept of the JavaServer Faces - Resource Handler:

- The default implementation must support packaging resources in the web application root under the path `resources/<resourceIdentifier>` relative to the web app root.
- Resources packaged in the classpath must reside under the JAR entry name `META-INF/resources/<resourceIdentifier>`

The SAPUI5 Resource Handler extends this concept to support standard and test-relevant resources. Therefore we package our resources into the following paths:

- `resources/**`
  Resources are all kind of JavaScript, CSS, Mimes, Resource Bundles, which are relevant for the runtime.

- `test-resources/**`
  Test resources are resources which are samples and only relevant for testing purposes e.g. the content of the SAPUI5 TestSuite.

Other additional features of the Resource Handler are:

- Theme fallback:
  If resources are not available for the current theme it automatically checks the base theme for such resources and returns this resource instead without returning a 404.

- Resource Bundle fallback:
  Similar to the client-side mechanism for loading Resource Bundles but it negotiates the request on the server and returns the best found Resource Bundle instead without 404, e.g.:
  `messagebundle_en_US.properties > messagebundle_en.properties > messagebundle.properties`

Resource Servlet

For Java Servlet containers SAPUI5 provides a `ResourceServlet` which manages the access to SAPUI5 resources within the web application and the various UI libraries in the classpath. The following snippet shows how to enable the resource servlet for SAPUI5:

```
<!-- ============================================================= -->
<!-- SAPUI5 resource servlet used to handle application resources  -->
<!-- ============================================================= -->

<servlet>
  <display-name>ResourceServlet</display-name>
  <servlet-name>ResourceServlet</servlet-name>
  <servlet-class>com.sap.ui5.resource.ResourceServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>ResourceServlet</servlet-name>
  <url-pattern>/resources/*</url-pattern>
</servlet-mapping>
<servlet-mapping>
  <servlet-name>ResourceServlet</servlet-name>
  <url-pattern>/test-resources/*</url-pattern>
</servlet-mapping>
```

Before you can use it you need to make sure that the ResourceServlet is available in the classpath as JAR file.

Configuration

The Resource Handler is configured via context parameter which are defined in the `web.xml`. The following table gives an overview about configuration parameters:

Key	Description
com.sap.ui5.resource.USE_CACHE	flag to enable resource cache or not (default: "true")
com.sap.ui5.resource.MAX_AGE	max age of resources in millis (default: "604800000" - 1 week)
com.sap.ui5.resource.ACCEPTED_ORIGINS	list of accepted origins, e.g. * or *sap.corp,vesapui5.dhcp.wdf.sap.corp (default: empty)
com.sap.ui5.resource.DEV_MODE	flag to enable the development mode (default: "false")
com.sap.ui5.resource.TEMPLATE_PATH	template for the resource listing (default: "/templates/listing.html")
com.sap.ui5.resource.VERBOSE	verbosity of the resource handler (default: "false")
com.sap.ui5.resource.REMOTE_LOCATION	remote location which is used to proxy requests to for resources not being located locally (default: empty)
com.sap.ui5.resource.PREFER_REMOTE_LOCATION	flag to prefer resolving the resource from the remote location before fallback to the classpath (default: false)

Configuration parameters are added as context parameters to the web.xml.

Development Mode

When starting to develop SAPUI5 controls and modules being located inside the servlet paths *resources/* or *test-resources/* it makes simplifies this development process by disabling the caching of such resources as well as enabling the resource browsing. To activate the development mode you need to add the following context parameter.

```
<!-- BEGIN: DEV MODE -->
<context-param>
  <param-name>com.sap.ui5.resource.DEV_MODE</param-name>
  <param-value>true</param-value>
</context-param>
<!-- END: DEV MODE -->
```

Resource Browsing

In case of having the development mode turned on you can browse resources via the Resource Browser:

- %SERVER_URL%*resources*
- %SERVER_URL%/test-resources/

Tunneling a Remote Location

The ResourceServlet offers the opportunity to tunnel/proxy requests to another server providing SAPUI5 resources. This is the alternative instead for referring to SAPUI5 from remote location inside the bootstrap script tag to avoid cross domain issues. To activate this remote location tunneling/proxying you need to add the following context parameter to the web.xml of your application:

```
<context-param>
  <param-name>com.sap.ui5.resource.REMOTE_LOCATION</param-name>
  <param-value>http://%server%:%port%/sapui5</param-value>
</context-param>
```

This will dispatch requests from `resources/sap/ui/commons/Button.js` to `http://%server%:%port%/sapui5/resources/sap/ui/commons/Button.js`.

Additionally the resource servlet provides the opportunity to prefer the resources from the remote location instead from the classpath of the project. Therefore you can use the following option to change the search order for the resources from `WEBAPP > CLASSPATH > REMOTE` to `WEBAPP > REMOTE > CLASSPATH`:

```
<context-param>
  <param-name>com.sap.ui5.resource.PREFER_REMOTE_LOCATION</param-name>
  <param-value>true</param-value>
</context-param>
```

If you are located behind a proxy and the remote location is outside your local network you can configure the proxy settings via the standard Java Networking and Proxy configurations by setting the system properties (for HTTP): `http.proxyHost`, `http.proxyPort`, `http.nonProxyHosts`, or (for HTTPS) `https.proxyHost`, `https.proxyPort`, `https.nonProxyHosts` of your Java runtime environment.

In general for the resources returned from the proxy the ResourceServlet is enabling caching. It by default uses the configured `com.sap.ui5.resource.MAX_AGE` to avoid to much load on the ResourceServlet.

Verify that a resource was retrieved from remote location

When in development mode it is possible to verify that a resource was retrieved from the desired remote location by checking the response header of the respective request. In this case the response header has an entry *x-sap-ResourceUrl = remote resource URL*, e.g.:

```
x-sap-ResourceUrl = http://%server%:%port%/sap/public/bc/ui5_ui5/resources/sap-ui-core.js
```

Resource Packaging

This section describes the resource packaing for web applications and Java modules which could be any kind of a JAR file (SAPUI5 UI library, ...) available in the classpath of the web application.

Web Application

For a web application this means you have to store the resources in the following way:

```
WebContent/
  resources/
    **/**
  test-resources/
    **/**
```

Java Modules (JARs / UI libaries)

For the SAPUI5 UI libraries we store the resources in the following way:

```
META-INF/
  resources/
    **/**
  test-resources/
    **/**
```

For custom JAR files you need to apply to this on your own.

OSGi Servlet Container

When running SAPUI5 as an OSGi Web Bundle and referencing the UI libraries as OSGi bundles you may need to know about a technical detail how SAPUI5 OSGi bundles are determined:

- In the OSGi Servlet Container we extend the `ResourceServlet` by using an OSGi fragment which is responsible to add the OSGi flavor for the determination of UI libaries. Now the `ResourceServlet` is aware about the OSGi bundles and can search within the OSGi Servlet Container for UI libraries.
- The `MANIFEST.MF` of UI library JAR files contains a special entry which is used for the determination:

  `x-sap-ui5-ContentTypes: UILibrary`

  This is used by the `OSGiResourceServlet` to determine the relevant UI libraries.

ABAP

SAPUI5 also provides a Resource Handler for the ABAP server which is used for SAPUI5 applications to serve the resources from the UI libraries properly. This Resource Handler comes with the UI Extension Program.

Cache Buster Concepts

A **Cache Buster** allows the application to notify the browser to refresh the resources only when the application resources have been changed. Otherwise the resources can always be fetched from the browser's cache.

Cache Buster

...in 5 secs

When you want to cache your resources permanently, you simply need to change the URL in the SAPUI5 bootstrap tag from

- `resources/sap-ui-core.js`

to

- `resources/sap-ui-cachebuster/sap-ui-core.js`

That's it!

Overview

The **Cache Buster** mechanism allows to always put the SAPUI5 resources into the browsers cache until a UI library or a web application has been changed. The default behavior of the SAPUI5 resource handler is either to cache the resources for a specific amount of time or alternatively in development mode it is using the `304/NOT MODIFIED` mechanism to check the SAPUI5 resources for being up-to-date. Both mechanisms are not optimal in a final, productive scenario - that is the reason for the implementation of the **Cache Buster** mechanism. Applications which want to use the **Cache Buster** mechanism have to explicitly decide to use it!

Usage

The **Cache Buster** mechanism is part of the Resource Servlet. In general requests to JavaScript resources can be handled via the **Cache Buster** mechanism. Typically this is used for the initial request for the bootstrap JavaScript:

```
  <script id="sap-ui-bootstrap"
    src="resources/sap-ui-cachebuster/sap-ui-core.js"
    data-sap-ui-libs="sap.ui.core,sap.ui.commons,sap.ui.table"
    data-sap-ui-theme="sap_goldreflection"></script>
```

The bootstrap JavaScript will be included via the URL `resources/sap-ui-cachebuster/sap-ui-core.js` instead of `resources/sap-ui-core.js`.

Mechanism

The basic mechanism is implemented in the `ResourceServlet`. For the request to the bootstrap JavaScript it now serves a JavaScript file with the following content:

```
(function() {
  var sTimeStamp = '~20120716-0201~';
  var sScriptPath = 'sap\x2dui\x2dcore.js';
  var aScriptTags = document.getElementsByTagName('script');
  for (var i = 0; i < aScriptTags.length; i++) {
    if (aScriptTags[i].src) {
      var iIdxCb = aScriptTags[i].src.indexOf('/sap-ui-cachebuster/');
      if (iIdxCb >= 0 && aScriptTags[i].src.substring(iIdxCb + '/sap-ui-cachebuster/'.length)
== sScriptPath) {
        var sBasePath = aScriptTags[i].src.substring(0, iIdxCb);
        sBasePath += '/' + sTimeStamp + '/';
```

```
        window["sap-ui-config"] = window["sap-ui-config"] || {};
        window["sap-ui-config"].resourceRoots = window["sap-ui-config"].resourceRoots || {};
        window["sap-ui-config"].resourceRoots[''] = sBasePath;
        document.write('<script type="text/javascript" src="' + sBasePath + sScriptPath +
'"></script>');
        break;
      }
    }
  }
})();
```

This script basically ensures that the global SAPUI5 configuration variable (`window["sap-ui-config"]`) exists, without modifying any existing values. It defines the resource root of SAPUI5 (the location where SAPUI5 loads all JavaScript modules, controls and control related resources from). Finally, another script tag is added to the page that points to the real bootstrap JavaScript. The new resource root and the request path to the bootstrap JavaScript now contain a timestamp. Additionally the cache headers of the reponses now look like the following:

```
Date: Mon, 16 Jul 2012 05:17:54 GMT
Expires: Thu, 14 Jul 2022 05:17:54 GMT
Cache-Control: max-age=315360000, public
```

By default all **Cache Buster** resources will be cached for one year.

Request Flow

When using the **Cache Buster** mechanism, the first request must never be cached because it is being used to determine the timestamp / and to finally redirect to the correct script. The following list explains the flow:

1. `resources/sap-ui-cachebuster/sap-ui-core.js` *NO_CACHE*
2. `resources/~201106210204~/sap-ui-core.js` **CACHE**

Timestamp

If you are interested in the timestamp of the **Cache Buster**, you can grab it with the following request:

- `resources/sap-ui-cachebuster`

The response is `text/plain` with such value:

```
~20120716-0201~
```

Application Cache Buster

The **Application Cache Buster** (short **AppCacheBuster**) is a similar caching mecahnism like the **Cache Buster** but for application resources.

Concept

Applications will provide an index file (created on the fly) containing the last modified timestamps of all included files (scripts, properties – files which we load via XHR programmatically). Technically this file is a mapping between the request path (below the context path of the application) and the last modified timestamp.

The server in general caches all the above resources (not using the 304/not modified mechanism). For the index file we are using the 304/not modified mechanism to avoid to load when it has not been changed.

On the client-side we initially load this file of the application when enabled via configuration option `sap-ui-appcachebuster` and use this for the XHR requests. If the request path is contained in the above mentioned index file we simply add the timestamp as leading path segment to this request. If the timestamp doesn't change the URL is unique and therefore it will be taken from cache. Once the file is

modified the URL parameter will be changed and therefore loaded again from the backend.

The server has to delete the timestamp from this URL to lookup the file properly. For the Java applications SAPUI5 provides a `AppCacheBusterFilter` and for ABAP the logic is implemented in the ICF handler. Both backend implementations also generate the index file on-the-fly.

Index File

In difference to the cache buster mechanism for the runtime resources the application files have an own timestamp for each file. Therefore the application provides the index file: **sap-ui-cachebuster-info.json** file. This file - in JSON format - includes all files that should use this mechanism. The index file looks like this:

```
{
  "mvc/MyMVC.view.js": "20120907134005",
  "mvc/MyMVC.controller.js": "20120907134005",
  "mvc/MyMVC.view2.js": "20120906113301",
  "mvc/MyMVC.controller2.js": "20120906113023"
}
```

Note: this file is generated by the server. There is no need to create it manually.

Configuration

To activate the **Application Cache Buster** the configuration `data-sap-ui-appCacheBuster="./"` must be added to the bootstrap script of the application page:

```
<script id="sap-ui-bootstrap"
        src="resources/sap-ui-cachebuster/sap-ui-core.js"
        data-sap-ui-libs="sap.ui.core,sap.ui.commons,sap.ui.table"
        data-sap-ui-theme="sap_goldreflection"
        data-sap-ui-appCacheBuster="./"></script>
```

The parameter `data-sap-ui-appCacheBuster` is a `string[]` which means you can pass a list of base URLs for other applications. By default it should contain the base path of your local application.

These base URLs are used to load the index files. This allows to handle not only the local resources via the Application Cache Buster. Furthermore other applications could then also be handled.

Request Flow

When using the **Application Cache Buster** mechanism, the first request must never be cached because it is being used to fetch the index file. The following list explains the flow:

1. `http://myserver/myapp/sap-ui-cachebuster-info.json` *NO_CACHE*
2. `http://myserver/myapp/~201106210204~/mvc/MyMVC.view.js` **CACHE**
    - `http://myserver/myapp/mvc/MyMVC.view.js` *internally resolve to this URL*

Enable the Filter for Java applications

To enable the server-side part of the Application Cache Buster mechanism the following filter needs to be configured in the web.xml:

```
<!-- ============================================================== -->
<!-- AppCacheBuster Filter                                          -->
<!-- ============================================================== -->

<filter>
    <display-name>AppCacheBusterFilter</display-name>
    <filter-name>AppCacheBusterFilter</filter-name>
    <filter-class>com.sap.ui5.resource.AppCacheBusterFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>AppCacheBusterFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

Not supported for NW AS Java: The Application Cache Buster requires an active component on the server side which is not available as compile time dependency on NW AS Java yet. We support this right now only for web applications build with LeanDI.

Enhanced concept (since 1.10)

The first iteration of the Application Cache Buster only supported files which have been loaded via `jQuery.ajax`. The enhanced concept now also supports to transform URLs for `jQuery.sap.includeScript`, `jQuery.sap.includeStyleSheet` and properties of the type `sap.ui.core/URI`. This ensures that the Application Cache Buster mechanism takes care about most of the URLs in a general way. Additionally the enhanced concept allows to register components or base URLs which are considered by the Application Cache Buster. This base URL is used to load the index file with the timestamp information.

Registration of external URLs

If you do not specify all the applications in the bootstrap configuration you can also register them during runtime. You can register additional locations via the following API:

```
sap.ui.core.AppCacheBuster.register("/sap/bc/my/other/component");
```

Avoid handling of specific URLs

To avoid handling of specific URLs you can override the default behavior in the following way:

```
sap.ui.core.AppCacheBuster.handleURL = function(sURL) {
  return sURL !== "my/specific/url";
};
```

How can I adapt the visuals of a control?

While there is always the option to create a new Theme, this is overkill for most minor style adaptations. For those minor changes, the recommendation is to include additional CSS into the page which changes the style of the respective tags of the SAPUI5 control. This allows complete, arbitrary changes of the visual design - after all it is the same technology that the UI5 controls use for their styling.
The main options are:

- Inspect the HTML and CSS of a control and write a similar, but adapted CSS rule for a CSS property you want to override for all controls of a type.
- Call `.addStyleClass("myStyle")` on some control instances if you want only those instances to look different than other instances - and then write CSS code that refers to the normal classes/tags AND to this CSS class you just added.

NOTE: with this high degree of power and flexibility comes quite some responsibility. With CSS you are perfectly capable of destroying the functionality of a control. This is no UI5-specific thing, but when you do CSS adaptions, you should always have good knowledge of this open standard.

NOTE 2: the inner structure of a control, the tag hierarchy, the IDs and CSS classes are NOT part of the public control API that we guarantee to stay stable. This is also the case in other UI libraries which may define some CSS classes as stable, but not everything else. As CSS can refer to the inner structures of a control, you have to accept the risk that your style changes break when we change the inner structure. Changing the inner structure is a freedom we absolutely need to reserve, so we can fix bugs and add features to a control, so this is nothing that can be discussed.

NOTE 3: when your CSS does not work as expected, use Firebug or the IE F12 developer tools or similar tools to inspect the page and check which CSS rules are applied to the respective tag - and which rules may be applied, but overridden by other rules. If your rules are overwritten by other rules, this is probably because of either the order of appearance (last rule wins) or because of the CSS selector specificity (more specific CSS selectors win). If the latter sounds strange to you, look it up in the CSS spec or on Google.

DON'Ts:

- Do not adapt the style attribute of HTML elements belonging to UI5 controls. When these controls are re-rendered, the changes will be lost.

Why do SAPUI5 controls not have a 'style' property where I can do arbitrary changes?

One tricky thing is that a control usually does not map to ONE HTML element, but to a whole tree of HTML elements. Whatever is set to the 'style' property we would probably add to the root element of this HTML tree – and only there, so you have no 'style' access to inner parts. If you just want to override the height of a Button, this would actually work. But as soon as things get more complex, it will not work that easy. And “more complex” already starts with the height of a ComboBox. The outer will get the proper height, yes. And incidentally also the <input> tag inside, as it has 100% height set. But the dropdown arrow and the respective button-kind-of-thing has a fixed height and the whole control will look pretty broken, then.

In other cases, when HTML elements are nested that break the CSS inheritance chain e.g. like a <table> does for font settings, you may change the “style” to a different font and text color, but it will just do nothing.

In general, we try to expose the obvious adaptation stuff in the API. Button height for example is no longer missing since May 13th. But the less obvious adaptations may require support inside the control to work properly and as we cannot foresee and support everything you can do with a 'style' property, we raise the bar a little bit by requiring you to write CSS (potentially using `.addStyleClass(…)` on the respective control). With CSS you can do what you cannot do with a 'style' property: tweak the inner HTML components of a control.

Plus: SAP applications (at least the more traditional ones – currently this seems to be less of a rule, but I'm not sure it will stay like this forever) need to conform to some visual design guideline and in general it is not even desired that applications change the TextField height or used font however they like. As you can use CSS, UI5 still supports that, but we shouldn't make breaking the visual design an option in our official API.

How can I provide additional CSS which is not overwritten by the UI5 CSS?

When UI5 is used "normally" (which means: loaded by a <script> element in the <head> of a page, all libraries declared in the respective attribute of the script tag), it is sufficient to just add the custom CSS at any place after the UI5 <script> element. UI5 will insert its CSS links immediately after the <script> tag, so any later CSS will appear later in the document and can thus overwrite the UI5 CSS.

However, it is important to understand the precedence rules of CSS!
The order of appearance is **not the only factor** deciding which one of two or more conflicting rules wins. Actually it is only the least important factor. In practice, the most important (and maybe least known) factor is the "specificity" of the selector belonging to a rule.
For example, if one rule says `button {color:red;}` to make all button texts red and a second rule says `div > button {color:green;}` to make all texts green in buttons which are direct children of a element, the second rule always wins, if applicable, because it is more specific. The order of appearance in the document does not matter in this case. It would only matter if both rules would start with an equal selector like `button{color:***}` .
The order of loading is completely irrelevant, only the position in the document counts in this case.

See http://www.w3.org/TR/CSS21/cascade.html#cascading-order for the respective part of the CSS spec and http://www.w3.org/TR/CSS21/cascade.html#specificity for more on specificity.

If you load UI5 not with a <script> tag in the <head>, or if you did not specify all used control libraries in the <script> tag, but loaded some later on when the body was already loaded, you can still make sure a custom CSS appears later in the document by loading it with `jQuery.sap.includeStyleSheet(stylesheetUrl[, id])` **after** loading UI5 or the dynamically loaded control library.

I am adding a style class, but it does not work! Why?

If you want to change some styling and use control.addStyleClass(...) to add a CSS class, but it does not seem to work, you first have to understand WHAT is not working:

1. Is the style class not added to the HTML?
2. Or is the style class correctly added, but the style you supplied is not applied by the browser?

You can check this by inspecting the HTML with your browser's developer tools/ web inspector.

- If the style class is really not added to a control (which is rather unlikely), it would be a bug that needs to be reported and fixed. Note: some entities are not Controls, but only Elements (inherit from sap.ui.core.Element). They only sometimes have support for addStyleClass.
- If the style class is there in the HTML, the bug is inside the CSS styles you supplied:
  - Are they loaded by the browser?
  - Are the selectors matching the element you want to style? You can again check in the developer tools: they mostly list all styles which would apply, but some are overriding others (those others are usually listed striked-through). If your style is not listed at all, your CSS selector seems to be wrong.
  - If your selector is fine, but other style rules override your styles (potentially those from the original UI5 theme), then the CSS precedence rules decided this. Please read the section "How can I provide additional CSS which is not overwritten by the UI5 CSS?" above and see http://www.w3.org/TR/CSS21/cascade.html#cascading-order for the respective part of the CSS spec and http://www.w3.org/TR/CSS21/cascade.html#specificity for more on specificity.

- It could also be that your browser does not understand the CSS styles you have written. Some browsers still display them in the developer tools, some don't, so you may want to try changing very common styles like the border to check whether selector and specificity are fine.

When my custom theme is located elsewhere, how can I load it?

When there is a UI5 theme that is NOT at the same location as the core and all the default libraries, you need to tell UI5 where to find that theme. This is very much like using `registerModulePath()` for libraries that are at a different location. This is done by either static declaration in the page or by using the `Core.setThemeRoot()` method.

setThemeRoot() works as follows:

```
sap.ui.getCore().setThemeRoot("my_theme", "http://url.to/the/root/dir");
```

This will cause UI5 to load all theme resources from below this URL. E.g. the library.css file of the "sap.ui.core" library will be loaded from:
`http://url.to/the/root/dir/sap/ui/core/themes/my_theme/library.css`

For easier usage this base directory can also be given as second argument to `core.applyTheme(...)`.

In case some parts of the theme are at different locations than others, you can use the above call to set the default, but override the theme location for specific libraries by specifying them in an array as second parameter:

```
sap.ui.getCore().setThemeRoot("my_theme", ["my.lib.one","my.lib.two"],
"http://url.to/the/other/root/dir");
```

The static configuration can be done on the global configuration object (that must be created before the UI5 bootstrap script tag):

```
<script type="text/javascript">
window["sap-ui-config"] = {
        themeRoots : {
                "my_preconfigured_theme" : "http://preconfig.com/ui5-themes",

                "my_second_preconfigured_theme" : {
                        "" : "http://preconfig.com/ui5-themes",
                        "sap.ui.core" : "http://core.com/ui5"
                }
        }
}
</script>
```

The first theme is completely at one location, while the second theme has the default location changed plus the location changed for the part of the theme that covers the "sap.ui.core" library.

Or it can be done (using the same object structure as JSON string) in an attribute of the UI5 bootstrap script tag. One example

```
<script id="sap-ui-bootstrap"
        type="text/javascript"
        src="......../sap-ui-core.js"
        data-sap-ui-theme-roots='{"my_theme" : "http://themes.org/ui5"}'>
</script>
```

Last but not least, the location of a theme can be specified via an URL parameter:

```
http://myserver.com/sap/myapp/?sap-ui-theme=my-theme@/sap/public/bc/themes/~client-111
```

For more details and restrictions see the Configuration documentation for parameter `sap-ui-theme`.

How can I create a new SAPUI5 Theme?

There is not one single way to create a new theme, but there are several options. Which one to choose depends on several factors:

- How different is the desired design from an existing theme?
- Should the theme be used across several applications or just in one?
- Are sufficient CSS skills available?
- How much effort can be invested?
- How structured and "clean" should the result be?

Depending on the answers it may be that one should not even create a new theme but just adapt an existing one.

Technically, there are these different choices available:

1. Adapt an existing theme by adding CSS on application level (as described above). This is the easiest option and still sufficient for many use-cases. One can technically adapt and change everything.
2. Adapt an existing theme by using the SAPUI5 Theme Designer (currently under development) which basically modifies the color scheme, but in a very easy, non-technical manner with instant live preview. Adaptation parameters are limited, but the Theme Designer also allows adding custom CSS, which gives the same freedom as in the first option.
3. Create a new theme as SAPUI5 library project in Eclipse (currently no official documentation available). This gives a clear development structure like separated CSS files per control, again complete freedom, but requires Eclipse and considerable CSS coding effort. This effort can be reduced by copying an existing theme project as starting point. Even without copying, such a theme automatically uses the so-called "base" theme as foundation, so even when starting to implement the theme, the UI is already functional (but looks quite plain ad ugly).
4. Create a new theme from scratch, possibly without Eclipse support, writing every piece of CSS which will be loaded later. The only requirement is to have "library.css" files within a certain folder structure (which also defines the theme name).

All options except the first result in a new standalone theme which needs to be deployed and referenced by its name in the application and which could be used by any application. In the first option, the adaptation is rather done on top of an existing theme and only available within the specific application.

In all options the CSS developer could choose to limit the styling effort to those controls which are actually used in the application (which of course limits the re-use value of the theme in other applications).

Right-to-Left (RTL) Support

Mini Introduction to RTL

What do the specs say?

- Unicode defines the directionality of characters which the browser must take into account when arranging characters as words.
- The HTML "dir" attribute overrides the overall directionality of blocks (and influences the text alignment if not explicitly set). In detail there are many more effects, e.g. parentheses are mirrored and viewed as a "word" on their own, which changes their position.
- the HTML "lang" attribute does NOT influence text directionality
- document.dir can be used (browser support is there, it can be set in the bootstrap already, but webkit seems to reset this to LTR, if set too early)
- There is a <BDO> HTML tag which turns off the bidirectional algorithm, i.e. the character order is not reversed if RTL and LTR words are mixed.
- CSS 2.1 has also a "direction" property

What does this mean in a nutshell?

Read this to (hopefully) understand in 1-2 minutes how RTL works:

- Each character inherently belongs to a RTL or LTR script (defined by Unicode). Some characters like parentheses and dots have no inherent directionality. Browsers know all of this.
- For single "words" (characters sequences with same directionality) the browser knows the text direction, and does it "right" automatically, handling them as sort of **blocks** that get their internal text direction ONLY from the used characters.
- These words/sequences/"runs" are separated by the direction-neutral characters like parentheses and dots - and obviously when character directionality changes.
- The direction in which these blocks are put *next to each other* depends on the **base direction**.
- The default base direction of HTML is left-to-right, but can be inverted by setting the attribute "dir='rtl'", either on the <html> tag or on any subregion which should have a different base direction.
- This base direction also determines the default text alignment, the order of columns in tables and the presentation of some direction-neutral characters: opening parentheses are still opening parentheses when RTL mode is switched! This means they render horizontally flipped!
- The algorithm for ordering words/runs/blocks according to the base direction only covers one level of mixed directionality. To achieve deeper nesting, spans with dir attribute can be used o define a subcontext with different base direction.
- While (as said above) words/runs internally always have the text direction implied by the used characters, this behavior can also be overridden (using the <bdo> tag or via CSS `unicode-bidi:bidi-override`) when the order of characters must follow the base direction regardless of the inherent character direction. E.g. because the characters do not form a word but are *really* just a list of characters, like e.g. in a part number.
- The "lang" attribute defining the document language does not have any influence on directionality.

UI5 Theming Concept - RTL Support

The UI5 theming concept defines RTL support to be achieved as follows:

- All controls are styled for the LTR case in their "normal" CSS files
  - Images are put into/below the "img" subfolder
- The CSS generator includes an RTL flipping algorithm (inherited from previous SAP UI technologies, proven over many years, similarly also used in the Open Source community, see CSSJanus)
  - `border-left:` is converted to `border-right:`, `padding: 1px 2px 3px 4px;` is converted to `padding: 1px 4px 3px 2px;`, `float:left;` is converted to `float:right;` etc. The assumption

and experience is that this automated conversion does almost the whole job.
    - Most CSS properties, including CSS3 ones are supported
- CSS generation also mirrors all images inside the "img" folder, as this has been found to be the right thing in most cases
    - Some images should not be mirrored for RTL. The RTL version of those images must then be put into the "img-RTL" folder with the same location and name. In this case the image reference in the CSS is modified to point to this image.
- A control should NOT write any special CSS classes to "notify" the CSS that RTL is turned on
    - In CSS the <HTML> tag's dir attribute can be checked to provide specific RTL style - but beware: the written style is still mirrored in the actual RTL case! Example:

      ```
      html[dir=rtl] .sapUiBtn {
         color: red;
      }
      ```

      However: this should only be required in rare cases! If you find yourself writing this kind of style more than 1-3% of the time, you should have another look at what UI5's automatic CSS mirroring offers.

Programmatic Access to RTL Mode

Some controls need to provide specific coding in case of RTL mode, e.g. because they are positioning or animating elements programmatically, not via CSS.

The UI5 RTL configuration can be read by calling

```
var bRtl = sap.ui.getCore().getConfiguration().getRTL();
```

What do applications need to know and do?

This wiki page will not supply the general knowledge to build RTL-aware applications, but with respect to UI5: the text direction is LTR by default but can be set using the various known configuration switches:

- URL parameter `sap-ui-rtl=true`
- Configuration object: `window["sap-ui-config"].rtl = true;`
- Script tag configuration: `data-sap-ui-rtl="true"`

UI5 will take care of setting the overall page direction to "RTL". All UI5 content will then be displayed in RTL mode. For self-written controls and content you need to test. If manual style adoptions are required, you need to provide style specifically for the RTL case by using `html[dir=rtl]`

Steps

If UI5 is configured to RTL mode, the core:

1. sets dir="rtl" on the HTML tag
    - The W3C officially recommends using the HTML attribute instead of the CSS properties, e.g. directionality is a matter of content, not of presentation, and because the CSS properties may even be ignored.
    - Using the <HTML> tag (not the BODY tag) is recommended as well
2. loads the respective library-RTL.css files

Resources

Standards:

- W3C bidi tutorial: http://www.w3.org/International/tutorials/bidi-xhtml/
- HTML 4.01 "dir" attribute/text direction: http://www.w3.org/TR/html401/struct/dirlang.html
- HTML 5 "dir" attribute: http://dev.w3.org/html5/spec/Overview.html#the-dir-attribute

- CSS 2.1 text direction: http://www.w3.org/TR/CSS21/visuren.html#direction
- CSS3 has its own "writing modes" module: http://dev.w3.org/csswg/css3-writing-modes/

Other Resources:

- A page with Hebrew RTL text: http://www.i18nguy.com/unicode/shma.html
- Good introduction: http://www.i18nguy.com/markup/right-to-left.html
- List of languages and their directionality: http://www.w3.org/International/questions/qa-scripts.en.php
    - Of the scripts commonly used in computer application localization, only Arabic and Hebrew are written right-to-left.
    - Scripts used in China, Hong Kong, Japan and Korea are written left-to-right or top-to-bottom. We happily choose the former...

- FAQ
- Right-to-Left supp

SAPUI5 Security Information

Introduction

Security With SAPUI5

This guide covers security aspects of the usage of SAPUI5. It is targeted to SAPUI5 application and control developers as well as to system administrators running applications based on SAPUI5.

It is important to understand that SAPUI5 is a clientside JavaScript library, so while the library itself is designed and tested to be secure, it can not ensure the application to be secure. Unlike Web Dynpro, where the application is built against an abstract programming model and the framework handles all the HTML rendering, JavaScript code and communication with the browser, in SAPUI5 the application controls the HTML output, it provides own JavaScript code which is executed on the client and it handles client/server communication.

While this brings a lot of freedom and possibilities for the application, it comes with a lot of responsibility with regards to security. Application developers need to understand the security threats and actively prohibit exploitation. Also the correct configuration of the HTTP server which is used is important.

This also means common security mechanisms, which are taken for granted, like User Authentication, Session Handling, Authorization Handling or Encryption are not part of SAPUI5 and need to be handled by the server side framework and/or custom code of the application.

Why Is Security Necessary?

With the increasing use of distributed systems and the Internet for managing business data, the demands on security are also on the rise. When using a distributed system, you need to be sure that your data and processes support your business needs without allowing unauthorized access to critical information. Protection of the user's personal data must be guaranteed and legal regulations regarding this must be complied with. User errors, negligence, or attempted manipulation on your system should not result in loss of information or processing time. These demands on security apply likewise to SAPUI5 . To assist you in securing SAPUI5, we provide this Security Guide.

About this Document

The security guide comprises the following main sections:

- Before You Start Links to general security guides
- Architectural Overview How SAPUI5 is embedded and interfacing with the application
- Browser Security Client related security aspects
- Transport Security Security of data transport between client and server
- Server Security Serverside security considerations with SAPUI5
- Third Party Libraries jQuery and datajs are part of SAPUI5
- Secure Programming Guide What needs to be done in the application
- Using the SAPUI5 Repository based on BSP Repository

Before You Start

SAPUI5 is not bound to any server implementation or serverside programming language, so can be used with Netweaver ABAP, Java, HANA XSEngine or any standard web server. Therefore, the corresponding Security Guides also apply to SAPUI5.

Fundamental Security Guides

Please also refer to the global and other security guides of involved technologies and platforms.

Additional Information

For more information about specific topics, see the Quick Links as shown in the table below.

Content	Quick Link on the SAP Service Marketplace or SDN
Security	http://sdn.sap.com/irj/sdn/security
Security Guides	http://service.sap.com/securityguide
Related SAP Notes	http://service.sap.com/notes
	http://service.sap.com/securitynotes
Released platforms	http://service.sap.com/pam
Network security	http://service.sap.com/securityguide
SAP Solution Manager	http://service.sap.com/solutionmanager
SAP NetWeaver	http://sdn.sap.com/irj/sdn/netweaver

Architectural Overview

SAPUI5 is a JavaScript library based on jQuery. It is embedded in the application using a script-tag and triggers additional requests for on-demand-loading of JavaScript classes, stylesheets and other resources. For Java and ABAP a special resource handler is offered, which provides extended capabilities, and is used for all SAPUI5 internal requests.

The application usually has a serverside part and a clientside part. The serverside part can be based on any web framework, the clientside part is a web application, which is utilizing SAPUI5 for its user interface.

Data Protection and Privacy

SAPUI5 does not store or provide access to any person-related data. If an application built with SAPUI5 deals with person-related data, it has to take care of the data protection rules of the target countries of the application. This includes usage of proper authentication, authorization and encryption (e.g. SSO and usage of https), as well properly securing and logging access to person-related data. For more inforamtion regarding data protection and privacy, please see the security guide of your server side framework.

Browser Security

First of all, the browser is an untrusted client by design. The server can not rely on any information sent from the browser, as a malicious user can use a JavaScript debugger to tamper with the client code or a proxy server like fiddler to modify request data. All input validation on the client is just for convenience, the server always has to validate all the data coming from the client again.

The browser also offers some possible attack vectors, especially Cross-Site-Scripting (XSS), which has to be taken care of by the application.

Cross-Site-Scripting

Cross-Site-Scripting is the most prominent security issue of web applications within the last years and also the mosts dangerous one, as it allows so many ways of exploitation. Once malicious code is running within your browser, it can be used to steal your session cookies, to trigger requests within the current session, or even to exploit a known browser vulnerability to do native code execution.

For SAPUI5 applications XSS vulnerabilities can exist on different levels:

- Within the HTML page or custom data transports sent to the browser from the server
- Within the JavaScript Code of the application, which is processing server responses
- Within the HTML renderers of SAPUI5 controls

SAPUI5 can only prevent cross site scripting in the processing and rendering of controls. For that purpose there is input validation on all typed Element properties and output encoding done in the renderer class of Controls (see Secure Programming Guide for Control Developers). Please be aware that there are exceptions to this, for controls which are especially built to include arbitrary HTML (like sap.ui.core.HTML).

Application is responsible for proper output encoding of all content embedded into the HTML page itself, as well as for encoding of JSON or XML data sent to the client and secure processing of this data. Of course the application also has to take care of security of custom controls provided by the application.

HTML5

HTML5 offers a lot of new functionality which also brings a lot of potential new security issues. This just an overview of some of the new features and possible security issues when they are used.

Local Storage

All browsers are now offering a local storage API. This API can be used to store a limited amount of data on the browser. Access to this data is limited to JavaScript code running from the same domain as it has been stored. SAPUI5 offers helper functions to access the local storage on different browsers.

The local storage of browsers is not a secure storage, so while it can be used for static data, like enumerations, applications must not store any user or application data within the local storage.

By default SAPUI5 is not using the local storage, but it can be enabled for the history-capability of Dropdown- and ComboBoxes for uncritical data.

WEBGL

While more and more browsers are supporting WEBGL by default, WEBGL allows accessing the graphics API of the computer on a very low level, which may also lead to low level exploits. This is the main reason Internet Explorer has no support for WebGL at all, Microsoft recently stated, that they are not going to support WebGL as they think it can not be implemented in a secure way.

SAPUI5 is currently not using WEBGL.

WebSockets

While WebSockets offer great new possibilities for the client/server communication of web applications, there have been many security issues rising while the first implementations were done by the browser vendors. Standardization of WebSockets has reached a stable state with RFC 6455 and is now implemented beginning with Chrome 16, Firefox 11 and Internet Explorer 10. Even if the browser implementations themselfes prove to be secure, using Web Sockets may require additional security measures on the client.

SAPUI5 is currently not using WebSockets.

postMessage/onMessage

This is another great feature in the HTML5 area, which can lead to massive security issues, when not used correctly. postMessage allows inter-window-communication between windows from different domains. So basically this opens a hole in the same origin policy currently implemented in the browser. As soon you are subscribing to the onMessage event, you can receive messages from any other browser window, the application is responsible to check the originating domain and only to process messages which have been sent by trusted domains.

SAPUI5 is utilizing postMessage for its debugging and tracing functionality.

Transport Security

Of course the best security on the client and server doesn't help, if the data transport between client and server can be read, intercepted or even modified by an attacker. Per default HTTP communication is stateless and unencrypted, which makes it necessary to configure it in a way that it is using encrypted connections and to add session handling on top using either cookies or URL rewriting.

Encryption

Sending the HTTP protocol over a SSL secured connection is not only standardized, but also required for SAP applications.

SAPUI5 fully supports usage of HTTPS, but there are some restrictions regarding the CDN version of SAPUI5 when HTTPS is used. It is recommended to enable or at least to test SSL connections in an early stage of application development, as usually switching to HTTPS causing some issues. First of all, when the application is started using HTTPS, the SAPUI5 library also has to be loaded from an HTTPS server. Second Internet Explorer 8 and 9 have some additional restrictions regarding cross origin requests with HTTPS, which are related to the security zone concept.

Session security

Even if the data transport is secured using SSL, there are possibilities to hijack such a secure connection and sending malicious requests from the client. Cross site request forgery and session fixation are two of the prominent examples of this class of attacks.

SAPUI5 does only provide XSRF prevention for the data which is sent to the server by SAPUI5. Currently this only happens in the OData Model, where a XSRF token is read from the server and used for subsequent write requests.

Application is responsible for using XSRF header or other mechanisms to prevent XSRF for all other server communication triggered by the application.

Server Security

SAPUI5 only ships a small serverside part, to support loading of resources by the client framework. Usage of the resource handlers is not mandatory, SAPUI5 also offers a static version of the libraries, which can be used with an arbitrary HTTP server.

Cross Origin Resource Sharing

Usually the XMLHttpRequest for security reasons does only allow accessing resources from the same domain as the originating document. As their are a lot of web based services available today, starting with RSS or Atom feeds, WebServices or OData services, there was a need to be able to also access data sources from different domains within the browser, which was adressed with the CORS (Cross Origin Resource Sharing) standard. This allows the server to set special headers on their responses, which are telling the XMLHttpRequest object, whether it is allowed to process the requested data or not.

This CORS capability also plays an important role in SAPUI5 based applications. In case the application itself and the data visualized are coming from different servers, the CORS header has to be configured correctly on the data providing server, to allow the application server domain to access the data.

SAPUI5 is using CORS header on its CDN based library, to be able to load additional scripts, styles and resources from the CDN server.

Resource Handler (Java)

The Resource Handler for Java provides some configuration options (context parameters in the web.xml). It is possible to enable CORS, configuring the cache control and activating a development mode. Details about the configuration options can be found here: Resource Handling.

In case of productive usage of the Resource Handler you must not have the development mode activated. You have to remove the configuration option `com.sap.ui5.resource.DEV_MODE` in the web.xml.

Resource Handler (ABAP)

The Resource Handler for ABAP is not configurable. It is used to serve the resources from the mime repository.

Resource Handler for Application Resources (ABAP, NW 7.X)

The Resource Handler for Application Resources for ABAP, NW 7.X is not configurable. It is used to serve the resources from the SAPUI5 Repository based on BSP Repository, see also Executing SAPUI5 Applications from the SAPUI5 Repository

Proxy Servlet (Java)

SAPUI5 is providing a `SimpleProxyServlet`. This proxy servlet can be used on a local server only for local requests to access data from other domains. This is useful to avoid cross domain issues when fetching data from another domain for testing purposes. Also for the SAPUI5 tools this is required since the local testing needs to access data from the remote ABAP server. Due to security reasons this proxy servlet is limited to localhost usage only and cannot be used by requests from other domains.

Third-Party Libraries

Libraries shipped with SAPUI5

SAPUI5 ships with a handful of third party libraries. jQuery is mandatory as SAPUI5 is based on it and datajs is needed in case OData services should be used.

jQuery

jQuery does not have any security related documentation on their site, but they are known to be aware of security and usually reacting quickly in case security issues are found within their library.

SAPUI5 is including jQuery in different versions together with their own libraries, so also has the possibility to add custom security fixes to jQuery, if necessary.

datajs

datajs does not have any security related documentation on their site.

SAPUI5 is including the datajs library and can add custom security fixes, if necessary.

Libraries included by the application

Applications based on SAPUI5 are allowed from a technical point of view to include arbitrary custom libraries within their application. Of course SAPUI5 can not give any statement about the security of third party libraries and can not ensure security of third party libraries. The application has full responsibility for doing an security assessment of third party libraries before using them and for embedding and using them in a secure manner.

Secure Programming Guide

Input Validation

Application point of view: Input validaton of user input, must be done on the server, optional on the client, can be achieved using two way data binding and model types. The application can create custom types by extending the SimpleType? class. This is explained in more detail in the databinding documentation.

- `oInput.bindValue("/path", new sap.ui.model.type.Float())` Ensures only a Float value can be entered, otherwise a ParseError will be thrown
- `oInput.bindValue("/path", new sap.ui.model.type.String({}, {maxLength: 20}))` Ensures that maximal 20 characters can be entered, otherwise a ValidationError will be thrown

Control point of view: Input validation of control properties, so integer proeprties only accept integers, enumeration properties only accept an existing enumeration value. While this sounds obivous, in JavaScript it is not. The type system of JavaScript doesn't do type validation on assignment.

Try to avoid string properties whenever possible. SAPUI5 offers special types like CSSSize or URL for control properties, which should be used when applicable. If there is a fixed set of possible values, please create an enumeration.

Output Encoding

All data sent from the server must be properly output encoded according to the context they are contained in. For more information please see the XSS Secure Programming Guide. Content which is created on the client side, either for display within the browser or for data transport, needs to be properly output encoded with the encoding methods provided by SAPUI5. There are methods for encoding HTML, XML, JavaScript, CSS and URI components.

When developing custom controls, the following two methods should be used within the control renderer for HTML encoding:

- `RenderManager.writeEscaped(sString)` Encodes the given string and writes it to the HTML output as content
- `RenderManager.writeAttributeEscaped(sString, sString)` Encodes the given string and writes it to the HTML output as an attribute

The encoding library does encode according to the XSS Secure Programming Guide and should be used whenever encoding is needed by the application. It offers the following API:

- `jQuery.sap.encodeCSS(sString)` Encode the string for inclusion into CSS string literals or identifiers
- `jQuery.sap.encodeHTML(sString)` Encode the string for inclusion into HTML content/attribute
- `jQuery.sap.encodeJS(sString)` Encode the string for inclusion into a JS string literal
- `jQuery.sap.encodeURL(sString)` Encode the string for inclusion into an URL parameter
- `jQuery.sap.encodeXML(sString)` Encode the string for inclusion into XML content/attribute

All controls from SAPUI5 libraries properly encode their data, except for HTML-control and XMLView. The latter two are explicitely built to display arbitrary HTML content. If applications use these two controls AND provide unsecure HTML content to them, they have to check/validate the content on their own. Note that using an XMLView with application controlled, secure HTML content together with standard UI5 controls (other than HTML and XMLView) containing potentially unsecure data, is also safe. Only untrusted HTML content is critical.

SAPUI5 includes the CAJA HTML sanitizer, which is used by the HTML control and the RichTextEdit?.

URL Validation

URL validation should take place on the server side when possible. In case URLs are entered on the

client side or are loaded from an external service, SAPUI5 offers an URL validator, which can be used to whether a URL is well formed and properly encoded. It also contains a configurable white liste to restrict URLs to certain protocols or certain hosts. Initially, the white list only checks for the protocols `http`,`https` and `ftp` but nothing else. Applications should define their own whitelist.

SAPUI5 is using URL validation for outgoing requests (e.g. links). SAPUI5 offers the following API for URL validation within the application.

- `jQuery.sap.getUrlWhitelist()` Gets the whitelist for URL validation
- `jQuery.sap.addUrlWhitelist(protocol, host, port, path)` Adds a whitelist entry for URL valiadtion
- `jQuery.sap.removeUrlWhitelist(iIndex)` Removes a whitelist entry for URL validation
- `jQuery.sap.validateUrl(sUrl)` Validates an URL.
- `jQuery.sap.clearUrlWhitelist()` clears the whitelist for URL valiadtion

Cache Settings

The application has to take care, that caching of data is disabled, by setting appropriate HTTP Headers on the server side.

Static resources from SAPUI5 or from the application are not security relevant and are excluded from this rule, so they can safely be cached on the client.

User Management / Authentication

SAPUI5 doesn't provide any authorization or user management. An application which implements such facilities based on SAPUI5 has to make sure that SSL is enabled to prevent cleartext passwords sent over the wire. Applications must not store any logon information on the client.

Local Storage

The local storage of browsers is not a secure storage, so while it can be used for static data, like enumerations, applications must not store any user or application data within the local storage.

The following API is offered by SAPUI5 to access the local storage

- `jQuery.sap.storage.get(sId)` Retrieves the state string stored in the session under the key sStateStorageKeyPrefix + sId
- `jQuery.sap.storage.getType()` Returns the type of the storage.
- `jQuery.sap.storage.put(sId, sStateToStore)` Stores the passed state string in the session, under the key sStateStorageKeyPrefix + sId
- `jQuery.sap.storage.remove(sId)` Deletes the state string stored in the session under the key sStateStorageKeyPrefix + sId
- `jQuery.sap.storage.clear()` Deletes all the entries saved in the session. CAUTION: This method should be called only in very particular situations, when a global erasing of data is required. Given that the method deletes the data saved under any ID, it should not be called when managing data for specific controls.

Using the SAPUI5 Repository based on BSP Repository

Using the SAPUI5 Repository Team Provider

The SAPUI5 Repository Team Provider connected against a NW 7.31 ABAP system with UI-AddOn or a NW 7.40 (from SP1) can be used to synchronize SAPUI5 application resources between Eclipse and the SAPUI5 Repository on the ABAP system.

For usage of the SAPUI5 Repository Team Provider, also see the Security Guide for ABAP Development Tools which is part of the ABAP Development User Guide and the SAP NetWeaver Security Guide.

Needed Authorization Objects

Authorization Object	Description
S_DEVELOP	The authorization object S_DEVELOP is needed to create, update and delete SAPUI5 applications in the SAPUI5 Repository.
S_ICF_ADM	The authorization object S_ICF_ADM is needed to create the SAPUI5 application specific ICF node under */sap/bc/ui5_ui5/*.
S_TCODE	The authorization object S_TCODE is needed to create the SAPUI5 application specific ICF node under */sap/bc/ui5_ui5/*.
S_TRANSPRT	The authorization object S_TRANSPRT is used to create new transport request or new task.
S_CTS_ADMI	The authorization object S_CTS_ADMI is needed to transport SAPUI5 applications.
S_CTS_SADM	The authorization object S_CTS_SADM is needed to transport SAPUI5 applications.
S_ADT_RES	The authorization object S_ADT_RES is used for the communication between Eclipse and the ABAP Backend via the SAPUI5 Repository Team Provider.
S_RFC	The authorization object S_RFC is used for the communication between Eclipse and the ABAP Backend via the SAPUI5 Repository Team Provider.

For more information about authority checks and working with authorization objects, see the following: *SAP NetWeaver 7.0x Security Guides (Complete)* on SAP Service Marketplace at http://service.sap.com/securityguide

Delivered Virus Scan Profile

When uploading files to the SAPUI5 Repository, you can perform a virus scan.

As of SAP NetWeaver 7.00 with UI AddOn, SAP delivers the following virus scan profile for ABAP within the UI AddOn (from 7.40 SP1 on the virus scan profile is already part of the software component `SAP_UI`):

- /UI5/UI5_INFRA_APP/REP_DT_PUT

This profile is used by the SAPUI5 Repository API to store files in the SAPUI5 Repository based on BSP Repository.
For example: Upload of a local file using SAPUI5 Repository API /UI5/CL_UI5_REP_DT, method /UI5/IF_UI5_REP_DT~PUT_FILE from 7.00 on, or the SAPUI5 Team Repository Provider in 7.31 or 7.40.

The profile is deactivated when delivered. To activate it, first create at least one basis profile and save it as the default profile. You can then activate one of the delivered profiles. By default, it links to a reference profile, which is the default profile. For more information, see SAP Help Portal:

- ABAP-Specific Configuration of the Virus Scan Interface (7.00)
- ABAP-Specific Configuration of the Virus Scan Interface (7.31)

Executing SAPUI5 Applications from the SAPUI5 Repository

The SAPUI5 Application can be executed from the NW 7.X ABAP System by retrieving the SAPUI5 Application Resources from the SAPUI5 Repository based on BSP Repository with the help of an ICF Handler.

Delivered ICF Nodes

For the execution of SAPUI5 Applications from the SAPUI5 Repository, SAP delivers ICF node */sap/bc/ui5_ui5/*. For every SAPUI5 applications an ICF node below that exists.

Note that all services delivered by SAP (such as the */sap/bc/ui5_ui5/* service for executing SAPUI5

Applications) are initially inactive. Also each new service that you create has status inactive. Before you work with the ICF, you must activate the services you require.

For more information, see also Activating and Deactivating ICF Services (7.00 EhP3) and SAP Library for SAP NetWeaver on SAP Help Portal at http://help.sap.com *SAP NetWeaver → SAP NetWeaver Library → SAP NetWeaver by Key Capability → Application Platform by Key Capability → Connectivity → Components of SAP Communication Technology → Communication between ABAP and Non-ABAP Technologies → Internet Communication Framework → Development → Activating and Deactivating ICF Services*.
For more information about ICF security, see SAP NetWeaver Security Guide on SAP Service Marketplace at http://service.sap.com/securityguide *SAP NetWeaver 7.0x Security Guides (Complete) → SAP NetWeaver 7.0x Security Guides (Online Version) → Security Guides for Connectivity and Interoperability Technologies → RFC/ICF Security Guide*

Needed Authorization Objects

No specific authorization objects are needed to execute SAPUI5 Applications from the SAPUI5 Repository.

As for ICF service nodes in general, authorization for specific ICF service nodes can be restricted, see also Defining Service Data (7.00 EhP3) or SAP Library for SAP NetWeaver on SAP Help Portal at http://help.sap.com *SAP NetWeaver → SAP NetWeaver Library → SAP NetWeaver by Key Capability → Application Platform by Key Capability → Connectivity → Components of SAP Communication Technology → Communication between ABAP and Non-ABAP Technologies → Internet Communication Framework → Development → Server-Side Development → Creating and Configuring ICF Services → Create Service → Defining Service Data* and Authorization Object S_ICF (7.00 EhP3) or SAP Library for SAP NetWeaver on SAP Help Portal at http://help.sap.com *SAP NetWeaver → SAP NetWeaver Library → SAP NetWeaver by Key Capability → Application Platform by Key Capability → Connectivity → Components of SAP Communication Technology → Communication between ABAP and Non-ABAP Technologies → Internet Communication Framework → ICF Administration → Server Function Administration → Authorizations → Authorization Object S_ICF*.

Tracking coding changes and text changes in the SAPUI5 Repository

Code changes can be traced by using the usual ABAP version control of the corresponding resource file. A new version is created when a new transport is written.
Text changes can be traced by using the "Table History" transcation(SCU3), the relevant tables for UI5 texts are /UI5/TREP_TEXT and /UI5/TREP_TEXT_T for the translated text. Table logging has to be activated in the system for this functionality.

Cross-Site-Scripting and how to Prevent it in Application Development

What is Cross-Site Scripting (XSS)

Cross-site scripting is a widely known vulnerability most web sites have. This page does not provide general information about cross-site scripting but focuses on what you as an application developer using SAPUI5 can do to avoid these security issues.

To give a short info on XSS: It is about injecting script code into a web page, which is then executed in the context of the page and therefore not only can access any information which currently displayed on the screen but can either access session information contained in cookies, or send new requests to the server within the current session, or even try to exploit browser vulnerabilities to get full access to the machine the browser is running on.

XSS in SAPUI5-Based Web Applications

AJAX frameworks in general are an interesting target for XSS exploits, as not only the HTML which is initially sent to the browser may contain vulnerabilities, but also the code which is used to visualize content on the client side may have bugs which can be exploited to get the JavaScript coding executed on the client side. In addition to that, once a script has injected an AJAX application, it will be alive for a long time, as usually navigation will not reload the whole page which would also clean up any running JavaScript code, but stays within the same HTML document and uses a delta update mechanism to show new content.

It is important to understand that UI5 is not involved in creating the HTML page which is sent to the client, so there is no way UI5 can prevent XSS vulnerabilities which are contained in the HTML page itself. The application which is using the UI5 rendering has to take care, according to the documentation of their server-side rendering framework, to properly escape user data, in a way that no JavaScript can be injected.

The UI5 framework will take care of proper escaping for all content which is created and displayed on the screen using the controls provided by UI5. There is no need for the application to HTML-escape user data, but the control API expects all data to be unescaped, so that it can be escaped as needed for the context it will be visualized.

URL White List Filtering

The SAPUI5 framework provides a client-side API to manage a whitelist for URLs. This white list can be used to validate arbitrary URLs if they are permitted or not. Internally controls which accept arbitrary HTML content like the `sap.ui.richtexteditor.RichTextEditor` or the `sap.ui.core.HTML` use the URL white list to check (sanitize) the URLs of their content and value to not infringe against the allowed URLs. The option to sanitize the value can be enabled or disabled in the respective control properly via property: RichTextEditor.sanitizeValue or HTML.sanitizeContent. For the HTML control it is disabled by default whereas for the RichTextEditor the sanitize option is enabled. When adding a Path to the Whitelist be aware to add a "/" at the start of the Path if necessary, so "/index.epx" would be the correct entry instead of "index.epx". The last example below shows this.

Maintaining the URL white list

The white list can be maintained with the following API:

- jQuery.sap.addUrlWhitelist
- jQuery.sap.clearUrlWhitelist
- jQuery.sap.getUrlWhitelist
- jQuery.sap.removeUrlWhitelist

Here is an example how valid URLs can be added to the white list:

```
// jQuery.sap.addUrlWhitelist(/* protocol */ undefined, /* host */ undefined, /* port */
undefined, /* path */ undefined);

jQuery.sap.addUrlWhitelist(undefined, "www.sap.com");
jQuery.sap.addUrlWhitelist("https", "sdn.sap.com");
jQuery.sap.addUrlWhitelist(undefined, "sap.de", "1080");
jQuery.sap.addUrlWhitelist("http", "www.sap.com", undefined, "/index.epx");
```

Validating a URL

A URL can be validated by using the following API:

- jQuery.sap.validateUrl

Here is an example how a given URL is validated against the before maintained white list:

```
jQuery.sap.validateUrl("http://www.sap.com"); // => true

jQuery.sap.validateUrl("http://sdn.sap.com"); // => false (wrong protocol)

jQuery.sap.validateUrl("https://sdn.sap.com"); // => true

jQuery.sap.validateUrl("ftp://sap.de:1080/anyftpfolder"); // => true

jQuery.sap.validateUrl("http://www.sap.com/index.epx"); // => true
```

If no white list is maintained the URL validity check also basically checks the URL for being defined in a valid format, but be aware that the specification of a URL is not very limited so nearly everything is a valid URL from a format perspective. It is not checked if the destination of the URL is valid or reachable.

SAPUI5 for Mobile

sap.m

SAPUI5 contains a dedicated control library for mobile devices like tablets and smartphones with the technical name `sap.m`.
Both `sap.m` and the control libraries for the desktop are based on the same Runtime which provides a common infrastructure for Data Binding, Model-View-Controller Concepts, Localization and other topics. In addition, both control sets have a harmonized API for the common parts. This makes it easy for the application developer to efficiently build applications for both scenarios. As SAPUI5 is based on web standards like HTML, CSS and JavaScript you can run the application on multiple platforms while the `Mobile Visual Identity` theme makes users feel comfortable on both Android and Apple devices.

- Application Development
  - Hello World Example
  - Best Practice
  - MVC (Model-View-Controller) for Mobile
  - Navigation and Lifecycle Events in the sap.m.App/sap.m.NavContainer
  - Adapting to Platform and Screen Size
  - Platform Attribute on Mobile
  - The adaptive SplitApp control
  - Layouting with FlexBox
  - Working with Lists
    - Swipe-for-Action in Lists
  - Triggering Phone/SMS/E-Mail Application
  - Scrolling in UI5 mobile
    - Pull to Refresh
  - Running in Web Containers
  - Image Densities
  - Navigation incl History Handling
  - How to theme your application

- Control Development
  - Mobile Events
  - Styling/Theming Mobile Controls

sap.makit

There is an additional library called `sap.makit` for building charts to visualize business data in mobile applications.

- Building Mobile Charts with MAKit

How to create a Mobile "Hello World" App in 23 seconds

This page explains **how to create a UI5 Mobile application - from scratch within 23 seconds** (with some practice...).

- If you are interested in exactly doing this without reading too much, you can **jump right down to the respective section** on this page.
- If you prefer a solid introduction, the SAPUI5 Mobile wiki area might be better suited as starting point.

Explanation

This page assumes you have seen and understood the SAPUI5 Hello World in 20 seconds example, so you are familiar with how UI5 is loaded, how controls are instantiated and how they are added to the HTML page.
This page therefore only explains the parts specific to the creation of a mobile app.

To make this clear up front: UI5 on mobile devices and on desktop browsers is pretty much the same thing. All the concepts like data binding, the MVC pattern and so on are available on mobile as well. UI5 just provides an additional control library ("sap.m") which is *optimized* for mobile devices by focusing on touch interaction, providing a mobile-style theme which makes users feel comfortable on both Android and Apple devices, and by using the most advanced features of the browsers available on modern smart phones and tablets.

NOTE: as this "sap.m" mobile library is optimized for mobile browsers which are based on Webkit or alternatively for IE10, it does currently **NOT run properly in other browsers**, like Internet Explorer 9 and Firefox. When testing on desktop PCs, please use Chrome or Safari.
Furthermore, the mobile library only uses touch events and deduces its appearance from the platform it is running on. Hence, when testing in desktop Chrome/Safari or IE10, **a URL parameter is needed** which suggests to emulate touch events from the mouse events and to apply the styling for either Android, iOS, BlackBerry or Windows Phone. The URL parameter is: `sap-ui-xx-fakeOS=android`, `sap-ui-xx-fakeOS=ios`, `sap-ui-xx-fakeOS=blackberry` or `sap-ui-xx-fakeOS=winphone`, respectively. (This parameter could also be added as "data-sap-ui-xx-fakeOS" attribute to the bootstrap script tag.) Note that "winphone" simulation only works in Internet Explorer 10 and "ios"/"android"/"blackberry" only in Chrome and Safari.

Creating the Mobile Application

Basic Structure of a SAPUI5 Application

Just like on desktop:

- HTML is the foundation of UI5 applications, so one needs an HTML file.
- Putting the HTML5 doctype "`<!DOCTYPE html>`" in the first line and the Internet-Explorer-specific meta tag "`<meta http-equiv="X-UA-Compatible" content="IE=edge" />`" to the beginning(!) of the <head> ensures that all browsers use the latest version of their rendering engine. Internet Explorer is not really used widely on mobile devices and not supported yet by the UI5 mobile library, but this meta tag does not hurt and in doubt it makes the page more future-proof.
- A second meta tag "`<meta http-equiv="Content-Type" content="text/html;charset=UTF-8"/>`" lets all browsers treat the file as UTF-8 encoded (assuming that you use this encoding when editing/saving the file)

- In the HTML <body> there needs to be a place where the App HTML will go. We will just add a element as only content to the HTML.
- The "`sapUiBody`" class should always be added to the <body> tag of a UI5 application to initialize font and colors for the whole page:

```
<body class="sapUiBody">
    <!-- This is where the App will live: -->
    <div id="content"></div>
</body>
```

- As UI5 is a JavaScript UI library, the UI5 JS file needs to be loaded. The following script tag in the <head> does this.
  - Make sure the URL is pointing to a UI5 installation accessible from your site.
  - Note how this is equal to writing desktop apps.
  - But also note that we are **only loading the "sap.m" control library and the "sap_mvi" theme** ("mvi" stands for: "Mobile Visual Identity" and is the name of the SAP Mobile design):

```
    <script src="http://<your server with SAPUI5 installed>/sapui5/resources/sap-ui-core.js"
            id="sap-ui-bootstrap"
            data-sap-ui-libs="sap.m"
            data-sap-ui-theme="sap_mvi">
    </script>
```

- At this point, UI5 (including the Mobile controls) is loaded and ready for use.

How to set up a SAPUI5 Mobile Application

The sap.m library provides a control called `App` which is meant to be the root control of a mobile application. It provides initialization of the HTML page, sets some meta tags to ensure as-native-as-possible look&feel and can manage different pages and the animations between them.
Just create the control and tell it which page should be displayed first.

```
// create a mobile App
// it initializes the HTML page for mobile use and provides animated page handling
var app = new sap.m.App("myApp", {initialPage:"page1"}); // page1 should be displayed first
```

Instead of using the App control, you can also use `jQuery.sap.initMobile()` to set up the HTML and use other full-screen controls like the `sap.m.Page` or `sap.m.Carousel` as root element of your app.

Adding Content Pages

Adding the Main Page

Typical mobile applications are often composed of a number of pages/views/screens between which the user can navigate. Two of them are now added to the App.

IMPORTANT: while sap.m.Page controls can be used as pages, and the aggregation is called "pages", other controls could be used as well'''

One `sap.m.Page` control is created, its title is set and the content is just one Button:

```
// create the first page of your application
var page1 = new sap.m.Page("page1", {
    title: "Initial Page",
    content : new sap.m.Button({   // content is just one Button
        text : "Go to Page 2",
        press : function() {
            app.to("page2");   // when pressed, it triggers drilldown to page 2
        }
    })
});
```

```
app.to("page2")
```

When the Button is pressed, it triggers drilldown navigation by calling ("page2" is the ID of the second screen/page).
Optionally, the animation type could be given here - the default is a "slide" animation from right to left.

Adding the Second Page

The second page is created similarly. In contrast to the main page "showNavButton" is set to "true" to get a "Back" button. And when this button is triggered, the handler takes care of calling `app.back()`. This causes an inverse animation leading back to the main page.

```
  // create the second page of your application
  var page2 = new sap.m.Page("page2", {
      title: "Page 2",
      showNavButton: true,       // page 2 should display a back button
      navButtonPress: function(){
          app.back();            // when pressed, the back button should navigate back up to
page 1
      },
      icon: "http://www.sap.com/global/ui/images/global/sap-logo.png",
      content : new sap.m.Text({text:"Hello Mobile World!"})
  });
```

A header icon (only visible on Android!) and some content saying "Hello Mobile World" is also given.

Placing the App onto the Screen

Finally, the two pages are added to the App and the App is placed into the HTML, just like a UI5 desktop control. The App takes care to cover the whole screen.

And how to do it in 23 seconds?

Assumption for these instructions to work exactly as described: you have a Windows Computer (other operating systems will work as well, but the instructions may differ), a current **Webkit-based browser (Chrome or Safari)** (the mobile library is currently not designed to run properly in Firefox or Internet Explorer) and you know where you can refer to SAPUI5 on some server.

Important: the URL in the script tag is pre-filled as " https://sapui5.netweaver.ondemand.com/resources/sap-ui-core.js" (which was at least available at the time of writing this document). Test this URL first: if it does not work, you need to find a different installation or deploy SAPUI5 on your own server.
Also note that the version of SAPUI5 deployed on https://sapui5.netweaver.ondemand.com/ may be updated with a delay of some days or weeks after a new release of SAPUI5, even though we try to keep them in sync. This example will work nevertheless.

1. Right-click your desktop, select "New" → "Text Document"
2. Name the new file e.g. "mobile.html" and accept the extension change warning
3. Right-click the new file and select "Edit" (make sure it opens in Notepad or so, NOT in MS Word!)
4. Copy&Paste the below HTML code and save the file (e.g. press Ctrl-S) (remember: the SAPUI5 URL may need to be adapted)
5. Drag the file to this browser window (remember: it must be Chrome or Safari!)
6. That's it. Well, that's only in a desktop browser so far, but to load it on a mobile device you'd need to put the file on a server which takes longer than a couple of seconds as long as you have not set up everything in advance.
7. But you want to play around with the app at least in your desktop browser, so add the following URL parameter to the file URL: `sap-ui-xx-fakeOS=ios`, so the URL should read something like: "mobile.html?sap-ui-xx-fakeOS=ios"
    - This enables simulation of touch events on desktop PCs
    - This also enables the iPhone/iPad styling; if you want to see the Android styling, use "sap-ui-xx-fakeOS=android" instead

```
<!DOCTYPE HTML>
```

```
<html>
  <head>
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta http-equiv="Content-Type" content="text/html;charset=UTF-8"/>

    <title>Mobile App in 23 Seconds Example</title>

    <script src="https://sapui5.netweaver.ondemand.com/resources/sap-ui-core.js"
        id="sap-ui-bootstrap"
        data-sap-ui-libs="sap.m"
        data-sap-ui-theme="sap_mvi">
    </script>
    <!-- only load the mobile lib "sap.m" and the "sap_mvi" theme -->


    <script>

      // create a mobile App
      // it initializes the HTML page for mobile use and provides animated page handling
      var app = new sap.m.App("myApp", {initialPage:"page1"}); // page1 should be displayed
first


      // create the first page of your application
      var page1 = new sap.m.Page("page1", {
          title: "Initial Page",
          content : new sap.m.Button({   // content is just one Button
              text : "Go to Page 2",
              press: function() {
                  app.to("page2");   // when pressed, it triggers drilldown to page 2
              }
          })
      });


      // create the second page of your application
      var page2 = new sap.m.Page("page2", {
          title: "Page 2",
          showNavButton: true,       // page 2 should display a back button
          navButtonPress: function(){
              app.back();            // when pressed, the back button should navigate back up
to page 1
          },
          icon: "http://www.sap.com/global/ui/images/global/sap-logo.png",
          content : new sap.m.Text({text:"Hello Mobile World!"})
      });

      app.addPage(page1).addPage(page2); // add both pages to the App

      app.placeAt("content"); // place the App into the HTML document

    </script>

  </head>
  <body class="sapUiBody">
    <div id="content"></div>
  </body>
</html>
```

Result

If you followed the steps above you should now see the Mobile App.

iOS appearance: Android appearance:

If you click the button (and have not forgotten the "sap-ui-xx-fakeOS" URL parameter), navigation to the second page will occur.

You can also launch this small application by pointing your mobile browser to this URL. Open it in a new window or tab to make sure it does not appear inside the demokit (the URL should start with "...demokit/content..." not with "...demokit/#content...").

To open the application on a real mobile device, you can also put the HTML document on a web server and load the resulting URL in your mobile browser.

Next Steps

You could now...

- Try different navigation animation types (e.g. add "fade" or "flip" as second parameter to the `app.to("page2")` call)
- Add more pages
- Add more content to the pages
- Try out other mobile controls like the `sap.m.Popover`
- ...

Why 23 seconds and not 20, 21, or any other number of seconds?

In contrast to the other examples you need to add the URL parameter to make the App really usable on a desktop browser, so this takes a couple of seconds longer than usual...

Best Practice for Building Mobile Applications

Goal of this Document

SAPUI5 is not a restrictive UI framework but more a collection of UI controls together with loosely coupled infrastructure components like basic MVC support, data binding, localization and others. This approach offers great flexibilty but also requires more care in application development and will most likely yield in heterogenous application structures.

This document provides a best practice how to structure application projects along the most typical scenarios with a clean and consistent structure. Technically, the whole application could also just be coded into one single HTML file - and the simple "Hello World" applications are built exactly like this to make basic functionality easy to understand. But when creating reasonably complex applications a clean architecture is key for efficient development and maintainability. There are many degrees of freedom where this documentation decides for one of the possible options. E.g. when it comes to the file structure or the granularity of Views or even the usage of the MVC concept at all. This does not mean that other options are wrong. In fact, other options may suit specific applications even better and some decisions also depend on project experience, team setup or plain taste. Nevertheless, this document should provide valuable suggestions for a good application structure, it's just important to know that this is not the only possible or correct way to build SAPUI5 applications.

This document contains sections that refer to concepts or controls which are only available in the mobile library of SAPUI5 ('sap.m'), but most suggestions are the same or very similar for desktop applications.

Shopping Cart App

The concepts are applied to the 'Shopping Cart' demo app, and its source code can also help understand the proposed concepts.

This picture provides an overview of the involved components:

In the beginning the **browser** loads the web applications's **index.html** which contains itself only minimal code. The index.html then loads the **Application.js** which actually launches the application and instantiates the **App View** which holds the **sap.m.App** control that serves as root container for all UI controls.

The application user interface is split into six pages: **Home, Category, Product, Cart, Entry** and **Order**. These pages are represented each by a view containing the **sap.m.Page** control and a controller handling the view events. When trigger a navigation to another page an event is send via the **EventBus** to the App Controller. There are no direct dependencies between views, and those EventBus events are only used for navigation and parameter handover between pages.

To enable the back button of Android devices the **jQuery.sap.history** plugin is used which listens to changes in the browser history and triggers the App Controller for backward navigation.

The **InstanceManager** tracks open instances of dialogs and popovers. The App controller checks on backward navigation for open instances and closes them on demand.

There are two data models: The **Product Model** containing the available goods. The **Cart Model** contains the items put into the shopping cart. In addition there are two models that contain resources used by the application: The **Image Model** abstracts from access paths and the **i18n Model** connects to translated texts.

The **Formatter.js** class is an example for utility coding that is referenced by all views.

Detailed Guidance

- File Structure – Proposed structure and naming for the file system layout
- index.html - Detailed description how to structure the index.html
- Application.js – An example of the Application.js that starts up the application

Connecting to the Back End – Guidance how to handle mock data and use HTTP Proxy to overcome the Same-Origin-Policy
- Custom HTML – How to include custom HTML in your apps
- Navigation – A deep dive into navigation handling explaing all about the navigation events, history handling and instance manager
- i18n Model – Explains how to localize your app and setup the i18n model
- Image Model – Explains how to setup the image model
- Utilities – Where to put all the utility stuff (aka reused javascript coding)
- Lazy Loading – How to achieve lazy loading of the pages in the App Controller
- Message Handling - Guides you in the process of showing messages to the user

Using the Model View Controller Concept in Mobile Apps

The UI5 Model View Controller concept as described here can be equally used in UI5 Mobile applications.

The granularity of Views can be freely chosen; one possible approach is to use one View per page. This page illustrates such a usage of the MVC concept in order to decouple the different parts of the application. It is not supposed to provide a best practice but rather to illustrate the interaction of views. For a more complex example, see the Best Practice for Building Mobile Applications.

An MVC Demo Application

Overview

A simple Mobile application consisting of two pages is built to illustrate the MVC usage.

Three Views are used in total: one XMLView for the App control which forms the root level of the application, and two for the pages: one XMLView for the home page and one JSView for the detail page.

The home page displays a list of countries and tapping any of the countries will navigate to the detail page, displaying more information about this country.

IMPORTANT: The App control holds its child controls in the "pages" aggregation. This does **NOT** mean that only "sap.m.Page" controls can be aggregated. In contrary, this demo application will put the **Views** into the aggregation! Calling app.getPages() will therefore **NOT return any sap.m.Page control, but both directly aggregated Views'''**

The following image depicts the overall structure of the application. Note that the "homePageView" is displayed initially and the "detailPageView" is displayed when drilling down to details. The "appView" depicted above just holds the sap.m.App control which manages the page navigation. It does not have any visual UI, but is always rendered as a sort of invisible frame containing the application.

The HTML Page

The "App_with_rootView.html" HTML page launches the whole application. It is very minimal: it just

- loads SAPUI5
- ensures that all resources starting with "sap.m.mvc" (=the Views and Controllers) are loaded from locations relative to the HTML page
- instantiates the root View and places it into the "content" div element

```
<script id="sap-ui-bootstrap"
        src="../../../../resources/sap-ui-core.js"
        data-sap-ui-theme="sap_mvi"
        data-sap-ui-libs="sap.m"
        >
</script>


<script>
    // let UI5 know that certain package (the one containing the Views/Controllers) is
available locally (next to the HTML file)
    sap.ui.localResources("sap.m.mvc");

    // define View and place it onto the HTML page
    sap.ui.xmlview("appView", "sap.m.mvc.App").placeAt("content");
    // all other initialization will be done by the App controller

</script>
```

The Root (App-) View

The root View and Controller could also be omitted by putting some code into the HTML file, but this example tries to use MVC as much as possible, so it shows how Views are embedded into others.

It is also very simple: one sap.m.App control which aggregates two sap.m.Pages. The first one is displayed initially:

```
<core:View controllerName="sap.m.mvc.App"
    xmlns="sap.m" xmlns:core="sap.ui.core" xmlns:mvc="sap.ui.core.mvc"
xmlns:html="http://www.w3.org/1999/xhtml">

    <App id="theApp">
        <mvc:XMLView id="homePageView" viewName="sap.m.mvc.HomePage"></mvc:XMLView>
        <mvc:JSView id="detailPageView" viewName="sap.m.mvc.DetailPage"></mvc:JSView>
    </App>

</core:View>
```

The viewNames define the View type and from which location each View is loaded. The AppView also defines a "controllerName", so a Controller is loaded and attached as well.

The Root (App-) Controller

The root Controller does two things:

- It takes care of loading the data (using an AJAX call, from a JSON file) and initializing the overall JSONModel
- It handles the page navigations by listening to events published by the Views and triggering the appropriate navigation on the App control using the EventBus

When the user taps a list item on the homePageView, it publishes a "to" event in the "nav" channel (see the respective section below) which this root controller listens to and then executes the "navToHandler" method. After lazy loading the view, if necessary, it calls `this.app.to(data.id, data.data.context)` to trigger navigation to the second page and to hand over the bindingContext (information about the selected item in the initial page).
By default this leads to a horizontal slide animation, but other animations could be chosen.

Back navigation is handled similarly, but simpler, as no data is needed to be passed and the App control keeps track of the page stack, so it automatically navigates back to the "homePageView" (using an inverse animation).

```
sap.ui.controller("sap.m.mvc.App", {

    onInit : function() {

        var view = this.getView();

        // Data is fetched here
        jQuery.ajax("Data.json", {   // load the data from a relative URL (the Data.json file
in the same directory)
            dataType: "json",
            success: function(data){
                var oModel = new sap.ui.model.json.JSONModel(data);
                view.setModel(oModel);
            }
        });


        // remember the App Control
        this.app = view.byId("theApp");

        // subscribe to event bus
        var bus = sap.ui.getCore().getEventBus();
        bus.subscribe("nav", "to", this.navToHandler, this);
        bus.subscribe("nav", "back", this.navBackHandler, this);
    },

    navToHandler : function(channelId, eventId, data) {
        if (data && data.id) {
            // lazy load view
            if (this.app.getPage(data.id) === null) {
```

```
                jQuery.sap.log.info("now loading page '" + data.id + "'");
                this.app.addPage(sap.ui.jsview(data.id, "sap.m.mvc." + data.id));
            }
            // Navigate to given page (include bindingContext)
            this.app.to(data.id, data.data.context);
        } else {
            jQuery.sap.log.error("nav-to event cannot be processed. Invalid data: " + data);
        }
    },

    navBackHandler : function() {
        this.app.back();
    }
});
```

The HomePage View

The "HomePage" View is the content displayed initially. It contains a full-screen Page control which displays a List of countries. This List is constructed from a StandardListItem template using UI5 data binding. So for each country in the data one list item will be created and displayed.

```
<core:View controllerName="sap.m.mvc.HomePage"
    xmlns="sap.m" xmlns:core="sap.ui.core" xmlns:mvc="sap.ui.core.mvc"
xmlns:html="http://www.w3.org/1999/xhtml">

    <Page id="homePage" title="Countries">
        <List items="{/countries}">
            <StandardListItem title="{name}" description="{short}" type="Navigation"
press="listItemTriggered">
                <customData>  <!-- this is an OPTIONAL way to bind additional data to each
list item; when the item is pressed, this data is read in the Controller -->
                    <core:CustomData key="id" value="{short}" />
                </customData>
            </StandardListItem>
        </List>
    </Page>

</core:View>
```

The attribute `press="listItemTriggered"` means that when a list item is pressed, the "listItemTriggered" method of the HomePage Controller should be called.

The HomePage Controller

The Controller connected to this View does nothing else than just handling the press on a list item by publishing the event to "nav" channel and handing over the relevant view (DetailPage) and the context of the pressed list item.

```
sap.ui.controller("sap.m.mvc.HomePage", {

    listItemTriggered: function(evt) {
        // Option 1: using custom data attached to the ListItem
        // The ID (abbreviation) of the country is available as a custom data object and...
        // ...we could use it to fetch detail data
        // ...or we could hand it over to the detail page with  .to("detailPage", {id: id});
        var id = evt.oSource.data("id"); // this id remains unused in this example, though!

        // Option 2:
        // In case of data binding we can get the binding context (a sort of pointer to the
data object to which the clicked ListItem is bound)
        var bindingContext = evt.oSource.getBindingContext(); // evt.oSource is the ListItem

        // The EventBus is used to let the Root Controller know that a navigation should take
place.
        // The bindingContext is attached to the data object here to be used in the Root
Controller's event handler.

        var bus = sap.ui.getCore().getEventBus();
        bus.publish("nav", "to", {
            id : "DetailPage",
            data : {
                context : bindingContext
            }
        });
    }

});
```

The DetailPage View

In order to also use View types other than XMLViews, the detail page has been built as a JSView. As always in JSViews, its "createContent" method constructs the UI of this View - which again contains one sap.m.Page control with a List and a VBox layout as content. The List displays some country details, using data binding, so the correct country data is displayed without actively fetching this data.

Two special parts of this View should be highlighted before listing the entire View content:

Triggering Back Navigation

While there are differences between platforms concerning the semantics of the button in the left of the page header (it is a "back" button on iOS and an "up" button on Android), the sap.m.Page just offers a "navigation button" and the application can decide whether there is a difference between "up" and "back" on the respective page and whether it wants to follow the platform guidelines to respect this difference.

In this case, up is the same as back, so we just display this button (giving the label "Countries", which only appears on iOS) and when pressed, the method "backTriggered" of the Controller of this View is called (see below). The second array entry is the context of this method, so when the method is executed, "this" is the Controller.

```
var oPage = new sap.m.Page({
    title:"Details",
    showNavButton:true,
    navButtonText: "Countries",
    navButtonPress:[oController.backTriggered,oController]
});
```

Of course an application could take shortcuts, e.g. by directly calling app.back() if the "app" object is known inside the View. This demo application, however, wants to demonstrate maximum decoupling.

Handling the Navigation Event

As described here, pages (or rather children of NavContainer/App controls, like this detail View) are informed about navigation activities. One can attach delegates to these pages to also get informed and do certain preparation or cleanup work.

The detail View wants to update the data binding every time before it is displayed, so it registers for "beforeShow" and sets the binding context (which has been passed by the root Controller as payload data of the "to(...)" call which triggered navigation to the detail page.

```
this.addEventDelegate({
    onBeforeShow: function(evt) {
        this.setBindingContext(evt.data); // evt.to is actually this View (the navigation
target). This should be "this" in the future to be more intuitive...
    }
}, this); // give this (= the View) as additional parameter to make it available inside the
delegate's functions as "this" object
```

Similarly, the "beforeFirstShow" event could be used by the application to lazily construct the UI only when it is actually required because the user navigates to this page - currently the View UI creation in "createContent" is executed on application startup.

The Entire View Code

```
sap.ui.jsview("sap.m.mvc.DetailPage", {

    getControllerName: function() {
        return "sap.m.mvc.DetailPage";
    },

    /**
     * Creates the UI of this View
     *
     * @returns {sap.ui.core.Control}
```

```
     */
    createContent: function(oController) {

        var oPage = new sap.m.Page({
            title:"Details",
            showNavButton:true,
            navButtonText: "Countries",
            navButtonPress:[oController.backTriggered,oController]
        });

        // create the page content structure
        jQuery.sap.require("sap.ui.core.format.NumberFormat");
        var oList = new sap.m.List({headerText: "Country Details", items:
            [
                new sap.m.DisplayListItem({label:"Capital:",value:"{detailInfo/capital}"}),
                new sap.m.DisplayListItem({label:"Population:",value:{
                    path:"detailInfo/population",
                    formatter:function(iValue){
                        var oFormatter = sap.ui.core.format.NumberFormat.getIntegerInstance({
// format the population count
                            groupingEnabled: true,
                            groupingSeparator: "."
                        });
                        return oFormatter.format(iValue);
                    }
                }}),
                new sap.m.DisplayListItem({label:"Currency:",value:"{detailInfo/currency}"}),
                new sap.m.DisplayListItem({label:"Area:",value:{
                    path:"detailInfo/area",
                    formatter:function(iValue){
                        var oFormatter = sap.ui.core.format.NumberFormat.getIntegerInstance({
// format the area
                            groupingEnabled: true,
                            groupingSeparator: "."
                        });
                        var formattedNumber = oFormatter.format(iValue);
                        return formattedNumber + " sq km";
                    }
                }})
            ]});
        oPage.addContent(oList);

        var oFlagArea = new sap.m.VBox({
            alignItems: sap.m.FlexAlignItems.Center,
            items:
                [
                    new sap.m.Label({text:"Flag:"}),
                    new
sap.m.Image({src:"{detailInfo/flagUrl}",decorative:true,densityAware:false})
                ]
        });
        oPage.addContent(oFlagArea);

        this.addEventDelegate({
            onBeforeShow: function(evt) {
                this.setBindingContext(evt.data); // give this (= the View) as additional
parameter to make it available inside the delegate's functions as "this" object
            }
        }, this);

        return oPage;
    }

});
```

The DetailPage Controller

The Controller of the detail page is extremely simple, it just has the method which published the "back" event to the "nav" channel of the EventBus. The root Controller listens to this event and triggers back navigation.

```
sap.ui.controller("sap.m.mvc.DetailPage", {

    backTriggered: function() {
        var bus = sap.ui.getCore().getEventBus();
        bus.publish("nav", "back");
    }

});
```

Navigation and Lifecycle Events in the App and the NavContainer

Many mobile apps are composed of some pages and the user can drill-down to detail pages and go back up again (visualized often by horizontal slide animations).
UI5 supports this Pattern by providing the sap.m.App control which can handle pages and navigation between them.
Internally sap.m.App inherits the navigation capabilities from the sap.m.NavContainer control, so with regards to navigation and navigation events both are equal and this documentation will only talk about the NavContainer - but everything also applies to the sap.m.App control.

While a mobile application could centrally control this navigation flow and directly trigger the initialization or refresh of the pages, it is also beneficial when this can be done non-centrally, e.g. to support better modularization of the application: the code that constructs a page should also be the code that handles e.g. loading data in this page.

To support this, SAPUI5 provides two types of events:

- Events fired by the App/NavContainer whenever it navigates
- Events fired ON the pages when they get shown or hidden by navigation

Events fired centrally by the App/NavContainer

Whenever navigation occurs, triggered by calling NavContainer.to(...) or NavContainer.back(...), the NavContainer triggers events to which an application can register. One event ("navigate") is fired before the transition animation starts and the other ("afterNavigate") once the animation has completed.
These events contain plenty of information about the page which is being left and the page which is the target of navigation and what kind of navigation is happening.

Usage example:

```
app.attachNavigate(function(evt) {
   var isBack = !evt.getParameter("isTo"); // there are several types of back animation, but
we want the general direction only
   alert("Navigating " + (isBack ? "back " : "") + " to page " + evt.getParameter("toId"));
});
```

Check out the official documentation of navigate and afterNavigate.

Events fired on the Pages

This is what allows decentral reaction to navigation. The NavContainer also fires events on its child controls when they are being displayed or hidden.

NOTE: while this documentation calls them pages and a sap.m.Page control may be the typical child of a NavContainer, any "full-screen" control can be used: e.g. a sap.m.Carousel control or a custom control. Therefore the Event cannot be fired BY the child control, but it is rather fired by the NavContainer ON the child control (whatever type it is). Hence the slightly different registration compatred to normal control events.

Before the navigation animation starts, the NavContainer fires the following events:

- "beforeHide" (on the page which is about to be left)
- "beforeFirstShow" (on the page which is about to be shown - but only if it has never be shown before)
- "beforeShow" (on the page which is about to be shown)

This event can be used to create or update the UI of the new page and to stop any activity (like

animations or repeated data polling) on the page which is left.

After navigation has completed and the new page has covered the whole sccreen, the following events are fired:

- "afterShow" (on the page which is now shown)
- "afterHide" (on the page which has been left)

The hidden page can be destroyed if desired and the now active page could start certain activity.

You can register to these events using the "addEventDelegate" function available on every Control.

```
page1.addEventDelegate({
   onBeforeShow: function(evt) {
      // page1 is about to be shown; act accordingly - if required you can read event
information from the evt object
   },
   onAfterHide: function(evt) {
      // ...
   }
});
```

You can find the complete documentation here.

Passing Data when Navigating

When using the "to(...)" and "back(...)" methods of the NavContainer to trigger navigation, an optional payload data object can also be given. This object is then available in the "beforeShow", "afterShow" etc. events described above. This mechanism can also be used to decouple page implementations.

Example:

```
app.to("detailPage", {id:"42"}); // trigger navigation and hand over a data object
                                 // this data object could also be a binding context when
dealing with data binding
...

// and where the detail page is implemented:
myDetailPage.addEventDelegate({
   onBeforeShow: function(evt) {
      var idToRetrieve = evt.data.id;
      // ...now retrieve the data element with the given ID and update the page UI
   }
});
```

When a page is navigated back to, it receives the original data object which has been given when the page was first navigted to, but you can also give an additional data object with the back navigation. Again, you can find the complete documentation in the NavContainer for firing the event and here for receiving the data.

Example Application

The Model-View-Controller example application illustrates the usage of these navigation events and how they help in modularizing an application. You can launch it from this page?.

Adapting to Platform and Form Factors

The SAPUI5 Mobile library targets devices running different operating systems and having very different screen sizes and pixel densities. SAPUI5 does some adaptations automatically and applications may wish to adapt to the current device even beyond this scope.
This page gives an overview of what is done by SAPUI5 and what the application can do in addition.

Built-in Adaptation of SAPUI5 Mobile

Platform-dependent Styling

The "sap_mvi" ("Mobile Visual Identity") theme delivered with the SAPUI5 Mobile library provides three different visual styles which closely mimic the native look of the Apple iOS, the Android and the Windows Phone platform. Compared to the native visuals they are adapted to match the SAP color scheme with black and golden color accents.

- The iOS design has the typical Apple gradients and rounded corners, but is tinted in black and grey with golden accents.
- The Android design is using the Android 4.0 ("Ice Cream Sandwich") design as base with rather flat elements with no gradients or rounded corners. While the ICS design already has Black as a base color, the MVI theme adds golden accents while maintaining Google's original blue style in the interactive controls
- The Windows Phone design mimics the typical chrome-less visuals, but does not adapt the entire interaction to feel 100% native

The controls look very much like the native UI controls on these platforms, so the user immediately recognizes them.

On BlackBerry 10 currently the Android look and feel is used.

Other Platform-dependent Adaptations

The SAPUI5 Mobile library provides platform adaptations beyond the sheer visual design. To a certain degree controls automatically adapt to the native UI concepts and user experience without any effort on application side.

Examples for this are:

- The `sap.m.Page` control by default creates a header which looks different on both platforms even though there is only one single Page API:
  - On iOS the navigation button is the typical Apple back button having a label and an arrow shape on the left side and the header title is in the center
  - On Android the navigation button appears as just an arrow with no label, there is an optional icon displayed next to it, and the header title is displayed left-aligned
  - On Windows Phone there is never a back navigation button. Instead, the phone's hardware button is used for back navigation.
- The `sap.m.SegmentedButton` looks like a tab when being used in a bar on Android.
- The `sap.m.Dialog` appearance not only differs between operating systems, but also between form factors:
  - On Android it is a bright popup with buttons filling the whole popup footer
  - On iPad it is a Popup with dark header and buttons in the left and right side of the header
  - On iPhone it looks like on iPad, but it covers the full screen and is sliding in from the bottom

The SplitApp Control - adapting automatically to different form factors

The `sap.m.!SplitApp` control can be used to implement multi-page scenarios that automatically adapt to the available screen size: on phones only one page is displayed, on tablets, especially in landscape

orientation, two pages are displayed simultaneously. It is described in more detail here.

Options for further Adaptation

Beyond the adaptations done automatically by SAPUI5, the application can choose to do much more platform adaptations. UI5 provides a variety of detection mechanisms and markers to make this easy:

Mobile vs Desktop and Phone vs Tablet

How can an application detect whether it is on a Mobile device?
This is not easy to answer as the border gets blurs more and more: there are now desktop computers supporting touch operation, tablets where keyboards can be attached and even laptops where you can detach the screen which gives you a tablet. So where is the border? There is no clear separation.
But depending on your own definition of the border you can detect e.g. touch capabilities by checking

```
var isTouchDevice = jQuery.support.touch;
```

Based on this check there is also the following flag provided by UI5:

```
var runningOnDesktop = jQuery.device.is.desktop;
```

Similarly, when not running on desktop browsers, one of the following is true:

```
var runningOnTablet = jQuery.device.is.tablet;
var runningOnPhone = jQuery.device.is.phone;
```

On iOS the separation is clear, on Android the border between these classes is assumed to be at 600 pixels virtual screen width.

For your CSS, the same information is available on the html root tag as one of these three CSS classes:

```
sap-desktop
sap-tablet
sap-phone
```

So you can provide style for the phone case using CSS cascades as follows:

```
.sap-phone .myControl {
      font-size: small;
}
```

Note that when using the "fakeOS" device simulation on desktop browsers, the platform will be recognized as either "tablet" or "phone".

You might also consider checking the screen size (using media queries in CSS or the browser/jQuery APIS in JavaScript) or the presence of certain APIs or features.

Further information in the jQuery.device.is object includes:

- iphone, ipad: flags whether the application runs on either of them

iOS vs Android and Version Information (as JavaScript API)

The jQuery.os object holds information about the operating system:

- os: the operating system as string ("ios", "android", "winphone", or "blackberry")
- ios, android, winphone, blackberry: boolean flags for the respective platform
- version: the operating system version as string
- fVersion: the operating system version as float

iOS vs Android vs Windows Phone vs BlackBerry - and Version Information (for CSS styling)

Platform and browser version information is also added to the root element (the <html> element of the page) by UI5. There are three distinct packages of information:

- The CSS class "sap-ios", "sap-android", "sap-winphone", or "sap-bb" (for BlackBerry) is added for easy platform-dependent styling
- The attribute "data-sap-ui-browser" is added, providing more information about browser and version
- The attribute "data-sap-ui-os" is added, providing more information about operating system and version

The CSS class can be used directly in cascades, the other two need attribute selectors which can also be used to match substrings:

```
.sap-ios .myControl {
   color: red;  /* make my control red on Apple devices only */
}

html[data-sap-ui-browser*=msf] .myControl {
   background-color: blue;  /* Control should have a blue background whenever Mobile Safari is
running... */
}

html[data-sap-ui-os="iOS5.0"] .myControl {
   background-color: green;  /* ...but when running on an Apple device with version 5.0 of the
operating system, it should rather be green */
}
```

You can find more details here.

Orientation and Change of Orientation

The jQuery.device.is object also includes orientation information:

- landscape: flag whether the current orientation is landscape
- portrait: flag whether the current orientation is portrait

When the orientation is changed because the user rotates the device, the "orientationchange" event is fired by the browser, so you can handle this case as well. This event seems to be delayed in some cases, so the sap.m.App control also fires its own "orientationChange" event based on the browser's window resize event (which on mobile devices happens when the orientation is changed):

```
myApp.attachOrientationChange(function(evt) {
   if (evt.getParameters().landscape) {
      // do something...
   }
});
```

In CSS you can use CSS media queries to check this.

Platform attribute on Mobile

When running on mobile devices, a platform attribute `data-sap-ui-os` is added to the HTML tag. This attribute provides information of the current platform and the corresponding version. As an example, if one is running on an IPad3, the value of this attribute would be "iOS5.1". Currently, SAPUI5 Mobile only supports iOS, Android, Windows Phone 8 and Blackberry 10. On top of that, a platform-dependent CSS class is also added to the HTML tag of the page so that control/application developers can create platform-dependent styling for their controls or applications.

Technical details

When does this happen?

On load of the SAPUI5 bootstrap script file, a check is performed to see if the application is running on a mobile platform, and if so, the attribute and CSS class are put to the HTML tag.

Possible values

The value of the platform attribute is constructed with the following connotation: *Operation System + version*. e.g. iOS6.0, Android4.1.1, bb10.0.9.2372, winphone8.0, etc.

Operation System has 4 possible values:

- iOS (Apple devices)
- Android (Android devices)
- bb (BlackBerry)
- winphone (Windows Phone)

As seen from the example above, version are numbers separated by dots.

For the CSS class, the 4 possible values are:

- sap-ios (Apple devices)
- sap-android (Android devices)
- sap-bb (BlackBerry)
- sap-winphone (Windows Phone)

How to use the platform attribute/CSS class

As an example, assuming you want to provide a different font for Android devices, you can specify your font by directly using the CSS class "sap-android".

```
.sap-android{
        font-family: Roboto;
}
```

Now you want to specify a different font when running on Android 2.x, in order to do that, you can write the following CSS

```
html[data-sap-ui-os^='Android2'] .sap-android{
        font-family: "Droid Sans";
}
```

SplitApp - Hello World

When developing applications for tablets, for example the iPad or Google Nexus7, we need to make good use of the extra available space compared to smartphones.
Porting existing mobile apps to tablets will lead to a lot of unused space, since the tablet will display the app.

One of the most used patterns to address this is called the "Master-Detail" pattern which is also often used in native iOS and Android development.
Good examples are the native "Settings" and "E-Mail" applications of iOS and Android tablets. This pattern can be used with the SplitApp control.

The image below shows the basic idea of this pattern. The app is divided in two views, the Master and Detail view.
The Master is used as the main navigation of the application. It mostly presents a list of items whereas the detail view then presents the detail of the selected item.

On a mobile devices the selection of an item would instead navigate the user to the detail page. So on a tablet the user can see the list of items and the detail view at the same time.
This will be different if the tablet device is used in portrait mode, since the tablet will have less available space in width.
Hence we need to decide how we deal with the reduced with.
Therefore the SplitApp? control provides three different modes how to display the master and detail view when the device is in portrait mode:

- **ShowHideMode**
  This is the default mode, which will hide the master view automatically when the user switches the device into portrait mode.
  To show the master view the user can swipe right on the detail view or use the button which will be placed in the header of the detail view.
  The master then will slide in from the right. The user can then chose another list item, which will update the detail view and automatically hides the master view again.

- **PopoverMode**
  This mode will place the master view inside a Popover, which can be opened via the button in the header of the detail view.

- **StretchCompressMode**
  This is the only mode where master is shown in both the portrait and landscape mode of the device. Hence the detail view will have less space when in portait mode.

All modes however will always show the master view when the device is in landscape mode.

So, do we need to develop two applications? One for mobile devices and one for tablet devices? The good news is when a SplitApp is run on a mobile device,
it automatically behaves like a standard mobile application. The difference can be seen in the image below:

Since we can only show one page per screen our master and detail view will automatically be shown on separate pages and the standard page navigation will be applied.

Now lets build our first SplitApp application.

The first SplitApp

In order to develop a SplitApp application we will use sap.m.SplitApp instead of the sap.m.App control which is used for smartphone applications only.

```
var oSplitApp = new sap.m.SplitApp("mySplitApp", {});
```

As with the App control, we can add pages to our control but we have to decide to which view we want to add them:

```
oSplitApp.addDetail() or oSplitApp.addMaster();
```

But first we need some basic pages, two for our master and two for our detail view:

```
var oDetailPage1 = new sap.m.Page("detail1", {
   title : "Detail 1",
   content : [ new sap.m.Label({
                  text : "this is Detail 1"
               })
             ]
});

var oDetailPage2 = new sap.m.Page("detail2", {
   title : "Detail 2",
   content : [ new sap.m.Label({
                  text : "this is Detail 2"
               })
             ]
});
```

The two master pages need a list so we can implement a basic navigation and to select our detail view.
We will use a simple StandardListItem with the type "Navigation", since we want to navigate or "drill down" only inside our master view.
So when the user taps on our list item we want to navigate to master page 2.
To do so we call the method toMaster() and the page we want to navigate to, in our example it is "master2".

```
var oMasterPage1 = new sap.m.Page("master1",{
   title : "Master",
   content : [ new sap.m.List({
                 items : [ new sap.m.StandardListItem({
                         title : "To Master 2",
                         type : "Navigation",
                         press : function() {
                                 oSplitApp.toMaster("master2");
                         }
                 })]
        }) ]
});
```

Our second master page will be the leaf point of our master navigation, because now we want to update our detail view, depending on which list item the user is pressing.
An example would be a mail application where the list is a list of emails and the display view would then show the mail's content.
That is why we set the mode of our list to "SingleSelectMaster", since we do not want any further navigation inside the master view.
To update the detail view we have to listen to the list's select events, so we can decide on which detail page to show.
Depending on the selected list item we will show either page "detail1" or "detail2". Hence we call the method toDetail, since we want to navigate inside our detail view.

```
var oMasterPage2 = new sap.m.Page("master2",{
   title : "Master2",
   navButtonPress : function() {
                    oSplitApp.backMaster();
                   },
   content : [ new sap.m.List({
                 mode:"SingleSelectMaster",
                 select: function(oEv) {
                            if(oEv.getParameter("listItem").getId() == "listDetail2") {
                               oSplitApp.toDetail("detail2");
                         }else {
                               oSplitApp.toDetail("detail1");
```

```
                        }
                },
                items : [ new sap.m.StandardListItem("listDetail2",{
                                title : "To Detail 2"
                        }), new sap.m.StandardListItem("listDetail",{
                                 title : "To Detail 1"
                }) ]
        }) ]
});
```

In order to make our SplitApp also run on mobile devices we need to add some more information to our pages.
Therefore we have to make sure we can navigate back from any detail page to the master page.

Now we can set everything together and put our pages to the SplitApp control:

```
//add the master pages to the splitapp control
oSplitApp.addMasterPage(oMasterPage1).addMasterPage(oMasterPage2);

//add the detail pages to the splitapp control
oSplitApp.addDetailPage(oDetailPage1).addDetailPage(oDetailPage2);
```

To set the initial master and detail page, which are shown first when the app launches, we can need to call:

```
oSplitApp.setInitialDetail("detail");
oSplitApp.setInitialMaster("master");
```

We also can define what kind of page transition we want to use. For more information, see the documentation for the NavContainer.

```
oSplitApp.setDefaultTransitionNameDetail("fade");
```

Finally we place our SplitApp control to the HTML body tag:

```
oSplitApp.placeAt("body");
```

This link provides a sample SplitApp application : SplitApp Demo

To test the other two SplitApp modes we can set the them via this method:

```
oSplitApp.setMode("PopoverMode");
//or
oSplitApp.setMode("StretchCompressMode");
```

List API

To get a better understanding of the different states a list can appear in, several areas und combinations are described in this page.

List

The mode property of a list defines in which way the left area of a list item will appear. You can show a single selection, multi selection, delete buttons or none of these.

- mode
  - None (default)
  - SingleSelect (on the right side)
  - SingleSelectLeft (on the left side)
  - SingleSelectMaster (without select control for use cases like the split app, by default the !includeItemInSelection = true)
  - MultiSelect
  - Delete

With the includeItemInSelection (default: false) property you can change the tap handling of a list item. By default an item can only be selected by tapping on the radio button or check box. If you want to use the whole list item tap for selecting an item, change includeItemInSelection to 'true'. This property will only be taken to account, when using a selection mode.

- includeItemInSelection (default: false)

The showUnread property (default: false) decides, if an unread indicator is added to each list item. Each list item has an unread property (default: false), which shows a blue bubble when active.

- showUnread (default: false)

If a list has no content a text will be shown to the user. The default text ('No data') can be customized with the noDataText property. By changing the showNoData property (default: true) you can decide, if a text should be shown.

- showNoData (default: true)
- noDataText (default: 'No data')

Swipe For Action

A user can swipe on a list item to bring in a control (e.g. a Button) to initiate an action for this item. This control is defined through the 'swipeContent' aggregation of the List and will be displayed on the right on center of the list item. Check here for details.

List Events

With the selection, deletion and swiping we also needed new events. Selection modes fires a 'select' event and the deletion a 'delete' event. A swipe left fires a 'swipe' event. These events contain the event causing list item within them.

- select (listItem)
- delete (listItem)
- swipe (listItem)

Re-Rendering

The List including all contained items is completely re-rendered when the data of a bound model is changed. Due to the limited hardware resources of mobile devices this can lead to longer delays for lists with many items. As such the usage of a list is not recommended for these use cases.

ListItemBase

All SAPUI5 list items inherit from ListItemBase. It contains the navigation, selection and event features.

By now you can choose between five types, which decide the way a list item is interacting. A list item has a content (main area) which may fire a "press" event and on the right side a navigation area, which might fire a "press" or a "detailPress" event. The "type" property on each ListItem defines which events are fired. This needs to be configured because it also decides the visual feedback a ListItem is giving when being touched.

Note that a ListItem (except for the ActionListItem) does by default not fire an event unless it is configured with a type that defines how events are fired.
The following table describes the different combinations (which event is fired when a list item is set to a certain type):

type	press Event	detailPress Event	icon	active Feedback
Inactive(default)	-	-	-	-
Active	yes	-	-	yes
Navigation	yes	-	>	yes
Detail	-	yes	(>)	-
DetailAndActive	yes	yes	(>)	yes(only content)

As mentioned above the ListItemBase has a unread indicator property (default: false), which shows a blue bubble. This has to be enabled by the list showUnread property. For the selections on each list item a selected property (default: false) exists. Another feature is the counter property (default: null), which shows integer numbers except zero. If the number is zero, the counter will be hidden.

- unread (default: false)
- selected (default: false)
- counter (default: undefined)

ListItemBase Events

The ListItemBase fires a "press" event, when tapping the content and the detailPress event is fired by tapping the detail icon.

- press
- detailPress

ActionListItem

The ActionListItem inherits all features from the ListItembase and provides following additional features:

1. A text can be set, which will be center aligned.

- text

This is a simple list item for triggering actions.

CustomListItem

The CustomListItem inherits all features from the ListItembase and provides following additional features:

1. A content can be aggregated.

- content (the CustomListItem can be used for all list items we don't provide. Build your own content and aggregate it.)

Custom HTML

There are several ways to use custom HTML in lists.

HTML Control

An easy way to show simple content, is adding an HTML control into the CustomListItem and there you can directly write your HTML. Either write directly HTML or if you are using a model and databinding integrate it between the HTML by using a formatter. Remember that all changes to the model will rerender the control.

E.g. Let's assume we have a model with first name, last name, age and city properties. We create a CustomListItem, add a HTML control to the list item content and than write a formatter with our HTML code.

```
var myCustomListItem = new sap.m.CustomListItem({
  content: new sap.ui.core.HTML({
    content: { parts: [
                          {path: "firstName"},
                          {path: "lastName"},
                          {path: "age"},
                          {path: "city"}
                      ],
            formatter: function(firstName, lastName, age, city) {
      return "<div><div class='myStyleClass'>Name: " + firstName + " " + lastName +
                  "</div><div>Age: " + age +"</div><div>City: " + city +"</div></div>";
    }
    }}),
  type : 'Navigation' //({type} if provided by the model)
  press : myTapEvent
});
```

This approach does only work when programming the UI in JavaScript. When using a declarative model - like XML Views - you should use an On-The-Fly-Control.

Notepad List Item Content Control

With notepad controls you have a very good way to create reuseable controls in a structured way. This also provides you with the chance to use UI5 controls inside of your custom coding.

Assume you want to build a product list item. It shows an image of the product and some product details. You can create a notepad control for the list item content and than add it to the CustomListItem:

```
jQuery.sap.declare("my.control.ListItemContent");
sap.ui.core.Control.extend("my.control.ListItemContent", {

metadata: {
        properties : {
            "name": {type: "string", defaultValue: ""},
            "description": {type: "string", defaultValue: ""},
            "price": {type: "string", defaultValue: ""},
            "currency": {type: "string", defaultValue: ""},
            "image": {type: "string", defaultValue: ""}
        },
                events: {
            "myTap": {},
        },
    },

    init: function(){
        this._image = new
sap.m.Image({src:"<myImageSrc>"}).addStyleClass("myImageCSS").setParent(this);
    },

    renderer: function(oRm, oControl) {
        oRm.write("<div class='listItemCSS'");
        oRm.writeControlData(oControl);
        oRm.write(">");
            oRm.renderControl(oControl._image);
            oRm.write("<div class='descCSS'>");
            oRm.writeEscaped(oControl.getDescription());
            oRm.write("</div>");

            oRm.write("<div class='priceCSS'>");
            oRm.writeEscaped(oControl.getPrice());
            oRm.write("</div>");

            oRm.write("<div class='curCSS'>");
            oRm.writeEscaped(oControl.getCurrency());
            oRm.write("</div>");

            oRm.write("<div class='nameCSS'>");
            oRm.writeEscaped(oControl.getName());
            oRm.write("</div>");
        oRm.write("</div>");
    oRm.write("</div>");
    }
});

//example how to avoid rerendering of text, when the name property is changed
my.control.ListItemContent.prototype.setName = function(sText){
    this.setProperty("name", sText, true);
    return this;
};

//example how to use events
my.control.ListItemContent.prototype.ontap = function(){
        //your own tap logic
    this.fireMyTap({});
};

var oCustomListItem = new sap.m.CustomListItem({ content: [new my.control.ListItemContent({
    //usual control setup
})]});
```

Notepad List Item Control

Instead of using the CustomListItem, where you add the list item content you could create a new notepad list item control. Therefore you have to inherit from ListItemBase and use the renderer hook. The benefit to the content control is, that you do not need the CustomListItem to your notepad control and you have control over the whole list item. On the other hand, you should know what you do, otherwise you can cause a lot of issues. Since this extends the ListItemBase you can accidentally overwrite properties/methods for all sap.m list items. If you overwrite methods without calling the base method the control might not work anymore. Therefore check the existing methods and if you need to extend them, make sure you call the base method, too.

Assume you want to build a product list item, which shows an image of the product and some product details:

```
jQuery.sap.declare("my.control.MyListItem");
sap.m.ListItemBase.extend("my.control.MyListItem", {

metadata: {
        properties : {
            "name": {type: "string", defaultValue: ""},
            "description": {type: "string", defaultValue: ""},
            "price": {type: "string", defaultValue: ""},
            "currency": {type: "string", defaultValue: ""},
            "image": {type: "string", defaultValue: ""}
        },
                events: {
            "myTap": {},
        },
    },

    init: function(){
        this._image = new
sap.m.Image({src:"<myImageSrc>"}).addStyleClass("myImageCSS").setParent(this);
    },

    renderer: {
            //this is the hook method of the !ListItemBase
            renderLIContent : function(oRm, oControl) {
        oRm.write("<div class='listItemCSS'");
        oRm.writeControlData(oControl);
        oRm.write(">");
            oRm.renderControl(oControl._image);
            oRm.write("<div class='descCSS'>");
            oRm.writeEscaped(oControl.getDescription());
            oRm.write("</div>");

            oRm.write("<div class='priceCSS'>");
            oRm.writeEscaped(oControl.getPrice());
            oRm.write("</div>");

            oRm.write("<div class='curCSS'>");
            oRm.writeEscaped(oControl.getCurrency());
            oRm.write("</div>");

            oRm.write("<div class='nameCSS'>");
            oRm.writeEscaped(oControl.getName());
            oRm.write("</div>");
        oRm.write("</div>");
    oRm.write("</div>");
    }
    }
});

//example how to avoid rerendering of text, when the name property is changed
my.control.MyListItem.prototype.setName = function(sText){
    this.setProperty("name", sText, true);
    return this;
};

//example how to use events
my.control.MyListItem.prototype.ontap = function(){
        //your own tap logic
    this.fireMyTap({});

        //this is very important, when the base method should be called
        //otherwise the base method won't be called anymore and depending on the method a
control might not work anymore
        sap.m.ListItemBase.prototype.ontap.apply(this, arguments);
};

var oMyListItem = new my.control.MyListItem({
    //usual control setup
});
```

DisplayListItem

The DisplayListItem inherits all features from the ListItembase and provides following additional features:

1. A label can be set.
2. A value can be set.

label
- value

InputListItem

The InputListItem inherits all features from the ListItembase and provides following additional features:

1. A label can be set.
2. A content can be aggregated.

- label
- content (To the content you can add a input-, radio button-, checkbox-, slider-, select-, search-control)

StandardListItem

The StandardListItem inherits all features from the ListItembase and provides following additional features:

1. An icon on the left side which can be shown with or without an inset.
2. A title can be set.
3. A description can be set.

- title
- description
- icon (Icon source path where all the images for each device resolution can be found.)
- activeIcon (Icon source path for images which should be shown while the list item is tapped.)
- iconInset (Default: false images takes the whole list item height, true for an inset.)
- iconDensityAware (If you won't provide specific device resolution icons, set it to false. Therefore no checks fpr other resolutions are made.)

Triggering Phone/SMS/E-Mail

With SAPUI5 for Mobile you can easily trigger phone's native applications like E-Mail, Telephone and SMS. This can be very useful because you can set predefined values for the application and users don't have to enter these fields by themselves. When displaying personal information like phone numbers and email you can initiate a phone call or email application with just one tap.

API

Let's first take a look at the API and in the next chapter we will build simple examples. You can find URLHelper API from

Trigger Telephone Application

```
sap.m.URLHelper.triggerTel( [Telephone Number] ); //Telephone number is optional
```

Trigger SMS Application

```
sap.m.URLHelper.triggerSms( [Telephone Number] ); //Telephone number is optional
```

Trigger Email Application

```
sap.m.URLHelper.triggerEmail( [Destination Email], [Subject], [Default Message Text], [CC], [BCC] ); // All
parameters are optional
```

Redirect To Custom URL

```
sap.m.URLHelper.redirect( URL ); //URL is required and can be used for custom protocols(e.g http, ftp, ...)
```

Examples

Firstly, let's create our sample data and then you will see the different possibilities in examples.

```
var person = {
    name : "John Smith",
    tel : "+49 62227 747474",
    sms : "+49 173 123456",
    email : "john.smith@sap.com",
    website : "http://www.sap.com"
};
```

Triggering an external application can be done at any time but most often it happens as a reaction to a UI control throwing an event. The next sections illustrate some of the most typical use cases.

Click Button To Trigger Phone Call

Lets build a button that the user can click to place a call. At the end the button will look like this:

```
new sap.m.Button({
    text : person.tel,
    icon : "images/action.png", /* Depends where your images are located */
    tap : function() {
        sap.m.URLHelper.triggerTel(person.tel);
    }
});
```

Click Image To Trigger Email

You can also set Subject and Message of the email app.

```
new sap.m.Image({
    src : "images/website.png",  /* Depends where your images are located */
    tap : function() {
        sap.m.URLHelper.triggerEmail(person.website, "Info", "Dear " + person.name + ",");
    }
});
```

Inside List

DisplayListItem with active feedback is the most popular use-case.

```
new sap.m.DisplayListItem({
    label : "Sms",
    value : "(  " + person.sms + " )",
    type : "Active",
    tap : function() {
        sap.m.URLHelper.triggerSms(person.sms);
```

```
    }
});
```

But if you want to use any other control inside the list this is still possible with InputListItem?.

```
new sap.m.InputListItem({
    label : "Website",
    content : new sap.m.Button({
        text : person.website,
        tap : function() {
            sap.m.URLHelper.redirect(person.website);
        }
    })
});
```

NOTES

1. If you have an "*" or "#" inside of the Telephone Number then iOS does not try to trigger phone call.
2. In Android you can add multiple recipients for an SMS by separating recipient numbers using ";".
3. According to "RFC 2368" you can set multiple subscribers for the email application by separating each with "," but please keep in mind this still depends on the application (e.g Outlook uses ";" as separator).
4. You can use the sap.m.URLHelper.redirect method to use custom URL Schemes:
    1. For iOS: http://developer.apple.com/library/safari/#featuredarticles/iPhoneURLScheme_Reference/Introduction/Introduction.html
    2. For Android: http://developer.android.com/guide/appendix/g-app-intents.html
    3. URI schemes: http://en.wikipedia.org/wiki/URI_scheme
5. If you just want to get a URI back but you want no redirect to happen, then you can use normalize methods which have the same parameter as trigger methods, like:

    ```
    /*
     * These methods do not redirect but return URI scheme back as string.
     * All parameters are optional
     */
    sap.m.URLHelper.normalizeTel( [Telephone Number] );
    sap.m.URLHelper.normalizeSms( [Telephone Number] );
    sap.m.URLHelper.normalizeEmail( [Destination Email], [Subject], [Default Message Text], [CC], [BCC] );
    ```

6. You can find URLHelper API under this link

Scrolling in SAPUI5 mobile

Because of limited size of mobile devices, scrolling is an essential topic in mobile user experience. Smooth and easy scrolling is important for user acceptance of mobile applications.

In general, application programmers do not need to take care of scrolling when using the `sap.m` control library - scrolling is provided automatically by the sap.m.Page, sap.m.Dialog, sap.m.Popover, and sap.m.ScrollContainer controls. However, if you are going to create a custom scrollable control, read the description below.

Implementation details

Unfortunately, scrolling support in mobile browsers is very weak and inconsistent. Only the newest platforms and browsers start to support partially usable scrolling functionality.

Hence, SAPUI5 embeds the open source iScroll4 library that takes care of scrolling in the application. Though the library is globally available in a UI5 application, programmers should not call it directly. The sap.ui.core.delegate.ScrollEnablement delegate that provides all needed functionality and smooth integration of iScroll4 into the SAPUI5 library should be used instead of this.

Do not use nested scrolling

We do not recommend to use nested levels of scrolling, for example, when a page with enabled vertical scrolling contains a scroll container that has vertical scrolling too. Such combinations may lead to behavior that is unexpected both for programmers and users.

Implement a custom scroll container

A custom control that needs to provide a scrollable area for its content should implement the following steps:

1. Instantiate a `sap.ui.core.delegate.ScrollEnablement` delegate, at best in the `.onAfterRendering` callback.
2. Implement a `.getScrollDelegate` method that returns the current instance of the delegate to other controls.
3. Destroy the `ScrollEnablement` delegate on exit.

Example:

```
myCustomScroller.prototype.onAfterRendering = function() {
  if(!this._oScroller){
    jQuery.sap.require("sap.ui.core.delegate.ScrollEnablement");
    // attach a scroller to the scrollable container DOM element
    this._oScroller = new sap.ui.core.delegate.ScrollEnablement(this, this._scrollContainerId,
{
      horizontal: false,
      vertical: true
    });
  }
};

myCustomScroller.prototype.getScrollDelegate = function() {
  return this._oScroller;
};

myCustomScroller.prototype.exit = function() {
  if(this._oScroller){
    this._oScroller.destroy();
    this._oScroller = null;
  }
};
```

Note: Though the UI5 library includes also a Zynga scroller, it is an experimental alternative to iScroll4 that is not supported in UI5 and may be used for testing only. The configuration parameter

`oConfig.zynga=true` of the scrolling delegate should therefore not be used.

Interaction with the scroll containers

There are cases, when an embedded control controls scrolling of the parent container, if required. These are

- a sap.m.ScrollContainer inside of a Page may block parent scrolling, if it scrolls in the same direction itself;
- a sap.m.TeaxtArea control in edit mode blocks parent scrolling, so that the user can scroll text contents during input;
- a sap.m.GrowingList control scrolls parent container to update positions of visible items after the new items have been loaded from the server.

When using a sap.m.FlexBox with `fitContainer` set to true inside of a Page, the `enableScrolling` property of the Page needs to be set to false for the FlexBox to fit the viewport.

Pull to Refresh

The SAPUI5 mobile library supports the "pull down to refresh" functionality that allows users to refresh lists or page content with fresh data from server. To implement it, create a sap.m.PullToRefresh control and put it as the first control into a page or a scroll container that contains the list that needs to be refreshed.

Example:

```
var pullToRefresh = new sap.m.PullToRefresh({
  description: getLastUpdatedTime(),
  refresh: function(){
    pullToRefresh.setDescription("loading from server...");
    // request new data from server
    getNewData({
      // when data comes from server
      onSuccess: {
        pullToRefresh.hide();
        pullToRefresh.setDescription(getLastUpdatedTime());
        redrawList();
      }
    });
  };
```

The application should request new data on the `refresh` event and call the `hide` method when the data is received and the list is refreshed. You can provide a URL to a custom logo image with `customIcon` or switch display of logo of by setting `showIcon` to `false`. The first line of text "Pull to refresh" is standard and cannot be changed. However, you may set an optional description text to display, for example, the last update time.

Note: PullToRefresh control is part of the scroll area and therefore its height is reflected in the scroll bar calculation and display. The user can see that the page can be scrolled down to reveal the pull-down area.

Carousel

Pull to Refresh does not work with a Carousel if both are contained in a page: in order to make Pull to Refresh work, the page has to enable scrolling which leads to problems with the Carousel (Carousel not visible). Suggested Workaround: Add a sap.m.PullToRefreshinstance to each page that you add to your Carousel.

Running in Hybrid Web Containers

General Considerations

SAPUI5 Mobile apps can be built either as web applications which are loaded from a certain URL, or as so-called "Hybrid Apps" which have a native app wrapper, for example PhoneGap, but the entire user interface consists of a HTML viewer where the normal SAPUI5 content is displayed.
Both have advantages and disadvantages. Hybrid Apps can be published in App Stores, the whole application code and the UI5 library files can also be embedded into this Hybrid container, so these files are rather installed once than downloaded on every application start. On the other hand this means that the total library size matters because every user has to install everything. In normal web applications the library deployed on the server may be huge, but only the needed parts are downloaded at runtime.

For these Hybrid Apps, the static package of SAPUI5 Mobile can be used to include all needed SAPUI5 resources, but still have a relatively small total library size because all test pages etc. are removed from this special package. The file is:

- sapui5-mobile-opt-static.zip (or sapui5-mobile-static.zip)

It contains the debug version of all Javascript files as well as the optimized and minified one (which are used by default) so it can be used for productive as well as debug purposes.

To use this package e.g. in PhoneGap, it just needs to be unzipped inside the respective resource location of the app development project. The PhoneGap build will then include the files and make them available at runtime.

Optimizing the Package Size: Delete Unneeded Files

While the above file is already pretty small (and small enough for inclusion in hybrid apps), the size can be further reduced and optimized for specific applications by deleting unneeded files:

- In `/resources` you can delete either `sap-ui-core-nojQuery.js` if you reference UI5 normally, or `sap-ui-core.js` if you reference the nojQuery version and reference jQuery separately (e.g. because you run in a Sybase Hybrid Container).
- In `/resources/sap` you can delete the entire `makit` folder when you do not need the makit charts.
- In `/resources/sap/ui/thirdparty` you can delete the entire `jquery` and `jqueryui` folders when you use the normal `sap-ui-core.js` file for bootstrap (they may only be needed when using the `sap-ui-core-nojQuery.js` script, and even then you may have another copy of jQuery available, so that folder can still be deleted).
- In `/resources/sap/ui/core/themes` and in `/resources/sap/m/themes` you can delete the "base" and the "sap_bluecrystal" folder (unless you are using this experimental theme, then you can delete "base" and "sap_mvi")

Of all JavaScript files there is an optimized and a "-dbg" version of each file, so when deleting the above files make sure you always delete both versions of them!
If you do not want to allow easy debugging and you want to go for minimum installation size, also ALL *-dbg.js files can be deleted from the "-opt-static" package.

Further files can be deleted, but the size reduction is limited and finding out what is not required gets increasingly difficult.

Device Ready Event

The hybrid web container takes some time to initialize itself. During the initialization, it will block sending AJAX request which means the Javascript code will stop at the line where AJAX request is sent and won't do further code execution. This leads to a UI freeze effect which isn't the purpose of sending AJAX request. Since the oData model in SAPUI5 uses AJAX request internally, the oData model initialization

must be done after the hybrid container is ready in order not to block the user interface.

Luckily, hybrid containers fire a special event when it's ready. This event is called *deviceready* in PhoneGap. Simply moving the code where the oData model is created and set to the core object or any other controls' model property to the deviceready event listener will fix the freezing UI issue. The code may look like below:

```
<script>
<!-- put the following code in the beginning of the application code -->
function appReady(){
    sap.ui.getCore().setModel(new sap.ui.model.odata.ODataModel(<ODATA_URL>, false));
}
<!-- bind to the deviceready event -->
document.addEventListener("deviceready", appReady, false);
</script>
```

Cross Domain Restriction

When data needs to be loaded from external server or service using AJAX, the external domain has to be configured inside the hybrid web container in order to make the AJAX request go through the cross domain restriction. Talking about PhoneGap, please see the reference page for detailed information.

We have integrated our demo applications into PhoneGap and here are our findings after doing the integration:

Android

There is no cross domain restriction any more if the AJAX code runs inside the webview in Android which means we can load data using AJAX from everywhere. This is what we found but in the PhoneGap documentation, it says the domain still needs to be configured in one XML file.

iOS

There's still this restriction in Webview in iOS, the domain that will be visited using AJAX should be added to a while list file in order to enable this to go through the restriction. For the detailed information about this white list file, please see the reference here.

Mobile Events

Comparison to Desktop Events

What is equal:

1. When running on mobile/touch devices, the application has still the possibility to access to all native browser events, including the touch-related ones.
2. Handling of control events fired by UI5 is also the same as on desktop PCs.
3. When implementing UI5 controls, some browser events can be handled very easily by implementing a method named `on<eventName>`, so all the bind/unbind effort is avoided. This is equally possible on mobile.

So basically the whole concept around events is identical.

What is different on Mobile?

However, the difference is that on mobile devices **additional** events are available within control implementations (see item 3. above).

Additional Mobile Browser Events

On touch-enabled platforms the following events are also provided within UI5 controls to be handled in "`on<eventName>`" methods:

- touchstart
- touchend
- touchmove
- touchcancel

Additional Mobile Pseudo Events

jQuery mobile event handling is used within UI5 when running on touch-enabled devices. From the basic browser events it creates semantically richer events. Some of them are also provided automatically within UI5 controls:

- swipe
- tap
- swipeleft
- swiperight
- scrollstart
- scrollstop

Please refer to the jQuery mobile documentation regarding details of these events.

Simulation of touch events on non-touch platforms

For testing or demonstration purposes, the events listed above can also be simulated on non-touch devices. When this simulation is enabled, the touch events will also be triggered by mouse interaction. **Important:** due to technical constraints the simulation cannot be perfect, so it may not be used in productive code.

To enable the simulation mode, set the UI5 configuration parameter"xx-test-mobile" to "true", e.g. by appending the url parameter "`sap-ui-xx-test-mobile=true`".

Events also available in Desktop apps

This section reiterates the list of events that are available within control implementations in **all** UI5 applications (even on non-touch platforms).

Browser Events

- click
- dblclick
- focusin
- focusout
- keydown
- keypress
- keyup
- mousedown
- mouseout
- mouseover
- mouseup
- select
- selectstart
- dragstart
- dragenter
- dragover
- dragleave
- dragend
- drop
- paste

Pseudo Events

The "pseudo events" can be used in `on<eventName>` methods inside controls like browser events, but are created by UI5 and typically have enriched semantics:

For the detailed meaning of each event please refer to the API documentation of jQuery.sap.PseudoEvents.

- sapdown
- sapdownmodifiers
- sapshow
- sapup
- sapupmodifiers
- saphide
- sapleft
- sapleftmodifiers
- sapright
- saprightmodifiers
- saphome
- saphomemodifiers
- saptop
- sapend
- sapendmodifiers
- sapbottom
- sappageup
- sappageupmodifiers
- sappagedown
- sappagedownmodifiers
- sapselect

- sapselectmodifiers
- sapspace
- sapspacemodifiers
- sapenter
- sapentermodifiers
- sapexpand
- sapbackspace
- sapbackspacemodifiers
- sapdelete
- sapdeletemodifiers
- sapexpandmodifiers
- sapcollapse
- sapcollapsemodifiers
- sapcollapseall
- sapescape
- saptabnext
- saptabprevious
- sapskipforward
- sapskipback
- sapprevious
- sappreviousmodifiers
- sapnext
- sapnextmodifiers
- sapdecrease
- sapdecreasemodifiers
- sapincrease
- sapincreasemodifiers
- sapdelayeddoubleclick

Styling / Theming Mobile Controls

Writing CSS

- CSS is used to provide the visuals of controls, just like for Desktop controls.
- The main differences are:
  - The mobile browsers supported by UI5 are all based on Webkit. This allows using advanced CSS3 features which are not available in all supported desktop browsers.
  - "em" and "rem" should be used for dimensions.

Using LESS Features like Theme Parameters

- When controls are created with Eclipse tool support, the CSS files are processed by the "LESS" Open Source CSS preprocessor. Just like in desktop control CSS files, LESS features - in particular variables or "theme parameters" - can be used.

Theming sap.m

Currently, we provide 2 major themes (SAP Mobile Visual Identity and Blue Crystal) for the sap.m library. If you want to modify a certain part of our controls to visually accommodate your application needs, this can be easily done with the Theme Designer.

Your step by step guide to theme your application

- Open the Theme Designer in a WebKit browser.
- Choose the theme you want to start as the basis with. Here, we will choose SAP Blue Crystal as an example.
- Then you put in the application you want to style. In our case, we are taking the shopping cart application.
  - Link to Application: https://sapui5.netweaver.ondemand.com/sdk/test-resources/sap/m/demokit/cart/index.html
  - Name of Application: give a name you wish, we take "shopping cart" here to keep it consistent.
- Now you should end up in the theme designer with a preview screen in the middle and, on the right hand side, a list of theme parameters you can use to adjust the current application you have specified.

Explanation of theme parameters

- Here, we want to focus on the different theme parameters and explain what effect each of them have.

Quick Panel

- These are the most commonly used color parameters of the currently selected theme as well as image related parameters in the quick panel.
- **Base Color**: It is used as base for many screen elements, including backgrounds, texts and borders. For example, if you change this color to bright red (#FF0000), you will see that the bar background as well as the border color of the list have been adjusted accordingly.
- **Brand Color**: This is the basic branding color of the theme. In our example, if you change the brand color to bright red (#FF0000), you will see that the top branding bar has changed its color from blue to red.
- **Highlight Color**: This parameter is used for highlighted screen elements. In our example, if you change the highlight color to bright red (#FF0000), you will see that the list header as well as the background highlight color are changed. When you click on the list, you will notice the highlight color is changed there too.

- **Link Color**: This is the color used for links.
- **Background Image**: Here, you can set your own background image to the app.
- **Background Image Opacity**: This parameter is only relevant if you have set a background Image, then it will change the translucency of the image.
- **Background Image Repeat**: This parameter by default is set to true for blue crystal theme and you can change it to false if you want to display 1 big image without repeating it.
- **Image Background Color**: This is the color that is behind the global background image.From our example, if you change the background color to bright red (#FF0000), you will see the background color is updated accordingly.
- **Logo**: You can define a logo image which can be used for branding. It may only appear in desktop browsers when the screen/window is large enough.

Expert Panel

- If you want to change more than what the quick panel could offer, you can switch to the expert panel. There you have more refined parameters to adjust the application.

- **sapUiBarBG**: This is the parameter to adjust the header bar background color.
- **sapUIBarText**: If you want to just change the bar title text, you can use this parameter for it.
- **sapUIBase**: This is the same as the "Base Color" in Quick Panel.
- **sapUIBaseBorder**: If you want to only change the border color, but not other base color, you can do that with this parameter.
- **sapUIBaseText**: This is the parameter to change the base color of text, this also change the color of icons, where icon font is used.

- **sapUIBrand**: This is the same as the "Brand Color" in Quick Panel.
- **sapUIGlobalBackgroundColor**: This is the same as the "Image Background Color" in Quick Panel.
- **sapUIGlobalBackgroundImage**: This is the same as the "Background Image" in Quick Panel.
- **sapUIGlobalBackgroundRepeat**: This is the same as the "Background Image Repeat" in Quick Panel.
- **sapUiGlobalLogo**: This is the same as the "Logo" in Quick Panel.
- **sapUIHighlight**: This is the same as the "Highlight Color" in Quick Panel.
- **sapUILink**: This is the same as the "Link Color" in Quick Panel.

CSS Panel

- In this panel, you can write your custom css to adjust style according to your needs.

Building Charts with MAKit

Overview

Our new MAKit Chart is a UI5 control that provides charting features. MAKit chart is designed for mobile/tablet use. It has unique features such as Value Bubble and Range Selector that make chart report much more rich and interactive. It supports touch interaction on mobile devices such as:

- tap to highlight certain part of the chart.
- double tap.
- longpress.
- two finger pince to zoom in or out.
- one finger drag to move chart's Value Bubble highlight.
- two finger swipe to scroll the chart when zoomed in.

Currently sap.makit.Chart supports the following types of charts:

- Column
- Horizontal Bar(Table Bar)
- Line
- Bubble
- Pie
- Donut
- Waterfall Column
- Waterfall Bar
- Stacked Column
- 100% Stacked Column

The Chart's **Value Bubble** provides an interactive way of showing details of the chart. It provides detailed information (i.e. the value) of the currently highlighted chart region. User can also hide/show certain series of the chart by toggling the legends on the Value Bubble.

The Chart's **Range Selector** gives an overview look of the whole chart to user and also allows them to zoom in or out to a specific part of the chart for more clarity.

The following is an overview look on sap.makit library Control and Elements:

- sap.makit/Chart – The SAPUI5 Chart control (type: sap.ui.core/Control)
- sap.makit/Axis – Base type for the Chart's Axis (type: sap.ui.core/Element)
- sap.makit/CategoryAxis – Properties of the Chart's Category Axis (type: sap.makit/Axis)
- sap.makit/ValueAxis – Properties of the Chart's Category Axis (type: sap.makit/Axis)
- sap.makit/Category– Properties of the Chart's Category Axis (type: sap.makit/Element)
- sap.makit/Value – Properties of the Chart's Value Axis (type: sap.makit/ Element)
- sap.makit/Series – Properties of the Chart's Series (type: sap.ui.core/Element)
- sap.makit/Row – Row collection for the data binding (type: sap.ui.core/Element)
- sap.makit/Column – Column mapping for the data binding (type: sap.ui.core/Element)
- sap.makit/ValueBubble – Properties for Chart's Value Bubble (type: sap.ui.core/Element)

Simple Types:

- sap.makit/ChartType – Enumeration for Chart type (type: string)
- sap.makit/SortOrder – Enumeration for Category's sortOrder (type: string)
- sap.makit/LegendPosition – Enumeration for legendPosition (type: string)
- sap.makit/ValueBubbleStyle – Enumeration for Value Bubble's positioning style (type: string) for Non-Pie/Donut/HBar chart.

sap.makit/ValueBubblePosition – Enumeration for Value Bubble's position on Pie/Donut Chart (type: string)

Creating a Chart

To create a chart you must specify its data region information and data binding.

Specifying data region

In MAKit chart data region are represented by the following 3 aggregations:

- Values - Value is required to determine the size of each element in the chart. For example, the height of column in a Column chart, or size of a slice in a Pie chart. You can have multiple values in the Values aggregation. This is useful for bubble chart as it is possible to set the size of the chart based on a different set of values
- Category - Category is required to group the chart's data. You can only have max 1 category in the Category aggregation.
- Series - (Optional) Series is used to add an extra dimension to the chart for example in this case to add product dimension to the chart. You can only have 1 series in the Series aggregation.

Example of specifying chart's data region at the chart's creation time:

```
var oChart = new sap.makit.Chart({
    height: "80%", width: "100%"
    category : new sap.makit.Category({ column : "yearCategory" }),
    series : new sap.makit.Series({ column : "productSeries" }),
    values : [new sap.makit.Value({ expression : "revenueValue", format : "currency" })]
});
```

The above codes will create an empty chart. To populate the chart you must bind it to a data source.

Populating the chart's data with data binding

Data table in MAKit is represented by:

- Rows
- Columns

Data of the chart must be provided via data binding so that the data table is always in consistent format.

To bind the chart to a data source you must bind the Rows to the your data source's collection and Columns to the column of the collection. For example:

```
var testData = {
    mycollection : [
        { year : 2008, product : "Prod 1", revenue : 900000 },
        { year : 2008, product : "Prod 2", revenue : 700000 },
        { year : 2009, product : "Prod 1", revenue : 100000 },
        { year : 2009, product : "Prod 2", revenue : 900000 },
        { year : 2010, product : "Prod 1", revenue : 110000 },
        { year : 2010, product : "Prod 2", revenue : 120000 }
    ]
};

var jsonModel = new sap.ui.model.json.JSONModel();
jsonModel.setData(testData);

oChart.addColumn(new sap.makit.Column({name:"yearCategory", value:"{year}"}));
oChart.addColumn(new sap.makit.Column({name:"productSeries", value:"{product}"}));
oChart.addColumn(new sap.makit.Column({name:"revenueValue", value:"{revenue}",
type:"number"}));
oChart.setModel(jsonModel);
oChart.bindRows("/mycollection");
```

Note: Each of the Column's 'name' property must match the 'column' or 'expression' property of each data region you specified when creating the chart. Note 2: Because data must be added via data

binding, rows modification using addRow, insertRow, etc is not supported.

Configuring the Chart's Properties

sap.makit Chart can be configured in various ways to fit different use cases. The following properties of chart can be modified:

- categoryAxis - This property is used to configure the property of the chart's Value axis. It must be an instance of sap.makit.CategoryAxis element. Normally it is the X-Axis but it can be Y-Axis in certain type of chart (e.g. horizontal bar chart). You can some of the generic properties such as showLabel, showPrimaryLine, etc. You can also modify property specific to Category Axis such as sortOrder of the axis.
- valueAxis - This property is used to configure the property of the chart's Value axis. It must be an instance of sap.makit.CategoryAxis element. Just a categoryAxis, you can some of the generic properties such as showLabel, showPrimaryLine, etc. You can also modify property specific to Value Axis such as min or max value of the axis.
- valueBubble - This property is used to configure the property of the chart's Value Bubble. It must be an instance of sap.makit.CategoryAxis element. Value Bubble is the pop-up sliding box that show the details of the currently highlighted chart item. You can modify its property such as position, showCategoryDisplayName, etc.

By default the Chart always creates a default instance of each properties if you did not specify one at chart's creation time. If you would like to configure one of the you can do:

```
var oChart = new sap.makit.Chart({
    width : "100%", height : "80%",
    category : new sap.makit.Category({ column : "yearCategory" }),
    series : new sap.makit.Series({ column : "productSeries" }),
    values : [new sap.makit.Value({ expression : "revenueValue", format : "currency" })]
    categoryAxis : new sap.makit.CategoryAxis({ displayLastLabel: true }),
    valueBubble : new sap.makit.ValueBubble({
        style: sap.makit.ValueBubbleStyle.FloatTop,
        showCategoryText: true
    }),
});
```

Or you can also modify them at runtime after the chart has been created:

```
oChart.getValueBubble().setShowCategoryText(false);
oChart.getCategoryAxis().setDisplayLastLabel(false);
```

The rest of the modifiable properties can be found in the MAKit API reference.

Combination Chart

A Combination Chart is a chart that allow multiple layers of graph to share the same category but have their own values and series group and drawn in different styles.

Combination Chart consist of 2 components:

1. sap.makit/CombinationChart control
2. sap.makit/Layer element

To create a MAKit combination chart, user creates a CombinationChart control and add multiple Layer elements to it.

CombinationChart acts as the container for the combination chart.

Adding multiple charts to the CombinationChart is through 'layers' aggregation. Each chart in the CombinationChart is represented by the Layer object.

CombinationChart control contains all the properties, aggregations, events, and methods that are shared by all the layers.

For example:

- CombinationChart will have categoryRegions aggregation which are shared by all the layers.
- Layer will have the 'type' property which apply to that particular layer only.

For details of the properties, aggregations, events, and methods of the CombinationChart and Layer please check the MAKit API reference.

How to create a Combination Chart:

1. Create a CombinationChart control.
    1. set the width and height
    2. define the categoryRegions
2. Create as the layers as required.
    1. set the chart type
    2. define the values regions
    3. define the series regions (if needed)
    4. Define columns data. Must also define the column that are used in the categoryRegions of the CombinationChart
3. Bind data to the layers (because each layer have their own set of data).
4. Add the layers to the CombinationChart control (you can add the layers before binding the data too).
5. place the CombinationChart control onto a place holder to render.

    Important Note: All the layers must be created and added before the CombinationChart is rendered. Once it is rendered, no new layers can't be added or removed, although chart type and data can still be changed. This is current limitation of the MAKit CombinationChart.

For a more detailed example please refer to the Demo Application?

Combination Chart - Dual Axis

An important features on Combination chart is the concept of dual axis. It allows user to define one or more layers to be scaled and drawn based on a secondary value axis. The secondary value axis usually positioned on the right hand side of the chart (bottom if on Bar chart).

To use dual axis feature, user only need to set a property of Layer object: drawOnSecondaryAxis to true

```
var layerObj1 = new sap.makit.Layer({
        type : "Column",
        drawOnSecondaryAxis: true
        });
```

OR

```
layerObj1.setDrawOnSecondaryAxis(true);
```

The above example will set that specific layer to be scaled and drawn on the secondary axis.

CombinationChart's 'secondaryValueAxis' property (type: ValueAxis) can be modified to alter the secondary value axis' properties.

Configuring the Chart's Theme & Styling

The MAKit Chart makes use of SAPUI5 Theme Parameters to allow dynamic styling and theming. You can adjust the following parameters in the Theme Editor to change the colors of the chart's item (e.g. bar's color). font size of the table view, etc.

Chart's Theme Parameter	Description
sapUiValueBubbleHightlight	Value Bubble highlight's color

sapUiValueBubbleLegendBG	Value Bubble Legend's background color
sapUiValueBubbleLegendTextColor	Value Bubble Legend's text color
sapUiChartRangeSelectorFontSize	Range Selector's font size
sapUiChartRangeSelectorFontFamily	Range Selector's font size
sapUiChartRangeSelectorFontColor	Range Selector's font color
sapUiChartTableFontSize	Table View's font size
sapUiChartTableFontFamily	Table View's font family
sapUiFillPaletteColor1 to sapUiFillPaletteColor16	Chart's color palette for drawing chart's item (e.g. bar)

Useful Links

Link	Decription
MAKit	MAKit API References
MAKit Chart	MAKit Chart API References

Creating a SAPUI5 Application Project

Prerequisites

- You have installed the *SAPUI5 Application Development* feature in your Eclipse installation.

Step 1: Start Application Wizard

You can start the "New SAPUI5 Application Project" wizard in the Eclipse IDE via *New* → *Other...* → *SAPUI5 Application Development* → *Application Project*:

Step 2: Fill project related data

- provide a project name
- optionally chose a location (prefilled from current workspace)
- choose the target device 'Desktop' (default) or 'Mobile', for Mobile see also SAPUI5 Mobile Library
- decide whether an initial view should be created (a view can also be created later on with the "SAPUI5 Application View" Wizard)

Step 3 (if initial view should be created): Fill view related data

- enter a folder in which the view should be created
- provide a view name
- choose the development paradigm (either JavaScript or XML or JSON or HTML view)

See also Creating a SAPUI5 Application View.

Step 4 (optional and only if initial view should be created): Confirmation page

Before finishing the wizard the summary information about selected data in previous pages is displayed. With ‘Back’-Navigation the entered date could be corrected and displayed/controlled again before application project is created.

Result

After finishing the wizard

- a new dynamic web project is created. All relevant files are created under the *WebContent* folder.
- prefilled index.html is created which contains sap.ui.commons lib and sap_goldreflection theme in the boostrap in case of desktop target device and sap.m lib and sap_mvi theme in case of mobile target device
- web.xml is created which contains settings for ResourceServlet and SimpleProxyServlet (usage of SimpleProxyServlet is restricted to localhost and only intended for testing purposes and not for productive use)
- installed SAPUI5 UI lib plugins are automatically added to the Java Build Path and added to the deployment assembly
- SAPUI5 Class Path Container (if available) is automatically added to the JavaScript Include Path
- page index.html is opened in standard editor
- inside the JavaScript block of the index.html there is **JavaScript code completion for SAPUI5 controls**
- an automatic switch to the J2EE perspective is performed
- if "Create an Initial View" was flagged on the first page of the "SAPUI5 Application Project" wizard, also a view and view controller are created, see View Wizard and the coding to call the view is added to the index.html, if mobile target device was selected, special coding instantiating sap.m.App and sap.m.Page is generated

Then the user can edit the index.html e.g. by adding the snippet between script tags inside the body tags:

```
  <script>
    // create a button
    var oButton = new sap.ui.commons.Button();
    oButton.setText("demo");

    // connect the button with the previously created <div>
    oButton.placeAt("content");
  </script>
...
  <div id="content"></div>
```

For a MVC example see Creating a SAPUI5 Application View.

As a dynamic web project is created, the user can export the project as war file via right click, press *Export...* and chose *Web → WAR file*. When importing the generated war the external referenced SAPUI5 libs get copied into the /WEB-INF/lib folder (or can be copied in the workspace as web projects) but the JavaScript Include Path is not available in the imported project.

Next Steps

- Try out Code Completion, Templates and Snippets
- Testing the SAPUI5 Application
- Enabling Cache Buster for productive use

Creating a SAPUI5 Application View

This page explains how to create a view which follows the UI5 Model-View-Controller (MVC) pattern.

Prerequisites

- You have installed the *SAPUI5 Application Development* feature in your Eclipse installation.
- You have created a *SAPUI5 Application* using the *SAPUI5 Application Development Tools*, see Creating a SAPUI5 Application Project.

Step 1: Start View Wizard

You can start the "New SAPUI5 Application View" wizard via *New* → *Other...* → *SAPUI5 Application Development* → *View*:

Step 2: Fill view related data

- choose a SAPUI5 Application Project in which the view should be created
- select a folder in which the view should be created (defaulted with *WebContent/<application name>*)
- provide a view name
- choose the development paradigm (either JavaScript or XML view or JSON or HTML view)

Result

After finishing the wizard

- inside the chosen SAPUI5 Application a view is created in the specified folder:
  - <viewname>.view.js in case of JavaScript view
  - <viewname>.view.xml in case of XML view
  - <viewname>.view.json in case of JSON view
  - <viewname>.view.html in case of HTML view
- if corresponding index.html file contains sap.m lib in the bootstrap (i.e. SAPUI5 Application Project has been created for mobile target device), the view contains coding for instantiating a mobile page control sap.m.Page
- also a controller file <viewname>.controller.js is created containing some draft coding
- for a JavaScript view there is JavaScript code completion for SAPUI5 controls

> If you rename the view or controller file or move them to a different folder, the coding in the view and controller and in the places where the view is used need to be adapted *manually*!

Integrate new View

If a new view has been created for an existing SAPUI5 application project the view needs to be **manually called either e.g. from index.html page or from another view via view nesting**.

Behavior

- view can only be created for a SAPUI5 Application Project (which has been created with the SAPUI5 Application Wizard) and not for other kinds of projects
- view name needs to be unique inside the project folder
- if positioned on a project, the project is defaulted as project for the view if it is a SAPUI5 Application Project, if positioned on a folder, the folder's project is defaulted as project for the view if it is a SAPUI5 Application Project and the folder is defaulted (in other cases the folder is defaulted with WebContent/<application name>)
- folder needs to be WebContent/<application name> or sub folder

Example Coding

Please also refer to Model-View-Controller Concept for more details and examples on the MVC concept.

Simple MVC Example

JavaScript view file

```
    createContent : function(oController) {
        var aControls = [];
        var oButton = new sap.ui.commons.Button({
                        id : this.createId("MyButton"),
                        text : "Hello JS View"
        });
        aControls.push(oButton.attachPress(oController.doIt));
        return aControls;
    }
```

XML view file

```
<core:View xmlns:core="sap.ui.core"
           xmlns:mvc="sap.ui.core.mvc" xmlns="sap.ui.commons"
           controllerName="myapplication.myview"
           xmlns:html="http://www.w3.org/1999/xhtml">
    <Button id="MyButton" text="Hello XML View" press="doIt"/>
</core:View>
```

JSON view file

```
{
    "Type":"sap.ui.core.mvc.JSONView",
    "controllerName":"myapplication.myview",
    "content": [{
      "Type":"sap.ui.commons.Button",
      "id":"MyButton",
      "text":"Hello JSON View",
      "press":"doIt"
   }]
}
```

HTML view file

```
<template data-controller-name="myapplication.myview">
    <div data-sap-ui-type="sap.ui.commons.Button" id="MyButton" data-text="Hello HTML View"
data-press="doIt"></div>
</template>
```

controller file

```
    doIt : function(oEvent) {
        alert(oEvent.getSource().getId() + " does it!");
    }
```

index file for JavaScript example

```
  <script>
        sap.ui.localResources("myapplication");
```

```
        var view = sap.ui.view({id: "idmyview1", viewName:"myapplication.myview",
                                type:sap.ui.core.mvc.ViewType.JS});
        view.placeAt("content");
  </script>
...
  <div id="content"></div>
```

index file for XML example

```
  <script>
        sap.ui.localResources("myapplication");
        var view = sap.ui.view({id: "idmyview2", viewName:"myapplication.myview",
                                type:sap.ui.core.mvc.ViewType.XML});
        view.placeAt("content");
  </script>
...
  <div id="content"></div>
```

index file for JSON example

```
  <script>
        sap.ui.localResources("myapplication");
        var view = sap.ui.view({id: "idmyview3", viewName:"myapplication.myview",
                                type:sap.ui.core.mvc.ViewType.JSON});
        view.placeAt("content");
  </script>
...
  <div id="content"></div>
```

index file for HTML example

```
  <script>
        sap.ui.localResources("myapplication");
        var view = sap.ui.view({id: "idmyview4", viewName:"myapplication.myview",
                                type:sap.ui.core.mvc.ViewType.HTML});
        view.placeAt("content");
  </script>
...
  <div id="content"></div>
```

Outline View for Controller and JavaScript View

From UI5 Tools 1.12.5.1 on the generated JavaScript files for the controller and the JavaScript view contain JS Doc for the methods with `@memberOf` with the benefit that the methods (if they are not commented) appear in the Eclipse Outline View. For already existing controllers and JavaScript views the following JS Doc snippet can be added to each method to make the Eclipse Outline View work (e.g. for view or controller name "myapplication.myview"):

```
/**
* @memberOf myapplication.myview
*/
```

Next Steps

Testing the SAPUI5 Application

Testing the SAPUI5 Application in Eclipse

Prerequisites

- You have installed the *SAPUI5 Application Development* feature in your Eclipse installation.
- You have created a *SAPUI5 Application* using the *SAPUI5 Application Development Tools*, see Creating a SAPUI5 Application Project.

There are two possibilities for testing the application locally in Eclipse:

- SAPUI5 Application Preview
- Using a Java Web Server

If you have created a *SAPUI5 Application* for *Mobile Target Device*, see Creating a SAPUI5 Application Project, such an application can only run within a WebKit browser, see SAPUI5 Mobile Library and supported browsers. But you can use the mentioned possibilities for testing the application locally in Eclipse and copy and paste the url into a WebKit browser.

If your application has to access resources on a backend system (OData), you have to enable the backend access for local testing.
If you deploy the SAPUI5 Application project on the ABAP Server then you have a possibility to test the application on the ABAP Server.

Testing Using SAPUI5 Application Preview

To test the new application with the Application Preview on an embedded Jetty server, right-click the HTML file or the project node and choose *Run As → Web App Preview*. Everything is configured automatically.

Refresh

After changing a file of your SAPUI5 application project, you can refresh the SAPUI5 preview by choosing the *Refresh* button on the left in the preview editor.

Open in External Browser

To check the files of your SAPUI5 application project in an external browser (IE/FF), choose *Open in external browser* on the right in the preview editor. This opens the external browser which is specified in the Eclipse preferences under *General → Web Browser* (usually this is the default browser of your PC). You can also copy the URL from the text field of the editor to an external browser (this is useful for external browsers other than the default browser).

Testing Using a Java Web Server(e.g. Apache Tomcat)

Ensure that the corresponding Server Adapter Eclipse Plugin is installed.

You can find instructions for setting up Tomcat here.

To test the new application, right-click the new project, choose *Run as ... > Run on Server*, select a server (for example, Apache Tomcat), and choose *Finish*.

UI5 Library Location Used for Testing

If the UI5 bootstrap tag contains `src="resources/sap-ui-core.js"`, the UI5 runtime libraries from the Eclipse plugin are used.

If you like to test your UI5 application in Eclipse against a different UI5 Library location (e.g. on the ABAP server when running in the SAP NetWeaver UI AddOn scenario), you can configure the ResourceServlet. For that, open the `web.xml` file located in the `<WebContent folder name>/WEB-INF` folder and configure the parameter `com.sap.ui5.resource.REMOTE_LOCATION` and `com.sap.ui5.resource.PREFER_REMOTE_LOCATION` of the ResourceServlet where the placeholders {protocol}, {host name}, {port number}, {path to UI5 library} are to be exchanged by the real protocol, host name, port number and path to the UI5 library, see also Tunneling a Remote Location:

```
<servlet>
  <display-name>ResourceServlet</display-name>
  <servlet-name>ResourceServlet</servlet-name>
  <servlet-class>com.sap.ui5.resource.ResourceServlet</servlet-class>
</servlet>
...
<!-- force to use the remote location -->
<context-param>
  <param-name>com.sap.ui5.resource.PREFER_REMOTE_LOCATION</param-name>
  <param-value>true</param-value>
</context-param>
<!-- add the remote location for the UI5 libraries -->
<context-param>
  <param-name>com.sap.ui5.resource.REMOTE_LOCATION</param-name>
  <param-value>{protocol}://{host name}:{port number}/{path to UI5 library}</param-value>
</context-param>
```

In case of issues when accessing HTTPS-systems see Accessing HTTPS-Systems.

Enabling Backend Access for local testing

If you are testing locally in your Java Eclipse environment, a proxy is needed to access an OData service in the ABAP system. For the purpose of local testing the SAPUI5 Application project uses a SimpleProxyServlet (usage of this is restricted to localhost and only intended for testing purposes and not for productive use). In case of issues when accessing HTTPS-systems see Accessing HTTPS-Systems.
Such a proxy is not needed if you run the application from the ABAP system and access an OData service in the same system.

Ideally, all OData service URLs should be in one file to make the exchange easier - either in the `index.html`, or in one separate .js file which needs to be included. You can use a helper function `getServiceUrl` to use the service url with 'proxy' if running locally in Eclipse or the service url without 'proxy' when running on the server, see the following code example:

```
<script>
//One place for defining service URLs - either in the index.html file,
//or in one separate .js file which needs to be included.
//The application is responsible for exchanging the URLs before checking in and
//after checking out to SAPUI5 Repository;
//or using a helper function getServiceUrl for which also the application
//is responsible.

//var serviceUrl = "/mypath/myservice";        //url when running on the ABAP system
//var serviceUrl = "proxy/mypath/myservice";  //url when running locally in Eclipse

var serviceUrl = getServiceUrl("/mypath/myservice");

function getServiceUrl(sServiceUrl) {
  //for local testing prefix with proxy
  //if you and your team use a special host name or IP like 127.0.0.1 for localhost please
adapt the if statement below
  if (window.location.hostname == "localhost") {
      return "proxy" + sServiceUrl;
  } else {
      return sServiceUrl;
  }
}
</script>
```

As parameter for the `getServiceUrl` helper method, use the URL of the OData service document without *{protocol}://{host name}:{port number}*:
Like: */mypath/myservice*

> Place this script tag **before** the script in which the view is called (*sap.ui.view*).

Simple Proxy Configuration

Be aware that due to security reasons the SimpleProxyServlet is **restricted to local testing purposes only**. It can only be used for local host scenarios (accessing Gateway services to avoid cross-domain issues) and will not work when deployed on an application server. For productive use, refer to a mature proxy servlet.

Intranet Servers

The SimpleProxyServlet allows proxy requests to an arbitrary server in the intranet.
The proxy URL that is used in the coding looks like this: *proxy/<service url>*.

For that, open the `web.xml` file located in the `<WebContent folder name>/WEB-INF` folder and configure the parameter `com.sap.ui5.proxy.REMOTE_LOCATION` of the SimpleProxyServlet where the placeholders {protocol}, {host name}, {port number} are to be exchanged by the real protocol, host name and port number:

```
  <servlet>
    <servlet-name>SimpleProxyServlet</servlet-name>
    <servlet-class>com.sap.ui5.proxy.SimpleProxyServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>SimpleProxyServlet</servlet-name>
    <url-pattern>/proxy/*</url-pattern>
  </servlet-mapping>
  <context-param>
    <param-name>com.sap.ui5.proxy.REMOTE_LOCATION</param-name>
    <param-value>{protocol}://{host name}:{port number}</param-value>
  </context-param>
```

Internet Servers

The SimpleProxyServlet can be configured for proxy requests to internet servers in the same way as for Intranet Servers. Additionally, proxy settings might be necessary.
As the SimpleProxyServlet automatically uses the proxy settings from your Eclipse this can be configured in Eclipse under *Window → Preferences*, and select the *General → Network Connections*. Here you can specify the proxy entries and the proxy bypass.

For example: set *Active Provider* to Manual, *Schema*=HTTP, *Host*=proxy, *Port*=8080 under proxy entries and localhost, *.sap.corp as *Host* under proxy bypass.

Simple Proxy Servlet - Restriction regarding DELETE requests

The simple proxy servlet does not currently support the sending of HTTP DELETE requests with content. This is due to restrictions of the Java SE functionality that is used. If an HTTP DELETE request with content is sent, an HTTP 500 result status is sent with the description: "The HttpUrlConnection used by the SimpleProxyServlet doesn't allow to send content with the HTTP method DELETE. Due to spec having content for DELETE methods is possible but the default implementation of the HttpUrlConnection doesn't allow this!"

For practical purposes, this restriction should have only minor effects. This is because:

1. When applying a REST-like programming style, an HTTP DELETE request would use the URL to transmit which objects should be deleted, but not the content.
2. When building your own protocol that uses the content of the HTTP request, you typically use HTTP POST.

Accessing HTTPS-Systems

For accessing HTTPS-systems via the ResourceServlet or the SimpleProxyServlet it might be necessary to import the root certificate into the Java keystore.

JavaScript Code Completion

Prerequisites

- You have installed the *SAPUI5 Application Development* feature in your Eclipse installation.
- If your Eclipse installation contains the feature *org.eclipse.wst.jsdt.feature* in Version *1.3.1* it is recommended to update it. In this version invoking the JavaScript code completion might cause Eclipse to crash.

Automatic Code Completion for SAPUI5 Application Projects

The Eclipse JavaScript Development Tools (JSDT) are providing an editor which parses scripts and offers such a functionality for code completion.

If a SAPUI5 Application has been created with the SAPUI5 Application Development Tools the core libraries are made available for the code completion **automatically**.

Enabling Code Completion for other Projects

If you are not working with a SAPUI5 Application Project, you can do the following preparing steps to add the required SAPUI5 core libraries to the JavaScript include path.

Ensure that the JavaScript Facet is set:

- Open *Project → Properties*
- Go to *Project Facets*
- If you do not see the list of all possible facets, click the link: "Convert to facet form" and wait a second to see all available facets.
- Mark JavaScript Facet on the same view
- Leave the project properties

Your project now has the JavaScript facet. Now you can add the UI5 core libraries:

- Open *Project → Properties*
- Go to *JavaScript → Include Path*
- Use *Add JavaScript Library...*
- Select *SAPUI5*

After having applied this configuration to your project, you should see the following JavaScript resources in your project now:

JavaScript Templates

Prerequisites

- You have installed the *SAPUI5 Application Development* feature in your Eclipse installation.
- If your Eclipse installation contains the feature *org.eclipse.wst.jsdt.feature* in Version *1.3.1* it is recommended to update it. In this version invoking the JavaScript code completion might cause Eclipse to crash.

There is a functionality to add SAPUI5 control specific templates for JavaScript code. These templates are available e.g. in JavaScript views of SAPUI5 Applications development. They are generated during startup of the Eclipse runtime.
The templates are a full blown overview over all available

- control properties
- aggregations
- associations and
- events

To insert a template do the following steps:

1. Open the Eclipse JavaScript editor
2. Type the name of the desired control or the name of the alias, e.g. *button*. Also just the beginning part of the name or alias is working.
3. Hit <CRTL>+<SPACE>
    - choose from code completion list

Finally delete the lines which are not needed from the pasted content.

Adjusting templates

You can change templates to adjust them to your needs just by editing them and saving them under a different name or description. **If the user misses to change the name or description the changes are lost on next startup.**

SAPUI5 Snippets

Prerequisites

- You have installed the *SAPUI5 Application Development* feature in your Eclipse installation.

There is a functionality to add SAPUI5 specific code parts, the so called "SAPUI5 Snippets". These code snippets are templates and examples on how to use the SAPUI5 runtime and controls and what can be done. They are available as prepared HTML pages with no separation between Model, View and Controller (MVC) and they are generated during startup of the Eclipse runtime.

To open the Snippets View do the following steps:

1. In Menu click: *Window* → *Show View* → *Other ...*

2. Select *General* → *Snippets* & Click 'OK'

3. The *Snippet* View appears:

To insert a snippet do the following steps:

1. Create a SAPUI5 Application Development -> Application Project
2. Open the *index.html* in the HTML Editor
3. Delete all content: <CRTL>+<A>+<ENTF>
4. Insert the Snippet code in the editor by double clicking on the snippet or by using Drag&Drop
5. Save the code and run it in the integrated browser

6. If you encounter problems with incorrectly rendered pages open the default external browser

7. Page displayed correctly:

Synchronizing with the SAPUI5 Repository

Purpose

This tutorial describes how to use the SAPUI5 Repository and how to deploy the SAPUI5 application project on the ABAP Server. For this application type, the ABAP Server is used as a file share to serve browser requests. The SAPUI5 Repository is technically based on the BSP Repository of the ABAP Server.

Availability

The SAPUI5 Repository is available in SAP Business Suite systems with version 7.00 or higher containing the NetWeaver UI Add-On in the software component `UI_INFRA`. The Team Repository Provider functionality, which is available in the Eclipse IDE, is available in SAP Business Suite systems with version **7.31** containing the NetWeaver UI Add-On in the software components `UI_INFRA` and `UI5_731`.

The used SAPUI5 Libraries are available in SAP Business Suite systems with version 7.00 or higher containing the NetWeaver UI Add-On in the software component `UISAPUI5`.

From 7.40 SP1 on all required SAPUI5 parts are already part of software component `SAP_UI`.

As an alternative for SAP Business Suite system of version **7.00** and higher - especially until 7.31 - you may use the interactive **ABAP report /UI5/UI5_REPOSITORY_LOAD** offering similiar functionality. Compared to the Team Repository Provider it does not offer a built-in code merge. Here an separate source code repository like git, Suberversion (SVN) etc. may be used. The report is shipped via SAP note 1793771 "Up/Download SAPUI5 Application into/from UI5 Repository" and within SP03 of the NetWeaver UI Add-On. Find the corresponding documentation in the note and attached to the report.

Setup

- Make sure you install the *SAPUI5 Application Development* feature as well as the *SAPUI5 ABAP Repository Team Provider* feature in you Eclipse installation.
- For 7.31:
  - Make sure that the software component SAP UI5 TEAM PROVIDER ON 731 (UI5_731) is installed on the 7.31 ABAP backend
  - Make sure that SAP Note 1684342 is applied in the 7.31 ABAP backend.

Enabling Virus Scan during Upload

When uploading files to the SAPUI5 Repository, you can perform a virus scan.

As of SAP NetWeaver 7.00 with UI AddOn, SAP delivers the following virus scan profile for ABAP within the UI AddOn (from 7.40 SP1 on the virus scan profile is already part of the software component `SAP_UI`):

- /UI5/UI5_INFRA_APP/REP_DT_PUT

This profile is used by the SAPUI5 Repository API to store files in the SAPUI5 Repository based on BSP Repository.
For example: Upload of a local file using SAPUI5 Repository API /UI5/CL_UI5_REP_DT, method /UI5/IF_UI5_REP_DT~PUT_FILE from 7.00 on, or the SAPUI5 Team Repository Provider in 7.31 or 7.40.

The profile is deactivated when delivered. To activate it, first create at least one basis profile and save it as the default profile. You can then activate one of the delivered profiles. By default, it links to a reference profile, which is the default profile. For more information, see SAP Help Portal:

- ABAP-Specific Configuration of the Virus Scan Interface (7.00)

Remarks

- Only CR+LF is supported as a new text file line delimiter.
  - To set the new text file line delimiter in Eclipse, go to *Window* → *Preferences*, under *General* → *Workspace* change *New text file line delimiter* → *Other:* to *Windows*.
  - To change the new text file line delimiter in Eclipse for an existing project, go to project *Properties*, under *Resource* change *New text file line delimiter* → *Other:* to *Windows*.
  - To change the new text file line delimiter in Eclipse for an existing file, open the file and choose *File* → *Convert Line Delimiters to* → *Windows*.
- Not all code pages (text file encoding) are supported.
  - To set the text file encoding for new files in Eclipse, choose *Window* → *Preferences*, under *General* → *Workspace* change *Text file encoding* → *Other:* to *UTF-8*.
  - To change the text file encoding in Eclipse for an existing project, go to project *Properties*, under *Resource* change *Text file encoding* → *Other:* to *UTF-8*.
  - To change the text file encoding in Eclipse for an existing file, go to file *Properties*, under *Resource* change *Text file encoding* → *Other:* to *UTF-8*.
- If you view the BSP pages on the ABAP server, the might look different to the version in Eclipse or at runtime:
  - Trailing spaces in text lines are escaped with " " in the respective BSP pages.
  - Lines longer than 254 are broken into several lines and have a *+* on position 255.

Sharing the SAPUI5 Application Project with the SAPUI5 Repository

Purpose

You want to upload a new SAPUI5 Application into a new BSP Application on the ABAP server.

Prerequisites

- You have created a *SAPUI5 Application* using the *SAPUI5 Application Tool*, see Creating a SAPUI5 Application Project.
- Optionally you have tested the *SAPUI5 Application* locally in Eclipse, see Testing the SAPUI5 Application in Eclipse.
- The ABAP system you are using needs to be available in your SAP GUI Logon group. It is recommended to enable Single-Sign-On.

Creation of BSP Application

It is possible to create the BSP application with all required ICF nodes on the ABAP server remotely from Eclipse. However there might be cases in which this is not possible automatically:

- if the ABAP system is an SAP system (not a customer system) and the name of the BSP application and ICF node is in the customer namespace (for example, ZMYAPP)
- if the BSP application contains a SAP customer / SAP "development" namespace and that namespace has already been created in another system (for example, /ABC/MYAPP).

In these cases the BSP Application has to be created **manually** with the following steps before sharing:

1. In the ABAP system, call transaction *SE80*. Go to the *Repository Browser*, and select *BSP Application* from the dropdown menu.
2. Create a BSP application and enter `/UI5/CL_UI5_BSP_APPLICATION` as the name of the application class.
3. Save and activate the application.
4. Next, call transaction *SICF*, where you create a SAPUI5 application-specific ICF node.

1. In the *Service Path* field, select `/sap/bc/ui5_ui5/sap` as the path (or alternatively the respective namespace */sap/bc/ui5_ui5/<namespace>*), and choose *Execute*.
2. In the context menu for the *sap* (or the respective namespace) node, choose *New Sub-Element*.
3. In the *Create a Service Element* dialog, use the same name as for the BSP application you created as the *Name of Service Element to Be Created* and choose *Enter*.
4. Provide a description text for the ICF node and choose *Save*.
5. Return to the tree and position the cursor on the new service node.
6. Right-click the new service node to call the context menu, and choose *Activate Service*, then *Yes*.

Procedure

The SAPUI5 application project can be shared with the SAPUI5 Repository as follows:

1. In Eclipse, go to the *Project Explorer view* and select the required *SAPUI5 Application Project*.
2. In the context menu of the selected *SAPUI5 Application Project*, choose *Team → Share Project...*.

3. Select *SAPUI5 ABAP Repository* as the repository type.

4. Choose *Next*. Configure the connection to the ABAP system by using the *Browse...* button. You can only select system connections that are configured in the SAP GUI Launchpad.

5. Choose *Next*. Provide the required information for client, user, password, and language.
   Note that choosing the correct logon language is important. When creating a new repository this will become the original language of the repository. For submitting files to the repository you have to be logged on in the original language, otherwise an error will occur.

6. Choose *Next*. The SAPUI5 application project can either be shared with an existing or with a newly created BSP application.
   6a. Select *Create a New BSP Application* and enter name, description, and package (manually or via the *Browse...* button).

Choose *Next*. Note: If the new BSP application belongs to a transportable ABAP package, choose/create a transport request before finishing the wizard. Since transport requests are not automatically released, explicitly release them later using transaction *SE09*.

6b. Alternatively, if the BSP Application has been created manually or you want to download an existing one, choose *Select a BSP Application* and select an existing BSP application

artifact for SAPUI5 from the list.

7. Choose *Finish*.

Result

The selected SAPUI5 application project is now connected to the SAPUI5 BSP application artifact of the SAPUI5 Repository.
A description containing the ABAP system, client, user, language, and SAPUI5 BSP application name is displayed next to the project name.
If you have shared a SAPUI5 Application project, the SAPUI5 runtime library version of the server will be compared against the ones which have been installed into Eclipse and a warning might be shown. You can find more details about this check here.

Submitting the SAPUI5 Application Project to the SAPUI5 Repository

Prerequisites

- *SAPUI5 Application* has been created with the *SAPUI5 Application Tool*, see Creating a SAPUI5 Application Project.
- *SAPUI5 Application Project* has been shared with Team Provider, see Sharing SAPUI5 Application with SAPUI5 Repository.

Procedure

1. In Eclipse, choose *Team* → *Submit* in the context menu of the selected *SAPUI5 Application Project*. If a logon is required, enter your password in the logon popup and choose *OK*.

If you are submitting changes of a SAPUI5 Application project, the SAPUI5 runtime library version of the server will be compared against the ones which have been installed into Eclipse and a warning might be shown. You can find more details about this check here.

2. Select all files. You will get a list of files that have been modified (added, updated, or deleted) in the client. The dialog shows the files for which submit conflicts exist, for example, if another user has submitted a newer file version in the meantime. Submit conflicts must be resolved before submission. If a file is already locked in a transport request, the corresponding request is shown in the dialog. If the SAPUI5 BSP application belongs to a transportable ABAP package, you have to choose a transport request. Note that transport requests are not automatically released when the files are submitted; you still have to release them using transaction SE09 in the underlying ABAP system.

3. Choose *Finish*.

Result

All files have been submitted to the SAPUI5 Repository and are available under the corresponding SAPUI5 BSP application.
The SAPUI5 application project can now be run from a Web browser calling the underlying ABAP system. You have the convenient option in the context menu for an HTML page of starting a browser with the correct URL from the BSP application displayed in the ABAP Workbench (SE80).

Remark

When using the team provider the file type is determined based on the Eclipse preferences. If you submit binary files and submit fails due to "End of line marker not supported" error, the binary file type is probably not registered as binary. To change that in Eclipse go to *Window* → *Preferences* and then to *Team* → *File Content* and register the binary file type as type binary.

Retrieving the SAPUI5 Application Project from the SAPUI5 Repository

Prerequisites

- You or someone else has created a *SAPUI5 Application* using the *SAPUI5 Application Tool*, see Creating a SAPUI5 Application Project.

- You or someone else has shared and submitted the *SAPUI5 Application* via the SAPUI5 Team Provider to the ABAP system, see Submitting SAPUI5 Application to SAPUI5 Repository.
- The ABAP system you are using needs to be available in your SAP GUI Logon group.

Procedure

1. Create a generic project in Eclipse: Choose *New* → *Project* → *General* → *Project*. Enter the same name that is used for the SAPUI5 BSP application artifact in the ABAP system. Choose *Finish*.
2. Share the *SAPUI5 Application Project* with Team Provider by choosing the existing BSP application, see Sharing SAPUI5 Application with SAPUI5 Repository.
3. Synchronize the SAPUI5 application project as follows: In the context menu of the selected project, choose *Team* → *Retrieve*. Next, choose *Select All* to select the conflicting files as well.

4. Choose *Finish*.

Result

Application-project-related files have been retrieved from the SAPUI5 Repository.
The SAPUI5 application project can now be run locally in Eclipse, see Testing the SAPUI5 Application in Eclipse.

Handling Conflicts

Submit / Retrieve Dialogs

When you are submitting or retrieving a change, the *Conflict* column on the *Submit Changes* or *Retrieve Changes* screen indicates whether there is a conflict for a file. The dialog shows the files for which submit/retrieve conflicts exist, that is, another user has submitted a newer file version in the meantime. Submit conflicts must be resolved before submission.
If retrieve conflicts cannot be solved, the local files will be overwritten.

To set the conflict files as resolved, select the file (or multiple files) and choose *Set Resolved*. The same functionality is also available in the context menu of the selected file or files.

To display the conflicts with the *Compare Editor*, choose *Edit Conflict*.
The *Edit Conflict* function is not supported for multiple file selection.

Once you have analyzed and edited the changes, you can choose *Save and Set as Resolved* for the conflict file.

Synchronize View

In the *Synchronize* view, you can compare the local and remote state.

1. Mark the SAPUI5 application project in the *Project Explorer* in Eclipse.
2. To open the *Synchronize* view, still in Eclipse, choose *Window* → *Show View* → *Others ...*, then choose *Team* → *Synchronize*.
    1. To connect to a synchronization type, choose the *Synchronize ...* icon, select *Synchronize with SAPUI5 Repository* as the synchronization type, and choose *Finish*.
    2. To update the *Synchronize* view, press *F5* in the *Synchronize* view.

3. The *Synchronize* view now shows differences between the local and remote state for all shared projects, by indicating whether the project is in sync with the repository or, if not, indicating the differences.

The synchronize view also allows you to schedule a periodic refresh of the state of the files in the backend.

Compare Editor

1. From the *Synchronize* view, you open the *Compare Editor* by right-clicking the conflicting file and choosing *Open in Compare Editor* (or double-clicking the file) .
2. You can now display and resolve the conflicts in the *Compare Editor*.
3. Once the conflict is resolved, you can set the status of the file to "Resolved" by right-clicking the file and choosing *Set Resolved*. The file is now ready to be submitted.

Further Information

The Eclipse Help provides information about:

- Synchronizing with the repository - information site in Eclipse: http://help.eclipse.org/indigo/index.jsp?topic=%2Forg.eclipse.platform.doc.user%2Ftasks%2Ftasks-115.htm
- Workspace synchronization - information site in Eclipse: http://help.eclipse.org/indigo/index.jsp?topic=%2Forg.eclipse.platform.doc.user%2Freference%2Fref-33.htm
- Compare editor - information site in Eclipse: http://help.eclipse.org/indigo/index.jsp?topic=%2Forg.eclipse.platform.doc.user%2Freference%2Fref-25.htm
    - Although the description is based on the CVS repository, it offers some insights into the principles of synchronization and three-way comparison.

Testing the SAPUI5 Application Project on the ABAP Server

Purpose

To test a *SAPUI5 Application* that you have uploaded to an ABAP system, you can directly launch its public URL in a browser window.

Prerequisites

- You have shared a *SAPUI5 Application* via the SAPUI5 Team Provider with the ABAP system
- You have synchronized the application with the ABAP system

Procedure

You have the following options to launch a synchronized file (e.g. the index.html) on the ABAP server:

- Right click the file in the Project Explorer and choose *Run As → Run On ABAP Server*
- Open the file in an editor and from the Eclipse *Run As* menu choose *Run On ABAP Server*

The launch option is only available if the file is accessible via URL e.g. if it is in the WebContent folder.

Result

The URL of the selected file is opened in a browser window. The URL has the following pattern:

```
<protocol>://<host name>:<port number>/sap/bc/ui5_ui5/<namespace>/<application name>/index.html?sap-client=<client>&sap-ui-language=<language>&sap-ui-appcache=false
```

Please note that this URL contains test parameters which should not be set for productive usage!

You can also start the SAPUI5 application as follows:
Select the SAPUI5 application-specific Internet Communication Framework (ICF) node in transaction SICF under sap/bc/ui5_ui5/<namespace>/<application name>, and choose the context menu option *Test Service*. This will then automatically start your *index.html* page.

Cache Behavior for Application Resources

The index.html page which is never cached.

By default, the application files are stored in the browser cache for one year to speed up the performance in a productive environment. To get the latest changes you need to force your UI5 start page to refresh (e.g. by pressing `CTRL + F5` on Windows systems). If forcing to refresh does not help you need to clear your browser cache.

The following is an **experimental feature** which might change in future versions of SAPUI5: If you are in development mode and like to immediately get the latest changes without refreshing your UI5 start page, you can add the experimental url parameter `sap-ui-xx-devmode` to the UI5 start page in order to force the browser to check if necessary whether there is a newer version of the application files available. When switching between development and productive mode, you need to force your UI5 start page to refresh. If this does not help you need to clear your browser cache.

Using the Application Cache Buster

If you activate the Application Cache Buster in the start page of the application in which the ui5 bootstrap is located by setting `data-sap-ui-appCacheBuster="./"`, see Application Cache Buster, the UI5 application runtime handler on ABAP side supports the Application Cache Buster. In that case the ICM server cache on ABAP side as well as the browser cache is used and the Application Cache Buster mechanism ensures that all those resources which are supported by the Application Cache Buster mechanism are up to date.

Some Technical Remarks ...

- Under typical conditions - e.g. when using UI5 applications or when doing development with the SAPUI5 ABAP Repository Team Provider or the UI5 repository load report - the application cache buster works fully automatically.
- After system- and language imports for a UI5 application into the SAP NetWeaver Web AS however it may be necessary (since the UI-AddOn 1.0 SP03) to reset its internal cache buster information. You may explicitly use or schedule the ABAP report /UI5/RESET_CACHEBUSTER to do so on all servers. As an alternative you can trigger the reset for a specific UI5 application from the browser using the URL "<application base path>/resetcachebuster".
- Also take note that the internal cache buster information for a UI5 application becomes outdated every two hours and is calculated anew then automatically. As a consequence the underlying server side cookies (with name 'cachebuster_info') get recalculated without the need of manual acitivity.

Creating Alias for ICF Node with SAP Corbu Logon Screen

To create an alias for the `ui5_ui5` ICF node with an SAP Corbu logon screen (available with version 7.31 and higher), do the following:

1. In the ABAP Suite system, call transaction `SICF` and choose *Execute (F8)*.
2. Choose → *External Aliases (Shift+F6)*

3. Select the host, for example `default_host`, and choose *Create New External Alias (F5)*.
4. Enter a name in the *External Alias* field. The name must start with a slash, for example `/ui5`.
5. On the *Trg Element* tab, select the target handler `/default_host/sap/bc/ui5_ui5` in the tree, and double-click it.
6. On the *Error Page* tab, select the *Logon Errors* tab, select *System Logon*, and choose *Configuration*.
7. In the *System Logon Configuration* dialog, choose *Define Service-Specific Settings*.
8. Under *Select Display*, select the fields that are to be displayed on the logon screen; these are generally System ID, Client, and Language.
9. Under *Logon Layout and Procedure* and *SAP Implementation*, select *Signature Design* for Screen and *SAP Corbu* for Theme.
10. If you choose *Adjust Links and Images*, you have the option under *Adjustment of the Logon Page* to *Specify the URLs for the images you want to use instead of the standard images*.
11. Choose *Input (Enter)* to confirm the selection.
12. Choose *Save (Ctrl+S)* to save the new alias.

For information on the ICF, see:

- System Logon
- Administration of Server Functions

Troubleshooting with Communication Log

ABAP Communication Log

For troubleshooting purposes, you can use the *ABAP Communication Log* to log all REST requests from and to the ABAP system.

1. To open the *ABAP Communication Log* view, go to Eclipse and choose *Window* → *Show View* → *Others ...*, then *ABAP* → *ABAP Communication Log*.
2. To start the logging, choose *Start logging* (and to stop it later, choose *Stop logging*).
3. The ABAP Communication Log displays all requests between Eclipse and the connected ABAP system, giving you information about the HTTP method (GET, PUT, POST, DELETE), the query parameters of the request, and the HTTP status code.

1. If you click a request in the list, the details of this request are displayed (such as Request, Response, Stack Trace), giving you information about the HTTP header fields and the content of the request and the response.

Next Steps

Translating Text Elements of SAPUI5 Applications in ABAP

Guide for SAPUI5 Application Developers

In the following section a text enclosed by angle brackets is meant as a placeholder.

Prerequisites

- You are using the SAPUI5 Repository to store your UI5 application artifacts, see Synchronizing with the SAPUI5 Repository.

In a Nutshell

Texts in SAPUI5 Application can be translated in an ABAP system. It is a requirement that the name of the file containing the text elements ends with '.properties' and a special key in the format `# SAPUI5 TRANSLATION-KEY <GUID with 32 characters>` must be provided in the first line (Exchange the complete palceholder <GUID with 32 characters>, including the angle brackets, with a vaild 32char GUID).
In addition, every text element needs a classification such as the text type.

Functional Overview

- The purpose of the text API in the SAPUI5 Repository is to enable an ABAP translation process for text elements of SAPUI5 applications developed in Eclipse that are stored in the SAPUI5 Repository.
- The text elements are located in a property file (here denoted as a '<name>.properties' file, where '<name>' stands for an arbitrary identifier).
- The text elements stored in '<name>.properties' files are transferred to a database table when a SAPUI5 application is deployed to a SAPUI5 Repository on an ABAP server.
- In addition, these table entries are written to a transport request.
- These texts can then be translated in the usual way.
- The master language of the SAPUI5 Repository is taken as the master language for the submitted text elements.
- The translated texts can then be accessed during runtime.
- The SAPUI5 Text Repository is updated each time the property file is submitted to the SAPUI5 Repository (creation, update and deletion of texts is supported)

Prerequisites for Translating Text Elements of '<name>.properties' Files in ABAP

The '<name>.properties' file must fulfill certain prerequisites for processing:

The table entries for the text elements are written if the following prerequisites are met:

- The file name must end with '.properties'.
- The first line in the '<name>.properties' file must contain a certain identifier: `# SAPUI5 TRANSLATION-KEY <GUID with 32 characters>`.
  Important Notice: This GUID serves as a unique identifier for the 'properties' file, therefore it must **never** be copied to other '<name>.properties' files and it must **never** be changed in this file.
  The GUID can be created with the ABAP function module 'GUID_CREATE', or the complete line can be generated with the ABAP report /UI5/TEXT_FILE_GEN_TRANS_KEY.
- The file must not contain any duplicate text elements.

Additionally, transport entries are created if:

- The corresponding SAPUI5 BSP application is not located in a local package ($TMP).
- A valid transport request is available.

Important update with SP05:
Until SP04 it was possible to rename or relocate a properties file and the text elements of it contained in the UI5 Text Repository were preserved after submitting again.
This was possible as the GUID serves as the only identifier of the properties file, as a consequence text entries will be overwritten if another properties file with the same guid is submitted (which is forbidden!).
From SAP05 on: Accidental overwriting of text elements in the UI5 Text Repository with text from another properties file is prevented.
If a duplicated guid is detected, it is not possible any more to synchronize the properties file.
As a consequence it is not possible any more to rename or relocate a properties file and to submit it once it is contained in the SAPUI5 Text Repository.
If a renaming or relocation of a properties file is required, the text elements have to be removed from the UI5 Text Repository with abap program /UI5/TEXT_ENTRIES_DELETE.
Caution: Translated texts get lost in that case, only used this deletion report if the texts are not yet translated.
Afterwards the renamed or relocated properties file must be submitted again.

Whenever the property file with the translation-relevant texts is relocated, whether to another folder, application, or package, the file must be submitted again to the SAPUI5 repository, from SAP05 on the notes from above (Important update with SP05) apply(
If the BSP application on the ABAP server is reassigned from a local ($TMP) to a non-local package, the file also needs to be resubmitted. In this case, all corresponding existing texts with local assignment are deleted from the table and the property file will be treated as a new file, which means that all texts are created again with a new creation timestamp.

Important Notice: As the master language of the SAPUI5 Repository serves as the master language of the texts, it is important that the correct language is used for the creation of the repository.
When using the SAPUI5 Team Repository Provider and creating the BSP application manually the correct language has to be chosen on the logon screen.

Structure of the property file

Classification of Text Elements in a '<name>.properties' File

Text elements in the '<name>.properties' file are simple value key pairs separated by an equals sign. However, in order to enable proper translation for these text elements, it is necessary to classify the texts elements with additional information. This additional information must be placed in the line directly above the text element and must begin with the number sign ('#').
The complete line must have one of the following structures:

- `#<TextType>`
- `#<TextType>,<MaximumLength>`
- `#<TextType>,<MaximumLength>:<AdditionalContextInformation>`
- `#<TextType>:<AdditionalContextInformation>`

Text Type

Each text element must have a text type assignment. Text types can be, for example, XBUT for button texts, XFLD for field labels, or XTXT for general texts. A list of the various possible text types is provided below.

Maximum Text Length

The optional maximum text length information (simple number) can be provided directly after the text type, separated by a comma.
The maximum text length must be above the original text length and must never exceed 255.

Notice:
In order to support a proper translatability of the text, the developer should set an appropriate

maximum text length following these rules:

- Source text length **< 8 characters** : max. text length: **multiply by 5**, min. 12 characters
- Source text length **between 8 and 30 characters** : max. text length: **multiply by 3**
- Source text length **> 31 characters** : max. text length: **multiply by 1.5**

So the max. text length should always be significantly higher than the original text length, whenever possible.

Additional Context Information

An optional additional comment (a note for the translator) can be provided for text elements after the text type (or after the length information), separated by a colon. Although this additional context information is not mandatory, it can help translators to find the most suitable translation for the text element.

List of Possible Text Types

- SAPUI5 short texts with length of less than 120 characters:

Text Type	Corresponding S2X Type	Description
XACT	accessibility	Accessibility
XALT	alternativetext	Alternative text
XBCB	breadcrumbstep	Breadcrumb step
XBLI	listitem	Bullet list item text
XBUT	button	Button text
XCAP	caption	Caption
XCEL	cell	Cell
XCKL	checkbox	Checkbox
XCOL	tableColumnHeading	Column header
XCRD	tabStrip	Tabstrip
XDAT	datanavigationtext	Data navigation text
XFLD	label	Label
XFRM	frame	Frame
XGLS	term	Term
XGRP	grouptitle	Group title
XHED	heading	Heading
XLGD	legendtext	Legend text
XLNK	hyperlink	Hyperlink text
XLOG	logentry	Log entry
XLST	listbox	List box item
XMEN	menu	Menu header
XMIT	menuitem	Menu item
XMSG	messagetext	Message text
XRBL	radio	Radio button
XRMP	roadMapStep	Roadmap step
XROW	tableRowHeading	Table row heading
XSEL	selectiontext	Selection text
XTBS	tab	Tabstrip text
XTIT	tableTitle	Table title
XTND	treeNode	Tree node text
XTOL	quickInfo	Quick info text

XTXT	generaltext	General text

- SAPUI5 long texts with length of more than 120 characters.
  Attention: The total line length of the text element must not exceed 255 characters.

Text Type	Corresponding S2X Type	Description
YACT	accessibilitylong	Accessibility (long)
YBLI	list	Bullet list item text
YDEF	definition	Definition
YDES	description	Description
YEXP	explanation	Explanation
YFAA	faqa	FAQ answer
YFAQ	faq	FAQ
YGLS	glossarydefinition	Glossary definition
YINF	informationtextlong	Information
YINS	instruction	Instruction
YLOG	logEntrylong	Log entry
YMSE	errorMessage	Error message
YMSG	messagetextlong	Message text (long)
YMSI	informationMessage	Information message (long)
YMSW	warningMessage	Warning message
YTEC	technicaltextlong	Technical text
YTIC	ticker	Ticker/marquee
YTXT	generaltextlong	General text (long)

NOTR No translation

Valid Example of a '<name>.properties' File

Note: Exchange the <GUID> in the first line with a real generated 32-character GUID.

```
# SAPUI5 TRANSLATION-KEY <GUID>
#XBUT,20
BUTTON_SAVE = Save
#XMSG
MESSAGE_ERROR1 = Error during creation of {0}
#XCAP,20: furniture element
CAPTION_TABLE1 = Table 1
#XFLD: first name in address input dialog
FORMULA_FIRSTNAME = First Name
```

Access of translated texts during runtime

The translated texts are directly read from the table and send to the browser during runtime.
The browser sends a request for a certain property file with a language extension to the backend, e.g. i18n_en_US.properties.
The handler class in the backend analyzes the file name and extracts the language key.
All text elements that are contained in the default property file are read with this language key from the corresponding SAPUI5 Text Repository tables.

These text elements are collected and send back to the browser, like if a property file with this language extension would physically exist, but it is not present in the repository.

Procedure for Development in the correction phase

If an application is in the correction phase, development has to conduct an additional step when performing text changes.
Note: In the correction system an additional task in the used transport request is required for the table entries of the SAPUI5 text repository (because it needs another classification than "Repair")
If there is double maintenace the changes should only be submitted to the SAPUI5 repository in one system and then applied to the other (correction) system via the SCWB (software correction workbench, transaction code scwb).
Since the SCWB does not support table entries, the abap report **/UI5/TEXT_FILE_SAP_UPDATE** has to be executed after applying the request with scwb.
This report requires the following import parameter:

- Name of the application
- The path of the property file which contains added or changed texts

The path of the property file is not required if the application contains only a single property file. The path of the property file can be copied directly from the abap workbench: if the property file is selected, the path within the application is contained in the "Page" filed at the top of the screen.
Texts that are only changed are then updated automatically in the text repository.
For new texts a popup will appear asking for a GUID for that text, here the original guid of the text element has to be entered from the system where the text has been originally created. This can be found in table /UI5/TREP_TEXT, attribute PROP_TEXT_GUID.
. This is needed to ensure key consistency and to avoid duplicate elements later on.

If a note for customers is created, the same steps have to be performed at customer site:
The note should contain a step by step description for the customer:

- Manual steps to perform after the note is applied:
  Execution of report **/UI5/TEXT_FILE_SAP_UPDATE**
- Name of the application and property file which contains changed text elements
- For each new text element: The name (key) of the text element from the property file and the corresponding GUID of that text element from table /UI5/TREP_TEXT in the original SAP system.

Since the translated text elements will come to the customer system only with the next support package, the customer needs to translate the new SAPUI5 text elements in transaction se63 beforehand, if required.[[BR]

Translation

The following information is relevant for translators.

- SAP Note **1686090** must be implemented in the translation system to enable the translation of SAPUI5 text elements. SAPUI5 text elements are treated as ABAP short texts with translation object type: **UI5T**.
- The translation object name is a GUID, which is the key taken from the original property file containing the text elements (the GUID from the first line) (see under Prerequisites:`# SAPUI5 TRANSLATION-KEY <GUID with 32 characters>`). Therefore, all text elements from a property file have the same (translation) object name.
- The text key of each text element consists of the text type and an individual GUID, separated by a blank.
- The UI5T texts are stored in the following database tables (all three tables also contain an indicator for the SAP/customer namespace):
  - **/UI5/TREP_TEXT** Master table with the text name, unique text GUID (contained in the text key displayed in SE63), text type, additional context information, and another GUID that is the (translation) object name (the GUID from the property file).

    - **/UI5/TREP_TEXT_T** Language-dependent table containing the original and translated text; the key is the text GUID as in table /UI5/TREP_TEXT and the language key.
    - **/UI5/TREP_FILES** Another table, which contains the (translation) object name (the GUID from the property file) and path information for the property file.
- The **UI5T** translation object type needs to be assigned to the respective languages for translation in table LXE_MASTER (table LXE_LA_OB), otherwise it will not show up in the translation worklist.

How the Find the "Where-Used" Location of a Text Element

Finding the origin of a text in the source code has to be done manually, until more tool support is delivered for this functionality:

- The translator sees a GUID as the object name of the translation object. This GUID can be used to find the corresponding property file (path information) in table /UI5/TREP_FILES; the GUID corresponds to the field PROP_FILE_GUID.
- The translator sees a key for each text element, which is a concatenation of the text type and another GUID. This GUID corresponds to the field PROP_TEXT_GUID in table /UI5/TREP_TEXT; here you can find the text name for a particular GUID.
- Finally, the developer can use this text name to search in the corresponding project in Eclipse to find the location where this particular text is used.